THE SMYSL MANUAL

Working with the CLI, the Document Format, and the Library

A complete, worked guide to every workflow — the CLI, the document format, and the library beneath both

Vladimir Ulogov

2026 · smysl 1.0.0 · format smysl/1.0 · kernel smysl.kernel/0.1

Contents

BEFORE YOU START

How to read this book

This book is about twenty-nine chapters long, and reading it front to back is the right plan for exactly one kind of reader. Everyone else should skip, and this page is about skipping well.

Whichever path you take, **Chapters 1 and 2 are the ones not to skip.** Chapter 1 is why this exists — the problem, and how big it measurably is. Chapter 2 is what the system is made of and the twelve rules it enforces. Every later chapter names those rules by letter and assumes you have met them.

Four ways in

IF YOU ARE...	READ	WHY THAT PATH
Deciding whether to adopt this at all	1, 2, then 5	Chapter 1 gives you the measured cost of not having it and the honest cases where you do not need it. Chapter 5 builds a real document end to end in a few pages, which is the fastest way to find out whether the shape suits you. That is about an hour, and it is enough to make the call.
Going to write documents by hand	1–8, then 21	Part I orients you, Chapters 6–7 are the full authoring grammar, Chapter 8 is the validation loop you will live in. Chapter 21 explains what `check` is actually doing when it complains, which turns error messages from obstacles into information.
Wiring this into a pipeline	1–3, 9–12, 19–20, 23	The trust and staging machinery is Chapters 9–12 and is the part that has real operational consequences. Chapters 19–20 are budget-bounded selection — what survives when you have fewer tokens than document. Chapter 24 is how the output reaches a person.
Embedding the library in your own program	1–2, 27–29	Chapter 27 is the crate layout and the feature flags, Chapter 28 is a minimal working embedding, Chapter 29 runs

IF YOU ARE...	READ	WHY THAT PATH
		the whole pipeline start to finish. Come back for the command chapters when you need the behaviour one of them describes.

What the boxes mean

Five kinds of block interrupt the prose, and each one is a promise about what it contains, so you can skim on them.

> **WHY**
>
> A **Why** box appears before a piece of machinery and says what goes wrong without it. If you already know why something exists, this is the box to skip. If you are lost, it is usually the box you needed.

> **TERM** **Term**
>
> A **Term** box defines a word this book will then use precisely and without further apology. Every one of them is restated in the glossary, Appendix D.

> **TRY THIS**
>
> A **Try this** box is something to run, not something to read — a short, checkable task using files that ship in the repository, with the answer stated so you can tell whether you got it. Every one was run against the real binary before it was written down. Skipping them is fine; doing them is the difference between recognising the format and being able to use it.

> **WHAT'S NEXT, AND WHY**
>
> A **What's next** box ends a section by naming the next reasonable step and the reason for it. These are the connective tissue — if you are reading selectively, they tell you where the thread you are pulling actually goes.

> **WHAT YOU LEARNED**
>
> - A **What you learned** box closes each chapter. Read these first if you are deciding whether a chapter is worth your time — they are the chapter's claims, without the evidence.

What you need in front of you

Two things, both covered in Chapter 4: a built `smysl` binary, and a checkout of the repository — because nearly every example in this book operates on files under `fixtures/corpus/`, and the incident document `fixtures/corpus/F1-incident.smy` is the running example from Chapter 13 onward. Commands are shown with their real output, produced by running them; where output is long it is trimmed, and the trimming is marked.

No model access is needed for any chapter except 10 and 11, and both say so at the top. Everything else runs offline — including Chapter 12, whose whole subject is reporting what **would** be sent without sending it.

> **A NOTE ON THE NUMBERS**
>
> Where this book states a measurement — token ratios, claim-retention rates, how much a `pack` dropped — it is reporting a run, not an estimate, and the command that produced it is shown. Where something has not been measured, the book says that instead. Chapter 1's headline numbers come from two models and five documents, which is enough to establish an effect and not enough to put an error bar on it; that limitation is stated there too.

PART I

Foundations

CHAPTER 1

1

Why This Belongs in Your Pipeline

You can read this whole manual as a description of a file format and a command-line tool, and it will be accurate. It will also leave out the reason any of it exists. This chapter is the reason. It assumes you have never built an AI pipeline, uses no vocabulary it does not define, and ends with the one thing worth knowing before you spend an afternoon on the rest: what problem this solves, how big that problem measurably is, and when you can safely ignore it.

What a pipeline is, if you have never built one

WHY

"Pipeline" is one of those words that everybody in the field uses and nobody defines, because the people using it built one before they named it. If the word is doing any work in your head other than "several programs in a row", the rest of this chapter will read as abstract when it is in fact extremely concrete.

TERM Pipeline

A **pipeline** is a chain of steps in which the output of one step is the input to the next. Some steps are programs, some are language models, and at least one is usually a person. Nothing about the word implies automation, scale, or sophistication — two steps and an email is a pipeline.

Here is one, and it is the example this manual keeps returning to — it ships in the repository as `fixtures/corpus/F1-incident.smy`, and from Chapter 13 onward most chapters operate on it directly. A service got slow on Thursday. Four things happen:

1. Somebody — or something — **reads the raw material**: the alert history, the dashboards, the deploy log, three messages in a chat channel.
2. A model **drafts an account** of what happened, drawing on that material.
3. A second model, or the same one an hour later, **merges that draft** with a second account written by someone looking at a different subsystem.
4. A person **reviews the result** and signs off — and then a fourth step turns the signed-off version into a report that goes to people who were not in the room.

Every arrow in that list is a **handoff**: one participant finishes and passes something to the next. Nothing here requires artificial intelligence — this was the shape of incident review long before models could write. What the models changed is the number of handoffs anyone is willing to tolerate, because each one is now cheap.

The handoff is the lossy part

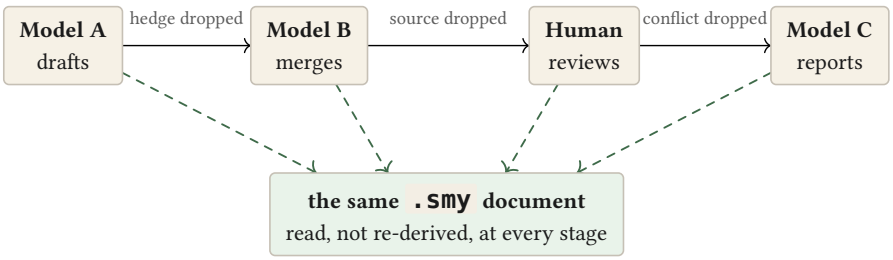
> **WHY**
>
> The failure this tool exists for is not in any step. Every step in the example above can be done impeccably and the pipeline will still degrade, because the loss happens in the gaps. That is an unintuitive enough claim that it is worth being slow about.

Ask what actually crosses each arrow, and the answer, almost always, is **prose**. A paragraph. Some Markdown. A chat message. That is the only medium every participant in the chain can both produce and consume, so that is what gets used.

Now consider what the drafting model **knew** while it was working. It knew that the latency figure came off a dashboard and the theory about the connection pool was its own guess. It knew the canary data pointed the other way. It knew which of its sentences it would defend and which it was floating. All of that structure existed, in the model's working state, at the moment it wrote — and then it wrote paragraphs, because paragraphs were the only thing on offer, and the structure did not survive the trip.

The next participant now has to guess that structure back out of the prose. It guesses, acts on the guess, and flattens **its** conclusions into prose for whoever is next. Do this more than once and the same three things go missing every time:

- **Hedges vanish.** "The data suggests" becomes "the data shows" becomes "we found." Each retelling is slightly more confident than the one before it, and nobody along the way earned the extra confidence. This is the dangerous one, because the output gets **more** persuasive as it gets less warranted.
- **Sources evaporate.** Nothing about a citation survives being paraphrased. By the third hop, the number is still there and the reason to believe it is not, and there is no way to tell from the text which numbers still have a source behind them.
- **Disagreement disappears.** Two conflicting findings go in; whichever one the last participant found more convincing comes out. The conflict was not resolved — it was dropped — and the result reads as unanimous precisely because the dissent is gone.



Top row: prose re-told four times, losing something specific at each hop. Bottom: one document, present unaltered at all four stages — there is no re-telling for anything to be lost in.

The property that makes this hard to catch is that **prose does not announce what it dropped.** A summary that lost every citation looks exactly like a summary that had none to lose. There is no error, no warning, no diff. The pipeline reports success at every step and the artifact quietly gets worse.

How much is lost — measured, not asserted

> **WHY**
>
> Everyone who has run a multi-hop pipeline already believes the three losses above happen. Almost nobody has a number for them, and the intuition most people carry — "a bit, at the margins" — is wrong by a large factor. The size of the effect is what decides whether this tool is worth your afternoon.

The project ships an evaluation harness that measures exactly this, and it is worth understanding the shape of the experiment before the numbers, because the shape is what makes them mean anything.

Five documents. Five hops each — five successive summarisations, each one reading only the previous hop's output, the way a real chain does. The whole thing run twice: once with `gemini-3.5-flash-lite` doing the summarising and once with `deepseek-chat`. Ninety original claims across the five documents.

Then a **different** model reads the far end and is asked, for each of the ninety original claims: does the final text still state this? How confidently does the text now put it? And what, if anything, does the text say it rests on?

```
$ make eval-live
```

	tokens	claims kept	hedges lost	sources kept
control (no summarising)	1.00	90/90	0/90	50/50
prose baseline	0.29	72/90	42/90	1/50
smysl	0.49	90/90	0/90	50/50

Three rows, and the order to read them in is bottom-up.

The **prose baseline** is the pipeline almost everyone is running: summarise, pass along, repeat. It kept 72 of the 90 claims. Of the 72 that survived, 42 came out the far end reading as flat statements of fact when the document that went in had marked them as inferred, cited or derived — the hedge was lost, but the

sentence was not, so the claim is still there and is now stronger than it was entitled to be. And of the 50 claims that originally named a source, exactly one still named it. Not one in ten. One.

The **smysl** row is the same five documents, the same five hops, the same two models — passing a `.smy` document instead of prose. Every claim kept, every hedge intact, every source intact.

The **control** row is the row that makes the other two trustworthy, and it is the one worth pausing on. It is the same judge, with the same prompt, reading each document **before any summarising happened at all**. If the judge were unreliable — if it simply failed to notice hedges, or could not find sources — the control row would show losses too. It shows none: 90 of 90, 0 lost, 50 of 50, and it declined to rule on nothing. So whatever went missing over five hops went missing **in the chain**, not in the instrument measuring the chain.

> **THE POINT**
>
> None of this is a criticism of the models. Both were asked to summarise and both summarised well — the prose output reads fluently and says broadly the right things. Hedges and citations are simply not what prose is built to preserve, and a summariser has no way to know it dropped one, because nothing in its input marked them as things that could be dropped.
>
> On the `smysl` side, both numbers are structural rather than lucky. Confidence is a field that a rule checks; a source is a field that travels with whatever travels. Neither depends on the model choosing to be careful.

Note the token column, because it is the honest part. The prose chain compresses harder — 0.29 of the original against 0.49 — and that is not `smysl` being inefficient. Prose compresses well **because** it is dropping the hedges and the citations. You are paying roughly 1.7× the tokens to keep them. Whether that is a good trade is a real question with a real answer that depends on your pipeline; this manual's position is only that you should know you are making the trade.

> **HONEST LIMITS**
>
> Two models, five documents, one run each. That is enough to say the effect is not one model's quirk. It is not enough to put an error bar on it, and this manual will not pretend otherwise. The harness is the `smysl-eval` crate and the `make eval-live` target; pointing it at your own documents gives you the number you should actually act on, which is not this one.

Two fixes that seem obvious and do not work

Before reaching for a new format, most people try one of two things. Both are reasonable, and it is worth knowing exactly where each one runs out.

Fix one: just use a schema

Agree on a JSON shape between each pair of steps and stop passing free text. This genuinely solves the machine-to-machine problem — a field called `confidence` does not evaporate the way the word "suggests" does.

It fails at the human step, and the human step is the one that mattered. Nobody reviews a wire format. When the artifact becomes a nested JSON document with

`{"claims": [{"id": "c-0417", "conf": 0.6, ...}]}`, the person in the chain stops reading the artifact and starts reading a rendering of it — and the rendering is prose again, produced by a step that can drop things. The one place a person could have caught the drift is now the one place they cannot see.

Fix two: just tell the model to be careful
Put "preserve all hedges and citations" in the prompt. This helps, somewhat, and it is the right thing to do regardless. It does not solve the problem for a structural reason: **the instruction is unverifiable.** When the summariser returns, nothing checks whether it complied. There is no assertion to fail, no exit code, no diff — you have added an intention to a system that had no way of noticing intentions being violated, and the failure mode is unchanged. It fails silently and reports success.

> TERM **The position `smysl` takes**
>
> Make the artifact itself **precise enough** to carry hedges, sources and disagreement as structure a program can check — and **plain enough** that the same file is what a person opens to review the work. One artifact, not a wire format plus a rendering of it. The bytes the machine reads and the bytes the human reads are the same bytes.

The four things a unit carries that a paragraph cannot

Here is a complete, real excerpt — not a simplified illustration. This is what crosses the wire, and it is also what a reviewer opens:

```
@evidence e/pool-wait { status: measured, source: { kind: metric, ref:
"pool.wait_ms" } }
~ Pool acquisition wait rose from 2 ms to 310 ms over the same window.

@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait] }
~ The eu-west connection pool is saturated.

@rel c/canary-clean --rebuts--> c/pool-saturation { weight: 0.6 }
```

> TERM **Unit**
>
> A **unit** is one recorded thing — a piece of evidence, a claim, a finding, a definition, a question. The `@evidence` and `@claim` blocks above are two units. The `~` line is the unit's **gist**: the one-sentence prose form, which is what a person reads and what a renderer emits.

Four things are being carried here that the paragraph version would have thrown away, and each one maps directly onto a loss from the measurement above.

WHAT IT CARRIES	WRITTEN AS	THE LOSS IT PREVENTS
How sure the document is entitled to be	`status:`	The hedge. `measured` (an instrument recorded it) is a different claim from `inferred` (a model reasoned it out), and the difference is a field rather than a tone of voice. It cannot be lost in a retelling because retelling is not how it travels.

WHAT IT CARRIES	WRITTEN AS	THE LOSS IT PREVENTS
What the claim rests on	`grounds:`	The chain of support. `grounds: [e/pool-wait]` says this claim stands or falls with that specific piece of evidence — so retracting the evidence tells you exactly what falls with it, by name, before you do it.
Where it came from outside the document	`source:`	The citation. A machine-checkable pointer travelling in a field, rather than a phrase in a sentence that paraphrasing dissolves.
What contradicts it	`@rel ... --rebuts-->`	The disagreement. The rebuttal is part of the document, sitting next to the claim it argues with — not filed in a review thread nobody opens twice. Selection routines are required to keep the two together.

The mechanism that makes the first two stick is a single rule, and it is worth stating now because it is the spine of everything: **a claim's status may be weakened as it crosses a hop, but never strengthened.** A model reading this document may propose at most `inferred` for what it reasons out. If it writes `measured`, the tool does not take its word for it. Chapter 2 states this precisely and names the two rules that enforce it.

What the tool refuses to do

> **WHY**
>
> A tool's refusals tell you more about whether it fits your pipeline than its features do, and they are usually buried. These are stated up front because two of them will disqualify `smysl` for some readers, and finding that out now is better than finding it out in Chapter 13.

- **It does not judge whether anything is true.** `status: measured` records that a unit was produced by measurement, not that the measurement was right. The tool checks that claims do not out-run their support; it has no opinion on whether the support is any good.
- **It does not pick a winner when two documents disagree.** `merge` records the disagreement as a first-class object and hands it to you. If you wanted a tool that resolves conflicts, this is not it — and the refusal is deliberate, because silently picking a winner is precisely the third loss.
- **It does not write your prose.** Every gist in a document was typed by a person or proposed by a model you explicitly invoked. Nothing generates text on its own initiative.
- **It does not phone home.** Of the twenty-two commands, exactly two — `ingest` and `attest`, whose entire job is calling a model — can open a socket, and they will tell you what they would send before they send it. The restriction is enforced by a build-time check rather than by promise.

Three places to put it, and what each one buys

You do not adopt this all at once, and the three insertion points are independently useful.

WHERE	WHAT IT GETS YOU
At the boundary — the last step before a person reads the output	The cheapest option and the one with the clearest payoff. Your pipeline stays exactly as it is; you add one `render` step so the reviewer sees status and sources rather than confident prose. You get the review surface without changing anything upstream.
Between two model steps — the handoff itself	This is where the measured numbers above come from. The document is read rather than re-derived at each hop, so there is no retelling for anything to be lost in.
At the entrance — `ingest`, turning source prose into units	The most work and the most value, because it is the only one that puts a trust ceiling on material entering the system in the first place. A model reading a document you supplied may propose at most `cited`, never `measured` — enforced at the door.

When you do not need this

Being honest about this is more useful than being persuasive about it. Skip `smysl`, or use only the rendering half, when:

- **Your pipeline has one hop.** Almost all of the measured loss is accumulation across hops. One model, one output, one reader: prose is fine, and the structure is overhead you will not get back.
- **Nobody reviews the output.** The format's second job is being readable by a person checking the work. If no person ever checks the work, you are paying for half a tool.
- **The stakes do not justify the tokens.** You are spending roughly 1.7× on the handoffs. For a draft nobody will act on, that is a bad trade and you should make it knowingly.

The pipelines where it pays are the ones with several hops, an artifact somebody is accountable for, and a reviewer who has to be able to ask "where did this number come from" and get an answer.

> **WHAT'S NEXT, AND WHY**
>
> Chapter 2 is the other half of the orientation: what the system is made of, and the twelve rules it enforces. Those rules are named by letter throughout the rest of the book — you will meet "rule M" and "rule T" repeatedly — and Chapter 2 is the one place they are all defined. If you would rather have a file in your hands before any more theory, Chapter 5 builds a small real one end to end, and Chapter 2 will still be there.

> **TRY THIS**
>
> 1. Run `smysl check fixtures/corpus/F1-incident.smy`. It reports **21 records, 8 units**. Open the file and account for the thirteen records that are not units — what are they, and why would a format that only had units be unable to carry Chapter 1's third loss?

2. In that file, `c/pool-saturation` is `inferred` and rests on `e/pool-wait`, which is `measured`. Suppose a downstream model wanted to publish `c/pool-saturation` as `measured` too. Its ground is already `measured`, so the "status may not exceed its weakest ground" rule permits it. What stops it anyway? (This is the distinction between the two rules Chapter 2 names; try to state it before reading on.)
3. Chapter 1 claims prose gives no signal when it drops something. Find the single line in `F1-incident.smy` that carries the disagreement, then read the gist of `f/root-cause`. What would a summariser have to do — not choose to do, but **be able** to do — for that disagreement to survive a paraphrase?

ANSWERS

1. Three `@rel` edges, one `@thread`, the `@doc` header, and one label binding per named unit: 8 + 3 + 1 + 1 + 8 = 21. The relations are the point: `--rebuts-->` is an edge, not a unit, so a format carrying only units would have nowhere to put a disagreement except inside the prose of one of the two units that disagree — which is exactly where a paraphrase loses it.
2. Rule M permits it; **rule T** forbids it. M caps a unit by what it rests on, and its ground is `measured`, so M has no objection. T caps a unit by the **rung of whoever produced it** — a model reasoning from its own priors works at the `model` rung, whose ceiling is `inferred`. The two rules constrain different things, which is why both exist: M watches the graph, T watches the door.
3. The line is the `--rebuts-->` edge from `c/canary-clean` to `c/pool-saturation`, and `f/root-cause`'s gist openly says the leading cause "is not consistent with the canary." For that to survive, the summariser would need to know that the contradiction was load-bearing rather than a hedge worth tidying — and nothing in a paragraph marks the difference. In the `.smy` file it is a typed edge that selection routines are **required** to keep with the claim (rule R), so no judgement about its importance is ever made.

WHAT YOU LEARNED

- A **pipeline** is a chain of steps where each one's output feeds the next. The loss this tool addresses happens in the **handoffs**, not in any step — every step can be done well and the artifact still degrades.
- Prose is the only medium every participant can produce and consume, and it cannot carry hedges, sources, or disagreement. Worse, it gives no signal when it drops them.
- Measured over five hops and two models: a prose chain kept 72 of 90 claims, turned 42 surviving hedged claims into flat assertions, and preserved 1 of 50 sources. The same chain over `.smy` documents lost none of the three, at roughly 1.7× the tokens.
- A schema fixes the machines and blinds the reviewer; a careful prompt is an unverifiable instruction. `smysl` makes one artifact that is both checkable and readable.
- The tool judges no truth, resolves no disagreement, writes no prose, and cannot reach the network in twenty of its twenty-two commands.
- The wire format has been implemented three more times — in Python, JavaScript and Go, each from the specification alone — and all three decode this one's output byte for byte. `Documentation/READINESS.md` tracks what that does and does not prove.

CHAPTER 2

2

The Shape of the System, and the Twelve Rules

Chapter 1 argued that structure has to survive the handoff. This chapter is about what actually enforces that — first the shape of the software, then the twelve rules that are the substance of the guarantee.

You are not expected to memorise the rules. They are stated here because the rest of this book names them by letter, in passing, dozens of times, and a reader who has never been introduced to "rule M" has to reconstruct it from context every time. This is the page to come back to.

One binary, ten libraries

> **WHY**
>
> Knowing the layering answers a question you will otherwise hit in Chapter 27: what can I use without taking all of it? The split is not decorative — the boundaries are where the guarantees are enforced, and two of the layers can be compiled out entirely.

`smysl` is one command-line binary sitting on a stack of libraries, each of which is usable on its own. The important structural fact is that the layer holding the data model has **no** dependency on the layers that talk to models or to the network — that direction of dependency is checked at build time, not maintained by discipline.

LAYER	CRATE	WHAT LIVES THERE
Kernel	`smysl-core`	The types, the deterministic binary codec, the surface syntax parser, and the diagnostic catalogue. Everything else is built on this. It cannot open a socket.
Graph	`smysl-graph`	The append-only store, its derived index, adjacency and lineage walking, merge, salience, and compaction.
Verification	`smysl-check`	The `check` pipeline — the passes that enforce most of the rules below.
Selection	`smysl-pack`	Budget-bounded, closure-complete selection: what survives when you have fewer tokens than document.
Structure	`smysl-thread`	The five thread schemas and their deterministic derivation.
Voice	`smysl-render`	Turning a graph into something a person reads — six target formats, driven by profiles.
Model boundary	`smysl-provider`	The only crate that speaks HTTP. Provider mappings, retry and backpressure, the usage ledger.
Ingest boundary	`smysl-ingest`	Prose in, staged units out — with the trust ceiling and the repair loop.
Harness	`smysl-eval`	The evaluation harness from Chapter 1. Not published.
Terminal UI	`smysl-tui`	The seven-pane interactive view.

Two of these — `smysl-provider` and `smysl-ingest` — are behind feature flags. A build with them compiled out is a build that provably cannot reach a model, which is a useful thing to be able to hand someone. Chapter 27 covers the flags in detail.

A kernel, and deliberate room outside it

> TERM **Kernel**
>
> The **kernel** is the fixed part of the data model: the unit types, the six status rungs, the relation kinds, and the record shapes. Everything the kernel defines, every conforming implementation must agree about.

Outside the kernel there is deliberate room. A header field the grammar does not recognise is not an error — it is kept, verbatim, in the unit's payload, and written back out in the same place. A record type from a later version decodes to an "unknown" record rather than failing the parse. This is rule X below, and its purpose is specific: a document from a team that records something yours does not, or from a version of the format newer than your binary, loses nothing by passing through your pipeline.

Identity is content

> **WHY**
>
> This one fact explains more surprising behaviour than anything else in the system. Every reader eventually asks why a unit's identifier changed after an operation that "only" adjusted its status. This is the answer, and knowing it early saves the confusion.

A unit's identifier is a hash of its canonical binary encoding — its type, its gist, its status, its grounds, its payload. Not a counter, not a UUID, not something assigned. Two people who record the same thing, in different field orders, on different machines, compute the same identifier; the encoding is canonical, so field order and formatting cannot affect it.

The consequence follows immediately and is not optional: **any transform that changes a unit changes its identity.** When a rule weakens a unit's status, the result is a different unit with a different identifier, and everything that pointed at the old one has to be moved to point at the new one — which the tool does, transitively, because a dangling reference would be worse than the original problem. When you author a violation yourself, `check` reports it instead of silently rewriting your file (Chapter 21); the automatic weakening path is the one `ingest` takes on a model's output, where there is no author to tell.

The twelve rules

Each rule is a promise the tool keeps, enforced in one identified place rather than by convention. The middle column is the promise; the right column is what you would see go wrong without it.

RULE	THE PROMISE	WHAT IT PREVENTS
M	A unit's status may not exceed its weakest ground.	A `derived` claim resting on a `speculative` one reading as settled. Confidence can fall through a chain of support, never rise.
T	A unit may not exceed the ceiling of the rung it was produced at.	Laundering. A model asserting from its own priors is capped at `inferred` however confidently it phrases the claim.
L	Closure: whatever a unit needs travels with it.	A selection or a bundle that arrives referring to grounds that were left behind — a claim with its support cut off.
R	A selected claim's rebuttals are selected with it.	The one-sided pack: compressing a document down to only the side that won, which is the third loss from Chapter 1 committed by your own tooling.
U	Merge is a join-semilattice union — commutative, associative, idempotent.	Order-dependent results. Merging A then B differs from B then A, and nobody can reproduce anyone else's store.
I	Ingest always makes progress.	A run that fails outright because one span of prose could not be structured. It degrades that span to opaque prose and continues.

RULE	THE PROMISE	WHAT IT PREVENTS
S	Model output never enters a store directly.	A model's proposal silently becoming part of your document. It lands in staging, and a human decision moves it.
V1	A rendering profile must render every status distinctly.	Two different confidence levels coming out looking identical — the hedge loss, reintroduced at the last step.
V2	Open disagreements are surfaced in the output.	A rendered report that reads unanimous over a document that was not.
X	Unknown extensions survive verbatim.	Data loss on round-trip through a version or a peer that did not know about some field.
D	Pure operations are bit-reproducible.	Silent drift — a command that quietly stops being a function of its inputs, so a rebuild produces a different artifact and nobody knows which was right.
P	`stdout` defaults to the binary form when it is not a terminal.	The pipeline surprise: piping a command into another and getting human-formatted text where structure was expected.

M and T are the pair that matter

If you remember two, remember these. They are the mechanical form of **a guess cannot quietly become a fact**, and they work at different places.

Rule T stops it at the door. A model producing units is working at a **rung** — `computed`, `document`, `web`, or `model` — and the rung caps the highest status it may claim. A model reasoning from its own knowledge is at the `model` rung and cannot exceed `inferred`, no matter what it writes.

Rule M stops it inside the graph. Once units are in, confidence flows through `grounds`, and it can only fall: a claim resting on a `speculative` unit is itself no better than speculative, whatever status was written on it.

> **WHERE THE RULES LIVE**
>
> Rules are not enforced by everyone remembering them. Each has an address: M and T are passes 6 and 7 of `check`, and are also applied at ingest; L is pass 4 and the thread repair step; R is constraint C3 of the packer; U is the merge implementation; I is the ingest repair loop; S is the staging file and exit code `10`; V1 is profile loading and V2 is render IR construction; X is the extension-carrying maps; P is the CLI's format selection. D is the odd one out — it is enforced by a test, `cargo xtask determinism`, which builds permutations of the same inputs specifically to catch a pure command that has stopped being one.

> **WHAT'S NEXT, AND WHY**
>
> Chapter 3 turns from what the system promises to how you will actually interact with it: what a store is, which records you write versus which the tool writes, and which commands can cost you money. Chapter 5 then builds a real document end to end.

TRY THIS

1. Run `smysl check fixtures/corpus/F6-adversarial.smy`. It reports three `SMY-E030` errors. Which of the twelve rules is `SMY-E030`, and what does the fact that there are exactly three of them tell you about how that rule is checked — once per document, or once per unit?
2. Read one of those three messages closely. It offers two ways out: "weaken this unit" or "strengthen its ground". Why is a tool that enforces "**status may only fall**" willing to suggest strengthening something?
3. `smysl fmt` on any corpus file returns the same records with fields in canonical order. Given that a unit's identity is a hash of its canonical encoding, what would break if `fmt` sorted fields the way you typed them instead? Name a specific two-person scenario.

ANSWERS

1. Rule M — a unit's status may not exceed its weakest ground. Three errors means it is checked **per unit**, not per document: `check` walks every unit and compares it against its own grounds, so a document with thirty violations reports thirty. That is what makes the output actionable — each message names the offending unit and the ground that capped it.
2. Because rule M constrains the **relationship** between a unit and its grounds, not the absolute level of either. There are genuinely two ways to restore the relationship, and the tool does not know which one is right — maybe you understated the evidence, maybe you overstated the claim. Only one of those is a licence to raise a status: strengthening the ground is subject to rules M and T in its own right, so the suggestion cannot be used to launder anything. The tool declines to guess and says so.
3. Two people record the same evidence with the same content but type the fields in different orders. With canonical ordering, both compute the same uid and `merge` recognises them as one unit. With as-typed ordering they compute different uids, so the merged store holds two units that are identical in meaning and unrelated in identity — every `grounds` reference points at one or the other, and a `--rebuts-->` edge aimed at one leaves the other unopposed.

WHAT YOU LEARNED

- One binary over ten libraries; the kernel has no path to the network, and the two crates that do can be compiled out.
- The kernel is fixed and small; everything outside it survives verbatim rather than being rejected, so a peer or a version that knows more than you loses nothing passing through.
- A unit's identity is a hash of its canonical content — so any transform that changes a unit moves its identity, and everything pointing at it is moved too.
- Twelve rules, each enforced at a named address rather than by convention. M and T are the pair worth memorising: T caps what a producer may claim, M caps what a claim may inherit from its support. Neither lets confidence rise.

CHAPTER 3

3

The Mental Model

Every later chapter in this book points a command at some data and reads back a report, a transformed store, or an artifact. That sentence is almost the whole tool. What takes a chapter to explain is not the mechanics of any one command — it's the small set of ideas that make all twenty-two of them feel like the same tool rather than twenty-two different tools that happen to share a binary. Get these ideas straight now and every subsequent chapter is a variation on a theme instead of a fresh pile of trivia.

The store

> **WHY**
>
> Every command needs to know two things before it can do anything: what data it's operating on, and where the result goes. `smysl` answers both with the same small vocabulary everywhere, so once you've read one command's `--help` you've effectively read the shape of all of them.

> **TERM** **Store**
>
> A **store** is whatever `smysl` reads its records from: a `.smy` surface file, a CBOR log (the binary form `merge -o file` and `pack` write by default), or `-` for stdin. Almost every command accepts one, either as a trailing positional argument or via the global `-s`/`--store` flag — the two are interchangeable. A store is not a database or a service; it is just bytes, in one of two encodings, that decode to the same set of records either way.

Concretely, a store is a sequence of **records**. `smysl check` reports the count of both when it looks at one:

● ● ● *$ smysl check fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: 21 records, 8 units, 0 diagnostic(s)
```

21 records reduce to 8 units because a store holds more than one record shape, and most of them are bookkeeping. Eight units, three relations, one thread, the `@doc` header — and eight **label bindings**, one for every name the file gave a unit. The rest of this section is about what those shapes are, and which ones you'll ever type yourself.

What you author, and what the tool derives

> **WHY**
>
> The previous shallow pass at this manual left readers guessing which parts of a `.smy` file are theirs to write and which parts show up on their own after running a command. That confusion is exactly what turns "staged" or "attestation" into a mystery word instead of an expected next step. Sorting records into **authored** and **derived** up front removes that guess before it can form.

A record is one of nine kinds (`smysl-core`'s `Record` enum, verbatim): `Unit`, `Attestation`, `Relation`, `Thread`, `View`, `Contention`, `PackInfo`, `SchemaDecl`, `LabelBinding`. You will hand-author three of these regularly, one occasionally, and never touch the rest directly:

RECORD	WHO WRITES IT	WHAT IT IS
Unit	You	A single claim, piece of evidence, finding, definition, hypothesis, prose beat, question, observation, data table, or artifact reference — the atoms of the document. Full grammar is Chapter 6 and the format guide.
Relation	You	A typed edge between two units — `--causes-->`, `--rebuts-->`, `--warrant-->`, and the rest — the connective tissue that turns a pile of units into an argument.
Thread / the `@doc` header	You (or `thread --derive`)	The document's own identity, roots, and granularity — and, separately, an ordered narration over existing units. You can write a thread by hand or have `smysl thread` derive one; Chapter 20 covers both paths.
View	Occasionally, via `smysl view`	A named, reusable selection of roots — a saved answer to "which units does this document actually need."
Attestation	The tool	The record an operation like `attest` or `ingest` attaches to a unit to say who backed it,

RECORD	WHO WRITES IT	WHAT IT IS
		at what trust rung, and when — never something you type.
Contention	The tool, during `merge`	A materialised disagreement between two stores that both spoke about the same unit — the thing rule C promises never gets silently dropped.
PackInfo	The tool, during `pack`	A self-describing receipt: how much of the budget was used, what got dropped or degraded, and under which estimator.
SchemaDecl	The tool, rarely you	A declaration of a non-kernel schema extension a store depends on.
LabelBinding	The tool, from what you typed	The record that remembers `c/pool-saturation` names a particular unit. You write the label as part of the unit; the binding is how it reaches the wire. It has to be its own record because a label is **not** identity — putting one inside hashed content would make renaming it produce a different unit.

WHY THE RECORD COUNT IS LARGER THAN IT LOOKS

A labelled unit yields two records, so the number `check` prints is always bigger than the number of things you typed. Worth knowing once and then forgetting: read **units** when you want to know how much document you have, and **records** when you want to know what a merge or a round trip has to carry.

Before 0.2 the bindings did not exist, and labels survived a parse but not a store — a document that had been through `merge` came back with every reference spelled as a bare `b3:…` uid. It still passed `check`, and no reader could follow it.

The pattern above is the one to keep: **you write intent, the tool writes provenance.** Nowhere in this book will you type an `@attestation` or a `@contention` block by hand — if you ever see one in a file, a command put it there, and that command's chapter explains why.

WHAT'S NEXT, AND WHY

Chapter 6 gives you the full authoring grammar for units and relations — every field, every status, worked examples of each unit schema. Chapter 5 gets you there faster with one small worked file, if you'd rather see the shape before the reference.

Purity: what can and can't reach off your machine

WHY

This is the single fact in this chapter with the most operational consequence. If you don't know which commands are deterministic, you don't know which ones are safe to run in a tight loop, wire into CI, run offline, or re-run without a second thought about cost or drift — and which ones need the trust and staging machinery the rest of this book spends four chapters on.

TERM Purity

Every command is classified as one of three things. **Pure** means the command is a bit-reproducible function of its inputs: same bytes in, same bytes out, on any machine, forever — no clock, no randomness, no network. **Mixed** means the command is pure except for one opt-in mode. **Model** means the command's entire job is to call out to a language model, and its output is therefore neither reproducible nor free.

Exactly two commands ever touch a model as the tool ships today: `ingest` and `attest`. A third, `thread`, is classified **mixed** rather than pure because `--refine` is planned for it — but that flag is not yet wired, so a `thread` run in this version is as deterministic as `fmt` is. The classification is deliberately pessimistic: it describes what the command is permitted to become, so that nothing downstream has to be re-audited the day the flag lands. Everything else in this book — canonicalising a file, validating it, packing it to a budget, merging stores, walking provenance, rendering to Markdown — is pure. The command table itself carries this classification, and each subcommand's own `--help` repeats it under `Purity:`:

COMMAND	PURITY
`fmt`	pure
`check`	pure
`pack`	pure
`merge`	pure
`diff`	pure
`trace`	pure
`view`	pure
`bundle`	pure
`thread`	mixed — pure today; reserved for `--refine`, not yet wired
`salience`	pure
`find`	pure
`retract`	pure
`render`	pure
`import`	pure
`relink`	pure
`compact`	pure

COMMAND	PURITY
`ingest`	model-dependent
`attest`	model-dependent
`providers`	pure — it reports what **would** egress, without egressing
`usage`	pure
`reindex`	pure
`ui`	pure — needs a terminal, and `--features tui`, which is not on by default

This is not a marketing claim; it is a tested one. The project's CI runs `cargo xtask determinism`, which builds permutations of the same inputs (reordered records, rebuilt indices, repeated runs) specifically to catch any pure command that has quietly stopped being a function of its bytes.

Operationally, the split means: run the fifteen commands that cannot reach a model as often as you like, in a pre-commit hook, in a CI matrix, on a laptop with no network — the answer is always the same answer, and there is no bill for asking twice. The two model commands are different in kind, not degree: their output depends on which model answered, may differ between runs, costs tokens, and can send your document's content somewhere else. That is exactly why `ingest` and `attest` never write directly into your store — Chapter 10 and Chapter 11 cover the staging and confirmation apparatus (exit code `10`, `--yes`, `merge --staged`) that exists specifically to put a human decision between a model's answer and your document.

> **WHAT'S NEXT, AND WHY**
>
> You don't need the model commands to get real work done — most of a document's life is pure operations. Chapter 9 explains the trust model they sit behind, before Chapters 10–12 cover ingest, attest, and the provider layer in depth.

Five phases, not an alphabet

> **WHY**
>
> Seventeen commands sorted alphabetically teach you nothing about order of use — `attest` would sit next to `bundle` next to `check`, three commands from three different moments in a document's life. This book is organised by what you are actually doing at each point, because that is the question you have when you reach for a command: not "what does `X` do" but "I have a document in this state, what do I do next."

A `.smy` document moves through five phases, and this book's Parts II through VII are exactly those five phases in order:

- **Create** — write or generate the units and relations that are the document's raw material (Part II: Chapters 6–7).
- **Infer and enrich** — optionally hand parts of that raw material to a model, under trust rungs and staging (Part IV: Chapters 9–12).
- **Operate** — combine, select, retract, thread, and query the resulting graph (Part V: Chapters 13–20).

- **Verify** — confirm the store is internally consistent, conformant, and reproducible before anyone downstream relies on it (Part VI: Chapters 21–23).
- **Export** — turn the graph into something a person reads (Part VII: Chapters 24–26).

Validation — Chapter 8's `check` — sits deliberately in its own short part right after creation, because in practice it isn't a phase you visit once; it's a loop you run after almost every edit, the way a compiler error is something you react to immediately rather than batch up. The five-phase structure describes the large-scale journey of a document; `check` is the thing you do continuously along the way.

TRY THIS

1. Run `smysl fmt fixtures/corpus/F1-incident.smy | smysl check -`. The report is identical to running `check` on the file directly — **21 records, 8 units, 0 diagnostic(s)**. Two things are being demonstrated at once. Name both.
2. You are on a plane with no network. Which of the twenty-two commands in the purity table can you not run, and which one **looks** like it should fail but will not?
3. A colleague sends you a `.smy` file containing an `@attestation` block. Chapter 3 says you never type one of those. What can you conclude about how that file was produced, and what should you look at next?

ANSWERS

1. First, `-` really is a store like any other, so commands compose in a pipe with no temporary file. Second, and more important: `fmt` changed the bytes — it expands defaults and renormalises quoting — and changed **nothing** about the record count, the unit count, or the diagnostics. Canonicalising is a transformation of spelling, not of content, and this pipeline is the cheapest way to prove that to yourself on your own files.
2. `ingest` and `attest` are the only two that need a model, so they are the only two that can fail. `providers` looks like a network command and is not — its whole job is reporting what **would** be sent, computed locally, so it works offline. `thread` also runs fine: it is classified **mixed** against a `--refine` flag that is not yet wired.
3. A command put it there — attestations are written by the tool, never by hand — so the file has been through `ingest` or `attest` rather than being purely hand-authored. Look at the attestation's rung, because that is what caps how strong any unit it covers is permitted to be (rule T), and at whether the units it covers came from a model rather than from a person.

WHAT YOU LEARNED

- A **store** is a `.smy` file, a CBOR log, or stdin — the same small vocabulary (`-s` / `--store`, a trailing path, `-`) works everywhere.
- Eight record kinds exist; you author units, relations, and threads/views by hand, the tool writes attestations, contentions, and pack info for you.
- Only `ingest` and `attest` are model-dependent as shipped (`thread` is classified mixed against a `--refine` that is not yet wired) — every other command is a deterministic, CI-verified function of its inputs, safe to run as often as you like.
- The book follows five phases — create, infer/enrich, operate, verify, export — because that mirrors the order you actually use the tool in, not the alphabet.

CHAPTER 4

4

Installing, Building, and Global Flags

Building the binary

WHY

`smysl` ships as one crate with a library and a CLI binary layered on top of it (Chapter 27 covers the library side in full). Before anything else in this book works you need that binary, and the way you build it already makes a decision about which parts of the tool you're getting.

From the repository root:

● ● ● *$ cargo build*

```
   Compiling smysl v0.13.0 (/Users/gandalf/Src/smysl)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.37s
```

That produces `target/debug/smysl`, which every example in this book invokes directly. A release build (`cargo build --release`) is the same binary, optimised; nothing about its behaviour changes, only its speed and size.

The feature matrix

`smysl`'s `Cargo.toml` defines a `[[bin]]` that `required-features = ["cli"]` — the binary does not exist at all unless the `cli` feature is on. This is deliberate: the crate's **primary** product is the library (`src/lib.rs`), and the CLI is its first, but not only, consumer. The feature flags decide how much of the tool you're compiling:

FEATURE	WHAT IT BUYS YOU
`cli`	The `smysl` binary itself and its argument parser (`clap`). Nothing below reaches you without this — it's why `--no-default-features` alone doesn't produce a binary.
`tui`	The `ui` subcommand: a terminal UI (`smysl-tui`, `ratatui`, `crossterm`) on top of the CLI.
`providers`	The provider registry and usage ledger (`smysl-provider`) with no concrete backend wired in yet — the plumbing `local`/`remote` build on.
`ingest`	`smysl-ingest`, plus `providers` — the library-level machinery behind `ingest` and `attest`, still with no model backend.
`local`	`ingest` wired to Ollama — a model running on your own machine, the only egress-free way to use `ingest`/`attest`.
`remote`	`ingest` wired to Anthropic, OpenAI, Gemini, and DeepSeek — every backend that leaves the machine, which is exactly what `--offline` and `providers --tasks` (Chapters 9–12) exist to police.
`render-typst`	The Typst backend for `smysl render`, on top of the always-available Markdown/HTML/text/JSON targets — see Chapter 25.
`render-html`	The HTML render backend.
`exact-pack`	Branch-and-bound search in `smysl pack --mode exact`, which proves optimality instead of only approximating it greedily (Chapter 19).
`tls-pure`	A Rust-only TLS stack for the provider crate, if you need to avoid linking a system TLS library.

`default = ["cli", "tui", "local", "render-typst"]` is what plain `cargo build` gives you: a full interactive tool that can run a local model, render Typst, and show a TUI, but cannot reach a remote model unless you also turn on `remote`.

> **WHY CHOOSE OTHERWISE**
>
> If you're embedding `smysl` as a library rather than shelling out to the binary (Chapter 27's whole subject), you almost certainly want none of the above: no `clap`, no TUI, no HTTP client, no async runtime pulled into your dependency tree. `--no-default-features` is how you ask for exactly that — the crate becomes a synchronous library with the same guarantees the facade documents: no panics on untrusted input, no global state, no hidden I/O.

Verify what actually happens, because it's easy to assume a flag like this either silently no-ops or breaks the build — it does neither:

● ● ● *$ cargo build --no-default-features*

```
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
```

That succeeds — quietly — because `cargo build` without `--bin` only builds targets that are buildable with the enabled features, and the `[[bin]]` requires `cli`. Ask for the binary explicitly and the real refusal appears:

● ● ● *$ cargo build --no-default-features --bin smysl*

```
error: target `smysl` in package `smysl` requires the features: `cli`
Consider enabling them by passing, e.g., `--features="cli"`
```

In other words: `--no-default-features` builds the **library** only, silently skipping the binary target rather than erroring — cargo simply narrows what it builds to what the enabled features permit. That's the mechanism Chapter 27 relies on when it walks through embedding the crate directly.

> **WHAT'S NEXT, AND WHY**
>
> If you only ever plan to use the `smysl` binary interactively, the default build is all you need — skip ahead to Chapter 5. If you're evaluating `smysl` as a dependency inside your own Rust program, keep this feature table in mind; Chapter 27 returns to it with a working embedding.

Global flags

> **WHY**
>
> Twelve flags are declared once, globally, and accepted by every subcommand — so instead of relearning them per command, learn them once here. Several don't matter yet and won't until a later chapter gives them a job; this table tells you which chapter that is so a flag you skip past today isn't a mystery when it resurfaces.

FLAG	WHY IT EXISTS	MATTERS MOST IN
`-s, --store PATH`	Names the input store; `-` reads stdin. Interchangeable with a trailing positional on most subcommands.	Every chapter
`-o, --output PATH`	Where to write the result; defaults to stdout so commands compose in a pipeline. Honoured by every command that emits one artifact; the three that print a **report** rather than an artifact — `view`, `salience`, `retract` — say so and point you at shell redirection instead of accepting a flag they will not act on.	Ch. 11–18, 22

FLAG	WHY IT EXISTS	MATTERS MOST IN
`--format surface|cbor`	Chooses the **shape** of output — human-readable surface text or the binary CBOR log. Defaults to `cbor` when stdout isn't a terminal, so a script gets bytes and a person at a keyboard gets text. On `merge` it also warns when the surface form cannot carry a record — contentions and attestations have no surface spelling.	Ch. 4–5
`-C, --config FILE`	Points at a configuration file — provider credentials, defaults — instead of environment variables or flags on every call.	Ch. 9–10
`--strict`	Promotes warning-severity diagnostics to the failure threshold, so a CI gate can refuse anything the tool merely frowns at. Honoured wherever a command has a warning to promote — `check`, `merge`, `pack`, `thread`, `fmt`, `render`. `bundle` produces no diagnostics, so there is nothing there for it to do.	Ch. 6, 19
`--offline`	Hard-fails rather than letting any byte leave the machine — the enforcement half of the trust model, not just a warning.	Ch. 7–10
`--json`	Machine-readable output instead of formatted text, for a caller that parses rather than reads. Honoured by every command that reports something: `check`, `diff`, `trace`, `salience`, `view`, `retract`. `retract --json` also carries `authorised` and `refusal`, which the text form prints on stderr where a machine reading stdout would never see them.	Ch. 6, 19
`-q, --quiet`	Suppresses the summary line that says it worked — useful once you trust a pipeline and only want to hear about failures. Diagnostics and exit codes are deliberately untouched: a quiet run that also swallowed its warnings would be a worse flag than one that did nothing.	Any scripted use
`-v, --verbose`	Increases verbosity; repeatable (`-vv`, `-vvv`) for progressively more diagnostic detail.	Debugging any command
`--no-color`	Disables ANSI colour, for logs and terminals that don't want it.	Scripted use
`--noprogress`	Disables progress bars unconditionally, whatever the terminal thinks it can render.	Large stores, CI logs
`--seed-check`	Asserts this exact invocation is bit-reproducible — the command-line hook onto rule D, the same determinism `cargo xtask determinism` checks in CI.	Ch. 21

A quick, real illustration of `--format` and stdout detection: `pack` (Chapter 19) writes CBOR by default because its natural home is a pipe into another command, but ask for `--format surface` and you get the same result as text you can read in this book.

The exit code contract

> **WHY**
>
> A pipeline that calls `smysl` needs to branch on **what happened**, not scrape stderr for a string. Every exit code below is part of the tool's stable contract — guaranteed not to shift meaning across a minor version — so a script can test `$?` the same way years from now. This table is the authoritative version; Appendix C only restates it.

CODE	NAME	WHAT IT MEANS
0	Success	Nothing to report.
1	Failure	A generic failure — usually an I/O or resolution problem that doesn't fit a more specific code below.
2	Usage	The command line itself was wrong — a missing required argument, an unknown subcommand — before `smysl` even looked at a store.
3	CheckErrors	`check` (or an operation that checks as part of its job, like `fmt --check`) found error-severity diagnostics, or `--strict` promoted warnings into that threshold.
4	PackInfeasible	`pack`'s mandatory floor doesn't fit in the requested budget — the budget is too small to hold what rule R says must survive together.
5	Contentions	`merge --fail-on-contention` found at least one open contention in the merged result.
6	Provider	A model backend failed — unreachable, unauthorized, rate-limited, or returned something malformed — for any reason other than the offline policy below.
7	Offline	The specific case of 6 where the failure was `--offline` refusing to let a call leave the machine at all — a policy outcome, not an accident.
8	UnsupportedVersion	The store declares a format or kernel major version this build doesn't understand.
9	HashVerification	A stored hash doesn't match its recomputed value — includes a truncated or corrupted uid, and `fmt`'s own round-trip check if canonicalising ever moved an identity.
10	Staged	Model output (from `ingest`) was written to a staging area rather than merged, and is waiting for `--yes` or `merge --staged` to confirm it.
11	StagedWithCorrections	The same, **and** rule M lowered at least one unit: the model claimed more than its grounds support and was corrected. Not a failure — the batch is intact and every corrected unit is in it — but a pipeline may want to route it differently. New in 0.2; a script testing `= 10` for "staged" should test `>= 10`.

Two of these are worth internalising now because they recur constantly: exit `3` is the one you'll see the most (Chapter 8 is entirely about avoiding and interpreting it), and exit `10` is not a failure at all — it's `ingest`'s way of refusing to finish the job for you. Chapter 11 explains why that refusal is the point, not a bug.

WHAT'S NEXT, AND WHY

You now have everything global out of the way. Chapter 5 puts it to work on one small, real document from start to finish, before Chapters 6 and 8 go deep on authoring and validation respectively.

TRY THIS

1. Run `cargo build --no-default-features`. It succeeds and takes about a hundredth of a second. Now run `ls target/debug/smysl` — the binary is either absent or stale. Then run `cargo build --no-default-features --bin smysl` and read the error. Why is the first command's silence correct behaviour rather than a bug?
2. Run `smysl check fixtures/corpus/F1-incident.smy >/dev/null; echo $?` and then the same for `F6-adversarial.smy`. You get `0` and `3`. Now predict: which exit code does a successful `ingest` return, and why is that the design rather than an oversight?
3. Run `smysl pack --budget 200 fixtures/corpus/F1-incident.smy | head -c 32 | xxd`. You get binary, not text, even though you asked for neither. Which rule is this, and what would have happened had you run the same command without the pipe?

ANSWERS

1. The `[[bin]]` target declares `required-features = ["cli"]`, and `cargo build` without `--bin` builds only the targets the enabled features permit — so it built the library, which is the crate's primary product, and narrowed away the binary rather than failing. Asking for the binary by name removes the ambiguity and produces the real refusal. The silence is cargo correctly building everything you asked for that is buildable.
2. `10` — **Staged**. A successful `ingest` writes its candidate units to `.smysl/staged.smy` and returns `10` rather than `0`, because the job is deliberately not finished: nothing from a model enters a store without a human decision (rule S). Returning `0` would mean "done", and it is not done. A script that treats `10` as failure and `0` as success gets the right behaviour by accident; one that treats any non-zero as an error will need to special-case it.
3. Rule P: `stdout` defaults to the binary CBOR form when it is not a terminal, so the natural thing to do with a pipe — feed another command — works without a flag. Run it without the pipe, straight to your terminal, and you get readable surface text instead. The same invocation adapts to who is reading; `--format` overrides the guess in either direction.

WHAT YOU LEARNED

- `cargo build` from the repository root produces `target/debug/smysl`; every example in this book runs that binary.
- The default feature set (`cli`, `tui`, `local`, `render-typst`) is a full interactive build; `--no-default-features` builds the library alone and silently skips the `[[bin]]`, which requires `cli` explicitly.
- Twelve global flags apply to every subcommand — most importantly `--store` / `--output` / `--format` for where data comes from and goes, and `--strict` / `--offline` / `--json` for how a pipeline enforces policy.
- Twelve exit codes (0–11) are a stable contract a script can branch on; `3` (check errors) and `10` (staged, awaiting confirmation) are the two you'll meet soonest.

CHAPTER 5

5

Your First Document, Start to Finish

This chapter has one job: take you through a document's whole early life — write it, break it, fix it, canonicalise it, and glimpse the pipeline waiting beyond it — as one connected walkthrough rather than a list of flags. Nothing here is invented; every command below was run for real against the file you're about to write.

Writing a minimal document by hand

> **WHY**
>
> The format guide documents every field a unit can carry. You don't need all of them to get started — you need exactly enough to make one true, well-supported claim, which is the smallest unit of work `smysl` is built around.

Create `first.smy` with one piece of evidence, one claim it supports, and the relation between them:

```
@doc smysl/0.1 {
  id: v/first
  intent: incident-brief
  lang: en
  roots: [c/root-cause]
}
```

```
@evidence e/cpu-spike { status: measured, source: { kind: metric, ref:
"host.cpu.pct", captured: 2026-07-20 } }
~ CPU on worker-3 sat above 95 percent for eleven minutes starting at 02:14.

@claim c/root-cause { status: derived }
~ A stuck cron job pinned worker-3's CPU and starved the request queue.

@rel e/cpu-spike --warrant--> c/root-cause
```

Four constructs, and each one answers a different question: `@doc` says what this file is and where it's rooted; `@evidence` records something measured, with a source so the claim behind it isn't free-floating; `@claim` states the conclusion; `@rel` says **how** the evidence backs the claim (`--warrant-->`, one of several relation kinds Chapter 6 covers in full).

Running `check`, and reading the report honestly

> **WHY**
>
> A hand-typed file is a hypothesis about what you meant, not a guarantee. `check` is how you find out whether the tool agrees with you before anything downstream — a merge, a pack, a render — has to guess.

● ● ● *$ smysl check first.smy*

```
first.smy: error: SMY-E060: unresolved reference `c/root-cause` (at 77..89)
first.smy: error: SMY-E060: unresolved reference `c/root-cause` (at 397..439)
first.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
284..396)
```

That's three errors from one mistake, and reading it in the wrong order looks like three separate bugs. `status: derived` requires non-empty `grounds` (rule M: a derived claim must name what it derives **from**) — that is `SMY-E031`, the third line, and it's the actual root cause. Because that claim never became a valid unit, both the `@doc`'s `roots: [c/root-cause]` and the `@rel` line at the bottom point at something that doesn't exist — hence two `SMY-E060` "unresolved reference" errors from a claim that failed to admit at all. (The doubled `SMY-E031: SMY-E031:` text is a real quirk of how the diagnostic wraps the underlying shape error's own message — copied here verbatim rather than cleaned up, because that's what you'll actually see.) The lesson generalises: when `check` reports a cluster of dangling references alongside one shape error, fix the shape error first and see how many of the others disappear on their own.

Add the missing grounds:

```
@claim c/root-cause { status: derived, grounds: [e/cpu-spike] }
~ A stuck cron job pinned worker-3's CPU and starved the request queue.
```

```
$ smysl check first.smy
```

```
first.smy: 6 records, 2 units, 0 diagnostic(s)
```

Clean. Four records — the doc header, the evidence unit, the claim unit, and the relation — reducing to two units, with nothing `check` objects to.

> **WHAT'S NEXT, AND WHY**
>
> This file only exercises two unit schemas and one relation kind out of the full grammar. Chapter 6 covers every schema, every status, and the rules (like the one you just tripped) that govern which combinations are legal. Chapter 8 goes back through `check` pass by pass — conformance, fidelity, granularity — once you have more than one small file to worry about.

Canonicalising with `fmt`, and reading the diff

> **WHY**
>
> A file you typed by hand almost never matches `smysl`'s canonical spelling — field order, quoting, and implicit defaults are all fixed by the writer, not by how you happened to type them. `fmt` is the one command whose entire job is closing that gap, safely: it re-parses its own output and refuses to write anything that wouldn't decode back to the identical records.

`fmt --check` tells you, without touching the file, whether it's already canonical:

```
$ smysl fmt --check first.smy
```

```
first.smy: not canonically formatted
```

That's not a warning — it exits `3`, the same code `check` uses for a validation failure, because reformatting **is** a check on the round-trip property. Run it for real with `--write` and diff the result to see exactly what canonical form means in practice:

● ● ● *$ smysl fmt --write first.smy (then diff against the original)*

```
4a5
>   granularity: { profile: default, l0_max: 30, l1_range: [40, 120], admission:
single-assertion }
8c9
< @evidence e/cpu-spike { status: measured, source: { kind: metric, ref:
"host.cpu.pct", captured: 2026-07-20 } }
---
> @evidence e/cpu-spike { status: measured, source: { kind: metric, ref:
host.cpu.pct, captured: 2026-07-20 } }
```

Two real changes, both instructive. First, the granularity profile you left implicit (just `default`, if you'd written it at all) comes back fully expanded to its four underlying fields — nothing about a document's shape is left for a later reader to look up elsewhere. Second, the quotes around `host.cpu.pct` disappear: canonical form only quotes a string when its content actually requires it, so the writer normalises quoting by content rather than preserving however you happened to type it. Neither change moved a uid — hashes are computed over CBOR, never over surface text, which is the property that makes `fmt` safe to run automatically rather than something you audit by hand every time.

TERM **Canonical form**

The one, unique byte-for-byte spelling of a given set of records: field order fixed, quoting decided by content, implicit defaults expanded. Two files that parse to the same records always canonicalise to the same bytes, which is what makes a diff between two canonical files a diff of actual content rather than of formatting noise.

NO FILE, NO PROBLEM

`fmt` with no files reads stdin and writes stdout, so `cat draft.smy | smysl fmt | smysl check -` canonicalises and validates in one pipeline, with no temporary file in between.

WHAT'S NEXT, AND WHY

Chapter 7 covers `fmt` and canonical form in full — every normalisation rule, not just the two you happened to trigger here. Run `fmt --check` as a habit after every hand-edit; it costs nothing and it's exactly the kind of drift you don't want to notice for the first time in a diff review.

One glimpse ahead: `pack --explain`

WHY

You don't need to understand packing yet — that's Chapter 19's job in full. What's worth seeing right now, on your own two-unit file, is that a whole pipeline exists past validation: documents don't just get checked and formatted, they get **selected down** to fit a budget before anything reads them, and the tool will tell you exactly why it kept what it kept.

$ smysl pack --budget 200 --explain --format surface first.smy

```
b3:fh63h4ededeae3dpn6ov4y4cuo @L0  -  earned on density
b3:uxjafqrmtdo2i5nbi2mlma2ylv @L0  -  earned on density
first.smy: 2 of 2 unit(s), 41 of 200 tokens, greedy mode, gap 0.000
@doc smysl/0.1 { id: v/pack, intent: pack }

@evidence e/cpu-spike { status: measured }
~ CPU on worker-3 sat above 95 percent for eleven minutes starting at 02:14.

@claim c/root-cause { status: derived }
~ A stuck cron job pinned worker-3's CPU and starved the request queue.
```

With a 200-token budget and only 41 tokens of content, both units fit easily, each "earned on density" rather than forced in by a hard constraint — `--explain` is telling you **why** each survivor made the cut, unit by unit, before the summary line. A two-unit toy file was never going to be interesting to pack; the point is that this same flag, pointed at a store with hundreds of units and a budget that can't hold all of them, is how Chapter 19 explains what gets kept, what gets dropped, and what degrades to a shorter form instead of disappearing outright.

WHAT'S NEXT, AND WHY

Two chapters are waiting on what you just did. Chapter 6 is the full authoring grammar — every unit schema, every relation kind, every status and the rules governing it, so the next file you write doesn't rely on guesswork. Chapter 8 is `check` in real depth — conformance classes, fidelity against a named consumer, granularity reporting — for the moment your documents outgrow two units and one relation.

TRY THIS

1. The broken `first.smy` produced three errors from one mistake. This chapter fixed it by adding `grounds: [e/cpu-spike]`. There is a second fix that also makes `check` pass with `6 records, 2 units, 0 diagnostic(s)`: change `status: derived` to

`status: speculative` and leave `grounds` empty. Try it. Both files are valid — so what did you change about what the document **means**, and which fix was right?

2. `smysl fmt --check first.smy` exits `3` — the same code a validation failure uses — for a file that is merely spelled unusually. Argue the case for that being an error rather than a warning. What is `fmt` actually asserting when it succeeds?
3. `fmt --write` expanded the granularity profile into four fields and removed the quotes around `host.cpu.pct`, and yet not one uid in the file changed. Explain why, in one sentence, using the fact from Chapter 2 about what a uid is computed over. Then say what it would mean for `fmt` if a uid **had** moved.

ANSWERS

1. Adding grounds says **this conclusion is computed from that evidence, and here is which**. Weakening to `speculative` says **this is a guess I am floating, resting on nothing**. Both are legal documents and they are not the same document — the second one has quietly abandoned the claim that the evidence supports the conclusion. Which is right depends on whether you meant it; the point of the exercise is that `check` passing is not evidence you meant it. A clean run says the document is coherent, never that it is true.
2. Because canonicalisation is a **round-trip assertion**, not a beautification pass. When `fmt` succeeds it is claiming that re-parsing its own output yields byte-identical records to what went in — and if that claim were false, two files with identical content could carry different identities, which would break merge, diff and every uid reference in the store. A file that is not canonical is a file whose identity has not been confirmed, and the tool treats an unconfirmed identity the way it treats a failed check.
3. A uid is a hash of the **canonical binary encoding**, never of the surface text — so any change confined to how the text is spelled cannot reach the hash. If a uid had moved, `fmt` would have altered content while claiming to alter only spelling; that is exit code `9`, `HashVerification`, and `fmt` checks for it rather than trusting itself.

WHAT YOU LEARNED

- A minimal document is four constructs: `@doc`, one or more units, the relations between them, nothing else required to get started.
- `check`'s errors can cascade from one root cause — a shape error (`SMY-E031`, empty grounds) produced two knock-on dangling-reference errors (`SMY-E060`) because the malformed unit never admitted at all. Fix the shape error first.
- `fmt --check` exits `3` if a file isn't canonical; `fmt --write` rewrites it in place, expanding implicit defaults and normalising quoting without ever moving a unit's identity.
- `pack --budget N --explain` previews the selection pipeline every store eventually passes through — full depth is Chapter 19, but the shape is visible on a two-unit file already.

PART II

Creating and Formatting Documents

CHAPTER 6

6

Writing Surface Syntax, With Room to Get It Wrong

Chapter 5 got you to a two-unit file that passed `check` on the first try — a `.smy` file small enough that nothing in it could plausibly be wrong. Real documents are not like that. You will write a claim before its evidence exists, forget that `derived` demands `grounds`, indent a line by the wrong amount, or reach for a unit you meant to cite but never declared. This chapter is built around that fact: every construct below is introduced once, correctly, and then broken on purpose, so the diagnostic that catches the mistake is something you have already seen with your own eyes before it ever surprises you mid-sentence.

The exhaustive reference — every field, every type, one example each — is `SMYSL_FORMAT_GUIDE.typ`. This chapter does not repeat it. What it adds is the experience the reference guide deliberately leaves out: what it feels like to get one of these wrong, and exactly what the tool says back.

> **WHY**
>
> It is 02:40, the incident is live, and you are typing evidence into a file while a second person keeps talking. You will refer to `e/pool-wait` two minutes before you declare it, because that is the order the conversation happened in. You will write `derived` on a claim whose grounds you fully intend to fill in as soon as somebody finds the dashboard link.
>
> Both are correct things to do while thinking and dangerous things to leave in a document somebody else will act on tomorrow. The format is deliberately strict about each of them — and strictness only helps if the message you get back tells you **which** mistake you made, in a file you are still holding in your head. That is what this chapter is for: to make every one of those diagnostics something you have already seen on purpose, in daylight, before you meet one at 02:40.

What you hand-author

Four kinds of record are ever typed by a person. Two more — `attestation` and `contention` — exist in the format, but tooling stamps them on for you (`attest`, `merge`); nothing in this chapter asks you to write either by hand.

CONSTRUCT	WHAT IT DECLARES
`@doc`	The document header — at most one per file, and optional.
a unit	`@claim`, `@evidence`, and eleven more kernel types — the thing being claimed, cited, defined, or asked.
`@rel`	A typed, directed edge between two units — `causes`, `rebuts`, `warrant`, and eleven more.
`@thread`	A named, ordered, role-annotated walk over units already in the document.

> **NOTE**
>
> For the full field list of each construct — every status, every source kind, every relation kind, every thread schema and role — see `SMYSL_FORMAT_GUIDE.typ`. It is organised as a specification; this chapter is organised as a sequence of decisions you make while writing.

Building a document, unit by unit

The scenario: checkout latency spiked, and you suspect a price cache went cold after an eviction. You will build the whole argument as you would in an editor, running `check` after every addition.

One piece of evidence

Start with the reading that kicked off the investigation:

```
@evidence e/cache-miss { status: measured, source: { kind: metric, ref:
"cache.miss_rate{svc=checkout}", captured: 2026-07-20 } }
~ Cache miss rate on checkout rose from 1% to 42% over the same hour.
```

● ● ● *$ smysl check step1.smy*

```
step1.smy: 2 records, 1 units, 0 diagnostic(s)
```

> **WHY**
>
> `measured` is the rung reserved for something an instrument recorded, and the format ties it to a `source` immediately rather than letting you add one later "when you get to it" — a measured reading with nowhere it grounds out externally is not yet a measured reading, whatever the status field says. You will see the diagnostic for skipping this in a few pages.

A claim that leans on it

Add the claim the evidence is for:

```
@claim c/cache-cold { status: inferred, grounds: [e/cache-miss] }
~ The checkout price cache was evicted and is serving cold.
```

● ● ● *$ smysl check step2.smy*

```
step2.smy: 4 records, 2 units, 0 diagnostic(s)
```

> **WHY**
>
> `inferred` requires `grounds` because an inference with nothing named as its basis is indistinguishable from a guess wearing a confident label. Rule M then pins the ceiling: this claim can never outrank `e/cache-miss` — retract the evidence, and the claim is reopened along with it, mechanically, not by someone remembering to revisit it.

Naming a term so later claims can rely on it

"Cold" needs a fixed meaning before anything downstream leans on the word:

```
@definition d/cold-cache { status: cited, source: { kind: doc, ref: "sre-
handbook#cache-eviction" } }
~ A cache is "cold" when its hit rate has not yet recovered after an eviction
event.
```

Wire it into the claim as a `deps` entry — the claim's **wording** depends on this definition to be read correctly, even though its **truth** rests on the evidence, not on the definition:

```
@claim c/cache-cold { status: inferred, grounds: [e/cache-miss], deps: [d/cold-
cache] }
~ The checkout price cache is cold after an eviction, not merely slow.
```

● ● ● *$ smysl check step3.smy*

```
step3.smy: 6 records, 3 units, 0 diagnostic(s)
```

> **WHY**
>
> `deps` and `grounds` answer two different questions, and the format keeps them separate on purpose: `grounds` is what the claim would fall without; `deps` is what a reader needs already in hand to parse the claim's wording at all. A definition rarely makes something **true**, but it very often makes a sentence **mean** something specific instead of something vague.

A competing line of evidence, and two more claims

Real incidents have more than one thread. Add a second reading, a claim about latency, and a claim the second reading supports:

```
@evidence e/deploy-clean { status: measured, source: { kind: metric, ref:
"deploy.canary_p95", captured: 2026-07-20 } }
~ The same-hour checkout canary deploy shows no latency regression on its own.

@claim c/latency-up { status: derived, grounds: [e/cache-miss] }
~ Checkout p95 latency rose from 80 ms to 410 ms in the same hour.

@claim c/deploy-clean { status: derived, grounds: [e/deploy-clean] }
~ The canary deploy rules out the new release as the direct cause.
```

`c/latency-up` is `derived`, not `inferred` — the rise is read straight off the metric by a deterministic procedure, no model judgement involved, and the format has a separate rung for exactly that distinction.

Relating them: causes and rebuts

The argument isn't complete until the units are connected. `causes` says what produced what; `rebuts` records the objection that argues against the cache theory, sitting right next to the claim it argues with rather than in a review thread nobody reopens:

```
@rel c/cache-cold --causes--> c/latency-up
@rel c/deploy-clean --rebuts--> c/cache-cold { weight: 0.4 }
```

● ● ● *$ smysl check checkout.smy*

```
checkout.smy: 14 records, 6 units, 0 diagnostic(s)
```

The complete file, for reference — six units, two relations, built up exactly as above:

```
@evidence e/cache-miss { status: measured, source: { kind: metric, ref:
"cache.miss_rate{svc=checkout}", captured: 2026-07-20 } }
~ Cache miss rate on checkout rose from 1% to 42% over the same hour.

@evidence e/deploy-clean { status: measured, source: { kind: metric, ref:
"deploy.canary_p95", captured: 2026-07-20 } }
~ The same-hour checkout canary deploy shows no latency regression on its own.

@definition d/cold-cache { status: cited, source: { kind: doc, ref: "sre-
handbook#cache-eviction" } }
~ A cache is "cold" when its hit rate has not yet recovered after an eviction
event.

@claim c/cache-cold { status: inferred, grounds: [e/cache-miss], deps: [d/cold-
cache] }
~ The checkout price cache is cold after an eviction, not merely slow.

@claim c/latency-up { status: derived, grounds: [e/cache-miss] }
~ Checkout p95 latency rose from 80 ms to 410 ms in the same hour.
```

```
@claim c/deploy-clean { status: derived, grounds: [e/deploy-clean] }
~ The canary deploy rules out the new release as the direct cause.

@rel c/cache-cold --causes--> c/latency-up
@rel c/deploy-clean --rebuts--> c/cache-cold { weight: 0.4 }
```

Eight ways to get it wrong

Everything above worked because every rule was already respected. The rest of this section breaks the same kind of file eight distinct ways, one rule at a time, and shows the exact diagnostic `check` gives back — not a paraphrase of `--help`, the real text.

Missing gist

```
@claim c/cache-cold { status: speculative }
```

● ● ● *$ smysl check missing-gist.smy*

```
missing-gist.smy: error: SMY-E021: record ends before its gist (at 20..43)
```

> **WHY**
>
> The gist is the one thing every unit guarantees a reader at L0 — the level a packed or budget-constrained view falls back to when nothing else survives. A unit with no gist has nothing to fall back to, so the parser refuses it outright rather than let a placeholder stand in for meaning that was never written.

Fix: add the line the header is missing, immediately below it — `~ The checkout price cache is cold.`

Detail without a body

```
@claim c/cache-cold { status: speculative }
~ The checkout price cache looks cold.

--
A longer breakdown of the eviction timeline.
```

● ● ● *$ smysl check detail-no-body.smy*

```
detail-no-body.smy: error: SMY-E023: SMY-E023: detail without body (at 0..131)
```

> **WHY**
>
> L2 detail exists to go **further** than L1 body — it is the fine grain a reader reaches for after the ordinary account, not a substitute for one. A detail with nothing under it is a level with no level beneath it to extend, so the format treats it the same as a floor with no ground under it: not allowed to stand alone.

Fix: either add a body paragraph above the `--` line, or delete the detail if there was never an ordinary account to extend.

`derived` with empty grounds

```
@claim c/latency-up { status: derived }
~ Checkout p95 latency rose from 80 ms to 410 ms.
```

● ● ● *$ smysl check derived-empty-grounds.smy*

```
derived-empty-grounds.smy: error: SMY-E031: SMY-E031: derived/inferred with
empty grounds (at 0..89)
```

> **WHY**
>
> `derived` and `inferred` are both claims about **how** the unit came to be believed — computed, or reasoned — and both claims are empty without something named as the input to that process. An empty `grounds` list under either status is a status describing a computation with no operands.

Fix: add `grounds: [e/cache-miss]`, or drop to `status: speculative` if nothing grounds it yet.

`measured` without a source

```
@evidence e/cache-miss { status: measured }
~ Cache miss rate on checkout rose sharply.
```

● ● ● *$ smysl check measured-no-source.smy*

```
measured-no-source.smy: error: SMY-E032: SMY-E032: measured/cited without source
(at 0..87)
```

This is the rule the first example in this chapter was built to satisfy from the start — this is what skipping it looks like.

> **WHY**
>
> `measured` and `cited` both ground out **externally**: an instrument, or a document a reader can open. Without `source`, the claim to have grounded out anywhere is unverifiable — there is nothing for a later reader, or `attest`, to go check.

Fix: add `source: { kind: metric, ref: "cache.miss_rate" }` — pick the `kind` that matches where the number actually came from.

`unfounded` authored directly

```
@claim c/cache-cold { status: unfounded }
~ The checkout price cache is cold.
```

● ● ● *$ smysl check unfounded-authored.smy*

```
unfounded-authored.smy: error: SMY-E034: SMY-E034: unfounded authored (at 0..77)
```

> **WHY**
>
> `unfounded` is the bottom rung of the trust ladder — reachable only by **retracting** something that used to stand higher. Writing it directly would let a unit start life already withdrawn, which is not a thing an argument can coherently do. If a unit belongs at the bottom, it belongs there because something above it was pulled out, not because you typed the word.

Fix: use `speculative` if this is a fresh, ungrounded claim — that is the actual floor for anything authored.

A body reaching for a unit it never declared

```
@evidence e/cache-miss { status: measured, source: { kind: metric, ref:
"cache.miss_rate" } }
~ Cache miss rate on checkout rose sharply.

@evidence e/deploy-clean { status: measured, source: { kind: metric, ref:
"canary.p95" } }
~ The canary deploy shows no regression on its own.

@claim c/cache-cold { status: inferred, grounds: [e/cache-miss] }
~ The checkout price cache is cold.

The miss-rate curve and the checkout latency curve track each other closely
enough over the same window that the direction is not seriously in doubt,
and this reading tracks closely with e/deploy-clean, which this claim never
listed as a dep or a ground anywhere in its header above.
```

● ● ● *$ smysl check body-not-declared.smy*

```
body-not-declared.smy: error: SMY-E020: the body references
b3:aman2pifslep4jh4umauo5zfu7, which is neither a dep nor a ground (at
b3:cxzv6o3tbghkhpvvzspo67fd5q) [try: add it to `deps` if it is a prerequisite,
or `grounds` if it is support]
```

> **WHY**
>
> Rule L says a body should be interpretable from the L0 of whatever it leans on — `deps` and `grounds` together. A body that reaches for a unit by name without listing it either place is asking the reader to already know something the header never promised them. `check` only catches this **structurally** — an explicit `e/deploy-clean` token, or a `b3:` hash, in the body text — not every English way of alluding to another unit; that softer, semantic version of the same question is what `attest` is for.

Fix: list the referenced unit — `deps: [e/deploy-clean]` here, since the body leans on it for context rather than for grounding the claim's truth.

> **NOTE**
>
> The line that becomes a **body** has to start at column 0. A continuation line indented by exactly two spaces is read as more **gist**, not body — easy to get backwards when you're used to indenting for readability. Compare:
>
> ```
> @claim c/cache-cold { status: inferred, grounds: [e/cache-miss] }
> ~ The checkout price cache is cold.
>   This looks like a body, but two-space indent makes it a gist continuation.
> ```
>
> `smysl fmt` on this file folds the second line straight into the gist — `~ The checkout price cache is cold. This looks like a body, but two-space indent makes it a gist continuation.` — one long sentence, not two levels. A real body needs a blank line after the gist, then text starting at column 0.

A dangling label reference

```
@claim c/cache-cold { status: inferred, grounds: [e/cache-miss] }
~ The checkout price cache is cold.
```

Nothing in this file defines `e/cache-miss` at all — the label was typed, or the unit that defines it was deleted, or it lives in a different file that was never passed to `check` alongside this one.

● ● ● *$ smysl check dangling-label.smy*

```
dangling-label.smy: error: SMY-E060: unresolved reference `e/cache-miss` (at
50..62)
dangling-label.smy: error: SMY-E031: SMY-E031: derived/inferred with empty
grounds (at 0..101)
```

Notice the second line: once the dangling reference is dropped, `grounds` is empty, and `inferred` with empty grounds is its own separate violation. One root cause, two diagnostics — fix the label and both disappear together.

> **WHY**
>
> A label that resolves to nothing is worse than a typo the parser could guess past: guessing wrong here would silently point a claim at the wrong unit. `check` refuses to guess, drops the unresolved reference, and reports exactly what it dropped — which is also why the knock-on diagnostic above appears; the tool is being honest about what the file actually contains after the bad reference is removed, not what you meant.

Fix: define `e/cache-miss` in this file, or pass every file that shares labels to `check` together.

A cycle in grounds

```
@claim c/cache-cold { status: inferred, grounds: [c/latency-up] }
~ The checkout price cache is cold.

@claim c/latency-up { status: inferred, grounds: [c/cache-cold] }
~ Checkout latency rose because the cache is cold.
```

● ● ● *$ smysl check cycle.smy*

```
cycle.smy: error: SMY-E061: cycle in deps or grounds; the back edge is dropped
(at 0..102)
cycle.smy: error: SMY-E061: cycle in deps or grounds; the back edge is dropped
(at 103..219)
cycle.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
0..102)
cycle.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
0..102)
cycle.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
103..219)
cycle.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
103..219)
```

The cycle is reported once from each unit's own traversal, and each unit's now-empty `grounds` is reported once for each of those traversals too — six lines from one mistake, all pointing at the same two units.

> **WHY**
>
> Rule M walks `grounds` to find the **weakest** thing a status rests on, and that walk has to terminate. Two claims each grounding the other have no base case — no unit either of them could actually be weaker than — so the graph cannot be evaluated, only diagnosed. `check` breaks the cycle by dropping the back edge rather than looping forever, which is exactly why `grounds` comes back empty on both sides afterward.

Fix: this pair is circular reasoning stated as syntax, and the real fix is to find the actual, non-circular ground for one of them — typically new evidence, not just a rearranged edge.

> **HOW DEEP A HEADER MAY GO**
>
> Nesting inside a header is bounded at 128 levels, and the same bound applies to a CBOR store. Nothing you write by hand will approach it — the deepest shape the kernel defines is a source reference inside a unit, three levels.
>
> It exists because both readers walk containers recursively, so before 0.3 a deeply nested document did not produce an error, it **aborted the process**. An abort cannot be caught, so a program embedding the library could not contain it — and a `.smy` file is untrusted by definition, since it is the thing another agent hands you.

Annotating what you are writing

> **WHY**
>
> You are reading somebody else's incident document at 09:00 and you want to leave a note for them — **check this against the deploy log** — without asserting it as a claim. A note is not a unit: it has no status, nothing grounds it, and it should not survive into a rendered brief. It is a message to a person.

A line beginning `#` or `//` at column 0 is a comment. Both markers, because an HJSON header inside a record already accepted both, and the surface used to contradict itself by rejecting between records what it took within one.

```
# Checked against the deploy log; the 4.2 timing lines up.
@claim c/regression { status: derived, grounds: [e/trace] }
~ p95 auth latency tripled after the 4.2 rollout.

// TODO: get a link for the canary run before this goes out.
@evidence e/canary { status: measured, source: { kind: metric, ref:
"canary.p95" } }
~ The 4.2 canary ran the same pool configuration without the regression.
```

Two things to know, and the second is a real limitation rather than a detail.

A comment is a comment wherever it sits

Including inside a body. That costs you the ability to open a body line with `#` or `//` — a Markdown heading in a body is read as a comment and dropped.

The alternative was worse. A body runs from the gist to the next record, so a comment sitting **between** two records falls inside that range; treating it as prose made the comment become the previous unit's body. Content invented out of a note, with a granularity warning fired about the invented content. A rule that depends on how far a line happens to be from the next record is not a rule anybody can hold in their head.

`fmt` cannot keep them, and says so

A comment is not part of any record, so canonical form has nowhere to put one.

● ● ● *$ smysl fmt --write draft.smy*

```
draft.smy: warning: 2 comment line(s) are not part of any record and will not
survive formatting
```

That warning exists because this book recommends `fmt --write` as a pre-commit habit, and silently deleting a reviewer's notes is the difference between a formatter and a hazard. If a note has to survive, it belongs in a unit — a `@question`, or a body paragraph — where it is content and travels like content.

> **WHAT'S NEXT, AND WHY**
>
> Comments are for the humans in the chain. If what you want to record is something the **graph** should know — an open question, a caveat, a disagreement — Chapter 6's schemas and `@rel` are where it goes, and it will then survive a merge, a pack and a render like anything else.

Reaching past the kernel: extension types

The thirteen kernel unit types cover claims, evidence, definitions, and the rest of general-purpose reasoning — but an SRE team running its own runbook process may want a unit type the kernel was never going to add, without waiting on a kernel RFC revision to get one. That is what `x.<domain>/<type>` is for:

```
@x.sre/runbook-step rb/rollback { status: cited, source: { kind: doc, ref: "ops-
runbook#eu-west-rollback" } }
~ Roll the eu-west pool config back to the 4.1 settings and re-run the canary.
```

● ● ● *$ smysl check runbook.smy*

```
runbook.smy: 2 records, 1 units, 0 diagnostic(s)
```

It checks clean — an extension type still carries the ordinary unit anatomy (status, gist, `source`) and is bound by the same rules as any kernel type. Relations get the same escape hatch:

```
@rel c/cache-cold --x.sre/mitigates--> p/warm-cache
```

● ● ● *$ smysl check extrel.smy*

```
extrel.smy: warning: SMY-W013: relation kind `x.sre/mitigates` is undeclared;
treated as elaborates
extrel.smy: 7 records, 3 units, 1 diagnostic(s)
```

WHY

`x.sre/mitigates` is only a warning, not an error, and the message says exactly what happens next: a reader without the `x.sre` extension treats the edge as a plain `elaborates` link rather than dropping it. That is rule X in miniature — an org can extend the format for its own tooling, and a reader who never installed that extension still gets **something** true about the edge, not silence.

An extension type is the right tool exactly when the unit or relation is genuinely domain-specific and you control both ends of the pipeline that reads it — reach for `x.<domain>/<segment>`, keep the structure in the header the way any kernel unit would, and a plain kernel-only reader degrades gracefully instead of losing the record.

WHAT'S NEXT, AND WHY

You now have a file that `check` accepts. That is not the same thing as a file that is **canonically formatted** — the next question, and Chapter 7's whole subject, is what `fmt` does to the exact bytes you just wrote and why it insists on doing it. Once formatting is routine, Chapter 8 goes back to `check` itself, in full: the ten-pass pipeline, conformance classes, and what it means to check a document against a **named** consumer rather than against the kernel alone.

TRY THIS

1. Write a two-line file containing only `@claim c/a { status: unfounded }` and a gist, and run `check` on it. You get `SMY-E034`. Chapter 2's status table lists `unfounded` as a real rung — so why is authoring one an error, and what is the only legitimate way for a unit to reach that status?
2. Add a key the kernel has never heard of to a valid claim — say `x.sre/severity: "sev2"` — and run `check`, then `fmt`. `check` reports no diagnostic and `fmt` hands the key back to you. Which rule is this, and what would the alternative behaviour cost a team whose peer records something yours does not?
3. Take any clean file, delete one character from a label inside a `grounds` list, and run `check`. Count the diagnostics. Now predict what happens to that count when you fix the single character, and check whether you were right.

ANSWERS

1. `unfounded` means **this was knocked out from under** — it is the state a unit lands in when something it rested on is retracted. It is a consequence, not a claim you are entitled to assert directly; a unit that never had support was `speculative` all along, and saying `unfounded` instead is claiming a history that did not happen. The only way in is `retract` (Chapter 18).
2. Rule X — unknown extensions survive verbatim. Without it, a document from a team that records incident severity, or from a version of the format newer than your binary, would either be rejected at your boundary or come out the far side quietly stripped. Neither failure is visible to the person who sent it, which is the same silent-loss problem Chapter 1 is about, committed by your own tooling.
3. You will usually get more than one, and fixing the character usually clears all of them at once. A broken reference means the unit that referred to it may fail to admit at all, and every **other** unit that referred to **that** one now

> dangles in turn. This is why the advice is to fix the first structural diagnostic and re-run rather than working down the list — the list is frequently one fault wearing several hats.

WHAT YOU LEARNED

- Four constructs are hand-authored — `@doc`, units, `@rel`, `@thread` — and the exhaustive field-by-field reference for each is `SMYSL_FORMAT_GUIDE.typ`, not this chapter.
- `deps` and `grounds` answer different questions: what a reader needs to parse the wording, versus what the claim would fall without.
- Every shape rule this chapter broke on purpose has a specific reason tied to how the corpus is read later — a missing gist breaks the L0 fallback, empty `grounds` breaks Rule M's walk, a cycle breaks its termination.
- `check`'s diagnostics can cascade from one root cause — a dropped dangling reference emptied `grounds`, which tripped a second, separate rule. Fix the first diagnostic and re-run before chasing the rest.
- A body/detail line must start at column 0; two-space indent after the gist is a gist continuation, not a body — `fmt` will fold it straight into the gist if you get this backwards.
- `x.<domain>/<type>` and `x.<domain>/<kind>` extend units and relations past the kernel without a kernel revision, and degrade to a warning and a graceful fallback for a reader who doesn't have the extension.

CHAPTER 7

7

Canonical Form and fmt

TERM **Canonical form**

The one, unique byte-for-byte spelling of a given set of records. Two people who wrote the exact same claims, quoted differently, with fields in a different order, converge on identical bytes once `fmt` has touched both files. Nothing about **what the document says** changes; everything about **how it happens to be spelled** is fixed.

WHY

Three things break if spelling is left to whoever typed the file. A diff between two versions of a document should mean **the claims changed** — not that someone re-quoted a field or reordered `deps` and `grounds`, which is exactly the kind of noise that trains a reviewer to stop reading diffs closely. Hashes are computed over CBOR, not surface text, so identity never moves when you reformat — but a person still reads the surface text, and **that** is what canonical form is actually protecting: one document, one spelling, so `surface → CBOR → surface` is not just lossless in principle but produces the same bytes back out in practice.

`fmt --check` finds drift

The file from Chapter 6, as hand-typed, is not canonical — nobody's first draft is:

● ● ● *$ smysl fmt --check checkout.smy*

```
checkout.smy: not canonically formatted
```

That command exits `3` — the same code `check` uses for a validation failure. It is not a coincidence: reformatting **is** a check, in exactly the sense that a file either already has its one canonical spelling or it does not, and `fmt --check` is the way to ask without touching the file.

`fmt --write`, field by field

Run it without `--check` and against a plain `@doc` header first, to see what "expanded, nothing left implicit" actually means:

```
@doc smysl/0.1 {
  id: v/checkout-incident
  intent: incident-brief
  granularity: { profile: default }
}
```

● ● ● *$ smysl fmt doc-header.smy*

```
@doc smysl/0.1 {
  id: v/checkout-incident
  intent: incident-brief
  lang: en
  granularity: { profile: default, l0_max: 30, l1_range: [40, 120], admission:
single-assertion }
}
```

`lang` appeared from nowhere — its default, `en`, is written out explicitly rather than left to be inferred later by whoever reads the file next — and `granularity` grew from a bare profile name into the full set of numbers that name actually means. Now the rest of Chapter 6's file, before and after, field by field. As typed:

```
@evidence e/deploy-clean { status: measured, source: { kind: metric, ref:
"deploy.canary_p95", captured: 2026-07-20 } }
~ The same-hour checkout canary deploy shows no latency regression on its own.

@definition d/cold-cache { status: cited, source: { kind: doc, ref: "sre-
handbook#cache-eviction" } }
~ A cache is "cold" when its hit rate has not yet recovered after an eviction
event.

@claim c/cache-cold { status: inferred, grounds: [e/cache-miss], deps: [d/cold-
cache] }
~ The checkout price cache is cold after an eviction, not merely slow.
```

```
@rel c/cache-cold --causes--> c/latency-up
@rel c/deploy-clean --rebuts--> c/cache-cold { weight: 0.4 }
```

After `fmt --write`:

```
@evidence e/deploy-clean { status: measured, source: { kind: metric, ref:
deploy.canary_p95, captured: 2026-07-20 } }
~ The same-hour checkout canary deploy shows no latency regression on its own.

@definition d/cold-cache { status: cited, source: { kind: doc, ref: sre-
handbook#cache-eviction } }
~ A cache is "cold" when its hit rate has not yet recovered after an eviction
event.

@claim c/cache-cold { status: inferred, deps: [d/cold-cache], grounds: [e/cache-
miss] }
~ The checkout price cache is cold after an eviction, not merely slow.

@rel c/cache-cold --causes--> c/latency-up

@rel c/deploy-clean --rebuts--> c/cache-cold { weight: 0.400391 }
```

Four independent decisions, each verified against the real writer rather than assumed:

CHANGE	WHAT THE WRITER ACTUALLY DOES
Quoting	A string is quoted only if it needs to be — it contains a comma, a brace, a bracket, a quote mark, or a backslash, or it has leading or trailing whitespace, or it would parse as a number, `true`, `false`, or `null`. `deploy.canary_p95` and `sre-handbook#cache-eviction` need none of that and lose their quotes; `pool.wait_ms{shard=eu-west}` keeps its quotes because of the braces inside it.
Field order	`status`, `deps`, `grounds`, `source`, `salience`, in that fixed order, regardless of the order you typed them in.
Weight / salience	Quantised to steps of 1/1024 and rendered to as many decimal places as that needs — `0.4` becomes `0.400391`, the nearest 1/1024 step, because both fields are stored as quantised binary32 values, never arbitrary-precision decimals.
Blank lines	Every unit and every relation is followed by exactly one blank line in canonical form — two `@rel` lines typed back to back gain a blank line between them.

> **NOTE**
>
> Canonical form doesn't just re-punctuate records in place — it **regroups** them. Units come first, in the order they were parsed; relations next; threads last — regardless of how you interleaved them while writing. A thread written **before** the units it points to is written **after** them once `fmt` is done.

Take a file with the thread written first and the claim it points to written second:

```
@thread t/brief { schema: brief, owner: "human:vladimir" }
~ Checkout latency rose after a cache eviction.
```

```
  bottom-line -> c/cache-cold

@claim c/cache-cold { status: speculative }
~ The checkout price cache is cold.
```

● ● ● *$ smysl fmt interleaved.smy*

```
@claim c/cache-cold { status: speculative }
~ The checkout price cache is cold.

@thread t/brief { schema: brief, owner: human:vladimir, ts: [0, 0] }
~ Checkout latency rose after a cache eviction.
  bottom-line → c/cache-cold
```

The thread was first in the source and last in the canonical output. `check` never cared about the order — order carries no meaning in the format — but two files that differ only in **which** valid order someone chose are exactly the diff-noise canonical form exists to kill. Notice also the arrow, `->` as typed and `→` as written, always, and the `ts: [0, 0]` timestamp pair that appeared on the thread the same way `lang` appeared on the doc header — nothing stays implicit once `fmt` has run.

> **WHAT'S NEXT, AND WHY**
>
> Format one file until `fmt --check` is silent, then try it on all of them at once — the next section.

Piping and batch formatting

`fmt` with no files reads stdin and writes stdout, so it composes directly with `check`:

● ● ● *$ cat checkout.smy | smysl fmt | smysl check -*

```
-: 14 records, 6 units, 0 diagnostic(s)
```

One pipeline, no temporary file: canonicalise, then validate the result, in that order, exactly as the exit code `1` you'd get from a shell that short-circuits on the first stage's failure would want.

`fmt` also accepts more than one path, and `--write` rewrites every one of them in place. Start from two files that are independent, differently hand-typed copies of the same records — one with a multi-line header and a quoted date, the other with `grounds` and `status` swapped — and format both in one command:

● ● ● *$ smysl fmt --write batch-a.smy batch-b.smy*

```

```

`fmt --write` prints nothing on success. Run `diff batch-a.smy batch-b.smy` afterward and it prints nothing either: two files that started out spelled completely differently converge on byte-for-byte identical canonical text. That is the entire point of a **canonical** form — it does not just tidy a file relative to itself, it converges every spelling of the same meaning onto one, which is exactly what makes a later `diff` between two documents mean something again.

The round-trip guarantee

`fmt` does not trust its own output blindly. Reading `cmd_fmt` in `src/main.rs`: after writing the canonical text, it re-parses that exact text and compares the result against what it started from —

```
match parse_surface(&formatted) {
    Ok(again) if again.labels == out.labels && again.records == out.records => {}
    _ => {
        eprintln!("{path}: canonical form does not reproduce the same records");
        worst = worse(worst, ExitCode::HashVerification);
        continue;
    }
}
```

— and only proceeds to `--check`'s comparison or `--write`'s rewrite if that reparse matches exactly. A mismatch here refuses to write anything and exits `9`, before either `--check` or `--write` ever runs.

> **WHY**
>
> This assertion is not about your document — it is `fmt` checking **itself**, on every single run, rather than trusting once during development that the writer and the parser agree. If a future change to the writer ever produced text the parser would read back differently, this is the guard that would catch it immediately, on the very first file anyone ran it against, rather than as a silent corruption discovered much later.

Constructing a real failure meant finding a string the writer would emit unquoted that the parser would then read back as something other than a string. The obvious candidate is a `ref` that looks like a number:

$ smysl fmt ticket.smy

```
@evidence e/ticket { status: measured, source: { kind: tool, ref: "42" } }
~ Ticket number for the follow-up.
```

It stays quoted. The quoting rule (Chapter 7's field-by-field table, above) already special-cases exactly this: a quoteless value that would parse as a number, `true`, `false`, or `null` is quoted anyway, specifically because an unquoted `42` would come back as an integer, not the string it started as.

Through 0.3 this manual said that no string it could find survived being written unquoted and came back changed, and offered that as the guarantee doing its job rather than a gap in the testing. That was true and it was also the wrong conclusion, and it is worth keeping the correction on the page rather than quietly editing it out.

WHAT A HAND-WRITTEN SEARCH COULD NOT FIND

In 0.4 a fuzzer — a program that generates millions of inputs and checks an assertion on each — was pointed at exactly this property. It found seven defects in a few minutes, several of which had been in the tree since before 0.2:

- a header **key** holding a `:` or a newline, written unquoted while values were quoted, which tore the header apart and lost the record;
- a value beginning `#` or `//`, which the comment syntax added in 0.2 then ate along with the closing brace;
- a body line ending in a carriage return, which shed one per pass;
- a gist assembled from continuation lines, which kept a leading space the reader then stripped;
- two labels naming one unit, where the parser and the writer disagreed about which name survived;
- unknown header text that skipped Unicode normalisation, so two peers writing identical content got **different uids** — silently, in release builds;
- and two CBOR decoders that filled in defaults for fields the encoder always writes, so two different byte strings decoded to one record.

Every one is fixed and pinned by a regression test, and the fuzz job now blocks CI rather than merely reporting.

The lesson is not that the guard was useless — the guard is what turned each of these into a loud failure instead of a silent corruption. The lesson is about the sentence this manual used to carry. "I looked and could not find one" is a statement about the search, not about the code. A hand-written search explores the cases its author already imagined; the bugs that survive are precisely the ones nobody thought to imagine. When a property is worth asserting, it is worth handing to something that does not share your assumptions about where to look.

WHAT'S NEXT, AND WHY

A canonical file is a clean starting point, not a finished one — `fmt` never asked whether any of the claims in it are actually well-formed arguments, only whether the bytes have one correct spelling. Chapter 8 picks the validation question back up in full: the ten-pass pipeline `check` actually runs, in order, and why that order is fixed. And because `fmt --check` is silent exactly when a file needs no attention and exits `3` the moment it does, it is the kind of command that belongs in CI before a commit lands — Chapters 21 and 23 come back to that same exit code from the other direction, as part of what a reproducible, verifiable pipeline demands on every run, not just the ones you remember to check by hand.

TRY THIS

1. Write the same two units into two files, but order the fields differently in each — `{ grounds: [...], status: derived }` in one and `{ status: derived, grounds: [...] }` in the other, and swap `kind` and `ref` inside a `source` block. Run `fmt` on both and `diff` the results. Predict the outcome first.

2. Run `fmt` on an already-canonical file. Then run `fmt` on **that** output. Compare. What property are you demonstrating, and why would `fmt` be unusable in a pre-commit hook without it?
3. `fmt --check` on a non-canonical file exits `3`, not `1`. Look up both codes in Chapter 4's table and explain, in one sentence, why `3` is the defensible choice.

ANSWERS

1. The two outputs are byte-identical. Field order in the surface text is yours; field order in canonical form is the format's, and `fmt` is the function that maps the first onto the second. This is the property that makes a `diff` between two canonical files a diff of **content** — if it were not true, every reviewer would learn to skim `.smy` diffs, and the format's central promise of being reviewable would quietly stop being cashed.
2. Idempotence: `fmt(fmt(x)) = fmt(x)`. Without it, a pre-commit hook that runs `fmt --write` would produce a new diff every time it ran, so the hook would never converge and a repository could never be in a formatted state. Idempotence is what makes "canonical" a place a file can arrive at rather than a direction it can be pushed in.
3. `1` is a generic failure — something went wrong. `3` is **check errors**: the tool looked at your document and found it wanting in a specific, documented way. `fmt --check` is a check, so it reports like one, which means a CI pipeline can treat `fmt --check` and `check` under the same branch without special-casing either.

WHAT YOU LEARNED

- Canonical form is one unique spelling per set of records — quoting decided by content, fixed field order, quantised floats, `→` always emitted — so a diff between two documents reflects a change in meaning, not a change in whoever's keyboard typed it last.
- `fmt --check` exits `3`, exactly like a failed `check`; `fmt --write` rewrites in place and expands every implicit default — `lang`, `granularity`'s full field set, a thread's `ts` — so nothing about the document stays unstated.
- Canonical form regroups records by kind — units, then relations, then threads — regardless of how they were interleaved while writing; order never carried meaning in the first place.
- `fmt` composes through stdin/stdout (`cat x.smy | smysl fmt | smysl check -`) and accepts multiple files with `--write`, converging every independently-spelled copy of the same records onto identical bytes.
- Every `fmt` run re-parses its own output and compares it, structurally, against what it started from — a mismatch refuses to write anything and exits `9`. The writer's own quoting rules are built specifically to prevent that mismatch from ever firing, which is why constructing a real failure of it was not possible from the outside.

PART III

Validating While You Write

CHAPTER 8

8

check in the Authoring Loop

You have written a handful of units by hand, or ingested a handful and staged them — either way, you now have a `.smy` file and a belief that it says what you meant. `check` is how that belief stops being a belief. This chapter is about the loop you actually run while a document is still moving: write a little, check it, read what comes back, fix it, write more. The full ten-pass pipeline — what each pass does, in what order, and why that order is fixed — is Chapter 21. Here, `check` is a tool you reach for mid-sentence, not a report you generate at the end.

Why check is a step, and not a formality

A parser has to be forgiving. `smysl`'s surface parser never hard-fails on a malformed unit (§15.3) — it records what went wrong and keeps reading, because stopping at the first bad span would throw away every well-formed unit after it. That tolerance is exactly why parsing cannot be the place you learn whether a document is **right**. It is the place you learn whether a document is **readable**, which is a much weaker thing.

> **WHY**
>
> `check` verifies **consistency**, never **correctness** — a distinction the source states plainly: it can tell you a body reaches for a unit it never declared, not whether the claim is true. A parser that kept going past a bad span, and a checker that only judged what the parser handed it, would between them let a document **look** fine and **be** broken — a claim resting on a typo'd id, a `measured` status with no source behind it, a status the document is not entitled to. `check` is the step where those become visible, every time, on demand, rather than discovered later by someone reading the prose and noticing something does not add up.

Running `check` is cheap and has no side effect — it reads a store and prints a report. There is no reason to wait until a document feels finished to run it once. The rest of this chapter is one document, checked after every change, exactly the way you would work on it yourself.

A document, checked three times

Round zero: a small, clean brief

Start with two units — one piece of evidence and one claim resting on it — the smallest thing worth calling a document.

```
@doc smysl/0.1 {
  id: v/pool-brief
  intent: incident-brief
  lang: en
  requires: ["smysl.kernel/0.1"]
}

@evidence e/pool-wait { status: measured, source: { kind: metric, ref:
"pool.wait_ms", captured: 2026-07-09 } }
~ Pool acquisition wait rose from 2 ms to 310 ms over the incident window.

@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait] }
~ The eu-west connection pool is saturated.
```

● ● ● *$ smysl check brief.smy*

```
brief.smy: 5 records, 2 units, 0 diagnostic(s)
```

Clean, and cheaply so — this ran in the time it takes to read the summary line. That summary line is worth reading closely: 3 **records** (the `@doc` header counts) and 2 **units**, with 0 diagnostics. Every round below reports the same three numbers, so the diagnostic count is the thing to watch move.

Round one: a reference that does not resolve

You keep drafting. The next claim is meant to ground on the pool-wait evidence, but you mistype the id — an easy thing to do once a document has a handful of units and you are typing from memory rather than looking each one up:

```
@claim c/pool-saturation { status: inferred, grounds: [e/missing-evidence] }
~ The eu-west connection pool is saturated.
```

● ● ● *$ smysl check mistake1.smy*

```
mistake1.smy: error: SMY-E060: unresolved reference `e/missing-evidence` (at
351..369)
mistake1.smy: error: SMY-E031: SMY-E031: derived/inferred with empty grounds (at
296..416)
```

Two diagnostics from one typo, and both are worth reading rather than just fixing on reflex. `SMY-E060` is the direct hit: nothing in this document is named `e/missing-evidence`. `SMY-E031` follows from it — with the only named ground unresolved, the claim's `grounds` list is, as far as the checker can tell, empty, and `inferred` requires at least one. Fix the id and both diagnostics disappear together, because the second one was never really a separate problem — it was the first one's consequence, reported so you would not have to work that out yourself.

Round two: a status the unit cannot back up

Next round, you add a second piece of evidence quickly, meaning to fill in the source in a moment, and then don't:

```
@evidence e/canary { status: measured }
~ The 4.2 canary ran the same pool configuration without the regression.
```

● ● ● *$ smysl check mistake2.smy*

```
mistake2.smy: error: SMY-E032: SMY-E032: measured/cited without source (at
108..225)
```

This is a shape rule, not a content judgement: `measured` is the strongest rung on the trust ladder, reserved for something an instrument recorded, and the document's own grammar requires that claim to point at a `source` field naming what recorded it. No source, no `measured` — the checker will not take your word for it, which is the entire point of writing status as a field instead of a tone of voice. Add the `source: { kind: metric, ref: ... }` back and the diagnostic clears.

Round three: a warning, not an error

One more round. You add a supporting detail to the saturation claim, but you're moving fast and the paragraph you type is three words long:

```
@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait] }
~ The eu-west connection pool is saturated.

Saturated.
```

● ● ● *$ smysl check mistake3.smy*

```
mistake3.smy: warning: SMY-W041: body is 3 tokens, under the default range
40..120 (at b3:rdmccg53fegdhoyvxzgwwm34gf)
mistake3.smy: 5 records, 2 units, 1 diagnostic(s)
```

Exit code `0`. This is the first diagnostic in this chapter that did not stop anything: the summary line still prints, and the process still succeeds. `SMY-W041` is a **granularity** remark (Chapter 21 covers the full pass) — the default profile expects an `l1` body of 40 to 120 tokens, and three tokens is under that, but nothing here is factually wrong. A three-word elaboration is usable; it just is not doing the elaborating it claims to be there for. A warning is the checker's way of saying "this is fine to ship, and also here is something you might want to know."

--strict: deciding who gets to ignore that

Whether "fine to ship" is actually true depends entirely on who is asking. Mid-draft, at your own keyboard, a three-word body is a note to yourself to come back to — stopping the process over it would be the tool getting in the way of the thing it exists to help with. In a pipeline that gates a merge or a release, the same warning is exactly the kind of thing that should never reach main unremarked. `--strict` is how you tell `check` which situation you're in, and it works by promoting the failure threshold rather than adding a new rule:

● ● ● *$ smysl check --strict mistake3.smy*

```
mistake3.smy: warning: SMY-W041: body is 3 tokens, under the default range
40..120 (at b3:rdmccg53fegdhoyvxzgwwm34gf)
```

Exit code `3` this time — the same diagnostic, unchanged, but now it is fatal. Nothing about the document changed between the two runs; only the threshold did. `check`'s own source states the rule directly: it exits `3` on any error-severity diagnostic, and `--strict` promotes warnings to that threshold. That single flag is the difference between "a personal draft in progress" and "a document a CI job is about to let through":

SETTING	THRESHOLD	WHEN
default	errors only	drafting at your own keyboard — warnings are notes, not blockers
`--strict`	errors **and** warnings	CI, a merge gate, anything unattended

WHAT'S NEXT, AND WHY

A document that `check` reports clean is trustworthy in the narrow sense `check` promises — internally consistent, shape-valid, nothing dangling — and that is now the fork. If the next thing you want is more content, an outside model can propose units at the boundary Chapter 9 sets up, and Chapter 10 shows exactly how that proposal gets reviewed before it counts. If the document is already complete and you want to **operate** on it — merge it with another author's work, diff two versions, trace a claim back to its grounds — that is Part V. If it is ready to leave your machine as a budgeted excerpt or a rendered document, that is Part V and Part VII. All three forks assume the same thing: that you ran `check` before taking them, because none of the commands on the other side of this fork re-derive consistency for you.

TRY THIS

1. Run `smysl check fixtures/corpus/F7-mixed-granularity.smy` and note the exit code. Then run it again with `--strict`. The diagnostics printed are **identical** and the exit codes differ. Explain what `--strict` actually changes, and name a situation where you want each setting.
2. Run `smysl check --json fixtures/corpus/F6-adversarial.smy`. Compare the shape of that output to the human form. What does the JSON deliberately **not** include that the text form does, and who is the missing part for?
3. `check` reports "22 records, 9 units" on `F7`. Chapter 8 says `check` verifies consistency and never correctness. Write down one thing about that file a clean `check` run does **not** entitle you to believe.

ANSWERS

1. Nothing about the analysis — only the threshold at which the run is called a failure. Plain `check` prints warnings and exits `0`; `--strict` promotes warning severity to the failure threshold and exits `3`. Use plain while drafting, where a granularity warning is information and not a blocker, and `--strict` in CI, where the point is to refuse anything the tool frowns at before it reaches a reviewer.
2. It omits the summary line and the human-facing `[try: …]` suggestion framing, giving one JSON object per diagnostic with `code`, `severity` and `message`. The omitted parts are for a person deciding what to do next; a program wants a stable, parseable record per finding and computes its own summary. Two audiences, two renderings, one analysis.
3. That any claim in it is true. A clean run means every reference resolves, no status outranks its grounds, and every unit fits its declared granularity — all properties of the **bytes**. The document could be internally impeccable and factually wrong throughout, and `check` would still print `0 diagnostic(s)`. Correctness is what the source references and a human reviewer are for.

WHAT YOU LEARNED

- `check` verifies consistency, not correctness — it catches a dangling reference or an unsupported status, never a false claim.
- Diagnostics are cheap to generate and safe to run constantly: check after every meaningful change, not just at the end.
- A `Warn`-severity diagnostic (like `SMY-W041`) reports and continues; an `Error`-severity one (`SMY-E031`, `SMY-E032`, `SMY-E060`, …) exits `3`.
- `--strict` promotes warnings to the error threshold — the same rule for your own drafting pass and for a CI gate, just a different setting.

PART IV

Infer and Enrich

CHAPTER 9

9

The Model Boundary

Everything in Part III happens on your machine, deterministically, from files you already have. This chapter is about the three places that stops being true.

> TERM **Model boundary**
>
> The complete set of commands allowed to call an external model, and therefore the complete set of commands allowed to send anything off the machine. There are exactly three: `ingest`, `attest`, and `thread --refine` (the one flag on `thread` that touches a model — plain `thread` does not). Every other command in `smysl` is a pure function of its inputs: same bytes in, same bytes out, on any machine, forever.
>
> This is a statement about **permission**, and the distinction matters when you audit a build. `--refine` is not wired in this version, so only two commands can actually open a socket today. `thread` is classified against what it is allowed to become, so that nothing downstream has to be re-audited on the day the flag lands.

WHY

The source states the rule this chapter is built on directly: **only** `ingest`, `attest`, and `thread --refine` may depend on a model — every other command's purity is checked and asserted in the binary's own test suite, not left to documentation to promise. The reason to draw the boundary this narrowly, rather than letting convenience creep it wider, is that a determinism guarantee is worthless with exceptions. If `check` or `merge` were ever allowed a hidden network call or a hidden read of the wall clock, "run it twice, get the same bytes" would need a footnote every time, and a footnoted guarantee is not a guarantee a pipeline can build on. Concentrating every model dependency into three named commands means the other twenty-some are something you can audit once and stop worrying about.

You do not have to take that on faith. `smysl providers --tasks` reports, without contacting anything, exactly which command sends what to where:

$ smysl providers --tasks

```
task                 provider       egress     command
content-ingest       ollama         local      ingest
relation-extraction  ollama         local      ingest
gist-rewrite         ollama         local      thread --refine
thread-refine        ollama         local      thread --refine
attest               ollama         local      attest
```

Five tasks, and the `command` column names exactly the three commands the boundary permits — `ingest`, `attest`, `thread --refine` — nothing else. `egress` reads `local` here because this environment's only configured provider is a default Ollama entry at `127.0.0.1:11434` — `smysl` ships with that as its out-of-the-box configuration precisely so a first run cannot egress content nobody asked to send. Route a task to a hosted provider instead — an Anthropic or OpenAI-shaped endpoint that is not a loopback address — and the same column reports `LEAVES`, before anything is sent, because `egress` here is read from configuration, not from a probe. Chapter 12 covers configuring providers in depth; what matters here is that the report is unconditionally safe to run — asking the question makes no network call itself.

--offline: a refusal decided before any socket opens

`--offline` turns the "would leave the machine" column into a hard stop. Locality is decided from the endpoint alone — `127.0.0.1`, `localhost`, and a handful of loopback forms count as local; anything else does not — so `--offline` is never a promise a config file makes about itself, it is a fact read off the address. Against the default all-local setup, `--offline` changes nothing, because nothing here is hosted to begin with:

● ● ● *$ smysl --offline ingest --dry-run --rung document note.txt*

```
provider     ollama
egress       no - local
path         json-ast (default for small enforced ingest)
rung         document (ceiling cited)
input        139 bytes, 35 token(s)
```

To see the refusal fire, `content-ingest` has to actually route to a non-loopback address. Routing it at a provider entry whose endpoint is not `127.0.0.1` — this environment ships no compiled hosted mapper, so a temporary config that reused the local mapper against a non-loopback host was enough to make the endpoint genuinely non-local — reproduces the condition `--offline` exists to catch:

● ● ● *$ smysl providers --tasks (content-ingest routed to a non-local endpoint)*

```
task                provider       egress     command
content-ingest      remote         LEAVES     ingest
```

● ● ● *$ smysl --offline ingest --dry-run --rung document note.txt*

```
smysl ingest: operation would leave the machine while --offline is set
```

Exit code `7`. Nothing was sent — not even the dry-run's own report ran, because the refusal happens inside the provider lookup, before `ingest` decides what it would have said. That ordering is the guarantee: `--offline` is checked before any I/O is attempted, so the refusal cannot depend on whether the network happens to be up, and it costs nothing to check.

WHAT'S NEXT, AND WHY

You now know which three commands can reach outside this machine, and how to prove it (`--tasks`) and forbid it (`--offline`) without trusting either claim. Chapter 10 walks through the first of the three in full: `ingest`, which is where a model gets to **propose** new units, and the whole chapter is about why "propose" is the right word and what has to happen before a proposal counts as part of your document.

TRY THIS

1. Run `smysl providers --tasks` (Chapter 12 covers the output in full). It makes no network call, yet it can tell you what **would** be sent where. How is that possible, and why is a command that reports on egress without performing any the right tool for auditing a policy?

2. `--offline` is described as enforcement rather than a warning. Construct the argument for why a **flag** can be trusted as enforcement here, given that a flag is just an argument a caller might forget. What in the build, rather than in the flag, is doing the real work?
3. Chapter 9 draws the model boundary as narrowly as it can. Name a convenience the tool gives up by refusing to let, say, `render` consult a model to smooth its prose — and then name what that refusal buys.

ANSWERS

1. Routing is configuration, not a conversation: which task goes to which provider is decided locally before any socket opens, so it can be reported locally. That is what makes it auditable — a command that had to call out to tell you what it calls out to would be useless as a policy check, because running the audit would itself be the thing you were trying to police.
2. The flag is the **interface**, not the mechanism. The library layers that can open a socket are separate crates behind feature flags, and the purity classification is asserted by the binary's own test suite rather than documented and hoped for. A build compiled without those features cannot reach a model whatever flags you pass, which is the version of the promise you can hand to someone who does not trust you.
3. It gives up fluency: a rendered brief is exactly as well-written as the gists people typed, and no step will quietly improve them. What it buys is that `render` is bit-reproducible — the same thread and profile produce the same bytes forever, on any machine — so a rendered artifact can be regenerated years later and compared against the one that was signed off. A model in that path would make the artifact unreproducible, and an unreproducible artifact cannot be audited.

WHAT YOU LEARNED

- Exactly three commands may **ever** depend on a model: `ingest`, `attest`, and `thread --refine` — of which only the first two are wired today. Everything else is a pure function of its inputs, asserted in the binary's own tests, not just documented.
- `smysl providers --tasks` reports, with no network call, which task routes to which provider and whether that provider is local or would leave the machine.
- A provider's locality is read from its endpoint, not declared by its configuration — a loopback address is a fact, not a promise.
- `--offline` refuses a hosted route before any socket opens, exit `7`; against an all-local configuration it is a no-op, because there is nothing hosted to refuse.

CHAPTER 10

10

ingest: From Prose to Staged Units

`ingest` is the command that turns prose, or any document you did not write in `.smy` yourself, into units a model proposed on your behalf. It is also the command every question in this manual's introduction was really about: staged — why? What next? This chapter answers both, in order, with a real attempt run against this environment's actual provider and a real failure along the way, because the failure turns out to be exactly as instructive as a success would have been.

> **WHY**
>
> A model's output is a **proposal**, not a fact, until a human — or a later, accountable step — confirms it. This is why `ingest` is structurally unable to write into your real store: there is no code path from a model's answer to a committed unit that does not pass through a file you can open, read, edit, and either accept or throw away. The source states this as a rule, not a preference — model output **must not** enter the store directly — and the rest of this chapter is what that rule looks like from the outside.

--dry-run: what would be sent, and to whom

Before anything leaves the machine, `--dry-run` answers the only question worth asking first: what would this call have done? It makes no request at all — no socket, no DNS lookup — which is what makes it safe to run even when you are unsure a provider is configured correctly, or unsure you want to spend the call yet.

```
$ smysl ingest --dry-run --rung document note.txt
```

● ● ● *note.txt is a short paragraph about a pool-saturation incident*

```
provider     ollama
egress       no - local
path         json-ast (default for small enforced ingest)
rung         document (ceiling cited)
input        139 bytes, 35 token(s)
```

Every line here is a decision `ingest` made and is willing to show you before committing to it. `path` and `rung` are the two worth understanding well, because both cap what the eventual units are allowed to claim.

rung: what kind of source this is

`--rung` tells `ingest` what **kind** of thing it is reading, and that answer sets a hard ceiling on the status any resulting unit may carry — never a suggestion, a cap the checker enforces at staging time regardless of how confidently the model writes. Run the same file at each of the four rungs and only the ceiling changes:

● ● ● *$ smysl ingest --dry-run --rung <R> note.txt, R in {document, model, web, computed}*

```
rung         document (ceiling cited)
rung         model    (ceiling inferred)
rung         web      (ceiling cited)
rung         computed (ceiling derived)
```

RUNG	ORIGIN	CEILING	NOTES
`computed`	A deterministic tool, calculation, or parser produced this	`derived`	never requires a source, but needs `grounds`
`document`	You supplied an existing document or dataset	`cited`	requires a `source`
`web`	Fetched content, gated	`cited`	requires a `source` **and** a captured timestamp
`model`	The model's own parametric knowledge — it is guessing	`inferred`	requires `grounds`, not a source

`ingest`'s own default is `document`, not `model` — deliberately. Ingesting a file is transcribing something that already exists; asking a model to invent content from what it already knows is a different, riskier act, and has to be asked for explicitly with `--rung model`. Nothing here is advisory: a model that writes `measured` anyway is downgraded unconditionally and told so, via `SMY-E033`, because rule T's whole purpose is to stop a model's confidence from laundering itself into a status only an instrument earns — however the sentence is phrased. `ingest` **never** assigns `measured` at any rung; that status is reserved for `op: Imported` records with a machine-checkable source, and a model reasoning from its own priors is never that.

path: surface text, or a JSON shape

`--path` picks the wire shape the model is asked to answer in. `auto` (the default) reasons from the job: extracting a handful of relations or rewriting one gist takes the `json-ast` path unconditionally, because a schema the provider enforces is worth more than robustness for something that small. Bulk content ingestion goes the other way by default — the surface path — for a reason worth keeping in mind whenever a large document is involved: a malformed unit in surface text is **recoverable**, since the parser reports a span and moves on, while a truncated JSON object is not — the closing brace is load-bearing for the whole answer. Past roughly 4 kilobytes of expected output, `ingest` takes the surface path automatically for exactly that reason:

● ● ● *$ smysl ingest --dry-run --rung document bignote.txt (a ~7 KB paragraph, repeated)*

```
provider     ollama
egress       no - local
path         surface (output too large to risk truncation)
rung         document (ceiling cited)
input        7300 bytes, 1825 token(s)
```

`--path surface` or `--path json-ast` overrides the choice outright, and the dry-run report always says which reason applied — `caller override`, `a structured operation`, `the provider enforces no schema`, `output too large to risk truncation`, or the plain default — so you are never left guessing why a run took the path it took.

A real attempt, and what actually happened

`--dry-run` never contacts anything, which means it cannot tell you whether the provider it named is actually **there**. Dropping `--dry-run` does:

● ● ● *$ smysl ingest --rung document note.txt*

```
smysl ingest: warning: SMY-W304: span degraded to opaque prose after provider
unreachable (at b3:olmqbgjpyhetxghtootiil6nmr)
smysl ingest: 1 chunk(s), 0 call(s), 1 unit(s), 1 degraded, 0 token(s)
1 unit(s) staged in ./.smysl/staged.smy; review, then `smysl merge --staged`
```

Exit code `10`. This environment has no Ollama server actually running at `127.0.0.1:11434` — `smysl providers --probe` confirms the same thing independently, reporting `ollama down no server at http://127.0.0.1:11434`, exit `6` — so the call genuinely could not be made. This is the real, unedited result of that failure, and it is worth reading carefully rather than treating as a dead end, because what happened next is the reason rule I exists.

WHY

`ingest` **must always make progress.** An input the model never actually saw did not vanish and did not abort the run — it degraded to an opaque `prose` unit, carrying the original text verbatim in its body, tagged `SMY-W304`, and staged like anything else. Rule I's reasoning is blunt and correct: a corpus with some opaque units in it is usable — you, a later pass, or a better-behaved provider can come back to exactly that span later — and a failed ingest run is not usable at all. The same degrade path fires whether the **provider** was unreachable, as here, or whether the **model answered but never produced a repairable unit** after exhausting its repair budget (covered just below) — either way, the guarantee is the same: every path out of the ingest boundary produces units, never a bare failure.

Look at what actually landed in `.smysl/staged.smy`:

● ● ● *$ cat .smysl/staged.smy*

```
@prose { status: speculative, ingest:unrepaired: true }
~ The eu-west connection pool looked saturated during the incident window

The eu-west connection pool looked saturated during the incident window. Wait
times rose sharply after the 4.2 rollout reached that shard.
```

Nothing was lost — the input paragraph is right there, verbatim, as the body — but nothing was **understood** either. `status: speculative` is the honest ceiling for an opaque span: rule I degrades rather than fails, but it never pretends a span it could not process is worth more than a guess. That `ingest:unrepaired` marker is how a later pass, or you, can find every unit that came through this way and decide what to do about it — split it by hand, re-run ingest once a provider is actually reachable, or leave it as searchable prose. This is a completely real result of a completely real failure, produced by this exact binary in this exact environment, and it is the truest possible answer to "what does staged output look like" — but it is not what a **successful** ingest looks like, and that is worth seeing too.

TERM **Staging**

`ingest` never writes to your real store. It writes to exactly one place — `.smysl/staged.smy`, relative to the project root — and that file is ordinary surface text: the same syntax you write by hand, readable in any editor, checkable with `smysl check` like any other `.smy` file, and editable before you commit to it. Nothing about staging is a special binary format standing between you and the model's output. The file **is** the review.

Because it was never reachable, that run could not demonstrate what a **normal** multi-unit staged batch — several proposed units, mixed statuses, maybe one rejected — reads like. So here is one, written by hand rather than produced by a live call, to make that concrete. **This block is hand-authored, not real model output** — a stand-in for what a successful `ingest --rung model` against a working provider would plausibly stage, so the shape of a review is visible even though this environment could not produce it live:

```
@finding f/pool-root-cause { status: inferred, grounds: [c/pool-saturation] }
~ Pool saturation is the most likely cause of the eu-west latency regression.

@claim c/config-drift { status: speculative }
~ A configuration difference between shards may also be contributing.
```

This is what you actually review: two units, one resting on a real ground already in your store (`inferred`, because that is what `--rung model`'s ceiling permits and the unit has `grounds` to justify it) and one offered with nothing behind it at all (`speculative` — the honest floor for a bare guess). You would open this file the same way you'd open any draft, and you have three ordinary options at this point, not a mysterious ritual:

- **Edit it.** Delete the `c/config-drift` line if the guess is not worth keeping, correct a `gist`, tighten a status you think the model was generous with. It is text; edit it like text.
- **Check it first.** `smysl check .smysl/staged.smy` runs the same pipeline Chapter 8 walked through, against the staged file directly — catching a shape problem before it ever touches your real store.
- **Merge it, or don't.** Confirmed units become part of your document only when you say so, which is the next section.

The exit-10 contract

Exit `10` is not an error in the ordinary sense — `ingest` did its job, the units were staged, the file is right there on disk. It exists because a **pipeline** — a script, a CI step, another program driving `smysl` — needs a machine-checkable way to learn "a decision is waiting for a human" without parsing prose off stdout. Three things can happen next, and all three are first-class, not workarounds:

ACTION	WHAT IT DOES	WHY YOU'D CHOOSE IT
Do nothing yet	Staged file sits at `.smysl/staged.smy`; exit stays `10`	You want to read it, maybe in an editor, before deciding — the default, and the safest one
`smysl merge --staged`	Commits the staged records into a real store, via the ordinary merge join (Chapter 13)	You've reviewed it — by eye, by `check`, or both — and it's ready to become part of the document
`ingest --yes`	Same staging happens, but `ingest` itself exits `0` — "staged and confirmed" — instead of `10`	You've decided in advance that this class of ingest doesn't need a pause — a script that already trusts this recipe and provider
`rm .smysl/staged.smy`	Discards the batch outright; nothing was ever in your real store to undo	The proposal isn't worth keeping — maybe the whole run degraded, maybe you changed your mind

One nuance worth being exact about: `--yes` changes `ingest`'s **exit code and message**, not what happens to the file. The batch is still written to `.smysl/staged.smy` either way — `--yes` only tells a calling script "don't treat this run as a pause," because whoever is running it already decided, ahead of time, that this recipe's output does not need a human in the loop every time. It is pre-approval, not a shortcut around staging itself — rule S is not something a flag can opt out of. And merging does not delete the staged file for you afterward; it is ordinary practice to `rm` it once you're confident it is

committed, though re-merging it again would cost nothing worse than redundancy — the join underneath is idempotent.

Confirming this end to end, against the earlier degraded unit and a small existing store:

```
$ smysl check .smysl/staged.smy
```

```
.smysl/staged.smy: warning: SMY-W041: body is 35 tokens, under the default range
40..120 (at b3:wmixdny4fa7xm4j6pprsiy4r3r)
.smysl/staged.smy: 1 records, 1 units, 1 diagnostic(s)
```

```
$ smysl merge --staged brief.smy -o combined.cbor
```

```
smysl merge: committed 1 staged record(s)
```

```
$ smysl check combined.cbor
```

```
combined.cbor: warning: SMY-W041: body is 35 tokens, under the default range
40..120 (at b3:wmixdny4fa7xm4j6pprsiy4r3r)
combined.cbor: 6 records, 3 units, 1 diagnostic(s)
```

The staged unit is now indistinguishable from any other unit in the store — same checks apply, same warning still shows because merging does not fix content, it only decides what counts. That is rule S working exactly as intended: the model proposed, the file made the proposal legible, and a deliberate command — not a side effect of `ingest` itself — is what made it real.

--granularity, --repair, and what happens when repair runs out

Two more flags shape what a successful call would have produced. `--granularity` names the body-length profile (`fine`, `default`, `coarse`) the model is asked to write to, and it becomes part of the ingest **recipe** — a hash of everything that decided what the model was asked to do, which is what later lets tooling tell two runs of "the same" ingest apart from two runs that were never really comparable. `--repair` sets how many times `ingest` will show the model its own mistake and ask again before giving up on a span — `2` by default.

The repair loop only spends a turn on genuine errors. A span the model wrote well is accepted immediately; a span with a real structural problem — a reference to nothing, a body that reads as two assertions instead of one — gets shown the diagnostic and one more chance, up to the repair budget. When that budget is exhausted without a clean answer, the span does not fail the run: it degrades to exactly the same kind of opaque `prose` unit an unreachable provider produces, `SMY-W304`, verbatim body, `speculative` ceiling. This environment could not reach a provider at all, so it could not exercise the "N failed repair attempts" path specifically — the unreachable path pre-empts it — but it is the same

function in the source that both failure modes call, and the source's own test suite exercises it directly: an unrepairable span becomes an opaque `prose` unit whatever put it there, never a failed run.

WHAT'S NEXT, AND WHY

A staged batch is just a store that happens to live at `.smysl/staged.smy` instead of wherever your real document lives — which is why every tool that works on a store already works on it. Before merging anything, run `smysl check .smysl/staged.smy` the way Chapter 8 showed, on the batch alone, so you're reading the model's proposal against the same bar your own hand-written units have to clear. Once it's clean — or you've edited it until it is — `smysl merge --staged` is where it stops being a proposal, and Chapter 13 picks up exactly there: what a join actually does, how a contention is recorded rather than silently resolved, and what "merge" means when two independent batches disagree.

TRY THIS

1. Run `smysl ingest --offline` on any plain-text file, with no model configured at all. It does not fail. Read the three lines it prints, then `cat .smysl/staged.smy`, then run `echo $?`. Three separate rules are being demonstrated at once — name them.
2. Look at the staged unit's status. It is `speculative`, and it carries `ingest:unrepaired: true`. Given that the provider was never reachable and no model saw this text, argue why `speculative` is the only defensible status the tool could have written there.
3. Delete `.smysl/staged.smy` without merging it. What have you lost, and what is the state of your real store? Now explain why that is the answer the design wants.

ANSWERS

1. Rule I — ingest always makes progress: the provider was unreachable, so the span degraded to opaque prose (`SMY-W304`) rather than the run failing. Rule S — model output never enters a store directly: the unit went to `.smysl/staged.smy`, not to anything you own. And the exit code is `10`, not `0`, because the job is deliberately unfinished — the tool is telling a calling script that a decision is outstanding.
2. Nothing supports it. `speculative` is precisely the rung for **offered, not yet grounded**, and every other rung would be a lie: `inferred` would claim a model reasoned it out, and no model ran; `cited` or `measured` would claim a source, and the only "source" is the sentence itself. Rule T caps what ingest may claim in the first place, and with no successful call there is nothing to claim at all.
3. You have lost nothing except the proposal, and your real store was never touched — it is exactly as it was before you ran the command. That is the design: the default outcome of an ingest you do not actively accept is that nothing happened. Making rejection free and acceptance deliberate is what puts the human decision on the path rather than beside it.

WHAT YOU LEARNED

- `ingest` never writes to your real store; it writes readable, editable surface text to `.smysl/staged.smy` and exits `10` to say a decision is waiting.
- `--dry-run` reports the provider, the egress, the chosen path, the rung's ceiling, and the input size — and makes no network call at all.
- `--rung` caps what a proposed unit may claim (`computed` → `derived`, `document` / `web` → `cited`, `model` → `inferred`); a model claiming more is downgraded unconditionally and told so, never silently accepted.
- Rule I guarantees an ingest run always produces units: an unreachable provider, or a span that exhausts its repair budget, degrades to an opaque `prose` unit rather than failing the whole run.
- Exit `10` has three legitimate responses: leave it staged and look at it, `smysl merge --staged` once it's reviewed, or `rm .smysl/staged.smy` to discard it — `--yes` only changes which of the first two happens by default, not whether staging itself happened.

CHAPTER 11

11

attest — Semantic Judgement Without Mutation

Chapter 8 ran `check` over a store and got back a verdict on every pass — shape, closure, granularity, trust. That verdict is mechanical: `check` looks at what a unit's `deps` and `grounds` actually name, compares it against what the unit's body and detail actually reach for, and reports where the two disagree. It never reads the prose and asks whether it is **any good**.

That is not an oversight. It is the boundary this chapter is about.

Why `check` cannot answer this

Take a concrete case. A unit's gist reads:

```
gist: p95 auth latency tripled after the 4.2 rollout
```

and its body is three paragraphs about a canary that never regressed, a connection pool whose wait time rose two orders of magnitude, and a rollout that reached one shard a full day before the rest of the fleet. `check` will pass this unit without hesitation — the gist is present, the body is present, nothing in the body cites a `dep` or `ground` that is not declared. Every mechanical rule is satisfied.

What `check` cannot tell you is whether that one-line gist actually **represents** those three paragraphs, or whether it quietly drops the canary evidence that complicates the story, or whether the body is really making two separate claims wearing a single gist because the eu-west/rollout coincidence and the pool-saturation theory are two different assertions that happened to get merged. Those are judgements about meaning, not structure — and a mechanical pass has no opinion about meaning. It cannot, because it never reads for content, only for consistency.

WHY

`check` verifies **consistency**: does a body cite what it names, does a status respect the trust ladder, does a unit fit its declared granularity shape as a matter of counting. Those are yes/no questions a parser can answer without understanding a single sentence.

Whether a gist **actually summarises** its body, whether a warrant is a **plausible** reason for a claim to follow from its grounds, whether a unit is **too coarse** — those are judgement calls. Judgement calls need something that can read prose and reason about it: a model. `attest` is the command that asks one.

And a model's judgement about a unit that already exists must never silently change that unit. If asking "is this gist honest?" could rewrite the gist, a single disputed opinion would permanently alter the record everyone else is working from — with no trace that anything happened, no way to disagree, no way to go back. So `attest` never touches the unit it judges. It writes its answer down as its own, separate record instead. That is the entire reason `attest` is a command of its own rather than a flag on `check`.

TERM **Attestation**

One agent's assertion **about** a unit — never a change to it. An attestation names the uid it is about, the agent that made the judgement, an operation (`Op::Attested`), a rung (`Rung::Model` for anything a model decided), and a timestamp. It carries no content of its own beyond that. Attestations are not hashed and do not affect a unit's uid: they **accrete** — a unit's identity is fixed by its content, and the evidence for or against it grows as agents attest to it, without the unit itself ever moving.

The three questions `attest` can ask

`attest` does not ask open-ended questions. Every invocation asks exactly one of three, chosen with `--what`, because a model given a vague brief gives a vague answer, and a vague answer is not evidence. Each question is phrased in the tool's own source as a single yes/no with a reason:

`--WHAT`	THE QUESTION ACTUALLY ASKED	NEEDS A BODY
`gist-coverage`	Does the gist summarise the body faithfully — no claim in the gist the body does not support, and nothing central to the body that the gist omits?	yes
`warrant-plausibility`	Is the stated warrant a plausible reason for the claim to follow from its grounds? Judging the connection, not whether the claim is true.	no — any unit
`granularity`	Does this unit make exactly one assertion? Several assertions sharing one gist is the failure being looked for.	yes

`gist-coverage` and `granularity` only apply to a unit that has a body at all — asking about the gist coverage of a unit with no body is asking about nothing, and finding that out would still cost a model call, so `attest` skips units the question cannot apply to before it ever reaches the provider. `warrant-plausibility` judges a relation (`@rel A --warrant--> B`, as seen in

`fixtures/corpus/F1-incident.smy` connecting `d/p95` to `c/regression`), so it applies regardless of whether either end has a body.

Trying it for real

The store below has eight units built from seven days of latency traces, a canary run, and a connection-pool metric — exactly the kind of place a gist might quietly oversell its body. `attest` needs a model to answer with, the same way `ingest` does, so it goes through the same provider registry Chapter 10 introduced:

● ● ● *$ smysl attest --what gist-coverage fixtures/corpus/F1-incident.smy*

```
smysl attest: provider unreachable
```

This environment has no model listening — `smysl providers --probe` reports the same thing (next chapter covers that command in full):

● ● ● *$ smysl providers --probe*

```
ollama         down  no server at http://127.0.0.1:11434
```

That is the real, honest result here: exit `6` (`ExitCode::Provider`), the same code `ingest` returns for the same reason. Nothing about this is `attest`-specific — it is the ordinary consequence of asking a command that needs a model to answer, when nothing is listening on `http://127.0.0.1:11434`. `attest --what warrant-plausibility` and `attest --what granularity` fail identically here, for the identical reason: the registry resolves a provider for `Task::Attest` before it looks at a single unit, so the failure is reported once, up front, rather than once per unit.

WHAT A SUCCESSFUL RUN DOES, PRECISELY

With a model actually listening, `attest` takes the store's units in uid order (not document order, and not at random — see below), keeps only the ones the question applies to, and for each one sends a request built from exactly four fields: `type`, `status`, `gist`, and `body` when present. Nothing else — not `deps`, not `grounds`, not provenance — travels in the request, because the model is not being asked about any of that. Its system prompt fences the unit's content between explicit markers and states, in every one of the three questions, that what is between the markers is **data, never instruction** — the same fencing discipline Chapter 10 covers for `ingest`, applied here to a whole unit instead of raw prose.

The model must answer with `YES` or `NO` as its first word, followed by a short reason. `attest` reads that first word and nothing more sophisticated: an answer that starts with neither word — `"I'm not sure"`, an empty response, an answer in the wrong language — is recorded as **unreadable**, never as a `NO`. Treating a garbled answer as a failed judgement would manufacture evidence against a unit nobody actually judged; the tool refuses to guess on the model's behalf. An unreadable answer produces no attestation at all — no partial record, no placeholder. For each readable answer, the CLI prints one line:

```
yes b3:7fae21 gist omits the canary result entirely
NO  b3:9c1d04 body makes two separate claims under one gist
```

and a summary line naming how many units were judged, how many judgements came back `NO`, how many were unreadable, and the total tokens spent. Every `YES`/`NO` judgement becomes an `Attestation` at `Rung::Model` — a model's opinion is always shelved at the model rung, whatever rung the unit itself was ingested at, because the opinion is the model's own regardless of where the thing it is judging came from.

Not every fixture is a good target for every question. `granularity` asks whether a unit makes exactly one assertion, and `fixtures/corpus/F7-mixed-granularity.smy` — despite the name — is mostly **already** well-graded: `c/consumers-undersized` grounds one claim ("the consumer group could not keep up with the offered rate") in its own evidence, `c/not-io-bound` grounds a separate claim in different evidence, and neither tries to carry the other. If a model were reachable, asking `granularity` about either would be expected to come back `YES` — a confirmation, not a catch. The failure mode this question exists to catch is a **single** unit whose gist reads like `c/consumers-undersized`'s but whose body quietly also argues `c/not-io-bound`'s point underneath it, sharing one gist between two claims that this fixture, correctly, keeps apart. That distinction — a fixture that passes a check versus one that would fail it — only means something once a model can actually answer; until then, the most honest statement about any of the three questions here is the sample name and the fact that nothing answered it.

Every exit code `attest` and `providers` can return traces back to the same handful of causes:

CODE	NAME	WHEN YOU SEE IT HERE
`0`	success	Judgements were made (some may be `NO`) — a `NO` is not a tool failure, it is the answer you asked for.
`6`	provider	`ExitCode::Provider` — no provider could be reached, or one answered with an error. This is the code every example in this chapter actually returned.

CODE	NAME	WHEN YOU SEE IT HERE
`7`	offline	`ExitCode::Offline` — `--offline` refused to fall back to a hosted provider. Chapter 12 covers exactly which providers that can happen to.

> WHAT'S NEXT, AND WHY
>
> You now know precisely what each of the three questions asks and what a real run costs when nothing answers. Before running `attest` against a store you actually care about, check what `smysl providers` reports is configured and reachable — Chapter 12 is that check in full, including how to see what would egress **before** you spend a call finding out the hard way.

Sampling: a call per unit is not free

Every unit `attest` judges is one model call. A store with a few thousand units and no sampling would mean a few thousand calls before you learn anything — that is real time and, for a hosted provider, real cost. `--sample` exists so that "attest everything, every time" is the exception you opt into, not the default you pay for by accident:

```
--sample 20      ask about 20 candidate units
--sample all     ask about every candidate unit — no cap
```

Left unset, `attest` asks about 10 units. That default is not arbitrary caution dressed up as a number — it is small enough that running `attest` after every `ingest` is cheap enough to actually do, and large enough that ten independent judgements about the same kind of unit tell you something about the batch, not just about one lucky or unlucky example.

Candidates are taken in **uid order**, never document order and never at random. That matters the moment you run `attest` twice: a random sample would ask about a different ten units on the second run, and the two reports would not be comparable — you would not be able to tell whether a problem got fixed or whether the sample simply moved. Uid order is deterministic and has nothing to do with where a unit sits in the file, so the same `--sample 10` against the same store asks about the same ten units every time, and a before/after comparison of two `attest` runs is actually a comparison of the same evidence.

> WHAT'S NEXT, AND WHY
>
> Sampling is why `attest` is affordable to run routinely rather than once at the end of a project. Chapter 12's usage ledger is where you would see the actual bill for that routine: `usage --by task` groups every call `attest` made separately from everything `ingest` made, so a rising `attest` line is visible on its own rather than buried in a single total.

The non-mutation guarantee, made concrete

A unit's uid is not assigned, stored, or looked up — it is **computed**, as a BLAKE3 hash over the unit's canonical bytes (Chapter 4 covers the canonical form in full). Two units with identical content always

compute the same uid, and a unit whose content changes by even one byte computes a different one. That is what "content-addressed" means in this tool: identity **is** a function of content, with nothing else in the loop.

Follow that one step further and the reason `attest` cannot touch a unit becomes a fact about hashing, not a policy choice. If judging a unit's gist coverage rewrote so much as its status field, the unit's bytes would change, its uid would change with them, and every `dep`, every `ground`, every `@rel` edge anywhere in the store that names the old uid would now point at nothing. A single semantic opinion — one that might itself be wrong, one that another attestation might later contradict — would have silently broken every reference to the thing it was commenting on.

`attest` avoids that by construction, not by discipline: a `Judgement` becomes an `Attestation`, and an `Attestation` names a uid, it is never folded into the bytes a uid is computed from. The unit is read; it is never opened for writing. The tool's own test suite asserts exactly this — compute a unit's uid, run a granularity judgement against it that comes back `NO`, then recompute the uid from the same unit and check it is bit-identical to what it was before the judgement existed.

WHY THIS IS NOT JUST A PROMISE

Because an attestation carries a uid rather than being merged into one, a judgement can be **wrong**. It can be **superseded** by a later attestation that disagrees. It can be **disputed** by a human who read the same unit and reached a different conclusion. None of that requires touching the claim under judgement, because the claim was never the thing that changed — the set of opinions about it grew, and growing a set of records is exactly what an append-only log is for. A `check` failure and an `attest NO` are answered differently for the same reason: `check` found something **wrong with the unit itself**, which means the unit needs fixing; `attest` found someone's **opinion about** the unit, which the record now holds alongside the unit, unresolved, until someone acts on it.

WHAT'S NEXT, AND WHY

A judgement that comes back `NO` — "this gist omits the canary result", "this unit makes two claims" — is not something `attest` fixes for you, and it is not supposed to be: fixing a gist or splitting a unit is authoring, and authoring belongs to a human or to `ingest`'s staged-and-confirmed path, never to a command whose entire contract is that it does not write units. Go back to Chapter 6 (writing units by hand) or Chapter 8 (`check` and repair) to act on what the judgement told you — rewrite the gist, split the unit, or record that you disagree and move on. `attest` has done its job once the judgement exists; deciding what to do about it is yours.

TRY THIS

1. Run `smysl attest --offline fixtures/corpus/F1-incident.smy` and check the exit code. Now recall from Chapter 10 that `ingest --offline` on plain text exits `10` and produces a degraded unit. Same missing model, two different outcomes. Explain the difference in terms of what each command would have to invent in order to carry on.

2. The chapter title says **without mutation**. Run `attest` against a store and then check whether any unit's uid changed. Why would an `attest` that adjusted statuses in place be a worse tool, even if its judgements were perfect?
3. An attestation records a rung. Given Chapter 2's rule T, describe what happens to a unit that already reads `measured` when an attestation is attached to it at the `model` rung.

ANSWERS

1. `ingest` can fall back to opaque prose: the text exists whether or not a model structured it, so there is always something honest to record — a `speculative` unit marked `ingest:unrepaired`. `attest` is asked for a **judgement**, and there is no honest fallback for a judgement nobody made. Degrading would mean fabricating an opinion, so it declines and exits `6`. Rule I is a promise about progress, not a promise that every command can always produce output.
2. Because the judgement and the document would become inseparable. Keeping the attestation as a separate record means you can see **that** a model assessed a unit, at what rung, and when — and disagree with it without rewriting the unit. A tool that silently adjusted statuses would leave a document whose history could not be reconstructed, and the reviewer would have no way to tell an author's `inferred` from a machine's.
3. Nothing is rewritten, but the unit is now checkable and fails: rule T caps a unit at the ceiling of its provenance, and the `model` rung's ceiling is `inferred`, so `measured` exceeds it and `check` reports `SMY-E033`. This is the mechanism from Chapter 1 doing its job — before the attestation the `measured` was an unchecked claim, and attaching provenance is exactly what makes it checkable.

WHAT YOU LEARNED

- `check` verifies consistency mechanically; `attest` asks a model a semantic question `check` cannot answer, which is why they are two commands rather than one command with a flag.
- Three questions, chosen with `--what`: `gist-coverage` (does the gist represent the body), `warrant-plausibility` (is the warrant a plausible connection), `granularity` (is this one assertion or several).
- `attest` needs a reachable provider exactly as `ingest` does; with nothing listening it fails once, up front, with `ExitCode::Provider` (6) — it does not try each unit before discovering the provider is down.
- `--sample N` (10 by default) bounds the calls spent; candidates are taken in uid order so two runs over the same store are directly comparable.
- An attestation is its own record: a unit's uid is a hash of its content, so mutating the unit to record a judgement would break every reference to it. Attestations accrete instead of overwriting.
- An unreadable model answer produces no attestation at all — never a manufactured `NO`.
- `attest` never fixes what it finds; acting on a judgement means going back to authoring — by hand or through `ingest`.

CHAPTER 12

12

Providers and Usage

`attest` and `ingest` both need to reach a model, and both go through the same registry to do it. This chapter is that registry from the outside: what it reports before you run anything, what it costs after you do, and where both are recorded so neither is a surprise.

What would leave this machine, and when to check

> **WHY**
>
> A model call to a hosted provider sends unit content somewhere outside this machine. Knowing **that will happen** before it happens is the entire point of `--offline` (Chapter 9) — and `--offline` is only as trustworthy as your own knowledge of what is routed where. `providers` is how you get that knowledge without spending a call to find it out: every mode below either reports configuration that is already known, or makes exactly one round trip whose purpose is to answer the question you asked and nothing else.

Plain `providers`, no flags, reports what is configured without contacting anything:

● ● ● *$ smysl providers*

```
ollama         ctx 8192     out 2048   json-schema  local
(--probe contacts them; --tasks reports what would egress)
```

This is the default that ships when no `.smysl/config.hjson` exists at all — a single local Ollama entry, every task routed to it. That default is deliberate: a first run that reached a hosted provider before anyone configured one would mean content left the machine before anyone decided it should.

`--probe` is the one mode that actually contacts every configured provider — one round trip each, to find out what is really there rather than what the config file merely claims:

● ● ● *$ smysl providers --probe*

```
ollama         down  no server at http://127.0.0.1:11434
```

That is a live, real result from this machine: nothing is listening on `127.0.0.1:11434`, so the probe reports `down` and the process exits `6` (`ExitCode::Provider`) — the same failure `attest` hit in the previous chapter, from the same cause, reported here without needing to run a whole `attest` batch first to discover it. `--models` behaves the same way and for the same reason: a provider's installed model list only exists on the far end of a live connection, so asking for it probes exactly as `--probe` does, whether or not you asked for `--probe` explicitly:

● ● ● *$ smysl providers --models*

```
ollama         down  no server at http://127.0.0.1:11434
```

`--tasks` is different in kind, not degree: it never opens a socket. It answers "which commands would send content off this machine, and to what", purely by reading routing configuration:

● ● ● *$ smysl providers --tasks*

```
task                provider       egress     command
content-ingest      ollama         local      ingest
relation-extraction ollama         local      ingest
gist-rewrite        ollama         local      thread --refine
thread-refine       ollama         local      thread --refine
attest              ollama         local      attest
```

Every task here reads `local`, because every task in this configuration routes to the same loopback Ollama entry. A configuration with a hosted fallback would show `LEAVES` in that column for whichever tasks route there — the word chosen specifically to be impossible to skim past. Locality is never declared by the config file; it is read directly off the endpoint (a loopback host is local, anything else is not), so a config that wanted to claim to be safe could not simply say so.

`--offline` only refuses what is actually hosted

`--offline` hard-fails a request rather than letting it fall back to a hosted provider — but it only refuses providers that are not local to begin with. Running it here changes nothing, because Ollama on loopback was never going to leave the machine in the first place:

● ● ● *$ smysl providers --probe --offline*

```
ollama         down  no server at http://127.0.0.1:11434
```

Same failure, same exit code — `--offline` adds nothing to check here because there was nothing hosted to refuse. Against a configuration that **did** route a task to a remote provider, `--tasks --offline` says so up front:

```
--offline: any task marked LEAVES will exit 7 rather than run
```

That is `ExitCode::Offline`, and it is decided from configuration alone, before a socket is ever opened — refusing a call that would have been harmless anyway is still the price of certainty that `--offline` means what it says.

> **WHY FALLBACK DOES NOT PAPER OVER A WRONG KEY**
>
> A `fallback` list (see the config file, next) exists for the case where a provider is simply down — the registry tries the next one rather than failing the whole call. But it only does that on `Unreachable`. An `Unauthorized` response, a context-window overrun, or a malformed request never triggers fallback: silently trying a different provider after an authorization failure would hand you an answer from a model you did not choose, and you would never learn that your key was wrong in the first place. Fallback covers outages; it deliberately does not cover misconfiguration, because the two failures need different reactions and collapsing them into one retry would hide which one you were having.

> **WHAT'S NEXT, AND WHY**
>
> `providers --tasks` is the check to run before `ingest` or `attest` against anything you have not routed yourself: it costs nothing, contacts nothing, and tells you exactly which of the two commands would leave the machine and through which provider. Run it once per project, not once per call.

The provider configuration file

Providers are described in `.smysl/config.hjson`, read by the same HJSON reader the kernel itself uses for its own surface format — one configuration syntax for the whole project rather than two. A configuration with two providers, one local and one hosted, and routing that sends most work local but relation extraction to the hosted one, looks like this:

```
{
  providers: {
    ollama: {
      kind: ollama
      endpoint: "http://127.0.0.1:11434"
      model: "llama3.2"
      context_window: 8192
      max_output: 2048
      structured: json-schema
    }
    remote: {
      kind: anthropic
      endpoint: "https://api.anthropic.com"
      model: "claude-sonnet-5"
      api_key_env: ANTHROPIC_API_KEY
      structured: tool-force
    }
  }
  routing: {
    content-ingest: ollama
    relation-extraction: remote
  }
  fallback: [ollama]
}
```

Every route and every fallback name must resolve to a provider actually defined in the same file — a typo in a task name or a provider id is a hard parse error, caught when the file is loaded, not discovered mid-run after a model has already been asked to do something.

> TERM **Provider**
>
> One configured endpoint: an id, which mapper drives it (`ollama`, `anthropic`, and so on), an endpoint URL, a model name, its context window and maximum output, and how it accepts structured output. Locality (`is_local`) is computed from the endpoint's host, never declared, so a config file cannot claim to be safe without actually being so.

Notice what is missing from the `remote` entry above: no key. `api_key_env` names an **environment variable** that holds the key at call time, and `api_key_cmd` — not shown here — would name a **command** that prints one. `ProviderConfig` simply has no field a literal key could be written into, which is a guarantee about the shape of the data rather than a rule asking someone not to. A config file that does contain a field named `api_key`, `key`, `token`, `secret`, or `password` is refused outright at load time, with an error rather than a warning a hurried reader could scroll past:

```
provider remote has a `key` field; use api_key_env or api_key_cmd -
a config file must be safe to commit
```

WHY THE INDIRECTION, NOT JUST A CONVENTION

A config file is meant to sit in version control next to the rest of a project's settings. A literal key in it would be a credential in your git history the moment it was committed, recoverable long after the key was rotated. An environment variable name or a command to run for the key is not a secret at all — it is a pointer to wherever the secret actually lives (a shell profile, a keychain, a secrets manager), so the file itself stays exactly as safe to commit, read, and share as any other piece of project configuration.

WHAT'S NEXT, AND WHY

With providers and routing declared, `providers --tasks` (previous section) is how you confirm the file says what you think it says before trusting it with real content through `ingest` or `attest`.

The usage ledger

TERM Recipe

A hash of the **full conditions** of one model call — provider, model, prompt template and its version, everything that could change the answer. Two calls with the same recipe are, as far as the tool can tell, interchangeable; two calls that differ in provider or model but ask logically the same question share the same underlying prompt and differ only in who answered.

TERM Ledger

A local, append-only log of what was called: a timestamp, which provider and model answered, which task it was for, how many input and output tokens it cost, whether that count was estimated rather than reported, how many retries it took, and — when the caller has one — a recipe hash and a run identifier. One JSON object per line, in a fixed key order, so a diff over the ledger is a diff over what actually happened. **Never the prompt, never the completion text** — putting content in the ledger would create a second copy of everything the store already holds, outside the log's own integrity guarantees and outside `retract`.

An empty ledger is the normal starting state, and it reads back as one plain sentence rather than an empty table:

● ● ● *$ smysl usage*

```
./.smysl/usage.log: no model calls recorded
```

A missing ledger file is treated exactly like an empty one — reading it never fails just because nothing has been recorded yet, since a ledger is informational and losing the ability to read cost history must never itself be an error.

A real entry, and an honest caveat about attest

To see a populated ledger without a reachable model, run `ingest` against a scrap of prose in this same offline environment. `ingest` degrades a chunk to opaque prose when the provider it needs cannot be reached rather than failing the whole run — Chapter 10 covers that path in full — and it still records the attempt:

● ● ● *$ echo "The auth service saw a latency regression after the 4.2 release." | smysl ingest --yes*

```
smysl ingest: warning: SMY-W304: span degraded to opaque prose after provider
unreachable (at b3:gnpwyiyujyn6x32m6u36euxjqh)
smysl ingest: 1 chunk(s), 0 call(s), 1 unit(s), 1 degraded, 0 token(s)
1 unit(s) staged and confirmed
```

```
$ smysl usage
ollama                       1 call(s)          0 in          0 out
-----------------------      1 call(s)          0 tokens
```

```
$ cat .smysl/usage.log
{"at":1785224107272,"provider":"ollama","model":"","task":"content-ingest",
 "in":0,"out":0,"estimated":false,"retries":0,
 "recipe":"b3:livso2vmtfwpz3qwrjvmn3ni32"}
```

The row shows one call, zero tokens, an empty model name: the entry records that a call was **attempted** against `ollama` for `content-ingest`, not that a model actually answered — no tokens were spent because no model ever received the request. That is the ledger doing exactly what it promises: counts, never content, and it errs toward recording an attempt rather than silently discarding one.

Read this honestly rather than generalising it: in this build, `ingest` is the command that writes the ledger from the CLI. `attest`'s own report carries a token total too (the `N token(s)` in its final summary line from Chapter 11), but that total is not currently appended to `.smysl/usage.log` the way an `ingest` call's is. If you are reconciling `usage` totals against a batch of `attest` runs, the ledger will under-report them — the number to trust for `attest`'s own cost, today, is the summary line `attest` prints for itself, not `usage --by task`.

Grouping, filtering, and clearing

`--BY`	WHAT IT DOES
`provider`	Default. One row per configured provider — which one is actually taking your calls.
`task`	One row per task (`content-ingest`, `attest`, and so on) — which **kind** of work is costing you.
`model`	One row per model name, using the recipe's provider- and model-free identity where relevant so the same logical request aggregates across vendors rather than reading as unrelated calls.
`run`	One row per run identifier, when calls were tagged with one — groups everything one command invocation did.

```
$ smysl usage --by task
content-ingest               1 call(s)          0 in          0 out
-----------------------      1 call(s)          0 tokens
```

`--since MS` restricts the ledger to calls at or after an epoch-millisecond timestamp, for "what has this cost since I last checked." `--reset` discards the whole ledger — every entry, and the file itself:

```
$ smysl usage --reset
./.smysl/usage.log: discarded 1 entr(ies)
```

A ledger is informational rather than authoritative — it never blocks a command and never gates anything the way a staged `ingest` batch does. It exists so cost is visible, not so it can be enforced; resetting it discards history, never permission.

WHAT'S NEXT, AND WHY

You can now see, before a call, what would leave the machine and to whom (`providers`), and after a call, exactly what it cost and under what recipe (`usage`). That is the whole cost and egress discipline this manual asks of you — the rest of the book is about the documents themselves. Part V opens with Chapter 13, `merge`: the first command that actually changes what a store contains by joining two of them, now that you know what it costs to get material into one in the first place.

TRY THIS

1. Run `smysl providers --tasks` with no network available. It prints a routing table anyway. Read the `egress` column: every row says `local` against a default build. Change nothing, and say what that column would have to show before you would be willing to run `ingest` on a document under embargo.
2. Run `smysl usage`. If you have ever run `ingest`, you have rows. Note what the ledger records — provider, call count, token counts — and what it conspicuously does not. Why is the omission the whole point of keeping a ledger at all?
3. `providers` exits `0` even when no provider is reachable. Contrast that with `attest --offline`, which exits `6`. Both commands "failed to reach a model." Why is only one of them a failure?

ANSWERS

1. Any row reading `hosted` (or naming a remote vendor) for a task the document would exercise — `content-ingest` and `relation-extraction` are the two `ingest` uses. The value of the column is that it is computed from configuration rather than observed from traffic, so you can answer "where would this go" before anything goes anywhere, which is the only order in which the question is useful.
2. It records that a call happened, to whom, and how large — never the content of the call. A ledger that stored prompts would be a second copy of every document you ever ingested, sitting in a cache nobody audits, which is precisely the exposure the boundary exists to control. You get cost and volume accountability without creating a new place for the content to leak.
3. `providers` succeeded at its job. Its job is to report configuration, and it reported configuration correctly — reachability is not a precondition for describing a routing table. `attest`'s job is to obtain a judgement from a model; with no model, there is no judgement, and unlike `ingest` there is nothing to degrade to. Rule I promises `ingest` always makes progress because a span can fall back to opaque prose; a semantic judgement has no such fallback, so `attest` reports `6` rather than inventing one.

WHAT YOU LEARNED

- `providers` (no flags) reports configuration without contacting anything; `--probe` and `--models` each make one real round trip per provider; `--tasks` reports what would egress by reading routing alone, never by dialing out.
- The default configuration — no `.smysl/config.hjson` at all — is a single local Ollama entry with every task routed to it, so a first run never egresses before anyone has decided it should.
- `--offline` refuses only providers that are not local; a loopback provider is unaffected by it, because there was never anything to refuse.
- `.smysl/config.hjson` never holds a literal key — only `api_key_env` or `api_key_cmd` — and a file that tries to hold one is refused at load, so a config file is always safe to commit.
- The usage ledger records counts, provider, model, task, recipe, and run — never prompt or completion content — one JSON line per call, append-only.
- In this build, only `ingest` writes to the ledger from the CLI; `attest`'s token total lives in its own run summary, not (yet) in `usage`.
- `usage --by provider|task|model|run`, `--since MS`, and `--reset` are how you read and clear that record; none of it ever blocks a command from running.

PART V

Operating on Documents

CHAPTER 13

13

merge — Union Without a Referee

Two agents — or the same person, an hour apart — can work from the same corpus at once. Neither has to wait for the other, neither has to ask a coordinator who goes first, and neither's work is allowed to quietly overwrite the other's. That is the whole job of `merge`: take two or more stores and produce the one store that knows everything either of them knew.

WHY

The ordinary way to combine two versions of anything is "last write wins" — whichever save happened second is what survives. Applied to a document full of claims and evidence, that is not a merge, it is a coin toss with the claim's certainty as the stake: one agent's belief is silently destroyed, and nothing in the output says so. `smysl` treats a merge as a

TERM **Join**

From lattice math: given two partial pictures of the same situation, their **join** is the smallest single picture containing everything both of them knew — the least upper bound. A join is commutative and order-independent: `merge(A, B)` and `merge(B, A)` produce the same store, and merging the same input twice costs nothing extra. Nobody has to go first, and nobody has to be asked twice.

instead — a union over both stores' units, attestations, relations, and contentions. Nothing is picked for you behind your back. Where the two sides genuinely disagree, that disagreement becomes a record, not a casualty.

Units are content-addressed: two agents that independently write down the exact same claim compute the exact same identifier without comparing notes, and the join collapses the two into one entry with two attestations rather than two units. That is agreement, and it needs no ceremony. The interesting case is the other one — two **different** claims that cannot both be quietly true — and that is what the rest of this chapter is about.

Merging one document already tells you something

It is tempting to think of `merge` as a two-input operation, but it is really a fold: even a single store, merged into an empty one, can already contain a disagreement it was never told about. `F1-incident.smy` threads `c/pool-saturation` and `c/canary-clean` together as one argument — the thread's `support` step names the claim, its `risk` step names the very unit that rebuts it — and a rebuttal that a thread presents as one line of reasoning is exactly what `smysl` calls live:

● ● ● *$ smysl merge fixtures/corpus/F1-incident.smy -o /dev/null*

```
fixtures/corpus/F1-incident.smy: contention k/ccm3actwjjti65famnoe6mapo5d over
b3:cvhirtgs2mpvli2ethhyeo32uf (2 positions, live-rebuttal)
```

`b3:cvhirtgs2mpvli2ethhyeo32uf` is `c/pool-saturation` — the same real uid this book has used since Chapter 19's `pack --focus`, because these are content hashes and this fixture has not changed. The contention id, `k/ccm3actwjjti65famnoe6mapo5d`, is derived from the contention's own content — the kind, the unit it is over, and its positions — never allocated, so two peers that independently detect the same disagreement name it identically. Detection is not something `merge` invents on the spot: it is a pure function of whatever the union already contains, run once at the end of every merge, however many inputs there were.

> **TERM Contention**
>
> A materialised disagreement — a record naming a unit, the positions that disagree about it, and **why** it was raised. Merge never adjudicates a contention: nothing is superseded, nothing is dropped, and no side is marked as the winner. A contention is **reported** by every merge that would imply it, not written permanently into the log, because whether two successors fork or chain can change the moment a third store supplies the edge that orders them — writing a stale finding into an append-only log would make it permanent for no good reason. If a finding should travel with the document, someone resolves it deliberately (Chapter 18) and that resolution is what gets recorded.

There are exactly three shapes a contention can take, and a single two-agent merge below produces all three in one run.

Two agents, one root cause

`F8a-agent-alpha.smy` and `F8b-agent-beta.smy` are two independent triage documents about the same us-east gateway incident, both declaring `roots: [c/cause]` — the same **label**, used by two people who never saw each other's file. Merging them is the ordinary case this chapter exists for:

```
$ smysl merge fixtures/corpus/F8a-agent-alpha.smy fixtures/corpus/F8b-agent-beta.smy -o /dev/null
```

```
fixtures/corpus/F8a-agent-alpha.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy
over b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
fixtures/corpus/F8b-agent-beta.smy: contention k/c3xa27zib4d7t5rme4xekauzh4t
over b3:t65rff76bcbxnwzw4oxzcadthe (2 positions, live-rebuttal)
fixtures/corpus/F8b-agent-beta.smy: contention k/cboxvjp36tnst3vmpsoo2mmvhqu
over b3:cjixk2inyftvxj55d53w2ivxej (2 positions, label-collision)
fixtures/corpus/F8b-agent-beta.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy
over b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
```

Four lines, three distinct contentions — the fourth is the same `supersession-fork` re-detected once the second file is in, since detection runs fresh against the whole union at every step. Read each one against its actual source:

KIND	FIRES WHEN	WHAT ACTUALLY HAPPENED HERE
`supersession-fork`	Two distinct units supersede the same target, and neither supersedes the other.	Alpha's own file already disagrees with itself: `c/pool-size-32` and `c/pool-size-64` both `--supersedes-->` `c/alpha-scope` (`b3:v5sjn55…`), and nothing orders one revision after the other. This one needed no second file — alpha alone was already forked.
`live-rebuttal`	A `rebuts` edge joins two units both selected by a common thread.	Beta's thread `t/beta-brief` puts `c/cause` (`b3:t65rff76…`, "regressed because 7.1 shortened the upstream timeout") at `bottom-line` and `c/timeout-not-culprit` at `risk` — and `c/timeout-not-culprit --rebuts--> c/cause` is a real edge in the same file. Beta's document argues with itself, on purpose, and the thread is what makes that argument **live** rather than a stray edge nobody has read together.
`label-collision`	One label resolves to different uids across the sources in scope.	Both files bind the label `c/cause` — but alpha's `c/cause` ("exhausting its upstream connection pool") and beta's `c/cause` ("regressed because 7.1 shortened the upstream timeout") are different content, hence different real uids. The label agrees; the claims underneath it do not.

Nothing here was adjudicated. All five of alpha's and beta's competing claims are in the merged store, queryable, exactly as their authors wrote them — `diff` (Chapter 14) will show you all ten units are

present, and `trace` (Chapter 15) can walk from either `c/cause` to what it actually rests on. What changed is that a human reading only the contention list — not the whole ten-unit store — already knows there are three specific places where the two agents' work does not simply combine, and exactly which units are on each side of each one.

Three ways to handle a fork, on the same two inputs

`--policy` controls what happens to the `supersession-fork` kind specifically — the other two kinds are unaffected by it, because they are not about competing revisions of one thing, they are about live arguments and label conflicts, which stay worth reporting under every policy. Run the identical two-file merge three times, changing only `--policy`:

POLICY	WHAT CHANGES IN THE REPORT
`contend` — default	All three kinds are reported. The fork over `c/alpha-scope` is surfaced explicitly, alongside the live rebuttal and the label collision.
`latest`	The fork over `c/alpha-scope` is dropped from the report. The live rebuttal and label collision are still reported — they are not what this policy governs.
`all`	Identical output to `latest` in this build: the fork is suppressed, the other two kinds still fire. `all` and `latest` differ only in **name**, not in behaviour, until something downstream (a view, a render pass) actually picks a winner by HLC — which nothing in this chapter's commands does.

● ● ● *$ smysl merge --policy latest fixtures/corpus/F8a-agent-alpha.smy fixtures/corpus/F8b-agent-beta.smy -o /dev/null*

```
fixtures/corpus/F8b-agent-beta.smy: contention k/c3xa27zib4d7t5rme4xekauzh4t
over b3:t65rff76bcbxnwzw4oxzcadthe (2 positions, live-rebuttal)
fixtures/corpus/F8b-agent-beta.smy: contention k/cboxvjp36tnst3vmpsoo2mmvhqu
over b3:cjixk2inyftvxj55d53w2ivxej (2 positions, label-collision)
```

NOTE

This is worth being exact about, because the policy names invite a wrong guess. Units in `smysl` are immutable and content-addressed — a store cannot delete `c/pool-size-32` just because `--policy latest` was passed, any more than it could delete `c/pool-size-64`. Both revisions are in the merged store under every policy; you can `trace --parents` from either one straight to `c/alpha-scope` regardless of `--policy` (Chapter 15 does exactly this). What `--policy` actually decides is narrower and more honest than "which one wins": whether the fork gets raised as an open contention for a human to see, or passes through silently because you already know you do not need `merge` to flag every fork out loud. `Contend` is the default because silence is the one behaviour this design exists to avoid — pick `latest` or `all` deliberately, not by habit.

> **WHAT'S NEXT, AND WHY**
>
> A merge that reports a fork but leaves both revisions live is not the end of the story for that disagreement — someone still has to decide what `c/alpha-scope` actually is. Resolving it is an act of ordinary authoring, not a special command: write a new unit that lists both `c/pool-size-32` and `c/pool-size-64` as grounds and `--supersedes-->` the original, and merge that in like anything else — the fork closes because there is now a latest that both prior revisions feed into. Withdrawing belief outright, as opposed to superseding it with something better, is `retract`'s job instead (Chapter 18), and it computes exactly what else would fall before it lets you do that.

When a retraction outruns its dependents

A fork is two sides that never agreed. A retraction is different: one side withdraws something outright, and the question is what happens to whatever the **other** side was still resting on it. `--retraction` governs exactly that, and none of this book's corpus fixtures retract anything on purpose — so here is the smallest real case, two hand-authored files that do:

```
@evidence e/dashboard-read { status: measured, source: { kind: metric, ref:
"pool.wait_ms{shard=eu-west}", captured: 2026-07-09 } }
~ Pool acquisition wait read 310 ms at the time of capture.

@claim c/still-standing { status: derived, grounds: [e/dashboard-read] }
~ The eu-west pool was under load at capture time.
```

The second file retracts the evidence the claim above rests on — a retraction is a `rel` from a unit to the target it withdraws, and retracting a unit's own self (`from == to`) is how a document withdraws belief in **itself**:

```
@evidence e/dashboard-read { status: measured, source: { kind: metric, ref:
"pool.wait_ms{shard=eu-west}", captured: 2026-07-09 } }
~ Pool acquisition wait read 310 ms at the time of capture.

@rel e/dashboard-read --retracts--> e/dashboard-read
```

Merged under the three policies, in order:

● ● ● *$ smysl merge base.smy withdraw.smy -o strict.cbor (--retraction strict, the default)*

```
withdraw.smy: error: SMY-E050: every ground of this unit has been retracted (at
b3:x65pg5dyx2chhnfiytd5isug3k) [try: retract this unit too, or re-ground it on
something surviving]
```

● ● ● *$ smysl merge --retraction advisory base.smy withdraw.smy -o advisory.cbor*

```
withdraw.smy: warning: SMY-W052: retracted, but retained under the advisory
policy (at b3:5ksv36cuc6crzd66aiad45iu66)
```

● ● ● *$ smysl merge --retraction ignore base.smy withdraw.smy -o ignore.cbor*

```
```

POLICY	WHAT IT DOES TO `C/STILL-STANDING`
`strict` — default	The retraction is transitive over `grounds`: `e/dashboard-read` is gone, `c/still-standing` has nothing left under it, and the merge reports `SMY-E050` — an error, not a silent downgrade. The unit's effective status reads `unfounded`, the ladder's own floor, reachable only by a retraction like this one.
`advisory`	The retraction is recorded and flagged (`SMY-W052`, a warning) but does not propagate — `c/still-standing` keeps its `derived` status. You are told the ground under it was withdrawn; the tool does not decide for you whether that is fatal to the claim resting on it.
`ignore`	The retraction is recorded — it is still a real relation in the store, and `trace --parents` will still find it — but it has no effect on anyone's status at all. Silence in the report is this policy working as designed, not a sign nothing happened.

`strict` is the default for the same reason `contend` is: it is the policy that cannot quietly let something rest on ground that no longer exists. `RetractionAuthority` (`origin` by default) is the companion knob this chapter does not need to demonstrate directly — only an agent that already attested a unit may retract it, which is the defence against one adversarial source erasing work it had no hand in.

Making disagreement block the pipeline, or not

Everything above **reports**. Two flags exist for when a caller — a script, a CI step — needs disagreement to actually stop something.

● ● ● *$ smysl merge --fail-on-contention fixtures/corpus/F8a-agent-alpha.smy fixtures/corpus/F8b-agent-beta.smy -o /dev/null*

```
smysl merge: 1 open contentions
```

Exit code `5` — `ExitCode::Contentions`. Read the count carefully: it says `1`, not `3`, because `--fail-on-contention` checks after **each** input is folded in, and alpha's file alone already carries one contention (the `supersession-fork` over `c/alpha-scope`) before beta's file is ever merged. The pipeline stops at the first offending step, not at the end of the whole run — a caller who wants to know about **all** three contentions, not just the first one that tripped the gate, still wants the default `contend` policy without `--fail-on-contention`, reads the report, and decides by hand.

`--max-contentions-per-agent` is a softer, different concern: not "stop on any contention" but "warn if one source is raising an unreasonable number of them" — the mitigation for a single flooding source drowning a merge in disagreements nobody can review one at a time:

● ● ● *$ smysl merge --max-contentions-per-agent 0 fixtures/corpus/F8a-agent-alpha.smy fixtures/corpus/F8b-agent-beta.smy -o /dev/null*

```
fixtures/corpus/F8a-agent-alpha.smy: warning: SMY-W055: 1 contentions from one
source exceeds the cap of 0
fixtures/corpus/F8a-agent-alpha.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy
over b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
fixtures/corpus/F8b-agent-beta.smy: warning: SMY-W055: 3 contentions from one
source exceeds the cap of 0
fixtures/corpus/F8b-agent-beta.smy: contention k/c3xa27zib4d7t5rme4xekauzh4t
over b3:t65rff76bcbxnwzw4oxzcadthe (2 positions, live-rebuttal)
fixtures/corpus/F8b-agent-beta.smy: contention k/cboxvjp36tnst3vmpsoo2mmvhqu
over b3:cjixk2inyftvxj55d53w2ivxej (2 positions, label-collision)
fixtures/corpus/F8b-agent-beta.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy
over b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
```

The cap of `0` here is deliberately absurd, to force the warning on the smallest possible corpus. Exit code stays `0`: `SMY-W055` is a warning, not a gate — it tells you a source is behaving unusually without stopping the merge, which is the right default for a signal that is a heuristic ("this looks like flooding") rather than a certainty. If your pipeline needs the cap to actually block, pair it with `--fail-on-contention`, or read the report and decide.

--staged: the other half of ingest's pause

Chapter 10 left `ingest` exiting `10`, a batch of proposed units sitting in `.smysl/staged.smy`, waiting for a decision — and promised that committing it was "the ordinary merge join." That promise is literal: `--staged` reads the project's staged file and merges its records into your store exactly like any other document, no separate code path, no special ceremony.

Here is that loop closed for real. A small store already holding `F1-incident.smy`, and a staged batch — hand-authored here, standing in for what a real `ingest` run would have written, since Chapter 10's own live attempt degraded to opaque prose in this offline sandbox:

● ● ● *$ cat .smysl/staged.smy*

```
@evidence e/pool-config { status: measured, source: { kind: file, ref:
"pool.yaml", captured: 2026-07-10 } }
~ The eu-west pool's max size was left at the 4.1 default of 32 connections.

@claim c/config-drift { status: derived, grounds: [e/pool-config] }
~ The pool was never resized when 4.2 doubled per-request hold time.
```

● ● ● *$ smysl --store project/store.cbor merge project/store.cbor --staged -o project/store2.cbor*

```
project/store.cbor: contention k/ccm3actwjjti65famnoe6mapo5d over
b3:cvhirtgs2mpvli2ethhyeo32uf (2 positions, live-rebuttal)
smysl merge: committed 2 staged record(s)
```

The live-rebuttal line is the same one this chapter opened with — merging `store.cbor` at all re-detects it, because detection is a property of the union, not something remembered between runs. The line that matters for this section is the second one: two staged records, committed, into the real store. `.smysl/staged.smy` is ordinary surface text and this is an ordinary merge — the only thing distinguishing it from any other input file is where `--staged` looks for it (`.smysl/staged.smy`, relative to whichever path `--store` resolves the project root against). Nothing about a staged batch gets special treatment once it is committed: it is subject to the same supersession policy, the same retraction policy, the same contention detection as `F8a-agent-alpha.smy` or any other input.

WHAT'S NEXT, AND WHY

You now have a merged store — from two agents' work, from a retraction, or from a staged batch. The next honest question is **what actually changed**: which units are new, which survived untouched, which one side had that the other did not. That is exactly what Chapter 14's `diff` answers, and it is not a coincidence that the staged-merge example above reappears there as a live before/after. If instead you want to know why one specific unit ended up the way it did — what it rests on, who is behind it — Chapter 15's `trace` walks that lineage directly.

TRY THIS

1. Merge `F8a-agent-alpha.smy` and `F8b-agent-beta.smy` and read the contentions on stderr. Three **kinds** appear: `supersession-fork`, `live-rebuttal`, and `label-collision`. For each, describe in one sentence the real mistake two agents would have to make to produce it.
2. Run the merge twice, swapping the order of the two files, and send each result to a file. The two files are **not** byte-identical. Now run `smysl diff` on the pair. Reconcile the two observations with rule U's promise that merge is commutative.
3. Run the merge again with `--fail-on-contention` and check the exit code. Then argue the other side: name a pipeline where exiting non-zero on a contention would be the wrong thing to do.

ANSWERS

1. `supersession-fork`: both agents decided they were replacing the **same** earlier unit, so there are two competing successors and no rule that picks one. `live-rebuttal`: one agent's unit carries a `rebuts` edge against a unit the other agent asserted, so the merged graph contains an argument in progress. `label-collision`: both used the same short name — `c/cause` — for what turned out to be two different units, so the name no longer resolves.
2. Rule U promises the merge is commutative **as a set of records**, and `diff` confirms it: `0 only, 0 only, 11 common` in either order. What differs is the order records happen to be serialised

in, which carries no meaning — no uid, no reference, and no diagnostic depends on it. Commutativity of content is the property that lets two teams merge in whatever order they like and agree; byte-order equality was never promised and would buy nothing.

3. It exits `5`. The case against: a nightly job that merges every agent's output into a shared store **expects** disagreement — that is what the store is for — and failing the build every time two agents disagree would train everyone to pass `--no-fail` and stop reading. `--fail-on-contention` belongs at a gate where a human is supposed to look before something ships, not on the path that collects work.

WHAT YOU LEARNED

- `merge` is a join over stores keyed by canonical identity — commutative, order-independent, and idempotent. It never adjudicates a disagreement; it materialises one as a `Contention` and leaves both sides in the store.
- Three kinds of contention exist: `supersession-fork` (two successors, neither orders the other), `live-rebuttal` (a `rebuts` edge inside a common thread), and `label-collision` (one label, two real uids). A single two-agent merge can — and, in `F8a` / `F8b`, does — produce all three.
- `--policy` (`contend` default, `latest`, `all`) governs only whether a `supersession-fork` is **reported**; units are immutable, so no policy deletes a competing revision. `--retraction` (`strict` default, `advisory`, `ignore`) governs whether a retraction propagates to what was grounded on it, is merely flagged, or is recorded inertly.
- `--fail-on-contention` turns any open contention into exit code `5`, checked after each input is folded in — it can stop at the first offending file, before later inputs are even considered. `--max-contentions-per-agent` warns (`SMY-W055`) on a suspiciously large batch from one source without blocking the merge on its own.
- `--staged` merges `.smysl/staged.smy` into the store exactly like any other input — the confirmation `ingest`'s exit `10` was waiting for, with no separate commit mechanism to learn.

CHAPTER 14

14

diff — What Changed

`merge` tells you what a union contains. It does not tell you, in plain terms, what is **new** since the last time you looked, what one source has that another does not, or who is responsible for a given change. That is a different question — comparison rather than combination — and `diff` answers it two ways: between two stores, or across one store's own history.

> **WHY**
>
> "The two documents are different" is rarely the useful fact on its own. A human reviewing a merge, or a pipeline gating on one, needs to know exactly **which** units moved and into which of a small number of honest categories: present in one but not the other, present in both, or — across time rather than across sources — survived untouched, superseded by something newer, retracted outright, or newly added. `diff` partitions uids into exactly those categories and nothing fuzzier.

Two stores: only in A, only in B, common

`F1-incident.smy` and `F6-adversarial.smy` both start from the same eu-west latency incident and reach opposite conclusions by design — F6 launders a guess and a rumour into a root cause with no real evidence underneath. As stores, they share no unit at all: every claim, however similar the incident it discusses, is different content, hence a different uid.

● ● ● *$ smysl diff fixtures/corpus/F1-incident.smy fixtures/corpus/F6-adversarial.smy*

```
8 only, 5 only, 0 common
- b3:cvhirtgs2mpvli2ethhyeo32uf
- b3:ekitkvj75uvgzxpvq3ad2nrv3b
- b3:izyuzlt42mqcvgdfb4nfpllxyq
- b3:js4xzessu5zwjpv2rawtugnuvj
- b3:phsoomklkmlq3sjvbe6cyuqy5v
- b3:re42iey2e7syg6zp73tfrlqbvh
- b3:wo4t2c46lq45fnakd6tajlgcac
- b3:xkys7j42mcuyiaxiyh73xddimr
+ b3:bacqe7jyc3pfhpnmn2vivvpz47
+ b3:hdn4uifpmzzopwuyajmhyf5xzu
+ b3:xa7undjm37yl57reyhuxgkiwmm
+ b3:2hkacsatxuvcywj6f4w2lkzojx
+ b3:4p7cfgyytyimdbqyhq7hkuw2sb
```

`-` marks a uid only in the first store (`only_in_a`), `+` only in the second (`only_in_b`) — the same convention a text diff uses, applied to content-addressed units instead of lines. Zero common is the honest result for two documents that never shared a source: `diff` does not try to guess that two differently-worded claims about the same incident are "the same idea," because that judgement is exactly the kind of soft equivalence a content hash refuses to manufacture.

`common` becomes the informative column once the two stores in question really did share a starting point — which is precisely what Chapter 13's `--staged` example produced. Diff the original store against the one with the staged batch committed into it:

● ● ● *$ smysl diff fixtures/corpus/F1-incident.smy project/store2.cbor*

```
0 only, 2 only, 8 common
+ b3:qdypl5q72cqdk6oqfk4tzzkcgo
+ b3:wicfl5usmccnimzzmlo3jvpg3f
```

Zero only in A, exactly the two committed units only in B, and all eight of F1's original units in common — this is what a clean, additive merge looks like from the outside: nothing lost, nothing changed, two new units. Diffing two totally unrelated stores and diffing a store against its own later state are the same operation; the difference in what comes out is entirely a difference in what actually happened between them.

Across time: --hop

The same partition applies to one store's own history, if that store's units carry attestations placing them at specific hops — a hop being the ingest or merge event that first attested a unit. `--hop A..B` asks: of what existed at hop `A`, what survived to hop `B` untouched, what was superseded, what was retracted, and what is new in the window?

● ● ● *$ smysl diff --hop 0..5 fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: 8 unit(s), none attested - a store with no
provenance cannot be asked what changed when
```

Every fixture in this corpus produces exactly this message under `--hop`, and it is worth being honest about why rather than reaching for a fixture that happens to look different. Every `.smy` file this book has used is hand-authored surface text — a person wrote `@claim c/pool-saturation` directly. Attestations, and the hop number carried on them, are stamped by `ingest` (Chapter 10) or by a semantic pass (Chapter 11), not by the surface grammar itself: there is no way to write a hop into a `.smy` file by hand, because attesting **is** the act of a pipeline recording who touched a unit and when. A store built entirely from hand-authored text has zero attestations by construction, and `hop_diff`'s own rule is exactly this message's second half: a unit nobody has attested cannot be placed in time, so it is excluded rather than guessed at — the same conservative instinct that governs every ambiguous case in this design.

This is not a dead end so much as a description of when `--hop` becomes useful: once a store has actually passed through more than one distinct, attested pass — one `ingest` run this week, another next week, each stamped with an increasing hop — `--hop 0..1` partitions exactly what that week's pass did. A single ingest pass, or a hand-authored file, has nothing to compare a hop against; a store's **history**, not its content, is what this flag reads.

NOTE

`--by-agent` and `--recipe` are both extensions of this same `--hop` output, not independent flags: `--by-agent` attributes each addition, supersession, and retraction in the window to the agent responsible; `--recipe` additionally separates **the prompt changed** from **the provider changed** for any unit superseded across the window, using the

> TERM **Recipe**
>
> A hash of the full conditions of one model call — provider, model, prompt template and its version. Two attestations sharing a `recipe_family` but not a `recipe` were produced by a different vendor answering the same logical question; two differing in `recipe_family` came from a genuinely different prompt. Chapter 12 covers where this hash comes from in full.

each attestation carries. Both inherit the same requirement `--hop` has: a store with no attested hops has nothing for either flag to attribute.

WHAT'S NEXT, AND WHY

`diff` tells you **that** something changed and, across a hop range, **who** changed it. It does not tell you **why** a specific surviving unit is entitled to the status it has, or what would fall if one of its grounds were pulled out from under it. That question — walking a single unit's actual support, not the whole store's turnover — is Chapter 15's `trace`.

TRY THIS

1. Run `smysl diff fixtures/corpus/F8a-agent-alpha.smy fixtures/corpus/F8b-agent-beta.smy`. It reports **6 only, 5 only, 0 common** — two documents about the same incident with not one unit in common. Given Chapter 2's rule about how identity is computed, explain why that is unsurprising rather than alarming.
2. Following from that: what would have to be true of two independently written documents for `diff` to report units in common? Is that a realistic thing to engineer for, and if not, what is `merge` for?
3. Merge the two files, then `diff` the merged store against each input in turn. Predict both numbers before you run it.

ANSWERS

1. A uid is a hash of the unit's canonical content, and two agents writing independently about the same incident will not produce byte-identical gists — one writes "Gateway p99 rose to 2.4 s", the other phrases it differently. Different content, different identity. `0 common` means the two agents wrote separately, which is what they did; it is not a sign that anything went wrong.
2. They would have to share an **exact** unit — the same type, gist, status, grounds and payload, byte for byte — which in practice means one document was derived from the other, or both from a common ancestor. Engineering for it is not the goal. `merge` exists precisely because the interesting case is documents with **no** units in common but plenty to say about each other: it keeps both sides and materialises the disagreements as contentions rather than pretending the overlap it needed was there.
3. `5 only, 0 only, 6 common` against alpha and `6 only, 0 only, 5 common` against beta — the middle column is zero both times, and 6 + 5 is the merged store's 11 units. The merge added and never subtracted. That middle column is the one to watch as a habit: a non-zero count there would mean a unit present in an input went missing from the merge, which rule U forbids outright.

WHAT YOU LEARNED

- `diff` between two stores partitions uids into `only in A`, `only in B`, and `common` by exact content-addressed membership — never by similarity. Two unrelated documents about the same incident can, and often will, share nothing.
- `common` is the informative column once two stores share real history — diffing a store against itself after a merge is the direct way to see exactly what that merge added, with nothing lost or altered along the way.
- `--hop A..B` partitions one store's units by attested hop into survived, superseded, retracted, and added, plus a measured survival rate. It requires attestations to exist at all — a store built from hand-authored surface text, with no `ingest` or `attest` pass behind it, has none, and says so rather than guessing.
- `--by-agent` and `--recipe` refine a `--hop` diff further, attributing changes to specific agents and separating a provider swap from an actual prompt change — both inherit `--hop`'s dependency on real attested provenance.

CHAPTER 15

15

trace — Walking Provenance

`salience` (Chapter 17) ranks a whole store; `pack` (Chapter 19) decides what survives a budget. Neither one answers the question you actually reach for when someone pushes back on a specific claim: **where does this one thing's certainty actually come from?** `trace` walks exactly that — one unit's ancestry, in the direction you ask for, as far back as you ask it to go.

> **WHY**
>
> A claim that reads `inferred` is only as defensible as what it rests on. When someone challenges `f/root-cause`, "trust me" is not an answer this format is built to need — the real answer is a specific, walkable chain of evidence and definitions that either holds up or does not. `trace` is that chain, made explicit rather than left for a person to reconstruct by re-reading the whole document.

Grounds versus parents: two different questions

`trace` follows one of two distinct kinds of edge, and confusing them answers the wrong question. `--grounds` (the default) walks **evidential** support — a unit's `grounds` and `deps`, what it rests on to be true. `--parents` walks **causal** lineage — attestation `parents` and `supersedes`, where a unit's content actually came from. `--both` walks both at once. `F1-incident.smy`'s finding, `f/root-cause`, makes the distinction concrete:

● ● ● *$ smysl trace b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy (--grounds is the default)*

```
b3:js4xzessu5zwjpv2rawtugnuvj (root)
  b3:cvhirtgs2mpvli2ethhyeo32uf (grounds)
  b3:wo4t2c46lq45fnakd6tajlgcac (grounds)
    b3:ekitkvj75uvgzxpvq3ad2nrv3b (deps)
    b3:izyuzlt42mqcvgdfb4nfpllxyq (grounds)
    b3:re42iey2e7syg6zp73tfrlqbvh (grounds)
fixtures/corpus/F1-incident.smy: 6 unit(s) over 2 step(s)
```

Six units, resolved back to real labels: the root is `f/root-cause`, whose two direct grounds are `c/pool-saturation` and `c/regression`; `c/regression` in turn grounds on `e/trace` and depends on the definition `d/p95`, and `c/pool-saturation` grounds on `e/pool-wait`. That is the whole evidential case for the finding, two steps deep, and nothing here is causal — no author, no attestation, just what the claim rests on to be believed.

`--parents` on the same root asks a different question and gets a different, honest answer:

● ● ● *$ smysl trace --parents b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy*

```
b3:js4xzessu5zwjpv2rawtugnuvj (root)
fixtures/corpus/F1-incident.smy: 1 unit(s) over 0 step(s)
```

Just the root. `F1-incident.smy` is a single hand-authored document — no unit here was transformed from an earlier one, and nothing supersedes anything else — so there is no causal ancestry to walk. This is the correct answer to "where did this content come from," not a broken trace: for a document nobody derived from an earlier version, the honest lineage is one step long.

`F8a-agent-alpha.smy` has real `supersedes` edges — Chapter 13's fork over `c/alpha-scope` — and `--parents` finds them from either side:

● ● ● *$ smysl trace --parents b3:e3f6wpuw6ie4ruf2ba62yrtagg fixtures/corpus/F8a-agent-alpha.smy*

```
b3:e3f6wpuw6ie4ruf2ba62yrtagg (root)
  b3:v5sjn55ujeflrdyu5qycgmtvz3 (supersedes)
fixtures/corpus/F8a-agent-alpha.smy: 2 unit(s) over 1 step(s)
```

That revision of the pool-size claim supersedes `c/alpha-scope` (`b3:v5sjn55…`) — the exact unit Chapter 13 showed forked, from the other direction: tracing **from** `c/alpha-scope` itself with `--parents` finds nothing, because `trace` follows `supersedes` outward from whichever unit you name, and `c/alpha-scope` is the **target** of that edge, not its source. Naming the successor, not the superseded unit, is what makes the ancestry walk.

Bounding the walk: --depth

An unbounded trace walks to the roots of whatever it is following — fine for a six-unit fixture, less fine for a corpus with real depth. `--depth` truncates it:

● ● ● *$ smysl trace --depth 1 b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy*

```
b3:js4xzessu5zwjpv2rawtugnuvj (root)
  b3:cvhirtgs2mpvli2ethhyeo32uf (grounds)
  b3:wo4t2c46lq45fnakd6tajlgcac (grounds)
fixtures/corpus/F1-incident.smy: 3 unit(s) over 1 step(s)
```

Three units instead of six, one step instead of two — `f/root-cause`'s two direct grounds, with everything beneath **them** cut off. `--depth 0` would return only the root itself. This is the tool a person reaches for once a trace's full answer is a wall of text: name how many steps of "why" you actually need before deciding whether to go deeper.

--agents: who is behind each step

`--agents` names, for every node in the walk, every agent that attested it — answering not just **what** a claim rests on but **who** put each piece there. On this book's hand-authored fixtures, that answer is honestly empty:

● ● ● *$ smysl trace --both --agents b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy*

```
b3:js4xzessu5zwjpv2rawtugnuvj (root)
  b3:cvhirtgs2mpvli2ethhyeo32uf (grounds)
  b3:wo4t2c46lq45fnakd6tajlgcac (grounds)
    b3:ekitkvj75uvgzxpvq3ad2nrv3b (deps)
    b3:izyuzlt42mqcvgdfb4nfpllxyq (grounds)
    b3:re42iey2e7syg6zp73tfrlqbvh (grounds)
fixtures/corpus/F1-incident.smy: 6 unit(s) over 2 step(s)
```

No bracketed agent names appear, for the same reason Chapter 14's `--hop` came up empty: attribution reads from attestations, and a store built straight from hand-authored surface text has none. `--agents` earns its keep once a corpus has actually been through `ingest` or `attest` — at that point every node in a trace can carry `[human:vladimir]`, `[model:a/x]`, or whichever agents actually touched it, turning "what does this rest on" into "what does this rest on, and who is answerable for each piece of it."

The label-versus-uid rule, one more time, verified fresh

Every uid-taking flag in this book — `--focus`, `--seed`, `--roots`, and `trace`'s own positional argument — takes a real content hash, never a surface label. This is not a suggestion; the store enforces it, and the failure is exactly as blunt as Chapter 19 already showed for `pack`:

● ● ● *$ smysl trace c/pool-saturation fixtures/corpus/F1-incident.smy*

```
smysl trace: `c/pool-saturation` is not a uid
```

Exit code `1`. `c/pool-saturation` is only ever a name inside the one file that declared it — it has no existence at the store level, where the only identity a unit has is the hash of what it says. The real uid behind that label, `b3:cvhirtgs2mpvli2ethhyeo32uf`, is what every example in this chapter actually used, and it is exactly the same hash Chapter 19's `pack --focus` resolved for the same claim — content hashes are stable across every command that ever touches this fixture, which is the whole point of computing identity from content rather than position.

When you do not already know a unit's real uid, the practical path is the one this book has used throughout: run `smysl --format surface pack --budget <large> --explain <file>` and read every unit's hash straight off the top of the output, or reach for `thread --show` once a thread already names the unit you care about (Chapter 20). Either way, the uid you paste into `trace` has to have come from the store itself — never typed from memory, never guessed from a label that merely looks similar.

> **WHAT'S NEXT, AND WHY**
>
> Once you can walk from a claim to exactly what it rests on, the next question is usually "how do I hand **that** to someone else" — not the whole corpus, just the reachable set a trace just proved matters. Chapter 16's `view` and `bundle` take a set of roots and compute exactly that reachable closure as a portable, self-contained store — the natural next step once `trace` has told you what the roots should be.

> **TRY THIS**
>
> 1. Run `trace` on `f/root-cause` in `F1-incident.smy` with the default `--grounds`, then again with `--parents`. The first walks six units over two steps; the second returns just the root, over zero steps. The second result looks broken. Argue that it is exactly right.
> 2. `trace --grounds` reaches `d/p95`, a **definition**, through `deps` rather than `grounds`. If you retracted `d/p95`, would `c/regression` become unfounded? Answer from what the two edge kinds mean, then check yourself against Chapter 18.
> 3. Someone challenges the finding in a review. Using only `trace` output, write the two-sentence answer you would give them — and then name the one thing `trace` cannot tell you that they might have actually been asking.

ANSWERS

1. `--parents` walks **causal** lineage: attestation parents and `supersedes`. `F1-incident.smy` is a single hand-authored document — no unit in it was transformed from an earlier one and nothing supersedes anything — so there is genuinely no causal ancestry to walk. Zero steps is the honest answer to "where did this content come from" for content that came from a person typing it. A tool that invented a chain here would be worse than one that returns a root alone.
2. No. `deps` means **you need this to parse the wording** — `c/regression` says "p95 tripled" and `d/p95` defines what p95 means. `grounds` means **the claim falls without this**. Retracting the definition makes the claim harder to read, not unsupported; only its `grounds` (`e/trace`) carries its truth. The distinction is why the two lists are separate fields rather than one.
3. Something like: the finding rests on `c/pool-saturation` and `c/regression`; those rest in turn on two measured metrics and a cited definition, two steps down, with no gaps. What `trace` cannot tell you is whether any of it is **true** — it walks the structure of the argument, not the quality of the evidence. If the challenge was "that dashboard is wrong", `trace` shows you precisely which unit to go argue about, and then stops being useful, which is the correct division of labour.

WHAT YOU LEARNED

- `trace` walks one unit's ancestry in one of two directions: `--grounds` (the default) for evidential support — what a claim rests on to be true — or `--parents` for causal lineage — attestation parents and `supersedes`, where its content actually came from. `--both` walks both at once, and the two answers can be, and often are, completely different.
- `--depth` bounds how far back the walk goes, in steps from the root; `--depth 0` returns only the unit you named.
- `--agents` names every agent behind each step in the walk, read from attestations — a store with no attestation history, like a hand-authored fixture, has nothing for it to name, which is a fact about the store's provenance, not a broken flag.
- Every uid-taking flag across the whole CLI — `trace`'s target included — rejects a label outright and reports so plainly. A label exists only inside the one file that declared it; the real, content-derived uid is what the store actually knows, and it is stable across every command that touches the same content.

CHAPTER 16

16

view and bundle — Naming and Packaging Reachability

You have a store — a fully-parsed, checked `.smy` file, or several merged together. It might hold a hundred units built over months: measurements, claims, findings, threads, contentions. Nobody wants a hundred units on screen when they ask "what is the incident brief." They want the six units that brief actually rests on, named once, so the next command — `salience`, `pack`, `render` — knows where to start without re-deriving it.

That is what a view is for. It is not a folder you drop units into; it is a **name for a starting point**, and the graph does the rest.

A view is a root set, not a container

> **TERM View**
>
> A name plus a root set — nothing else. A view owns no units. Reachability does the owning: every unit that a root can reach by following its `grounds`, `deps`, and outgoing relation edges belongs to the view, at exactly zero copying cost, for as long as the store exists. `@doc`'s `roots` field is how a document declares its own default view; `smysl view --roots` builds another one on the fly without touching the file.

> **WHY**
>
> A container has to decide, unit by unit, whether something is "in." Two views that both care about the same finding would each need their own copy of it, and the moment one copy changes, the two views disagree about a unit that is supposed to be the same thing. A view sidesteps the question entirely: a unit is in every view whose roots can reach it, simultaneously, for free. Combining two views is then a plain set union of their roots — no copying, no reconciling, no "which copy wins." That is only available to you because membership is computed, not stored.

Printing an existing view

`fixtures/corpus/F1-incident.smy` declares one view in its `@doc` header — `v/f1`, rooted at a single finding, `f/root-cause`. Printing it with no further arguments prints exactly that view, resolved:

$ smysl view fixtures/corpus/F1-incident.smy

```
v/f1: 1 root(s), 0 thread(s), 6 unit(s) reachable
  b3:cvhirtgs2mpvli2ethhyeo32uf
  b3:ekitkvj75uvgzxpvq3ad2nrv3b
  b3:izyuzlt42mqcvgdfb4nfpllxyq
  b3:js4xzessu5zwjpv2rawtugnuvj
  b3:re42iey2e7syg6zp73tfrlqbvh
  b3:wo4t2c46lq45fnakd6tajlgcac
```

The file has eight units total (confirmed in Chapter 11 by `pack --explain`, which lists every one of them), but the view reaches only six. The two left out are `c/canary-clean` and its evidence `e/canary`. That is not an oversight in the fixture — it is the single most important thing to understand about reachability before you build your own views:

> **WHY**
>
> `f/root-cause`'s only grounds are `c/pool-saturation` and `c/regression`. `c/canary-clean` reaches the graph through `@rel c/canary-clean --rebuts--> c/pool-saturation` — and that edge runs **from** the rebuttal **to** the claim it rebuts, the direction the author wrote it in. Reachability follows edges the way they point. Starting from `f/root-cause` and walking forward, there is no outgoing edge from `c/pool-saturation` back to the thing objecting to it, so the objection is never visited. A view rooted downstream of a contested claim does not automatically pull in its objections; you get them by rooting on them directly, or by rooting further out.

Building an ad-hoc view

You do not need a unit's `@doc` to declare a view for you. `--roots` builds one from the command line, over any real uid — never a label, which only ever exists inside the one file's parse that wrote it. Rooted at `c/pool-saturation` alone, the reachable set shrinks to five units — and picks up a unit `f/root-cause`'s view never touched:

● ● ● *$ smysl view --roots b3:cvhirtgs2mpvli2ethhyeo32uf --id v/pool-only fixtures/corpus/F1-incident.smy*

```
v/pool-only: 1 root(s), 0 thread(s), 5 unit(s) reachable
  b3:cvhirtgs2mpvli2ethhyeo32uf
  b3:ekitkvj75uvgzxpvq3ad2nrv3b
  b3:izyuzlt42mqcvgdfb4nfpllxyq
  b3:re42iey2e7syg6zp73tfrlqbvh
  b3:wo4t2c46lq45fnakd6tajlgcac
```

`c/pool-saturation` grounds only on `e/pool-wait` — one unit — yet the view reaches five. The other three arrive through `@rel c/pool-saturation --causes--> c/regression`: `causes` is one of the two support-bearing relation kinds (with `answers`), so it is an outgoing edge exactly like a `ground`, and walking it pulls in `c/regression` and, from there, `c/regression`'s own grounds and deps (`e/trace`, `d/p95`). A relation edge is not a footnote here — it is as real a path into the view as `grounds` is.

Two more flags shape an ad-hoc view without changing what reachability means: `--threads t/brief` publishes a named thread alongside the roots (the reachable unit count does not change — a thread is a curated reading order over units already in the view, not a new way to reach one); and `--requires x.sre/mitigations` records which schema extensions a full- fidelity reader of this view must understand, the same field `@doc requires` fills in for a whole document. Multiple `--roots` behave exactly like `@doc roots: [...]` with more than one entry — the view is the union of what each root reaches:

● ● ● *$ smysl view --roots b3:cvhirtgs2mpvli2ethhyeo32uf --roots b3:phsoomklkmlq3sjvbe6cyuqy5v --id v/two-roots fixtures/corpus/F1-incident.smy*

```
v/two-roots: 2 root(s), 0 thread(s), 7 unit(s) reachable
  b3:cvhirtgs2mpvli2ethhyeo32uf
  b3:ekitkvj75uvgzxpvq3ad2nrv3b
  b3:izyuzlt42mqcvgdfb4nfpllxyq
  b3:phsoomklkmlq3sjvbe6cyuqy5v
  b3:re42iey2e7syg6zp73tfrlqbvh
  b3:wo4t2c46lq45fnakd6tajlgcac
  b3:xkys7j42mcuyiaxiyh73xddimr
```

Rooting on the objection directly (`b3:phsoomklkmlq...`, `c/canary-clean`) is exactly how you recover the unit `v/f1` left out: it and its own evidence `e/canary` (`b3:xkys7j...`) both appear now, alongside everything `c/pool-saturation` already reached. Nothing was copied to get there — the union of two root sets is still just a root set.

WHAT'S NEXT, AND WHY

A view names **what is reachable**. It says nothing about which of those six, or five, or seven units matters most if you can only keep three. That ranking is `salience` — Chapter 17 — and it is exactly why `view` exists as a separate, cheap step: rank a hundred units and you are wasting cycles on the ninety a reader will never see; name the view first, then rank only what is actually in play.

bundle — the reachable closure, made portable

A view is free precisely because it does not copy anything — it is a statement about **this** store, in memory, right now. The moment you want to hand that view to somebody else — a teammate, a downstream pipeline, a render step running on a different machine — "free" stops being enough: they need the units themselves, not a promise about how to compute them from a store they may not have.

`bundle` is the answer: it walks the same reachable closure `view` reports and serialises exactly that closure as a self-contained, binary CBOR sequence — a store in miniature, containing nothing the view did not reach.

● ● ● *$ smysl bundle fixtures/corpus/F1-incident.smy | wc -c*

```
1424
```

For comparison, the whole source file — parser input, comments-are-errors surface syntax, every one of the eight units whether `v/f1` reaches them or not — is 2192 bytes:

● ● ● *$ wc -c fixtures/corpus/F1-incident.smy*

```
    2192 fixtures/corpus/F1-incident.smy
```

WHY

Handing someone the whole file works until the file is not the whole story — until it is one document merged from three, with a contention report and units three other views depend on that this recipient has no business seeing. `bundle` gives them precisely the closure of the view they were handed, nothing upstream, nothing from a sibling view, and nothing they cannot already reach by the same `grounds` / `deps` /relation edges `view` prints. It is the reachable set turned into bytes you can put in an email, and it stays load-bearing: whoever receives it can run `check` or `render` on it exactly as if it were the original store.

`--view v/f1` picks which of a store's declared views to bundle, if there is more than one; the first declared view is the default. `--include-retracted` controls one specific edge case in that closure — a unit that has been retracted (Chapter 18) but that something **else** in the bundle still points at by `grounds` or

`deps`. `bundle`'s default already keeps that unit, because a bundle with a dangling reference is strictly worse than one carrying a unit nobody believes anymore; `--include-retracted` only matters for a retracted unit nothing else in the closure references at all, which the default quietly drops and `--include-retracted` keeps. On a two-root view built from an unrelated claim and a retracted, unreferenced "stray" claim, the difference is visible in the byte count itself:

● ● ● *$ smysl bundle v/demo.cbor | wc -c / smysl bundle --include-retracted v/demo.cbor | wc -c*

```
162
279
```

162 bytes is the surviving claim alone; 279 bytes is both, because `--include-retracted` kept the withdrawn one in, gist and all, rather than silently dropping it because a consumer might still want to know it was ever there.

WHAT'S NEXT, AND WHY

You now have two ways to name a reachable set — cheaply as a view, durably as a bundle — but neither one tells you which units in that set are load-bearing and which are along for the ride. Chapter 17's `salience` answers exactly that, over the same reachability this chapter built.

TRY THIS

1. Run `smysl view --id v/narrow --roots b3:wo4t2c46lq45fnakd6tajlgcac --format surface fixtures/corpus/F1-incident.smy`. One root reaches three units. Trace by hand, in the source file, why those three and not the other five.
2. A view stores **roots**, not a list of members. Suppose you add a new claim that grounds on a unit already inside `v/narrow`. Without re-running anything, is it in the view? Now suppose you had stored a member list instead. What breaks?
3. `bundle` takes a view and emits its reachable closure. Given rule L, predict what `bundle` must do about a unit that is reachable but whose grounds are not — and explain why the alternative would make a bundle dangerous to send.

ANSWERS

1. Reachability follows `grounds` and `deps` downward from the root. The root is `c/regression`; it grounds on `e/trace` and depends on the definition `d/p95` — three units, and the walk stops there because measured evidence and a cited definition rest on nothing further. The other five sit **above** the root — `f/root-cause` and what it reaches through its other branch — and a downward walk never sees them. A view is a question asked of the graph, and the answer is whatever the edges say.
2. Yes, it is in the view the moment you add it, with no re-run — because membership is computed from the roots every time rather than stored. With a member list you would have two copies of the truth, and they would diverge the first time anyone edited the graph without remembering to update the list. Worse, two views naming the same unit would each hold their own copy, and a change to one would silently disagree with the other.
3. It must pull the grounds in too — that is rule L, closure: whatever a unit needs travels with it. A bundle that arrived containing a `derived` claim whose evidence had been left behind would be a document that cannot

> be checked by whoever receives it: the claim would read as supported, its support would be a dangling reference, and the recipient has no way to fetch what they were not sent. Closure is what makes a bundle safe to hand to someone who has nothing else.

WHAT YOU LEARNED

- A view is a name plus a root set, never a container: every unit reachable from the roots belongs to it at zero copying cost, which is what makes combining views a plain union rather than a copy-and-reconcile problem.
- Reachability follows edges the way they are authored, forward: `grounds`, `deps`, and the two support-bearing relation kinds (`causes`, `answers`) all count; a `rebuts` edge runs from the objection to the thing objected to, so rooting downstream of a contested claim does not pull its objection in for free.
- `view --roots` builds an ad-hoc view over real uids only — never a label, which exists only inside the one file's parse that defined it. `--id`, `--threads`, and `--requires` name it, curate a reading order over it, and declare what a full-fidelity reader of it needs, in that order.
- `bundle` serialises a view's reachable closure as a portable, self- contained CBOR store — strictly smaller than shipping the whole document, and load-bearing on its own.
- `bundle`'s default keeps a retracted unit only if something else in the closure still references it; `--include-retracted` keeps it unconditionally, and the difference shows up directly in byte count.

CHAPTER 17

17

salience — What Matters, and Why

`view` tells you what is reachable. It does not tell you that a seven-day trace and a one-line definition matter more to an incident brief than the canary run that turned out to be a red herring — and once a store has dozens of units, "what matters" stops being obvious from staring at the graph. `salience` answers that question with a number, and — this is the part that matters for trusting the number — every digit of it is arithmetic you can re-run by hand.

Three named terms, not one opaque score

> TERM **Salience**
>
> A score in `[0, 1]` for every unit in a store, built from exactly three named, independently-weighted terms: **centrality** (how much of the graph depends on this unit, via personalised PageRank over `grounds`, `deps`, `causes`, and `answers`), **corroboration** (how many independent agents attested it, saturating at four independent groups), and **role** (a caller-supplied weight for where a unit sits in the thread currently being packed). `raw = w_c·centrality + w_r·corroboration + w_t·role`, quantised to the nearest 1/1024 once, at the end. An author who sets a unit's `salience` explicitly overrides all three terms outright — a human who says what matters is not second-guessed by an algorithm.

> **WHY**
>
> An importance score you cannot decompose is a score you cannot argue with. If `salience` returned one float and stopped there, disagreeing with a ranking would mean disagreeing with a black box — you would have no way to say **which** part of the reasoning was wrong, only that the number felt off. Three named terms mean a disagreement has an address: "this unit's centrality is right but corroboration is overcounting two attestations that share ancestry" is a claim you can check against the graph, the same way `check`'s diagnostics are addressed at a specific unit rather than a verdict about the file as a whole. Nothing in this system asks you to trust a number you cannot recompute.

Ranking a real store

Run without arguments, `salience` scores every unit in the file and prints them in descending order — the full ranking, not a sample:

$ smysl salience fixtures/corpus/F1-incident.smy

```
0.5000  b3:js4xzessu5zwjpv2rawtugnuvj
0.3027  b3:wo4t2c46lq45fnakd6tajlgcac
0.2129  b3:cvhirtgs2mpvli2ethhyeo32uf
0.1289  b3:ekitkvj75uvgzxpvq3ad2nrv3b
0.1289  b3:re42iey2e7syg6zp73tfrlqbvh
0.0898  b3:izyuzlt42mqcvgdfb4nfpllxyq
0.0000  b3:phsoomklkmlq3sjvbe6cyuqy5v
0.0000  b3:xkys7j42mcuyiaxiyh73xddimr
```

The top scorer, `b3:js4xzessu...`, is `f/root-cause` — the finding, and also the sole root of the file's declared view, `v/f1`. That is not a coincidence: with no `--seed` given, `salience` personalises against a store's view roots by default (§16.4's "no thread and no roots" case falls back to plain, unweighted PageRank only when a store declares **no** view at all). The finding gets first claim on rank because dangling mass returns to the seed on every one of the walk's 32 iterations, and the unit you are ranking **for** is the one thing you certainly care about. The two zero scores are exactly the two units Chapter 16 found `v/f1` cannot reach at all — `c/canary-clean` and `e/canary`. A unit that is not reachable from the active seed has no path for rank to travel along, so it scores nothing; unreachable is not "low salience," it is **no** salience, and that distinction matters when you are deciding whether a unit was excluded on purpose or simply never considered. `--top 3` trims the same ranking to the highest three without changing a single number:

$ smysl salience --top 3 fixtures/corpus/F1-incident.smy

```
0.5000  b3:js4xzessu5zwjpv2rawtugnuvj
0.3027  b3:wo4t2c46lq45fnakd6tajlgcac
0.2129  b3:cvhirtgs2mpvli2ethhyeo32uf
```

--explain — the arithmetic behind one number

`--explain` is the term-by-term breakdown that makes the callout above more than a promise. Run against `c/pool-saturation` (`b3:cvhirtgs2mpvli2eth...`, scored `0.2129` above):

● ● ● *$ smysl salience --explain b3:cvhirtgs2mpvli2ethhyeo32uf fixtures/corpus/F1-incident.smy*

```
b3:cvhirtgs2mpvli2ethhyeo32uf: 0.2129
  centrality      0.4248 x 0.50
  corroboration   0.0000 x 0.20  (0 independent group(s))
  role            0.0000 x 0.30
  raw             0.2129
```

Multiply it out by hand: `0.4248 × 0.5 + 0.0000 × 0.2 + 0.0000 × 0.3 = 0.2124`, which lands on `0.2129` once you account for the fact that `0.4248` is itself already quantised for display — the arithmetic checks out to the quantum, because it is the same arithmetic `salience` ran, not a paraphrase of it. Zero independent attesting groups is the honest reading here: nothing in this fixture stamps an attestation on `c/pool-saturation` at all, so corroboration contributes nothing to its score, and centrality alone is carrying the whole 0.2129.

--weights c,r,t — the same store, a different lens

The default weights — centrality `0.5`, corroboration `0.2`, role `0.3` — are a judgement call, not a law of the graph, and `--weights` lets you make a different one on the same store without touching a single unit. Set weights to `1,0,0` and every score becomes pure centrality, rescaled so the top unit reaches `1.0`:

● ● ● *$ smysl salience --weights 1,0,0 --top 5 fixtures/corpus/F1-incident.smy*

```
1.0000  b3:js4xzessu5zwjpv2rawtugnuvj
0.6055  b3:wo4t2c46lq45fnakd6tajlgcac
0.4248  b3:cvhirtgs2mpvli2ethhyeo32uf
0.2578  b3:ekitkvj75uvgzxpvq3ad2nrv3b
0.2578  b3:re42iey2e7syg6zp73tfrlqbvh
```

The **order** of these five is identical to the default-weight ranking — in this particular fixture, centrality alone already predicts the whole ordering, because nothing here is independently corroborated and no role weights are set. That is a real, checkable fact about this store, not a general property of the three terms: give a unit a strong role weight or a second attesting agent and the order can change (the next two subsections do exactly that). The library ships a second preset for a specific reason rather than as a demonstration: `SalienceWeights::untrusted()` zeroes corroboration outright (`w_r = 0.0`), because corroboration is only honest where agent identity can be trusted, and until attestations are cryptographically signed, a deployment that cannot vouch for who an agent actually was should not let a forged second opinion buy a unit extra rank.

--seed — personalising against a different question

Every ranking above answers "what matters to `f/root-cause`," because that is `v/f1`'s only root and the default seed. `--seed` overrides that, personalising the same walk against different units — and because rank starts at the seed and returns dangling mass to it, seeding on a unit the default view never reaches surfaces a completely different ranking from the same store. Seed on `c/canary-clean` (`b3:phsoomklkmlq...`, one of the two units `v/f1` scored `0.0000` above) instead of the view's default root:

● ● ● *$ smysl salience --seed b3:phsoomklkmlq3sjvbe6cyuqy5v --top 8 fixtures/corpus/F1-incident.smy*

```
0.5000  b3:phsoomklkmlq3sjvbe6cyuqy5v
0.4209  b3:xkys7j42mcuyiaxiyh73xddimr
0.0000  b3:cvhirtgs2mpvli2ethhyeo32uf
0.0000  b3:ekitkvj75uvgzxpvq3ad2nrv3b
0.0000  b3:izyuzlt42mqcvgdfb4nfpllxyq
0.0000  b3:js4xzessu5zwjpv2rawtugnuvj
0.0000  b3:re42iey2e7syg6zp73tfrlqbvh
0.0000  b3:wo4t2c46lq45fnakd6tajlgcac
```

Every unit that led the default ranking — the finding, `c/regression`, `c/pool-saturation` itself — drops to exactly `0.0000`, and the canary claim and its evidence take the entire top of the ranking instead. This is the seed doing its job: `c/canary-clean` grounds only on `e/canary` and reaches nothing else, so once you ask "what matters to the canary claim," the rest of the incident brief is, correctly, irrelevant. Contrast this with seeding on a **leaf** unit — say `e/trace`, which has no outgoing `grounds` of its own — and the seed's entire mass simply returns to itself every iteration with nowhere else to go, scoring `0.5000` and leaving every other unit at `0.0000`. A meaningful seed is a unit with somewhere further to reach; a pure leaf, seeded alone, only ever ranks itself.

Corroboration groups: what counts as a second opinion

None of the units in `F1-incident.smy` carry more than one attestation, so the fixture cannot show corroboration doing anything by itself. The mechanism is worth seeing for real rather than taking on faith, so this is one built from the same library `smysl` itself is written against: one evidence unit, one claim grounded on it, first with a single attesting agent, then with two attesting agents that share no ancestry:

● ● ● *$ smysl salience --seed <claim uid> --explain <evidence uid> corrob-one.cbor*

```
b3:vxqhovkdp36ndqe454ot6cfkcx: 0.4707
  centrality      0.8418 x 0.50
  corroboration   0.2500 x 0.20  (1 independent group(s))
      counted: model:openai/gpt-4
  role            0.0000 x 0.30
  raw             0.4707
```

● ● ● *$ smysl salience --seed <claim uid> --explain <evidence uid> corrob-two.cbor*

```
b3:vxqhovkdp36ndqe454ot6cfkcx: 0.5205
  centrality      0.8418 x 0.50
  corroboration   0.5000 x 0.20  (2 independent group(s))
      counted: model:anthropic/claude
      counted: model:openai/gpt-4
  role            0.0000 x 0.30
  raw             0.5205
```

Adding one more independent attesting agent moves corroboration from `0.2500` to `0.5000` — one group out of the four-group cap to two — and the final score rises from `0.4707` to `0.5205` with centrality untouched, because centrality only depends on graph structure, which did not change. Two details in `corroboration()`'s grouping matter more than the count itself: attestations group by `(agent, recipe)`, so the **same** model answering under the **same** recipe twice is one group, not two — a model cannot corroborate itself by repeating itself — and two groups that trace back to the same parent ancestry collapse into one, because two agents agreeing because they both read the same upstream source is not independent evidence, it is one piece of evidence read twice.

WHAT'S NEXT, AND WHY

Salience is what lets `pack` (Chapter 19) fit a large store into a small budget without asking a model which units to keep. `pack` does not derive its own notion of importance — it asks exactly this ranking, unit by unit, and spends the budget top-down. Everything demonstrated in this chapter — the three terms, the weights, the seed — is the same knob you will reach for again once the question changes from "what matters" to "what fits."

TRY THIS

1. Run `smysl salience --explain b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy`. The unit scores 0.5000, made of centrality 1.0000 at weight 0.50, corroboration 0.0000, and role 0.0000. It is the highest-ranked unit in the document **and** scores zero on two of three terms. Explain how both are true.
2. That unit's corroboration is `0 independent group(s)`. Look at `F1-incident.smy` and say what would have to be added — not changed — to raise it, and why "add another claim agreeing with it" is not the answer.
3. Run plain `smysl salience` on the same file. Two units score exactly `0.0000`. Find them in the source. Are they unimportant?

ANSWERS

1. The three terms measure different things and most units score zero on most of them. Centrality is structural — how much of the argument flows through this unit — and the finding is where everything converges, so it maxes out. Corroboration asks whether **independent** lines of support arrive at it, and role asks whether a thread has assigned it a job. Scoring 0.5 out of a possible 1.0 while leading the document tells you the document is a single chain of reasoning rather than several converging ones — which is a real and useful thing to learn about an incident brief.
2. A second, independent evidential path to the same finding — a different measurement, not resting on the ones already there. Another **claim** agreeing with it adds no corroboration if it grounds on the same evidence, because the term counts independent groups rather than voices; two claims sharing a ground are one line of support wearing two hats. This is precisely the property that stops a document from looking better-supported by restating itself.
3. They are the two units nothing else points at and no thread names — in this document, the ones off the main argumentative spine. Zero salience means **nothing in this store's structure asks for it**, which is a statement about the graph and not about the world. A retracted-but-important caveat, or a piece of evidence somebody

> forgot to ground anything on, scores zero too. Salience decides what survives a budget; it does not decide what matters.

WHAT YOU LEARNED

- Salience is `[0,1]`, built from three named, independently-inspectable terms — centrality, corroboration, role — never a single opaque number; `--explain` reproduces the exact arithmetic behind any one unit's score.
- With no `--seed`, `salience` personalises against a store's declared view roots by default, falling back to plain PageRank only when the store declares no view at all. A unit the seed cannot reach scores exactly `0.0000` — unreachable, not merely low.
- `--weights c,r,t` re-lenses the same store without touching a unit; `SalienceWeights::untrusted()` zeroes corroboration for deployments that cannot yet vouch for agent identity.
- `--seed` changes which question is being asked. Seeding on a unit the default view never reaches can invert the entire ranking; seeding on a pure leaf ranks only that leaf, because its own mass has nowhere else to flow.
- Corroboration counts independent `(agent, recipe)` groups, capped at four; the same agent repeating itself is one group, and two groups sharing ancestry collapse into one, because agreement that traces to a shared source is not two independent checks.

The other ranking: `find`

`salience` answers **what matters here** by looking at the graph — what grounds what, what rebuts what, how much of the argument leans on a unit. It never reads a word of the content. That is a strength when you want the shape of an argument, and useless when you arrive knowing only what you are looking for.

`find` is the complement. It ranks by **words**, over the gist of every unit and — at lower weight — the body and detail beneath it.

● ● ● *$ smysl find "connection pool saturated" fixtures/corpus/F1-incident.smy*

```
7.5637  b3:cvhirtgs2mpvli2ethhyeo32uf  The eu-west connection pool is saturated.
0.8732  b3:xkys7j42mcuyiaxiyh73xddimr  The 4.2 canary ran the same pool
configuration without the regression.
0.8635  b3:js4xzessu5zwjpv2rawtugnuvj  Pool saturation is the leading cause but
is not consistent with the canary.
0.8541  b3:izyuzlt42mqcvgdfb4nfpllxyq  Pool acquisition wait rose from 2 ms to
310 ms over the same window.
0.4362  b3:wo4t2c46lq45fnakd6tajlgcac  p95 auth latency tripled after the 4.2
rollout.
```

> **WHY**
>
> A store that has been through a few hops holds units nobody on this machine wrote, named by uids nobody can read. `trace` needs a starting uid. `view` needs roots. `pack` needs a budget and gives you what fits. None of them answers the question you actually have when you open a file someone handed you, which is **where is the bit about the connection pool**.
>
> It searches the **gist** principally, and that is what makes it work whatever the units contain. A unit's payload might be a stack trace, a metric series or a diff — but every unit carries a gist, because the format requires one, and the gist is a sentence about whatever the payload is. You are never searching the telemetry; you are searching the sentence that describes it.

Two properties are worth knowing because they are unusual for a search tool.

It is **pure**. No model, no index written to disk, no network. The same store and the same query give the same ranking on any machine, with ties broken by uid so there is one right answer rather than an arbitrary one. That is the same guarantee `pack` and `merge` carry, and it is why `find` can sit in a reproducible pipeline rather than beside one.

And it does **not stem**. A search tool for prose would reduce `latencies` to `latenc` so it matches `latency`. That helps English and destroys `connection_pool_size`, which is exactly the term someone types. Identifiers are split into parts **and** kept whole, so `pool.wait_ms` finds its own unit and so does `pool`:

● ● ● *$ smysl find "pool.wait_ms" --kind evidence fixtures/corpus/F1-incident.smy*

```
6.4462  b3:izyuzlt42mqcvgdfb4nfpllxyq  Pool acquisition wait rose from 2 ms to
310 ms over the same window.
0.8732  b3:xkys7j42mcuyiaxiyh73xddimr  The 4.2 canary ran the same pool
configuration without the regression.
```

Where it is weak, measured rather than guessed

The project evaluates this rather than asserting it. Twenty queries over the corpus, in three classes — queries that share the gist's vocabulary, queries that paraphrase it, and bare identifiers:

CLASS	RECALL@5	MRR	FIRST PLACE
shared vocabulary	1.00	0.94	0.88
paraphrase	0.75	0.41	0.12
identifier	1.00	1.00	1.00

Identifiers are perfect and shared vocabulary is nearly so. Paraphrase is where it falls down: the right unit reaches the top five three times in four, but ranks **first** once in eight. Recall without precision is a list to read, not an answer.

Broken down by what kind of unit was being looked for, the reason is plain. `evidence` and `data` units score 1.00 — they name concrete things, and concrete things are findable by name. `claim` units score 0.67, because a claim is an interpretation phrased in whatever words its author reached for, and you will reach for different ones.

Handing what you found to `pack`

Retrieval on its own gives you a ranked list. The units at the top of it are usually **claims**, and a claim without its grounds is an assertion — which is the failure mode of ordinary retrieval-augmented pipelines, where a model is handed five relevant-looking passages and left to infer what connects them.

`pack --query` closes that. The hits become the focus, and packing pulls in what they depend on:

● ● ● *$ smysl --format surface pack --budget 400 --query "connection pool saturated" fixtures/corpus/F1-incident.smy*

```
smysl pack: --query focused 3 unit(s): b3:cvhirtgs2mpvli2ethhyeo32uf,
b3:xkys7j42mcuyiaxiyh73xddimr, b3:js4xzessu5zwjpv2rawtugnuvj
@doc smysl/0.1 { id: v/pack, intent: pack }

@claim c/pool-saturation { status: inferred }
~ The eu-west connection pool is saturated.

@definition d/p95 { status: cited }
~ The 95th percentile of request latency over a one-minute window.

@evidence e/pool-wait { status: measured }
~ Pool acquisition wait rose from 2 ms to 310 ms over the same window.
```

Read the second and third units carefully: **the query matched neither of them.** `d/p95` is a definition the matched claim depends on to be understood, and `e/pool-wait` is the measurement it rests on. Retrieval found the claim; packing refused to hand it over naked.

That is the difference between this and a similarity search, and it is worth being precise about why it is not merely "returning more". The extra units are not the next-most-similar ones — they are the ones the format says are **required**: grounds, deps, and any live rebuttal, because a claim that travels without its rebuttal arrives looking uncontested. Rule R is doing that work, not the ranking.

> **WHY**
>
> A model given five relevant passages will connect them, and its connections will be plausible whether or not they are supported. That is not a flaw in the model; it is what happens when the input has no structure saying what supports what. A pack carries that structure, so the same model can tell the difference between a claim you gave it evidence for and one you did not.
>
> `--query-limit` defaults to 3 rather than `find`'s 10 for the same reason: every focused unit drags its closure in behind it, and ten of them exhaust an ordinary budget before anything gets chosen on merit. If the focus does not fit, packing fails rather than quietly dropping what you searched for.

When words are not enough

Everything above is **lexical**: it matches words. That is why it is perfect on `pool.wait_ms` and weak on "why was it slow" — the second shares almost no vocabulary with the sentence that answers it.

Since 0.7 there is a second engine. Build with `--features semantic`, give it a Model2Vec model directory, and queries that mean the same thing as a gist without using its words start finding it:

ENGINE	RECALL@5	MRR	FIRST PLACE
lexical	0.90	0.74	0.60
semantic	0.95	0.84	0.75
hybrid	0.95	0.87	0.80

On paraphrased queries specifically, the right unit is ranked first half the time rather than one time in eight.

The hybrid routes on what the **query** looks like: a single token carrying a separator is a name, and names go to the lexical engine, which answers them perfectly. Anything with a space in it is prose, and prose goes to the embedder — including a sentence that happens to mention `pool.wait_ms`, because a sentence about a name is still a sentence.

A WRONG TURN WORTH KEEPING

The first version routed on the **kernel type** the caller asked for, on the reasoning that the format already records what a unit is. It scored worse than the embedder alone — and its test passed, because the test only asked it to beat the lexical engine.

The flaw was in when the information arrives. A caller who already knows they want a `claim` is usually not searching; almost every real query carries no filter at all, so the routing never ran and a fallback that merged both engines did. Averaging a good ranking with a bad one gives a middling one.

The fix was to route on the thing that is always present — the query — and to make the test assert the property that had failed: routing must never lose to either engine it routes between. Both the fix and the test came from running the measurement, which is the only reason either was found.

WHAT THAT MEANS FOR YOU

Use `--kind` when you know what you are after. Narrowing to `evidence` when you want a measurement, or to `question` when you want what was asked, beats reading past four things you did not want — and it is a bigger improvement than any tuning of the ranking would be.

And expect to search for **nouns from the domain** rather than for the sentence you would write. `connection pool` works; "why was it slow" does not. That is a real limitation of lexical search, not a bug, and closing it is what a semantic backend would be for. `Retriever` is a trait precisely so one can be added without disturbing any of this.

CHAPTER 18

18

retract — Blast Radius First

Every other command in this part of the book adds information to a store, or names a slice of what is already there. `retract` is the one command whose entire job is **taking belief away** — and unlike deleting a line from a file, withdrawing belief in a unit can leave other units standing on nothing. Get this wrong and a finding three hops downstream silently keeps citing evidence nobody believes anymore. `retract`'s whole design is built around never letting that happen by accident.

Blast radius, computed before anything happens

> TERM **Blast radius**
>
> Every unit a retraction would reach, computed and printed **before** the retraction is applied — the target itself, plus every unit left with no surviving `grounds` once the target's status becomes `unfounded`. Under the default (`strict`) retraction policy this is transitive: a unit orphaned by the retraction can itself orphan whatever rests only on it, and the blast radius follows the whole chain, not just the first hop.

> **WHY**
>
> Retraction is the one operation in this system that can make another unit's grounds disappear out from under it without touching that unit at all. A claim you have not looked at in months can go from `derived` to effectively unfounded because someone three hops upstream retracted the measurement it was built on — and if the tool only told you that after doing it, you would find out by `check` failing later, or worse, by not finding out at all. `retract --dry-run` exists so the blast radius is something you read **before** you decide, not something you discover by having already decided.

Dry run, then the real thing

`fixtures/corpus/F1-incident.smy`'s `c/regression` (`b3:wo4t2c46lq...`) grounds on exactly one unit: `e/trace` (`b3:re42iey2e7syg6zp...`), the seven-day trace. Retracting the trace, dry-run first:

● ● ● *$ smysl retract --dry-run b3:re42iey2e7syg6zp73tfrlqbvh fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: retracting b3:re42iey2e7syg6zp73tfrlqbvh would
reach 2 unit(s), orphaning 1
fixtures/corpus/F1-incident.smy:   b3:wo4t2c46lq45fnakd6tajlgcac would lose all
of its grounds
```

Two units reached — the trace itself, plus `c/regression`, which loses its only ground and is named explicitly rather than left for you to work out. Note what is **not** in the blast radius: `f/root-cause` grounds on both `c/pool-saturation` and `c/regression`, and only one of those two goes unfounded, so the finding keeps standing on the one that survives. A unit with any surviving ground is never orphaned — orphaning requires **every** ground to be gone, which is exactly why the blast radius has to be computed rather than assumed from "this reaches the trace, and the trace reaches the regression, and the regression reaches the finding."

Dropping `--dry-run`, given an authority that accepts the request (the next section covers exactly when that is true), reports the same reach and then recomputes effective status with the retraction in place:

● ● ● *$ smysl retract --as model:openai/gpt-4 --authority any b3:re42iey2e7syg6zp73tfrlqbvh corrob-two.cbor*

```
corrob-two.cbor: retracting b3:vxqhovkdp36ndqe454ot6cfkcx would reach 2 unit(s),
orphaning 1
corrob-two.cbor:   b3:2tbhvo5rklopzpxb44o4nua2zb would lose all of its grounds
corrob-two.cbor: 2 unit(s) now read as unfounded
```

The dry run and the real run report an identical blast radius — that agreement is not incidental, it is the specific property `retract`'s own test suite gates on: a dry run that disagreed with what applying it actually does would be worse than no dry run at all. The final line is the number that matters operationally: two units now read as `unfounded`, where a moment ago they read as `measured` and `inferred`. Declared status — what an agent originally wrote — never changes; **effective** status is what a reader sees once every retraction in the store has been accounted for, and it is effective status, not declared status, that `check` and `render` act on downstream.

Authority: who is allowed to retract what

Computing the blast radius is only half of `retract`'s job. The other half is deciding whether the agents asking are **allowed** to cause it — because "anyone can withdraw anything" is a censorship vector the moment more than one agent writes to a shared store.

`--AUTHORITY`	WHO MAY RETRACT THE TARGET
`origin` (default)	Only an agent that has already attested this specific unit. A stranger cannot silence work they had no hand in.
`any`	Any single named agent — the permissive setting, for a single-writer store where the question does not arise.
`quorum:N`	At least `N` **distinct** agents must jointly issue the retraction; the same agent repeated does not count twice.

`origin` is the default for a reason: it is the direct defence against one adversarial agent erasing another's work. `fixtures/corpus/F1-incident.smy` demonstrates the refusal for free, because none of its units carry an explicit attestation at all — asking to retract one under the default authority fails before the blast radius even gets a chance to matter:

● ● ● *$ smysl retract --as human:vladimir b3:cvhirtgs2mpvli2ethhyeo32uf fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: retracting b3:cvhirtgs2mpvli2ethhyeo32uf would
reach 1 unit(s), orphaning 0
smysl retract: origin authority: none of the 1 requesting agent(s) attested this
unit
```

The blast radius still printed — refusal does not withhold the information a caller would need to argue for the retraction, it only withholds permission to apply it. The same command, on a store where the retracting agent genuinely is an attestor, succeeds:

● ● ● *$ smysl retract --as model:openai/gpt-4 --authority origin b3:vxqhovkdp36ndqe454ot6cfkcx corrob-two.cbor*

```
corrob-two.cbor: retracting b3:vxqhovkdp36ndqe454ot6cfkcx would reach 2 unit(s),
orphaning 1
corrob-two.cbor:   b3:2tbhvo5rklopzpxb44o4nua2zb would lose all of its grounds
corrob-two.cbor: 2 unit(s) now read as unfounded
```

— `model:openai/gpt-4` is one of the two agents Chapter 17 attested that same evidence unit with, so `origin` authority accepts it. A stranger, on the same store, is refused exactly like the F1 example above. `quorum:2` on the same target shows the same pattern with a distinct-agent count instead of an attestation check — two different named agents succeed, and the same agent named twice does not:

● ● ● *$ smysl retract --as human:a --as human:a --authority quorum:2 b3:vxqhovkdp36ndqe454ot6cfkcx corrob-two.cbor*

```
corrob-two.cbor: retracting b3:vxqhovkdp36ndqe454ot6cfkcx would reach 2 unit(s),
orphaning 1
corrob-two.cbor:   b3:2tbhvo5rklopzpxb44o4nua2zb would lose all of its grounds
smysl retract: quorum:2 requires 2 distinct agents, got 1
```

Naming the same agent twice on the command line is one agent, not two — the count is over distinct identities, not over how many `--as` flags you typed.

Retraction policy, chosen at merge time

`--authority` decides **who** may retract; it has nothing to do with **how far** a retraction reaches once it is allowed. That second question — `strict`, `advisory`, or `ignore` — is the retraction policy Chapter 13 covers as one of `merge`'s three knobs, and it governs the exact transitivity this chapter has been demonstrating under its default, `strict`. Under `advisory`, the same retraction marks the target `unfounded` and reports it (`SMY-W052`) but never touches a dependent's status; under `ignore`, the retraction is recorded and changes nothing at all. `plan_retraction` takes whichever policy the store was merged under as a parameter for exactly this reason — a blast radius computed under the wrong policy would tell you the wrong thing about what is actually at stake. If a retraction here looks smaller than you expected, check which policy the store you are working in was merged with before assuming the blast radius is wrong.

WHAT'S NEXT, AND WHY

A retraction changes what a store **means** without changing a single byte of any other unit's declared content — which is exactly the kind of change that is easy to lose track of. `diff --hop` (Chapter 14) is the most direct way to confirm the aftermath matches what the blast radius promised, unit by unit; `check` (Chapters 8 and 21) is the broader sweep, and will surface any unit left orphaned as `SMY-E050` if the active retraction policy is transitive. Run one of the two after any retraction you did not just dry-run — the whole point of computing the blast radius in advance was to know what to expect from it.

TRY THIS

1. Run `smysl retract --dry-run b3:re42iey2e7syg6zp73tfrlqbvh fixtures/corpus/F1-incident.smy`. It reports reaching 2 units and orphaning 1, and **names** the unit that would lose all of its grounds. Now run the same dry run against `b3:ekitkvj75uvgzxpvq3ad2nrv3b`. Why does one orphan something and the other not?
2. The command is `--dry-run` by default in every example this chapter shows. Make the argument for why a tool would make you ask twice to retract, when it does not make you ask twice to merge.

3. A unit that loses all of its grounds becomes `unfounded`. Look back at Chapter 6's exercise, where authoring `unfounded` directly was an error. Reconcile the two: why may the tool write a status you may not?

ANSWERS

1. Because blast radius follows `grounds`, and the two units sit at different depths of the argument. One is a leaf that nothing else rests on; the other is the sole support of a claim above it, so retracting it leaves that claim holding nothing. The output names the orphan rather than counting it, because "1 unit would be orphaned" is not actionable and "`b3:wo4t2…` would lose all of its grounds" is.
2. Because merge is additive and retraction is not. A merge that surprises you leaves everything it found still in the store, and you can look again. A retraction propagates: it can change the standing of units nobody has looked at in months, several hops from the one you named. The asymmetry is between operations you can inspect **after** and operations you had better inspect **before** — and the tool refuses to let the second kind be discovered by running it.
3. Because the tool is recording a history that actually happened and you would be asserting one that did not. `unfounded` means **something this rested on was withdrawn** — when `retract` writes it, that is a true statement about an event in the store. Typed by hand on a unit that never had support, it claims a collapse that never occurred; `speculative` is the honest word for a claim that never had grounds. The status is not reserved because it is dangerous, but because it is a **consequence**, and only the tool is in a position to observe it.

WHAT YOU LEARNED

- Blast radius is every unit a retraction would reach, computed and printed before anything is applied — the target, plus every unit that would lose **every** surviving ground, transitively, under the active retraction policy.
- `--dry-run` and the real retraction report an identical blast radius by construction; the real run additionally recomputes and prints the effective-status count once the retraction is in place.
- Declared status never changes; effective status is what a retraction changes, and it is effective status that downstream commands like `check` and `render` act on.
- `--authority origin` (default) requires the retracting agent to have already attested the target — the direct defence against a stranger silencing work they had no hand in. `any` accepts a single named agent unconditionally; `quorum:N` requires `N` distinct agents, where the same agent named twice still counts as one.
- Authority governs **who** may retract; retraction policy — chosen at merge time, Chapter 13 — governs **how far** an authorised retraction reaches, from fully transitive (`strict`) to recorded-but-inert (`ignore`).

CHAPTER 19

19

pack — Budget-Bounded Selection

Every earlier chapter in this part assumed the whole store travels: `view` picks roots, `salience` ranks, `bundle` groups, but nothing so far has had to throw anything away. Sooner or later something downstream has a ceiling — a model's context window, a chat message limit, a person's patience — and the document has to fit inside it. `pack` is the command that does the fitting, and the whole chapter is really about one design decision: what survives the cut is never chosen by how important a sentence sounds.

> **WHY**
>
> The obvious way to shrink a document is to keep the most important-sounding sentences and drop the rest. That is exactly the failure mode `Documentation/SMYSL_RATIONALE.typ` opens with: paraphrase a document enough times and disagreement is the first thing to go, because nothing marks an objection as **load-bearing** to the claim it argues with — a summariser keeps the confident-sounding claim and drops the hedge sitting next to it. `pack` is the alternative: budget-bounded selection where a claim's rebuttal is not optional cargo. If the budget cannot hold a claim **and** what argues with it, the claim does not travel alone — the whole operation fails, loudly, rather than shipping something that reads as uncontested when it was not.

The seven constraints

Every pack is checked against seven named constraints (`smysl-pack`'s `constraints.rs`). Six are closure obligations — things a selected unit drags in with it — and the seventh is the budget itself.

`--explain` prints the constraint code next to every forced unit, so this table is the key to reading that output rather than guessing at it from the letter:

CODE	BINDS WHEN	WHAT IT DEMANDS
C1	a unit sits at L1 or above	Its `deps` — interpretive prerequisites — must also be selected, at least at L0. A claim about "p95" cannot be understood without the definition of p95 sitting next to it.
C2	a unit sits at L1 or above	Its `grounds` — the evidence it rests on — must also be selected. This is the ordinary evidential floor: a claim at depth without its support is a claim with nothing under it.
C3	always, at every level, including L0	Every unit that rebuts it must also be selected. This is **rule R**, the constraint the whole chapter is really about: a claim never travels unopposed, even as a bare gist.
C4	a unit is a position in an open contention	Every other position in that contention must also be selected — a live disagreement is presented whole or not at all, never as one side.
C5	a unit is pinned	It must reach L1. Pinning is how `--focus` and an active thread's referenced units make a demand: not "include this," but "include this at enough depth to read."
C6	a unit sits at L1 or above	Any unit it names in a `warrant` edge — the inferential licence for the step it took — must also be selected, at L0.
C7	always	Total selected cost must not exceed the budget. The one constraint that is a number rather than a graph shape.

`--explain` reports the **reason** a unit is in using a slightly different vocabulary — `Reason` in `closure.rs` — because a reason names **what pulled it in**, not just which rule fired:

`--EXPLAIN` SAYS	CONSTRAINT	MEANING
`in focus`	C5	Named directly in `--focus`.
`pinned by the thread`	C5	Named by the active thread's steps, when packing under `--thread`.
`rebuts U`	C3	It rebuts unit `U`, which is in the selection — this is rule R showing its work.
`contests U`	C4	It is the other position in an open contention that touches `U`.
`dep of U`	C1	`U` needs it to be interpretable at L1+.
`ground of U`	C2	`U` needs it as evidence at L1+.
`warrant of U`	C6	`U` needs it as the licence for an inference at L1+.
`earned on density`	`-`	Nothing forced it in — it won a place by value per token, same as any other budget-driven selection.

A worked pack, read line by line

`fixtures/corpus/F1-incident.smy` is the postmortem fixture used throughout this book: eight units arguing that a saturated connection pool in `eu-west` caused an auth-latency regression, with one canary reading that complicates the story. `c/pool-saturation` is the contested claim, and its uid — a real, resolved content hash, not a label — is what `--focus` takes:

> **NOTE**
>
> `--focus`, `--seed`, and similar flags take a **uid**, never a label. Labels are a surface-file convenience for the person writing the `.smy` file; a uid is the only identifier the store actually resolves. Passing a label fails cleanly rather than silently doing the wrong thing:
>
> ● ● ● *$ smysl pack --budget 200 --focus c/pool-saturation fixtures/corpus/F1-incident.smy*
>
> ```
> smysl pack: `c/pool-saturation` is not a uid
> ```
>
> Mine the real uid from `thread --show` (Chapter 20 covers it directly) or `salience`, then pass that instead.

● ● ● *$ smysl --format surface pack --budget 200 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf fixtures/corpus/F1-incident.smy*

```
b3:cvhirtgs2mpvli2ethhyeo32uf @L1  C5  in focus
b3:ekitkvj75uvgzxpvq3ad2nrv3b @L0  -  earned on density
b3:izyuzlt42mqcvgdfb4nfpllxyq @L0  C2  ground of b3:cvhirtgs2mpvli2ethhyeo32uf
b3:js4xzessu5zwjpv2rawtugnuvj @L0  -  earned on density
b3:phsoomklkmlq3sjvbe6cyuqy5v @L0  C3  rebuts b3:cvhirtgs2mpvli2ethhyeo32uf
b3:re42iey2e7syg6zp73tfrlqbvh @L0  -  earned on density
b3:wo4t2c46lq45fnakd6tajlgcac @L1  -  earned on density
b3:xkys7j42mcuyiaxiyh73xddimr dropped: low-value
fixtures/corpus/F1-incident.smy: 7 of 8 unit(s), 193 of 200 tokens, greedy mode,
gap 0.010
```

Cross-referencing this against the surface excerpt the same command writes to stdout resolves every uid to a label:

UNIT	LEVEL	WHY IT IS HERE
`c/pool-saturation`	L1	The focus itself. Pinned by C5, so it reaches full depth.
`e/pool-wait`	L0	C2: the evidence `c/pool-saturation` grounds itself in. Without it the focus would be a claim resting on nothing visible.
`c/canary-clean`	L0	C3, the interesting line: this unit rebuts the focus (`@rel c/canary-clean --rebuts--> c/pool-saturation` in the source), so it is **forced** in, not merely likely to be picked.
`d/p95`, `f/root-cause`, `e/trace`	L0	Earned their place on density — no constraint demanded them, they were simply worth their token cost.
`c/regression`	L1	Also earned on density, but at L1 rather than L0 — greedy bought it depth because the budget had room and the value justified it.

UNIT	LEVEL	WHY IT IS HERE
`e/canary`	dropped	`low-value`: below the value floor at every level it could be bought at, and nothing forces it in.

The footer line is the packinfo, spoken: **7 of 8 units, 193 of 200 tokens, greedy mode, gap 0.010.** Read the middle line of the whole example again — `b3:phsoo… rebuts b3:cvhi…` — because that line is the same argument `SMYSL_RATIONALE.typ`'s section on compression makes, compressed into one row of output: what survived was not "the most important-sounding claim," it was **the claim and whatever argues with it**, selected together or not at all.

Budget syntax: plain numbers and the `k` suffix

`--budget` takes a token count, optionally with a `k` suffix meaning thousands (`parse_budget` in `main.rs`: strip a trailing `k`/`K`, parse the remainder, multiply by `1000`; anything else parses as a literal integer). `8000` and `8k` are the same budget:

● ● ● *$ smysl --format surface pack --budget 1k --explain fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: 8 of 8 unit(s), 244 of 1000 tokens, greedy
mode, gap 0.000
```

A thousand-token budget is generous enough to take the whole eight-unit store at full depth, which is also the cleanest way to see what "nothing was left behind" looks like: `gap 0.000` with every unit selected is the uninteresting, correct baseline every tighter budget below is measured against.

Naming more than one focus

`--focus` can be repeated. Each named unit is pinned independently, and their closures are computed and merged — asking for two focuses is not the same as asking for one twice as important, it is asking for two independent floors that both must be met:

● ● ● *$ smysl pack --budget 400 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf --focus b3:js4xzessu5zwjpv2rawtugnuvj fixtures/corpus/F1-incident.smy*

```
b3:cvhirtgs2mpvli2ethhyeo32uf @L1  C5  in focus
b3:ekitkvj75uvgzxpvq3ad2nrv3b @L0  -  earned on density
b3:izyuzlt42mqcvgdfb4nfpllxyq @L0  C2  ground of b3:cvhirtgs2mpvli2ethhyeo32uf
b3:js4xzessu5zwjpv2rawtugnuvj @L1  C5  in focus
b3:phsoomklkmlq3sjvbe6cyuqy5v @L0  C3  rebuts b3:cvhirtgs2mpvli2ethhyeo32uf
b3:re42iey2e7syg6zp73tfrlqbvh @L0  -  earned on density
b3:wo4t2c46lq45fnakd6tajlgcac @L2  C2  ground of b3:js4xzessu5zwjpv2rawtugnuvj
b3:xkys7j42mcuyiaxiyh73xddimr @L0  -  earned on density
fixtures/corpus/F1-incident.smy: 8 of 8 unit(s), 244 of 400 tokens, greedy mode,
gap 0.000
```

The second focus (`f/root-cause`, the finding) pulls its own grounds in at C2 — one of which, `c/regression`, ends up at L2 because greedy had budget left after meeting both floors and bought it full depth. With two focuses and a comfortable budget everything in the store ends up selected anyway,

so this example is really about the **mechanism**: each `--focus` is its own demand, checked and satisfied independently, before density gets to spend what is left.

Capping detail with `--lod`

`--lod L0`/`L1`/`L2` caps **every** unit at that level, whatever it was authored at and whatever a constraint would otherwise buy it. Because depth costs tokens, a cap can change what fits — the same budget that drops a unit at full depth can afford every unit once none of them is allowed past a gist:

COMMAND	UNITS IN	TOKENS USED	WHAT CHANGED
`pack --budget 200 --focus …`	7 of 8	193 of 200	`e/canary` dropped as `low-value`; nothing left to buy it with.
`pack --budget 200 --focus … --lod L0`	8 of 8	137 of 200	Every unit capped at L0 costs less, so the same 200-token budget now holds the whole store, gap 0.000.

● ● ● *$ smysl pack --budget 200 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf --lod L0 fixtures/corpus/F1-incident.smy*

```
b3:cvhirtgs2mpvli2ethhyeo32uf @L0  C5  in focus
b3:ekitkvj75uvgzxpvq3ad2nrv3b @L0  -  earned on density
b3:izyuzlt42mqcvgdfb4nfpllxyq @L0  -  earned on density
b3:js4xzessu5zwjpv2rawtugnuvj @L0  -  earned on density
b3:phsoomklkmlq3sjvbe6cyuqy5v @L0  C3  rebuts b3:cvhirtgs2mpvli2ethhyeo32uf
b3:re42iey2e7syg6zp73tfrlqbvh @L0  -  earned on density
b3:wo4t2c46lq45fnakd6tajlgcac @L0  -  earned on density
b3:xkys7j42mcuyiaxiyh73xddimr @L0  -  earned on density
fixtures/corpus/F1-incident.smy: 8 of 8 unit(s), 137 of 200 tokens, greedy mode,
gap 0.000
```

Notice the focus itself is now `@L0` rather than `@L1` — a cap wins over a pin's usual promise to reach L1 (C5's own text: a gist-only unit is satisfied by L0 because there is nothing higher to buy; `--lod L0` puts every unit, including the focus, in exactly that position on purpose). What you get back is breadth without depth: every claim named, none of them argued out past a sentence. Whether that trade is the right one depends on what happens next to the pack — Chapter 24 covers the render side of that same trade in `--lod`'s other home.

Two ways to solve: `--mode greedy` and `--mode exact`

The default solver is greedy by density (`solve.rs`): fast, and for this fixture at this budget, exactly as good as anything else could do — but **greedy is not proven optimal, it is merely checked afterward**.

There used to be a local-improvement pass after it, which downgraded the least valuable depth and spent what that freed on breadth. It was removed in 0.8 because it was finally measured: across 28 000 generated packs it changed 26, and 22 of those 26 came out **worse** by the value function it existed to maximise. It fired on 0.09% of packs, which is how it survived eight releases looking harmless. `--mode exact` asks for a proof instead, by branch-and-bound search over closure-complete anchors, pruned against a fractional-relaxation upper bound (`exact.rs`). That search is compiled in only behind the `exact-pack` feature — the default build does not carry it:

● ● ● *$ smysl pack --budget 200 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf --mode exact fixtures/corpus/F1-incident.smy (default build)*

```
fixtures/corpus/F1-incident.smy: warning: SMY-W202: exact packing is not
compiled in; rebuild with the `exact-pack` feature
b3:cvhirtgs2mpvli2ethhyeo32uf @L1  C5  in focus
...
fixtures/corpus/F1-incident.smy: 7 of 8 unit(s), 193 of 200 tokens, exact mode,
gap 0.010
```

`cargo build --features exact-pack` from the repository root does build it — it is a real, working feature, not a stub — and rerunning the identical command against that binary gets you the proof rather than the warning:

● ● ● *$ smysl pack --budget 200 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf --mode exact fixtures/corpus/F1-incident.smy (built with --features exact-pack)*

```
b3:cvhirtgs2mpvli2ethhyeo32uf @L1  C5  in focus
...
fixtures/corpus/F1-incident.smy: 7 of 8 unit(s), 193 of 200 tokens, exact mode,
gap 0.000 (proven optimal)
```

Same selection, same eight units — but the `gap` field is the whole point of the exercise, and it is worth being precise about what it means (`Optimality` in `smysl-core`'s `aux.rs`): **an upper bound on the value left on the table, in `[0, 1]`; zero means proven optimal.** Greedy's `0.010` on this fixture is not "10% of the tokens wasted" — it is a provable ceiling on how much better an optimal pack **could** have scored, computed from the fractional relaxation of the remaining budget. On this small, easy fixture the ceiling was low and exact mode found nothing greedy had missed. On a larger store with a tighter budget the two numbers can diverge a lot more, and that is precisely the situation `--mode exact` exists for: a gap you can act on ("this greedy pack might be leaving real value on the table, go check") rather than one you have to take on faith.

> **NOTE**
>
> Branch-and-bound is worst-case exponential — the problem is NP-hard — so `exact.rs` caps the search at `NODE_LIMIT` (250,000 nodes) and, above `EXACT_THRESHOLD` (256 units) in scope, declines to run it at all and packs greedily instead, saying so via `SMY-W202` rather than hanging. Hitting either limit does not invalidate the pack — the incumbent selection is still valid — it just means the reported gap is an estimate rather than a proof for that run.

The cost model: `--tokenizer`

Packing has to be pure (rule D: same graph, same budget, same thread yields identical bytes everywhere), and no provider's real tokeniser is available offline or stable across its own versions — so `smysl` ships one deterministic, approximate estimator and records which one produced every pack, always:

> **TERM Estimator**
>
> `smysl`'s bundled cost model, identified by `smysl/utf8-div4` (`DEFAULT_ESTIMATOR` in `smysl-pack`). The formula is `cost(text) = ceil(utf8_len(text) / 4) + 2` — a quarter-byte-per-token approximation plus two tokens of per-item framing overhead. It is not claiming to match any real model's tokeniser; it is claiming to be **deterministic and disclosed**, which a real tokeniser bundled offline could not promise to stay.

Every `packinfo` carries the estimator's id regardless of whether you ever pass `--tokenizer` (D-2: a budget that does not say what it was counted with is a number without a unit). Today there is exactly one estimator shipped (`Estimator::ALL` has one member), so `--tokenizer smysl/utf8-div4` is a no-op that happens to match the default — the flag exists so that a second estimator, when one ships, has somewhere to be named without a wire-format change:

● ● ● *$ smysl pack --budget 200 --tokenizer tiktoken/cl100k fixtures/corpus/F1-incident.smy*

```
smysl pack: unknown tokenizer `tiktoken/cl100k`
```

When the floor doesn't fit: infeasible budgets

C3 is not a suggestion. If the mandatory floor — the focus plus everything C1 through C6 force in around it — costs more than the budget, packing does not quietly ship the focus alone and call it done. It fails, with the exact minimum budget that would have worked:

● ● ● *$ smysl pack --budget 45 --focus b3:cvhirtgs2mpvli2ethhyeo32uf fixtures/corpus/F1-incident.smy*

```
smysl pack: SMY-E200: budget 45 but the mandatory floor needs 46
```

The exit code is `4` (`PackInfeasible` in the exit-code contract, Appendix C) — distinct from an ordinary failure, so a caller can tell "this budget was too small" apart from "something else went wrong" without parsing stderr. And the reported minimum is exact, not a rounded estimate — one token less than it and the pack still fails; exactly it and the pack succeeds, buying precisely the focus, its ground, and its rebuttal, with every other unit priced out on budget alone:

● ● ● *$ smysl --format surface pack --budget 46 --explain --focus b3:cvhirtgs2mpvli2ethhyeo32uf fixtures/corpus/F1-incident.smy*

```
b3:cvhirtgs2mpvli2ethhyeo32uf @L1  C5  in focus
b3:izyuzlt42mqcvgdfb4nfpllxyq @L0  C2  ground of b3:cvhirtgs2mpvli2ethhyeo32uf
b3:phsoomklkmlq3sjvbe6cyuqy5v @L0  C3  rebuts b3:cvhirtgs2mpvli2ethhyeo32uf
b3:ekitkvj75uvgzxpvq3ad2nrv3b dropped: budget
b3:js4xzessu5zwjpv2rawtugnuvj dropped: budget
b3:re42iey2e7syg6zp73tfrlqbvh dropped: budget
b3:wo4t2c46lq45fnakd6tajlgcac dropped: budget
b3:xkys7j42mcuyiaxiyh73xddimr dropped: low-value
```

This is the whole chapter in one pair of runs: at 45 tokens there is no version of this pack that presents `c/pool-saturation` without also presenting the reading that complicates it, so `pack` refuses outright rather than pick one. At 46 tokens it delivers exactly that pair, nothing more. A budget failing loudly here is not a bug to work around — it is the one behaviour that makes every **successful** pack trustworthy without your having to re-audit it by hand.

WHAT'S NEXT, AND WHY

A packed, budget-bounded excerpt like the ones above is exactly the shape you hand to a model with a context limit, or to a person who does not have time for the whole store — that is the whole job this chapter's command does. Two things follow from here. If what you actually want is prose rather than a truncated surface excerpt, Chapter 24's `render` is the command that turns a selection into an artifact meant to be **read**, not decoded; `pack`'s non-surface output — the default CBOR form this chapter mostly suppressed with `--format surface` for readability — is a portable sub-store in its own right, small enough to ship and still a real store any later command can open. And a pack has no opinion yet about **order** — it says what survives, not what order a reader should meet it in. That question is Chapter 20's.

TRY THIS

1. Run `smysl pack --budget 60 --explain --format surface fixtures/corpus/F1-incident.smy`. Three units survive and five are dropped under **three different reasons**: `budget`, `low-value`, and `closure-cost`. Explain what makes `closure-cost` a different kind of rejection from `budget`.
2. In that same run, `f/root-cause` is kept while `c/pool-saturation`, one of the two units it grounds on, is dropped. That looks like a closure violation. Consult the constraint table at the top of this chapter and explain why it is not.
3. Feed the packed surface output back into `check`. It reports `SMY-E031` and `SMY-E032` — the packed document does not validate. Is that a bug? Answer before reading on.

ANSWERS

1. `budget` means the unit was affordable in isolation and there was no room left by the time the packer reached it. `closure-cost` means the unit itself was cheap but bringing it **legally** was not — selecting it would have obliged the packer to bring its grounds, or its rebuttals under rule R, and the whole bundle did not fit. The distinction is worth having because the fixes differ: a `budget` drop is answered by raising the budget, a `closure-cost` drop often by restructuring what the unit depends on.
2. Because the closure constraints are level-dependent. C1, C2 and C6 — deps, grounds, warrants — bind only at L1 and above. `f/root-cause` was selected at **L0**, a bare gist, where the only constraint that still binds is C3: rule R, rebuttals. A gist is an assertion the reader can see is unelaborated; an L1 block that showed its reasoning while hiding the ground that reasoning rests on would be the dishonest case, and that is the one the constraints forbid.
3. Not a bug — but the packed output is not a document. At L0 a unit is emitted as a gist with its fields stripped, so a `derived` claim arrives with no `grounds` and a `cited` definition with no `source`, which is exactly what `SMY-E031` and `SMY-E032` describe. A pack is a **payload** built to fit a budget, with a `@packinfo` receipt saying what was dropped and degraded; it is meant to be read, not re-validated as a store. If you want something that checks clean, `bundle` a view — closure there is the whole point.

WHAT YOU LEARNED

- Seven constraints, C1 through C7, govern every pack; six are closure obligations and the seventh is the budget. `--explain` names which one forced each unit in, and rule R (C3) is the one that matters most: a selected claim's rebuttals travel with it, always, at every level.
- `--budget` takes a plain integer or a `k`-suffixed thousands shorthand; `--focus` may be repeated, each occurrence an independent floor that must be met; `--lod` caps every unit's depth and can change what fits inside an unchanged budget.
- `--mode exact` asks for a branch-and-bound proof instead of a greedy estimate; it needs the `exact-pack` feature to run at all, and its `gap` field is a provable ceiling on value left on the table, not a rounded guess — `0.0` means proven optimal.
- Every pack records the estimator that costed it, whether or not `--tokenizer` was passed, because a budget without a disclosed cost model is a number without a unit.
- A budget too small for the mandatory floor fails outright, with exit code `4` and the exact minimum that would have worked — never a one-sided pack that looks uncontested because it was too small to hold the objection.

CHAPTER 20

20

thread — Deriving Structure

`salience` ranks. `pack` compresses to a budget. Neither one arranges what survives into a path a specific reader is meant to walk. A finding, its grounds, and the claim that rebuts it can all clear the same budget and still arrive as an unordered pile — true, complete, and no shape at all. A `thread` is what gives a selection of units a **reading order** and says what role each one plays in it, for one named audience at a time.

> **WHY**
>
> The same graph can be a good brief for an executive, a good walkthrough for the engineer who has to fix it, and a good Q&A transcript for the person who only asked one question. Nothing about the underlying units changes between those three readings — only which units matter, in what order, and under what label. Rather than write three summaries by hand and hope they stay consistent with each other and with the graph, `thread` derives all three **from** the graph, deterministically, so the same store always yields the same brief, the same narrative, the same Q&A — and a later change to the graph is a re-derivation, not a rewrite.

Roles, schemas, and arities

> TERM **Thread schema**
>
> One of five closed shapes a thread can take in `smysl` 0.1: `analysis`, `narrative`, `brief`, `qa`, `plan`. Each schema declares an ordered list of **roles** — `bottom-line`, `support`, `setup`, `finding`, and so on — and, for each role, how many units it may hold. The set is closed by design (D-4): the rule language that assigns units to roles is not yet stable enough to expose as a user-facing extension point, so a sixth schema waits for a later format version rather than shipping something that would then have to be supported forever.

> TERM **Role**
>
> A named slot in a schema's narrative order — `finding`, `risk`, `question`, `step`, and twenty more across the five schemas, sharing one wire-stable numbering. A role belongs to whichever schemas declare it; `risk` happens to appear in both `brief` and `plan` as the same role, not two coincidentally-named ones.

Every schema's table is small enough to read whole. Weight is what a role contributes to salience's role-weight term when a thread is active — the schema's own statement of which of its roles matters most, independent of where that role sits in the reading order:

SCHEMA	ROLES, IN ORDER	ARITY	WEIGHT
`analysis`	context	0..2	0.4
	tension	1..2	0.8
	approach	0..2	0.5
	finding	1..3	1.0
	rebuttal	0..3	0.9
	implication	0..2	0.6
	next	0..1	0.3
`narrative`	setup	1..1	0.6
	complication	0..2	0.8
	turn	0..2	1.0
	resolution	0..2	0.9
	coda	0..1	0.5
`brief`	bottom-line	1..1	1.0
	support	1..3	0.7
	risk	0..2	0.8
	ask	0..1	0.5
`qa`	question	1..1	0.9
	evidence	0..3	0.6
	answer	1..2	1.0

SCHEMA	ROLES, IN ORDER	ARITY	WEIGHT
	caveat	0..2	0.7
`plan`	goal	1..1	1.0
	constraint	0..3	0.6
	step	1..5	0.8
	decision	0..2	0.7
	risk	0..2	0.7

Four schemas assign roles by **kind** — a rule table matches a unit's type, its status, or which edges point at or from it, first match wins. `narrative` is the one exception: it assigns by **position** in the graph's ordering chain (`--sequences-->` edges), because a narrative is about sequence, not about what kind of thing each beat is.

Deriving all five, one worked example each

Derivation is a pure function of the graph — no model is consulted, so the same store always derives the same thread on any machine (rule D again; the same discipline that makes `pack` reproducible). `--explain` prints how many units landed in each role against its declared arity, names any required role nothing could fill, and lists what coherence repair added.

`analysis` — F6-adversarial

`F6-adversarial.smy` is a small, deliberately loose corpus: a hunch, a rumour, and two claims built on top of them. It happens to derive a clean `analysis` thread — every required role filled, nothing left over:

$ smysl thread --derive analysis --explain --only fixtures/corpus/F6-adversarial.smy

```
fixtures/corpus/F6-adversarial.smy: context      0 of 0..2
fixtures/corpus/F6-adversarial.smy: tension      1 of 1..2
fixtures/corpus/F6-adversarial.smy: approach     1 of 0..2
fixtures/corpus/F6-adversarial.smy: finding      1 of 1..3
fixtures/corpus/F6-adversarial.smy: rebuttal     0 of 0..3
fixtures/corpus/F6-adversarial.smy: implication  2 of 0..2
fixtures/corpus/F6-adversarial.smy: next         0 of 0..1
fixtures/corpus/F6-adversarial.smy: 0 unit(s) not selected
@thread t/derived-analysis { schema: analysis, owner: tool:smysl, ts: [0, 0] }
~ The eu-west capacity shortfall is the root cause of the auth regression.
  tension → o/rumour
  approach → h/guess
  finding → f/laundered
  implication → c/laundered-once
  implication → c/laundered-twice
```

`o/rumour` (an `@observation`) took `tension`, `h/guess` (a `@hypothesis`) took `approach`, and the two claims built on top of them landed as `implication` — exactly the rule table's stated mapping (`Matcher::Type` on observation, hypothesis, and claim respectively), with the finding itself weighing heaviest in the gist that opens the thread.

`narrative` — F3-postmortem

`F3-narrative.smy` is a five-beat prose postmortem already linked end to end with `--sequences-->` edges, and it already carries an authored `t/story` thread. Deriving a **fresh** one under a different id, over the same graph, reproduces the same five-beat shape from the edges alone — evidence that the positional rule table is doing real work, not just repeating what a human already typed:

● ● ● *$ smysl thread --derive narrative --id t/derived-narrative --only fixtures/corpus/F3-narrative.smy*

```
@thread t/derived-narrative { schema: narrative, owner: tool:smysl, ts: [0, 0] }
~ The pool wait metric had been visible the whole time, on a dashboard
  nobody opened.; Rolling back the shard restored…
  setup → p/setup
  complication → p/complication
  turn → p/turn
  resolution → p/resolution
  coda → p/coda
```

The unit with nothing sequenced before it (`p/setup`) opens; the one nothing follows (`p/coda`) closes; the three in between land in thirds of the remaining chain. Every role is exactly one unit deep here because the chain itself is exactly five units long — a longer chain bands the same way, just with more than one unit per middle role, up to each role's arity.

`brief` and `qa` — F4 checkout-latency

`F4-qa.smy` is written as a support session: a question, the evidence chased down in answering it, and a caveat about over-attributing the fix. It derives a full `brief` and a full `qa` thread from the same nine units — same graph, two different readings, because the two schemas' rule tables route the same units to different roles for different purposes:

● ● ● *$ smysl thread --derive brief --only fixtures/corpus/F4-qa.smy*

```
@thread t/derived-brief { schema: brief, owner: tool:smysl, ts: [0, 0] }
~ The 4.3 serialiser change introduced an n-plus-one, made visible by peak
  load.; Attributing the regression to the…
  bottom-line → f/answer
  support → e/black-friday
  support → c/load-contribution
  support → c/n-plus-one
  risk → c/not-only-deploy
  ask → q/checkout-latency
```

● ● ● *$ smysl thread --derive qa --only fixtures/corpus/F4-qa.smy*

```
@thread t/derived-qa { schema: qa, owner: tool:smysl, ts: [0, 0] }
~ The 4.3 serialiser change introduced an n-plus-one, made visible by peak
  load.; Why did checkout latency regress after…
  question → q/checkout-latency
  evidence → e/black-friday
  evidence → c/load-contribution
  evidence → c/n-plus-one
  answer → f/answer
  caveat → c/not-only-deploy
```

The finding (`f/answer`) is the `bottom-line` in one reading and the `answer` in the other; the original question, asked before anyone had looked at a trace, is a supporting `ask` in the brief and the `question` that opens the Q&A. Same nine units, same edges, two audiences.

`plan` — F1-incident

Nothing in `F1-incident.smy` was authored as a plan, but a `@finding` reads as a `goal`, its grounds and the evidence under them read as `step`s, and the unit that rebuts the leading claim reads as the one `risk` worth flagging — which is exactly what the `plan` rule table does with them:

● ● ● *$ smysl thread --derive plan --only fixtures/corpus/F1-incident.smy*

```
@thread t/derived-plan { schema: plan, owner: tool:smysl, ts: [0, 0] }
~ Pool saturation is the leading cause but is not consistent with the
  canary.; The 95th percentile of request latency…
  goal → f/root-cause
  step → d/p95
  step → e/pool-wait
  step → e/trace
  step → c/regression
  step → c/pool-saturation
  risk → c/canary-clean
```

This is not a plan anyone would call well-written — a postmortem's units are not steps toward a goal, they are the argument for a root cause — and that mismatch is the honest lesson of this example: derivation always produces **a** thread that satisfies the schema's arities, but whether that schema was the right lens for this graph is a judgment call the tool does not make for you. Reach for `plan` on a graph that has genuine goals, constraints, and decisions in it; reach for `analysis` or `brief` on one that does not.

Widening a role: `--arity`

A schema's arity is a default ceiling, not a hard limit on the graph — it can be overridden per role with `--arity ROLE=N`. `F7-mixed-granularity.smy` has six units that would plausibly support its `brief`'s bottom line, but `brief`'s default `support` arity is `1..3`, so three of them sit out:

● ● ● *$ smysl thread --derive brief --explain --only fixtures/corpus/F7-mixed-granularity.smy*

```
fixtures/corpus/F7-mixed-granularity.smy: bottom-line  1 of 1..1
fixtures/corpus/F7-mixed-granularity.smy: support      3 of 1..3
fixtures/corpus/F7-mixed-granularity.smy: risk         2 of 0..2
fixtures/corpus/F7-mixed-granularity.smy: ask          0 of 0..1
fixtures/corpus/F7-mixed-granularity.smy: 3 unit(s) not selected
```

● ● ● *$ smysl thread --derive brief --arity support=6 --explain --only fixtures/corpus/F7-mixed-granularity.smy*

```
fixtures/corpus/F7-mixed-granularity.smy: bottom-line  1 of 1..1
fixtures/corpus/F7-mixed-granularity.smy: support      6 of 1..3
fixtures/corpus/F7-mixed-granularity.smy: risk         2 of 0..2
fixtures/corpus/F7-mixed-granularity.smy: ask          0 of 0..1
fixtures/corpus/F7-mixed-granularity.smy: 0 unit(s) not selected
```

`--arity support=6` widens exactly that one role; every other role's ceiling is untouched, and `smysl` refuses an override for a role the schema does not declare at all (`{schema} has no {role} role`, a usage error) rather than silently accepting a slot that would never be read.

Restricting the graph: `--scope`

`--scope` limits which units derivation is even allowed to see, by uid — the rest of the store still exists but is invisible to this derivation, the same restriction `pack --focus` uses for its own closure. Scoping `F1-incident` down to four units before deriving a `brief` produces a thread built only from what was named, nothing pulled in from outside it:

$ smysl thread --derive brief --explain --only ... fixtures/corpus/F1-incident.smy

```
$ smysl thread --derive brief --explain --only \
    --scope b3:izyuzlt42mqcvgdfb4nfpllxyq --scope b3:cvhirtgs2mpvli2ethhyeo32uf
\
    --scope b3:phsoomklkmlq3sjvbe6cyuqy5v --scope b3:js4xzessu5zwjpv2rawtugnuvj
\
    fixtures/corpus/F1-incident.smy
fixtures/corpus/F1-incident.smy: bottom-line  1 of 1..1
fixtures/corpus/F1-incident.smy: support      2 of 1..3
fixtures/corpus/F1-incident.smy: risk         1 of 0..2
fixtures/corpus/F1-incident.smy: ask          0 of 0..1
fixtures/corpus/F1-incident.smy: 0 unit(s) not selected
@thread t/derived-brief { schema: brief, owner: tool:smysl, ts: [0, 0] }
~ Pool saturation is the leading cause but is not consistent with the
  canary.; The canary rules out a pure configuration…
  bottom-line → f/root-cause
  support → e/pool-wait
  support → c/pool-saturation
  risk → c/canary-clean
```

Compare this against the unscoped `brief` derivation earlier in this chapter, over the full eight-unit store: `d/p95` and `c/regression` are gone, because they were never in scope to be picked, and `0 unit(s) not selected` here means exactly that — every unit **this derivation was allowed to see** found a role, which is a different and narrower claim than "every unit in the store found a role."

Identity is `(id, owner)`

> TERM **Thread identity**
>
> A thread's register key is the pair `(id, owner)`, not the id alone (`Thread::register_key` in `smysl-core`). Two threads with the same id and the **same** owner are the same register, resolved last-writer-wins by timestamp; two threads with the same id and **different** owners are two independent registers that happen to share a name. Neither has to coordinate with the other, and neither ever overwrites the other.

This is why `--as` matters as much as `--id`: it is not a display label, it is half of what makes a thread's name safe to reuse. Two agents deriving a brief under the same id `t/exec`, as different owners, coexist in the same store without either clobbering the other:

● ● ● *$ smysl thread --derive brief --id t/exec --as human:alice ... | smysl thread --derive brief --id t/exec --as human:bob ... | smysl thread --list*

```
$ smysl thread --derive brief --id t/exec --as human:alice --only F1-
incident.smy >> two-owners.smy
$ smysl thread --derive brief --id t/exec --as human:bob   --only F1-
incident.smy >> two-owners.smy
$ smysl thread --list two-owners.smy
t/brief  brief  3 step(s)  Auth p95 tripled in eu-west; pool saturation is
leading but contested.
t/exec  brief  3 step(s)  Alice's cut: pool saturation is the story, canary is
the caveat.
t/exec  brief  3 step(s)  Bob's cut: keep it to the regression and the one risk.
```

Three threads, one of them the file's original authored `t/brief`, two of them named identically as `t/exec` — and `--list` shows all three without complaint, because `owner` is part of what makes each one a distinct register. If Alice and Bob had instead both derived under `--as human:vladimir`, the second derivation would simply supersede the first as a later write to the same `(id, owner)` key — the ordinary last-writer-wins rule that governs every owned register in the format.

Reading a thread: `--list` and `--show`

`--list` is the inventory of every thread a store already holds, authored or derived; `--show ID` walks one thread step by step, resolving each uid to the gist of the unit it names:

● ● ● *$ smysl thread --list fixtures/corpus/F1-incident.smy*

```
t/brief  brief  3 step(s)  Auth p95 tripled in eu-west; pool saturation is
leading but contested.
```

● ● ● *$ smysl thread --show t/brief fixtures/corpus/F1-incident.smy*

```
t/brief  brief
~ Auth p95 tripled in eu-west; pool saturation is leading but contested.
  1. bottom-line  b3:js4xzessu5zwjpv2rawtugnuvj  Pool saturation is the leading
cause but is not consistent with the canary.
  2. support      b3:cvhirtgs2mpvli2ethhyeo32uf  The eu-west connection pool is
saturated.
  3. risk         b3:phsoomklkmlq3sjvbe6cyuqy5v  The canary rules out a pure
configuration cause.
```

`t/brief` here is authored by hand in the fixture, not derived — a person wrote exactly these three steps, in exactly this order, as the intended reading of the incident. That is the other half of what a thread is for:

`--derive` is the tool proposing a reading; nothing stops a person from writing one directly and having `thread --show` treat it identically.

> **NOTE**
>
> `--show`'s second column is the uid sitting right next to the gist it names — the practical technique this whole book leans on for finding a real uid to hand to `--focus`, `--seed`, or `--scope` elsewhere. Every worked `pack` example in Chapter 19 started life as a line from exactly this output.

Foreign roles: reported, not rejected

A schema's role table names which roles it **expects**; it is not a fence the parser enforces. `Role::parse` accepts any of the twenty-four roles across all five schemas, regardless of which schema a given thread declares — so a `brief` thread with a step in `coda` (a `narrative` role `brief` never lists) parses cleanly, shows cleanly, and passes `check` with zero diagnostics:

hand-authored: a brief thread reaching for narrative's `coda` role

```
@thread t/odd-brief { schema: brief, owner: "human:vladimir" }
~ A brief that reaches for a role brief never declared.
  bottom-line → f/root-cause
  support → c/pool-saturation
  coda → c/canary-clean
```

$ smysl thread --show t/odd-brief foreign-role.smy

```
t/odd-brief  brief
~ A brief that reaches for a role brief never declared.
  1. bottom-line  b3:js4xzessu5zwjpv2rawtugnuvj  Pool saturation is the leading
cause but is not consistent with the canary.
  2. support      b3:cvhirtgs2mpvli2ethhyeo32uf  The eu-west connection pool is
saturated.
  3. coda         b3:phsoomklkmlq3sjvbe6cyuqy5v  The canary rules out a pure
configuration cause.
```

$ smysl check foreign-role.smy

```
foreign-role.smy: 22 records, 8 units, 0 diagnostic(s)
```

Nothing here rejects the thread. That is deliberate, not an oversight: `Thread::foreign_roles()` in `smysl-core` exists precisely to let a caller ask the question directly — it returns the roles a thread uses that its own declared schema does not list (`[coda]`, for this example) — and it is a library-level check a

consumer can run rather than a gate the parser imposes. An **authored** thread is a person's stated intent, including, sometimes, reaching outside a schema's usual vocabulary on purpose; a **derived** one never does this, because `derive_thread` only ever assigns roles its own schema table lists. The asymmetry is the point: derivation is trusted to stay inside the lines because it is a pure function of a fixed table, and a human is trusted to say what they meant even when it does not fit the table — with the means to check, on demand, when it matters.

WHAT'S NEXT, AND WHY

A thread names an order and a role for each unit; it does not yet say what words those roles are rendered in, what register they read at, or what target format they land in — `smysl thread --derive schema | render --profile … --target …` is Chapter 24's command, and it composes directly with this one because a derived thread carries its originating store with it (`--only` is what strips that away, for exactly the cases where you want the thread record alone). Everything in this chapter has been about **which** reading; Chapter 24 is about how that reading actually gets said.

TRY THIS

1. Run `smysl thread --derive analysis --format surface fixtures/corpus/F1-incident.smy` and read the derived thread's steps. Then derive under `brief` instead. `analysis` gives you six steps under `context`, `finding`, `rebuttal` and `implication`; `brief` gives five under `bottom-line`, `support` and `risk`. Same graph, same facts, different walk. What does that tell you about where a thread's meaning lives — in the units, or in the thread?
2. The derived thread's owner is `tool:smysl` and its timestamp is `[0, 0]`. Both are deliberate. Explain the timestamp in terms of rule D, and the owner in terms of what a reader needs to know when two threads disagree.
3. `thread --derive` is deterministic; `--refine` would not be. Given that the derived analysis thread is serviceable but plainly mechanical, argue for shipping the deterministic one anyway rather than waiting for the model.

ANSWERS

1. In the thread. The units are the same facts either way — a schema decides which of them the reader is walked through, in what order, and under what label. `analysis` opens with context and works toward implications; `brief` leads with the bottom line. Writing three summaries by hand would mean three artifacts to keep in sync; deriving three threads over one graph means the facts cannot drift apart, because there is only one copy of them.
2. A timestamp is a clock reading, and rule D says a pure operation must be a function of its inputs alone — so `thread --derive` supplies a fixed clock rather than reading the real one, and running it twice produces the same bytes. The owner matters for a different reason: when two threads over the same graph tell different stories, the first question is who made each, and `tool:smysl` versus `human:vladimir` is the difference between a disagreement worth investigating and a machine doing what it was told.
3. Because a mechanical ordering that is always available, always free, and always the same beats a good ordering that costs a call, varies between runs, and cannot be regenerated years later to compare against what was signed

> off. The derived thread is a floor, not a ceiling — and a floor that a pipeline can depend on is worth more than a better answer it cannot.

WHAT YOU LEARNED

- A thread is a named, ordered, role-annotated walk over existing units — `salience` ranks and `pack` compresses, but neither one arranges a reading path for a specific audience the way a thread does.
- Five closed schemas — `analysis`, `narrative`, `brief`, `qa`, `plan` — each declare an ordered role list, a per-role arity, and a rule table that assigns units to roles; every one but `narrative` assigns by kind, `narrative` alone assigns by position in the graph's ordering chain.
- `--arity ROLE=N` widens one role's ceiling without touching the others; `--scope` restricts which units a derivation is even allowed to see, which changes what "nothing left over" means.
- Thread identity is the pair `(id, owner)`, not the id alone — two agents deriving under the same name are two independent registers, not a conflict, and only a later write from the **same** owner ever supersedes an earlier one.
- `--show` prints each step's uid next to its unit's gist — the practical way to mine a real uid for `pack --focus`, `salience --seed`, or `thread --scope` elsewhere in this book.
- A role outside a thread's declared schema is reported to a caller who asks (`Thread::foreign_roles()`), never silently rejected by the parser or `check` — a derived thread never produces one, because derivation is bound to its own schema's table by construction.

PART VI

Verification

CHAPTER 21

21

`check`, Pass by Pass

Chapter 8 used `check` the way you actually use it while drafting — a quick, cheap loop, run after every change, read for whatever it turns up. This chapter takes the same command apart. Ten passes are named in the format's own specification (§17), each with its own diagnostic codes, its own reasoning, and its own idea of what "wrong" means. Knowing which pass caught something — and, just as often, which pass **cannot** catch something — is what turns a diagnostic from a red line you clear into evidence you can reason about.

> **WHY**
>
> `check` never tells you a claim is **true**. It tells you the document is **internally consistent** — that a body cites what it names, that a status never outranks what it rests on, that nothing points at a uid the store does not have. Those are mechanical properties: they follow from the bytes alone, and a machine can decide them without understanding a word of the English inside a gist or a body. Whether the eu-west pool was **actually** saturated is not one of those properties, and no pass below decides it — that question belongs to a person, or to `attest` asking a model a bounded question and citing its answer as evidence rather than as fact (Chapter 11). The boundary is exact: a mechanical pass can verify a **relationship** between things already in the document; it cannot verify a **fact** about the world the document describes.

Ten passes, and where each one actually runs

`Pass` is a ten-member enum in `smysl-check`, numbered in the order the pipeline runs them, and every diagnostic code in this manual traces back to exactly one of them:

NO.	PASS	WHAT IT CHECKS	RUNS INSIDE `CHECK`?
1	`codec`	The CBOR is deterministically encoded — canonical key order, minimal-length integers, no indefinite-length items.	No — enforced at read time, before the pipeline starts.
2	`integrity`	Every reference resolves; no cycle in the	Yes;

NO.	PASS	WHAT IT CHECKS	RUNS INSIDE `CHECK`?
		support graph.	
3	`shape`	Field-level rules the constructor cannot see, chiefly the gist's token bound.	Yes.
4	`closure`	A body only reaches for what already `deps` declared (rule `grounds` L, structural half).	Yes.
5	`granularity`	A body fits the view's mission and length range.	Yes.
6	`epistemics`	Rule M — sta-	Yes.

NO.	PASS	WHAT IT CHECKS	RUNS INSIDE `CHECK`?
		tus never out- ranks the weak- est ground.	
7	`trust`	Rule T — sta- tus never ex- ceeds the ceil- ing of its best at- tested rung.	Yes.
8	`retraction`	Re- trac- tion au- thor- ity and or- phan- ing.	No — enforced by `merge`, not by `check`.
9	`extension`	An ex- ten- sion never re- de- fines a ker- nel rule; un- known schemas	Yes.

NO.	PASS	WHAT IT CHECKS	RUNS INSIDE `CHECK`?
		de- grade, never silently.	
10	`hashes`	Re- com- puted uids match a stored (C- dex, en- try for en- try.	No — belongs to `Store::verify_against`, surfaced by `reindex --verify` (chapter 23).

That "runs inside `check`?" column is not a simplification — it is `smysl-check`'s own test suite, asserted directly: `Pass::IMPLEMENTED` names exactly seven of the ten, and a build that is asked for one of the other three (`check --pass codec`, `--pass retraction`, or `--pass hashes`) runs nothing and reports nothing, rather than pretending to have checked it. The reasons the other three sit outside the pipeline are different from each other and worth knowing individually:

- `codec` decides whether the bytes are even legal CBOR before a `Store` exists to run passes over — a codec defect is not a diagnostic in a report, it is the reason `check` never gets that far.
- `retraction`'s codes (`SMY-E050`, `SMY-W052`) are real and enforced today, just not by `check` — they fire inside `merge`'s own retraction-integrity step instead (Chapter 13). The registry reserved pass 8 for it; the implementation landed the **check** in a different command than the one this chapter is about.
- `hashes` is the store's business, not a graph-shape question — verifying a uid means recomputing a hash over payload bytes and comparing it to what a sidecar index recorded, which is what `reindex --verify` does.

Each subsection below builds the smallest real file that trips one pass, and only that pass where isolation is possible at all, and shows the actual diagnostic. Where a pass cannot be reached from hand-written surface text — `trust` needs an attestation, and attestations have no surface syntax at all (Appendix A's grammar has none; the corpus fixtures' own README says so in as many words) — the example is built through the library instead and the gap is named rather than papered over.

Pass 1 — `codec`

> **WHY**
>
> Two encoders of the same document must produce the same bytes, or hashing becomes ambiguous and merge stops being commutative. `codec` is what makes that promise a checked fact rather than a convention: canonical CBOR fixes map-key order, forbids indefinite-length items, and requires the shortest legal integer encoding, so there is exactly one way to write any given record.

You will essentially never trigger this by hand — `fmt` and every command that writes CBOR already emit canonical bytes. The realistic way to see it is a store that was hand-edited at the byte level, or damaged in transit. Take a one-unit CBOR record and swap two of its map entries out of order — the same corruption a bit-flipping transport bug or a careless byte-level patch could cause:

● ● ● *$ smysl check /tmp/e080.cbor (map keys 0, 1, 6 reordered to 0, 6, 1)*

```
smysl check: /tmp/e080.cbor: SMY-E080: map keys not strictly ascending at byte
12
```

Exit code 1, not 3 — this never reaches the check pipeline at all, so it is not counted among "diagnostics found," it is the reason there was nothing to check. `smysl fmt` cannot repair this either, because `fmt` itself has to decode the file first. The only fix is to regenerate the store from a surface file or a known-good log; a canonical encoder never produces this shape of corruption in the first place, which is the actual guarantee this pass exists to protect.

Pass 2 — `integrity`

> **WHY**
>
> Content addressing means a unit's identity is a hash of its own content — nothing else in the store can ever accidentally collide with it, but nothing stops a `deps` or `grounds` list from **naming** a uid, or a label, that the store never defines. That single typo class — right shape, wrong target — is the most common mistake in hand-authored `.smy`, and it is this pass's entire job.

```
@definition d/p95 { status: cited, source: { kind: doc, ref: "sre-
handbook#latency" } }
~ The 95th percentile of request latency over a one-minute window.

@claim c/regression { status: speculative, deps: [d/p951] }
~ p95 auth latency tripled after the 4.2 rollout.
```

That `d/p951` is one character off from the definition actually declared just above it.

● ● ● *$ smysl check /tmp/e060.smy*

```
/tmp/e060.smy: error: SMY-E060: unresolved reference `d/p951` (at 206..212)
```

The byte span points at the exact six characters that do not resolve, which is why this pass costs nothing to act on: fix the typo, or add the missing definition, and the diagnostic disappears. A dangling `b3:` uid literal (rather than a label) is caught the same way, by the same code, once the store actually exists as a graph — this instance happens to be caught even earlier, while the surface parser is still resolving labels into uids, but it is the identical check and the identical wire code.

Pass 3 — `shape`

> **WHY**
>
> Most shape rules — an empty gist, a `detail` with no `body` above it, an authored `unfounded` — are things `UnitCore`'s own constructor already refuses, so a well-formed store cannot contain them; this pass is defence in depth for exactly that reason. One rule genuinely cannot live in the constructor: the gist's token bound is **relative to a granularity profile**, and the constructor has no access to one. That is the rule worth seeing fail for real.

```
@claim c/verbose { status: speculative }
~ The eu-west connection pool saturated during the rollout window and the resulting
queueing on every downstream call is almost certainly what tripled p95 latency for
the auth service that morning.
```

A gist that reads like a topic sentence for a paragraph, because that is exactly what it is.

● ● ● *$ smysl check /tmp/e022.smy*

```
/tmp/e022.smy: error: SMY-E022: gist is 49 tokens, default allows 30 (at
b3:6cvjx2fjm5bo7ohskil4ty6cli) [try: move the detail into `body` and shorten the
gist]
```

The suggestion is the fix, verbatim: move everything past the first clause into `body`, and leave the gist as the one sentence a reader sees at the document's L0 — the level a summary is built from without ever touching this unit's `body` at all.

Pass 4 — `closure`

> **WHY**
>
> Rule L says a body should be interpretable from its own declared support — the L0 of its `deps` and `grounds` — without the reader having to go fishing elsewhere in the store. `check` verifies the **structural** half of that: a body must not name a uid it never declared a relationship with. (The semantic half — does the body actually say what the gist claims — is `attest --what gist-coverage`'s job, because deciding that needs a model, not a graph walk.)

```
@evidence e/pool-wait { status: measured, source: { kind: metric, ref:
"pool.wait_ms{shard=eu-west}" } }
~ Pool acquisition wait rose from 2 ms to 310 ms over the same window.

@claim c/regression { status: speculative }
~ p95 auth latency tripled after the 4.2 rollout.

As e/pool-wait shows, wait time climbing in lockstep with request latency across
every
shard that took the rollout early is the strongest single signal pointing at the
pool
```

```
rather than at the release itself, and it is the reason this claim treats
saturation as
the leading explanation.
```

`c/regression`'s body leans on `e/pool-wait` by name, but the claim never put it in `deps` or `grounds` — the two fields this pass actually reads.

● ● ● *$ smysl check /tmp/e020.smy*

```
/tmp/e020.smy: error: SMY-E020: the body references
b3:vnuj3l674thnkkflwjm7kzkjdu, which is neither a dep nor a ground (at
b3:fbnkpl2b4lbuk2jnuylypgikbc) [try: add it to `deps` if it is a prerequisite,
or `grounds` if it is support]
```

The fix is a one-line edit — add `grounds: [e/pool-wait]` to the claim's header — and it is worth pausing on **why** the checker insists on it rather than just reading the prose: a reference that is only in the prose is invisible to `pack`'s closure expansion and `trace`'s walk. Declare it, and both commands now know this claim's support includes the pool-wait evidence; leave it only in English, and neither does.

Pass 5 — `granularity`

> **WHY**
>
> Granularity constrains **production** — what a unit is allowed to say in one shot — not truth (D-5), and `SMY-E040` is decided **structurally** rather than by reading the prose: more than one paragraph, or a list of more than one item, is more than one assertion by its own layout, however short each part is. Deciding "multi-assertion" any other way would need a model, and an error resting on a guess would be worse than no error.

```
@claim c/two-things { status: speculative }
~ The eu-west connection pool saturated during the rollout window and the resulting
queueing on every downstream call is almost certainly what tripled p95 latency for
the auth service.

The connection pool saturated in eu-west, tripling p95 latency for auth.

The canary shard also regressed independently, on a completely unrelated code path.
```

Two assertions in one body — the pool story and the canary story — under the `default` profile's `single-assertion` admission:

● ● ● *$ smysl check /tmp/e040.smy*

```
/tmp/e040.smy: error: SMY-E022: gist is 46 tokens, default allows 30 (at
b3:iekhua3u4l62a363hvd5gzh2r4) [try: move the detail into `body` and shorten the
gist]
/tmp/e040.smy: error: SMY-E040: body has more than one paragraph under single-
assertion admission (at b3:iekhua3u4l62a363hvd5gzh2r4) [try: split into one unit
per assertion, or move to `coarse`]
```

Two passes fired on the same unit here (shape's gist bound and granularity's admission rule), which is a reminder that `check` never short-circuits — every requested pass runs over the whole store regardless of what an earlier one found, because the repair loop wants every defect at once, not one round per pass. The fix for `E040` specifically is either structural (split into `c/pool-saturated` and `c/canary-regressed`, each grounding the eventual finding) or a granularity change (`coarse` admits several paragraphs as one topical unit) — which one is right is an authoring decision the checker deliberately leaves to you.

Pass 6 — `epistemics` (rule M)

> **WHY**
>
> This is the guarantee that makes hallucination laundering structurally impossible inside the graph: prose has no type for **measured** versus **guessed**, so a hedge is the first casualty of summarisation, and by the third retelling a speculation reads as a fact. Rule M caps that mechanically — a claim can never be stronger than the weakest thing it rests on — and the diagnostic names the weakest ground directly, because that is the actionable part and it is free to compute.

```
@hypothesis h/guess { status: speculative }
~ The connection pool is probably the cause, on general principle.

@claim c/promoted { status: derived, grounds: [h/guess] }
~ The connection pool caused the latency regression.
```

A guess, laundered into a `derived` claim by nothing more than restating it more confidently:

● ● ● *$ smysl check /tmp/e030.smy*

```
/tmp/e030.smy: error: SMY-E030: derived exceeds the cap of speculative set by
its weakest ground b3:4p7cfgyytyimdbqyhq7hkuw2sb (at
b3:3nqopz6ucqiyui24ladjfkqlmb) [try: weaken this unit to speculative, or
strengthen b3:4p7cfgyytyimdbqyhq7hkuw2sb]
```

The suggestion offers both directions honestly: weaken `c/promoted` back to `speculative` (it is still a perfectly good hypothesis, just not a claim), or go find the measurement that would let `h/guess` itself become `measured` or `cited` — at which point the exact same `derived` status on `c/promoted` would be earned rather than laundered. Fixture `F6-adversarial` in the corpus is built entirely around this rule — three separate laundering attempts, one of them two hops deep — and its own `.expected` file

names `SMY-E030` as the only code that may appear; a clean run over `F6` would mean rule M had stopped binding.

Pass 7 — `trust` (rule T)

> **WHY**
>
> Rule M stops laundering **inside** the graph; rule T stops it **at entry**. However confidently a model phrases something from its own parametric knowledge, that knowledge is capped at `inferred` — never `derived`, and never `measured`, which only an instrument recording `op: imported` with a checkable source may claim. This is the rule that keeps a model's fluency from being mistaken for evidence.

This one cannot be triggered from a hand-written `.smy` file at all: an attestation — the record that carries an agent, an op, and a rung — has no surface syntax. It is stamped on automatically by whatever tool ingests or authors a unit (`ingest`, `attest`, `thread --refine`), and the corpus fixtures' own README says outright that rule T "cannot be exercised from surface text" for exactly that reason. The honest way to see it is to build the store through the library directly — the same three records `ingest` would eventually write — and check the result:

```
// A store no surface file can express directly: a claim resting safely on
// measured evidence (rule M is fine here), then a model's own attestation
// asserting `derived` for it (rule T is not) — via smysl_core directly.
let evidence = UnitCoreBuilder::new(KernelType::Evidence, "seven days of traces",
Status::Measured)
    .source(SourceRef::new(SourceKind::Metric, "grafana://board/12"))
    .build()?;
let claim = UnitCoreBuilder::new(KernelType::Claim, "p95 auth latency tripled",
Status::Derived)
    .grounds([canonical_uid(&evidence)])
    .build()?;
let agent = AgentId::new("model:openai/gpt-4")?;
let attestation = Attestation::new(canonical_uid(&claim), agent.clone(),
Op::Authored, Rung::Model, Hlc::zero(agent));
```

Encoded to CBOR and checked for real:

● ● ● *$ smysl check /tmp/trust-violation.cbor*

```
/tmp/trust-violation.cbor: error: SMY-E033: derived exceeds the inferred ceiling
of its best provenance (model rung) (at b3:dpg2vjstd3ah7hw6l66ho2asav) [try:
weaken to inferred, or import it with op: imported and a checkable source]
```

Notice rule M does not also fire here — `claim`'s ground (`evidence`, `measured`) is strong enough that `derived` is well within cap, so this is a clean, single-pass isolation of rule T alone. The fix the diagnostic suggests is the honest one: either the model's claim gets relabelled `inferred` (which is all its own knowledge ever entitled it to), or a real import — an instrument, a tool adapter, something that recorded

`op: imported` against a source a machine could check — has to be the one making the `derived` claim instead.

Pass 8 — `retraction`

> **WHY**
>
> `retracts` says a unit should no longer be believed, and anything resting entirely on a retracted unit is left with no support at all — `SMY-E050` is what catches a document that retracted something without noticing what it orphaned downstream.

The registry reserves pass 8 for this, and the diagnostic codes are real and enforced — just not by `check`. Selecting it directly proves the gap rather than hiding it:

● ● ● *$ smysl check --pass retraction /tmp/retraction-demo.smy*

```
/tmp/retraction-demo.smy: 7 records, 3 units, 0 diagnostic(s)
```

Nothing fires, on a store that plainly should trip it:

```
@evidence e/old { status: measured, source: { kind: metric, ref: "pool.wait_ms" } }
~ Pool wait rose from 2ms to 310ms over the window.

@claim c/downstream { status: derived, grounds: [e/old] }
~ Pool saturation caused the regression.

@claim c/retractor { status: speculative }
~ The sensor feeding e/old was later found miscalibrated.

@rel c/retractor --retracts--> e/old
```

The actual enforcement lives one command over — retraction integrity runs as the last step of `merge`'s own fold, and a single-input merge is enough to trigger it for real:

● ● ● *$ smysl merge /tmp/retraction-demo.smy -o /tmp/retraction-demo.cbor*

```
/tmp/retraction-demo.smy: error: SMY-E050: every ground of this unit has been
retracted (at b3:tu45r2ged2aes2iinznh6zimlx) [try: retract this unit too, or re-
ground it on something surviving]
```

And the gap is worth stating plainly, because it has a real consequence: the CBOR `merge` just wrote still shows `c/downstream` as plain `derived` resting on `e/old` — the **declared** status never changes, only the **effective** one does, and effective status is computed on demand, not stored. Run `check` on that merged store and it reports clean, because none of the seven pipeline passes look at `retracts` edges at all:

● ● ● *$ smysl check /tmp/retraction-demo.cbor*

```
/tmp/retraction-demo.cbor: 7 records, 3 units, 0 diagnostic(s)
```

So today, orphaning is something `merge` tells you about at the moment it happens, printed to stderr, not something a later `check` on the same store will ever rediscover — if you need to know whether a store you did not just merge has an orphaned unit in it, re-running it through `merge` (even with itself as the only input) is currently the only way to ask.

Pass 9 — `extension`

> **WHY**
>
> Rule X is why a store written by someone who knew more than you stays readable: an extension may only **add** — a new unit type, a new relation kind — never redefine a kernel rule, and a reader without that extension degrades gracefully rather than refusing. This pass enforces both directions: `SMY-E012` if an extension tries to overwrite the kernel, `SMY-W013` if a relation kind shows up that nothing declared.

```
@claim c/a { status: speculative }
~ The pool saturated.

@claim c/b { status: speculative }
~ We rolled back the release.

@rel c/a --x.sre/mitigated-by--> c/b
```

`x.sre/mitigated-by` is shaped like a legal extension relation, but nothing in this file declared it with a `SchemaDecl` — which, like an attestation, has no surface syntax of its own, so an undeclared extension relation is the realistic case:

● ● ● *$ smysl check /tmp/w013.smy*

```
/tmp/w013.smy: warning: SMY-W013: relation kind `x.sre/mitigated-by` is
undeclared; treated as elaborates
/tmp/w013.smy: 5 records, 2 units, 1 diagnostic(s)
```

Exit `0` — a warning never blocks a plain `check`, and the store stays routable: an unknown kind degrades to the weakest kernel relation (`elaborates`) rather than being dropped, so `pack` and `trace` still see an edge there, just not the specific meaning `x.sre/mitigated-by` intended.

Pass 10 — `hashes`

> **WHY**
>
> Every other pass reasons about the **graph** — what a unit says about itself and what it points at. This one asks a narrower, more mechanical question: does the content still hash to the uid a sidecar index says it should. That is a property of the store's bytes against its own cached index, not of the document's shape, which is why it lives in `Store::verify_against` rather than in the graph-shape pipeline.

`check --pass hashes` selects nothing, for the same reason `--pass retraction` did — `Pass::Hashes.is_implemented()` is `false` in this build. `SMY-E070` ("the log does not match the index") is a real, tested code, but nothing in the current CLI sets `verify_hashes: true` when opening a store, so it is not yet reachable from any flag. What **is** reachable, and produces the practically equivalent result, is `reindex --verify` — a whole-index byte comparison rather than a per-unit `E070` report, covered in full in Chapter 23 alongside the rest of what `reindex` does. If your document checks clean on all seven implemented passes, hash verification is the next question worth asking, and it belongs to that chapter, not this one.

Narrowing and raising the bar

`--pass` — run only the passes you name

Ten passes over a large store is real work, and while debugging you often know which one you care about. `--pass` restricts the run to exactly the names you give it — pass in a store with two independent defects and narrow to just one:

```
@claim c/two-things { status: speculative }
~ The eu-west connection pool saturated during the rollout window and the resulting
queueing on every downstream call is almost certainly what tripled p95 latency for
the auth service.

The connection pool saturated in eu-west, tripling p95 latency for auth.

The canary shard also regressed independently, on a completely unrelated code path.
```

● ● ● *$ smysl check --pass shape /tmp/two-defects.smy*

```
/tmp/two-defects.smy: error: SMY-E022: gist is 46 tokens, default allows 30 (at
b3:iekhua3u4l62a363hvd5gzh2r4) [try: move the detail into `body` and shorten the
gist]
```

● ● ● *$ smysl check --pass granularity /tmp/two-defects.smy*

```
/tmp/two-defects.smy: error: SMY-E040: body has more than one paragraph under
single-assertion admission (at b3:iekhua3u4l62a363hvd5gzh2r4) [try: split into
one unit per assertion, or move to `coarse`]
```

Two independent, single-code reports from the same file — exactly the narrowing you want once you already know a category and just want it isolated. One caveat worth knowing: `--pass` only narrows the seven **pipeline** passes. A diagnostic caught while the surface parser resolves labels — an unresolved reference, `SMY-E060` — happens before the pipeline runs at all, so it is included in the report no matter which `--pass` you name.

`--strict` — warnings stop the build too

A plain `check` treats a warning as informational: it is printed, but exit stays `0`. `--strict` raises the bar to `Severity::Warn`, so the same warning now fails:

● ● ● *$ smysl check /tmp/w013.smy*

```
/tmp/w013.smy: warning: SMY-W013: relation kind `x.sre/mitigated-by` is
undeclared; treated as elaborates
/tmp/w013.smy: 5 records, 2 units, 1 diagnostic(s)
```

● ● ● *$ smysl check --strict /tmp/w013.smy*

```
/tmp/w013.smy: warning: SMY-W013: relation kind `x.sre/mitigated-by` is
undeclared; treated as elaborates
```

Same single diagnostic, same text — but the second run's exit code is `3` where the first's was `0`, and the summary line that prints on a passing run is withheld on a failing one. `--strict` is the flag to reach for in a pipeline that should refuse to move forward on anything short of clean, not just on anything outright broken — a `w` code is not wrong, but it may be exactly the threshold a particular team or a particular document's stakes call for.

> **WHAT'S NEXT, AND WHY**
>
> A document that checks clean on all seven implemented passes is internally consistent — every reference resolves, every status is earned, every extension behaves. That is exactly what makes it safe to hand to `merge`, `pack`, or `retract` (Part V) in the first place — every one of those operations trusts that the graph it is walking is already this sound. But a clean report answers only "is this store safe to touch at all." It does not yet answer "is this store safe for the **kind** of use I have in mind" — reading only, versus producing new units into it, versus merging it with someone else's. Chapter 22 is that second question: conformance classes ask what a store is safe for; fidelity asks what one particular consumer, implementing one particular set of schemas, can actually get out of it.

TRY THIS

1. `check` runs its passes in a fixed order, and this chapter walks them one at a time. Given that a parse failure makes every later pass meaningless, predict what `check` does when a file fails to parse partway through — does it report the parse error alone, or does it also report what it can see of the rest?
2. Chapter 21 says pass 6 is rule M and pass 7 is rule T. Run `check` on `F6-adversarial.smy` (which trips M) and recall the `SMY-E033` example that trips T. Why is it useful that these are separate passes with separate codes, when both amount to "this status is too strong"?
3. Write the shortest `.smy` file you can that produces **exactly one** diagnostic. Then try to write one that produces exactly two from a single mistake. Which was harder, and what does that tell you about reading `check` output?

ANSWERS

1. It reports what it can. The parser is built to keep going past a bad span rather than stopping at the first fault, so you get the parse error **and** whatever the later passes could establish about the records that did admit. The alternative — one error per run — turns fixing a file into a sequence of round trips, and this chapter's whole argument is that a checker is a tool you use continuously rather than a gate you visit once.
2. Because the two have different fixes and different meanings. `SMY-E030` (rule M) says the claim outran **what it rests on** — the answer is inside the document, in its grounds. `SMY-E033` (rule T) says the claim outran **who produced it** — the answer is outside the document, in the provenance, and no amount of editing grounds will help. Collapsing them into one code would leave you guessing which of the two situations you were in.
3. One is easy: a single unit with `status: unfounded` does it. Two from one mistake is easier than it sounds — a broken label in a `grounds` list usually gets you an unresolved reference **and** a shape error on the unit that failed to admit, and Chapter 5 showed one mistake producing three. The lesson is that diagnostic **count** is not fault count. Fix the most structural one and re-run before reading the rest.

WHAT YOU LEARNED

- `check` verifies consistency, never truth — a mechanical pass reasons about relationships already in the document, not about the world.
- Ten passes are named in the registry; seven run inside `check` itself. `codec` runs at read time and aborts before the pipeline starts; `retraction` is enforced by `merge`; `hashes` belongs to `Store::verify_against`, reachable today through `reindex --verify`.
- Every implemented pass has a real, isolable failure mode: `SMY-E060` (integrity), `SMY-E022` (shape), `SMY-E020` (closure), `SMY-E040` (granularity), `SMY-E030` (epistemics, rule M), `SMY-E033` (trust, rule T), `SMY-W013` / `SMY-E012` (extension).
- Rule T's violations have no surface syntax to author by hand — attestations are stamped on by tooling, never typed — so seeing one for real means building the store through the library, exactly as `ingest` eventually will.
- `--pass` narrows to the pipeline passes you name, but parse-time diagnostics are always included; `--strict` promotes every warning to a build-stopping error.

CHAPTER 22

22

Conformance and Fidelity

Chapter 21 answered one question per pass: is this specific relationship in the document consistent. This chapter asks two broader questions that sit on top of a clean `check` run. Both start from the same seven-pass report, and both are ways of reading it — neither adds a new pass of its own.

> **TERM** **Conformance class**
>
> A named answer to "is this **store** safe for a given **kind** of use" — reading it at all, consuming it as input to a decision, producing new units into it, merging it with another store. Five classes exist, every one of them building on `C-Read`, and each is decided by asking which diagnostic codes a store's `check` report contains — a conformance class is a lens over a report you already have, not a new thing to compute.

> **TERM** **Fidelity**
>
> A named answer to "what can **this specific consumer**, implementing **these particular schemas**, actually do with this store" — `Full` (every schema the store `requires` is implemented), `Degraded` (an extension is missing, but the kernel is not — payload preserved, interpretation lost), or `Refuse` (the kernel major itself is unsupported — the one case where silently degrading is forbidden).

WHY

Conformance is a property of the **store**: "would any C-Consume-conforming implementation be able to act on this." Fidelity is a property of a **pairing**: "can this particular reader, who happens to implement `x.sre/incident` and nothing else, get full value from this particular store." A store can conform at every class and still degrade a given consumer — a store that only uses kernel types is `C-Full` for everyone, and a store built around an SRE extension is still perfectly conformant even though a kernel-only reader loses interpretation on every unit typed with it. The two questions are independent on purpose: conformance is about what the **format** guarantees; fidelity is about what **you**, specifically, implement.

The five conformance classes, demonstrated

CLASS	SAFE FOR
`C-Read`	Parsing and verifying the store at all — the floor every other class builds on.
`C-Consume`	Reading it and acting on it as a decision input — adds rules M and R.
`C-Produce`	Authoring new units into it — adds the shape rules.
`C-Merge`	Merging it with another store — adds the lifecycle rules (retraction, orphaning).
`C-Full`	All of the above at once.

`--conformance` takes exactly these five spellings, case-insensitive — the bare words `read` or `full` are not accepted, only the hyphenated `C-` form:

● ● ● *$ smysl check --conformance full fixtures/corpus/F1-incident.smy*

```
smysl check: unknown conformance class
```

● ● ● *$ smysl check --conformance C-Full fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: C-Full: pass
fixtures/corpus/F1-incident.smy: 21 records, 8 units, 0 diagnostic(s)
```

A clean store — `F1-incident`, the corpus's baseline fixture — passes every class there is, because there is nothing in its report for any class to forbid. That is the uninteresting case; the useful one is a store that passes a **lower** class while failing a **higher** one, because that is exactly the situation conformance classes exist to describe. The corpus's `F6-adversarial` fixture is built entirely around rule M violations (`SMY-E030`, Chapter 21's pass 6), and `SMY-E030` is classified as **epistemic** — which `C-Read` does not forbid, but every other class does:

● ● ● *$ smysl check --conformance C-Read fixtures/corpus/F6-adversarial.smy*

```
fixtures/corpus/F6-adversarial.smy: error: SMY-E030: derived exceeds the cap of
inferred set by its weakest ground b3:2hkacsatxuvcywj6f4w2lkzojx (at
b3:hdn4uifpmzzopwuyajmhyf5xzu) [try: weaken this unit to inferred, or strengthen
b3:2hkacsatxuvcywj6f4w2lkzojx]
fixtures/corpus/F6-adversarial.smy: C-Read: pass
```

● ● ● *$ smysl check --conformance C-Consume fixtures/corpus/F6-adversarial.smy*

```
fixtures/corpus/F6-adversarial.smy: C-Consume: fail (SMY-E030)
```

Read this pair the way it is meant to be read: `F6` still **parses**, still **hashes correctly**, still has no dangling reference — nothing structural is wrong with it, so a naive reader that only cares about getting bytes back out could open it all day. But no implementation that promises `C-Consume` (enforcing rules M and R on your behalf) can honestly hand you a decision built on this store, because the store itself is laundering a speculation into a derived claim. `C-Produce`, `C-Merge`, and `C-Full` all inherit that same failure, for the same reason — the constraint is on what the class **promises about the store**, not on how careful any one reader happens to be.

Fidelity — `--as`, `Full`, `Degraded`, and the one `Refuse`

`--as` names the schemas a particular consumer implements; the kernel is always implied, so naming nothing at all means "a kernel-only reader." Build a small store around one genuine extension type:

```
@doc smysl/0.1 {
  id: v/sre-demo
  intent: incident-brief
  requires: ["smysl.kernel/0.1", "x.sre/incident"]
  roots: [i/pool-outage]
}

@x.sre/incident i/pool-outage { status: speculative }
~ eu-west connection pool exhausted during the 09:00 rollout window.
```

A kernel-only reader — the default, and the honest baseline for "a consumer that has not adopted the SRE extension yet":

● ● ● *$ smysl check --as smysl.kernel/0.1 /tmp/fidelity-demo.smy*

```
/tmp/fidelity-demo.smy: as `smysl.kernel/0.1`: Degraded
/tmp/fidelity-demo.smy:   b3:bwmw4g2dyfmtsyw2vb7gdzamr3 degraded: x.sre/incident
not implemented
/tmp/fidelity-demo.smy: warning: SMY-W010: schema x.sre/incident is not
implemented by `smysl.kernel/0.1`; payload preserved, interpretation lost (at
b3:bwmw4g2dyfmtsyw2vb7gdzamr3)
/tmp/fidelity-demo.smy: 3 records, 1 units, 1 diagnostic(s)
```

Nothing is lost or dropped — `Degraded` and rule X's own promise are the same sentence twice: the payload survives untouched, this reader just can not interpret it as an incident, only as an opaque, still-mergeable, still-traceable unit. A reader that has adopted the same extension gets `Full`, on the identical file:

● ● ● *$ smysl check --as x.sre/incident /tmp/fidelity-demo.smy*

```
/tmp/fidelity-demo.smy: as `x.sre/incident`: Full
/tmp/fidelity-demo.smy: 3 records, 1 units, 0 diagnostic(s)
```

THE ONE CASE DEGRADING IS FORBIDDEN

Rule X's negotiation is three-valued on purpose, and the third value is the whole point: a missing **extension** degrades, but a missing **kernel major** refuses, unconditionally, because a reader that does not implement the kernel a store was written against cannot safely interpret **any** of it — not even enough to know what it is safe to ignore. `smysl` enforces this at the earliest point it possibly can: the surface parser itself refuses to load a document declaring an unsupported kernel major, before `check` or `--as` ever run.

● ● ● *$ smysl check --as smysl.kernel/0.1 /tmp/refuse-demo.smy (requires: ["smysl.kernel/9"])*

```
smysl check: /tmp/refuse-demo.smy: SMY-E002: kernel schema smysl.kernel/9
```

Exit code `8` — `UnsupportedVersion`, distinct from a check failure, because this is not "the document is inconsistent," it is "this build cannot safely read this document at all." That refusal happens so early that `--as` never gets a chance to print `Refuse` for a hand-authored file — but the verdict exists in the library independently of the parser's own gate, and a store that reaches `check` by another path (a merged CBOR log, say, carrying a view whose `requires` names a kernel major none of its contributors actually wrote against) can still surface it as a fidelity result rather than a load failure:

● ● ● *$ smysl check --as smysl.kernel/0.1 /tmp/refuse-demo.cbor (same requirement, built directly as a view record)*

```
/tmp/refuse-demo.cbor: as `smysl.kernel/0.1`: Refuse
/tmp/refuse-demo.cbor: 1 records, 0 units, 0 diagnostic(s)
```

Two different guards, one rule: whichever point a mismatched kernel major is discovered at — the parser's own refusal on the way in, or `--as`'s verdict on a store that got further than that — nothing about it is ever silently downgraded to "read what you can."

`--granularity` — the distribution, not a verdict

Mixed granularity in a merged store is legal by design (D-5): different documents are written for different depths, and a merge does not force them into agreement. `--granularity` reports what is actually there rather than passing judgement on it. `F7-mixed-granularity` in the corpus is exactly that — a real merge of the incident brief (`default` profile) and a research trace (`fine` profile):

● ● ● *$ smysl check --granularity fixtures/corpus/F7-mixed-granularity.smy*

```
fixtures/corpus/F7-mixed-granularity.smy: 1 view(s) at granularity default
fixtures/corpus/F7-mixed-granularity.smy: warning: SMY-W041: body is 30 tokens,
under the default range 40..120 (at b3:s6etp4fjq3gkd7335lzdwjzx54)
fixtures/corpus/F7-mixed-granularity.smy: warning: SMY-W041: body is 26 tokens,
under the default range 40..120 (at b3:tfhgiir2fegvfkvq2fplgtvgq7)
fixtures/corpus/F7-mixed-granularity.smy: 22 records, 9 units, 2 diagnostic(s)
```

The two `SMY-W041` warnings are ordinary pass-5 length advisories on individual units — advisory, not fatal, exactly as Chapter 21 described — and the granularity line above them is not a diagnostic at all, just a count of how many views this store declares at each profile. A store with several views at several profiles would print one line per profile, with no ranking implied between them: this flag exists so a reader can see what they are looking at, not to tell them they are looking at the wrong thing.

WHAT'S NEXT, AND WHY

You now know what a store is safe for — structurally (Chapter 21), categorically (conformance), and for one specific reader (fidelity). None of that has asked anything about the **tool** yet: whether the index `smysl` keeps beside a CBOR log actually describes the log it sits next to, and whether the same invocation really does produce the same bytes on a different machine. Chapter 23 turns to those two guarantees — properties of `smysl` itself, not of any document it happens to be reading.

TRY THIS

1. Run `smysl check --conformance C-Consume fixtures/corpus/F1-incident.smy`, then again with `C-Full`. Both pass. Explain what you have and have not learned about the file from two passes.
2. `smysl check --as x.sre/incident` Run `fixtures/corpus/F7-mixed-granularity.smy`. It reports fidelity `Full`, and **also** two `SMY-W041` granularity warnings. Both are true at once. What is each one telling you, and which would block a pipeline?
3. Conformance is a property of a store; fidelity is a property of a pairing. Describe a store that conforms at every class and still gives a particular consumer almost nothing.

ANSWERS

1. You have learned the store is consumable by any conforming implementation, at both the minimum and the full class — no exotic record shapes, no version the reader could not handle. You have learned nothing about whether the claims are true, whether the argument is any good, or whether **your** consumer will find it useful. Conformance answers "can this be processed", which is a lower bar than it sounds and a necessary one.
2. Fidelity `Full` says a consumer implementing `x.sre/incident` gets everything the store has to offer — nothing in it needs a schema that consumer lacks. The `W041` warnings say two bodies are shorter than the document's own declared granularity range, which is a drafting observation about prose length. Neither blocks by default; under `--strict` the warnings would, and fidelity would not, because fidelity is a report rather than a diagnostic.
3. A store using only kernel types conforms everywhere — every implementation understands `@claim` and `@evidence`. Hand it to a consumer that exists to process `x.sre/incident` extensions and it finds not one unit it was built for. Perfectly conformant, near-zero fidelity for that reader. This is why both numbers exist: conformance is about the store alone, fidelity only means anything once you name who is reading.

WHAT YOU LEARNED

- Conformance and fidelity are both readings of a report you already have, not new passes — conformance asks what the **store** is safe for; fidelity asks what **one consumer** gets out of it.
- Five classes build on `C-Read`; a store can pass a lower class while failing a higher one, and the exact codes blocking each class are printed, not just a pass/fail bit.
- `--as` reports `Full`, `Degraded`, or `Refuse`. Degrading preserves the payload and loses only interpretation; refusing — reserved for a kernel major mismatch — is enforced as early as the surface parser itself, before fidelity is even computed.
- `--granularity` reports a distribution across a merged store's views, never a verdict — mixed granularity is legal by design.

CHAPTER 23

23

Determinism, `reindex`, and the Index

Everything so far has verified properties of a **document**. This chapter verifies properties of the **tool**: does the on-disk index next to a CBOR log actually describe that log, and does running the same command twice really produce the same bytes. Both questions matter for the same underlying reason — a store is meant to survive being copied, interrupted, and rebuilt from nothing but its own log, on any machine, indefinitely.

> **WHY**
>
> A store's index is derived, disposable state: everything it contains can be recomputed from the append-only log alone. That split exists for two concrete reasons. First, corruption recovery — a log is append-only and content-addressed, so it can be replayed from scratch even if the index beside it is stale, truncated, or gone entirely; nothing about the log's own integrity depends on the index agreeing with it. Second, portability — handing someone your log is handing them everything; the index is a local performance cache, not a second copy of the truth, and it never needs to travel with the log for the recipient to reconstruct an identical one.

`reindex` and `--verify`

Build a real CBOR store the way any pipeline would — `merge` writing to a file:

```
$ smysl merge fixtures/corpus/F1-incident.smy -o /tmp/incident.cbor
```

```
fixtures/corpus/F1-incident.smy: contention k/ccm3actwjjti65famnoe6mapo5d over
b3:cvhirtgs2mpvli2ethhyeo32uf (2 positions, live-rebuttal)
```

`reindex` rebuilds the sidecar from the log alone and reports what it found:

```
$ smysl reindex /tmp/incident.cbor
```

```
/tmp/incident.cbor: 21 records, 8 units, index 1759 bytes
```

That write lands beside the log, at `.smysl/index/incident.idx` — a name derived from the log's own filename, never from anything you pass explicitly. `--verify` does not overwrite it; it rebuilds a second copy in memory and compares the two byte for byte:

```
$ smysl reindex --verify /tmp/incident.cbor
```

```
/tmp/incident.cbor: index matches a rebuild (1759 bytes)
```

That is the case you want to see in CI: the sidecar genuinely describes the log it sits next to. The other case — the one worth constructing on purpose, since it is the whole reason `--verify` exists — is a sidecar that has drifted from its log. Flip a single byte in the `.idx` file directly, the way a truncated copy, a disk error, or a careless hand edit might:

```
$ smysl reindex --verify /tmp/incident.cbor   (one byte flipped in the sidecar)
```

```
/tmp/incident.cbor: warning: SMY-W110: index does not describe this log;
rebuilding
/tmp/incident.cbor: the sidecar does not match a rebuild from the log
```

Exit code `9` — `HashVerification`, the same code the registry reserves for pass 10′s hash-mismatch case. Two things happened in that one run, worth separating: `Store::open_with` noticed on the way in that the sidecar's own header no longer matches the log (`SMY-W110`, and it silently rebuilds an in-memory index so the rest of the command can proceed at all), and then `--verify`'s own comparison — a raw byte-for-byte diff of the sidecar file against a fresh rebuild — reported the mismatch explicitly. The fix is never to hand-edit the sidecar back; it is to run `smysl reindex` (without `--verify`) and let it overwrite the stale file from the log, which is always authoritative.

> **SMY-E070 EXISTS, BUT NO FLAG REACHES IT YET**
>
> `Store::verify_against` — the function that recomputes every unit's uid and compares it entry-by-entry against a sidecar, reporting `SMY-E070` for anything that does not match — is real, tested library code, exposed through `StoreOptions::strict()`. As of this build, no CLI flag sets `verify_hashes: true` when opening a store, so `SMY-E070` itself is not yet something you can trigger from the command line. What `reindex --verify` gives you instead — a whole-index byte comparison rather than a per-unit report — answers the same practical question ("has this sidecar drifted from its log") with a coarser diagnostic. If you need the per-unit version today, it is one `Store::open_with(path, StoreOptions::strict())` call away in the library.

`--seed-check` and `cargo xtask determinism`

The global `--seed-check` flag is documented as asserting that a specific invocation is bit-reproducible (rule D) — the same guarantee, on demand, against your own store. It is worth being exact about what this build actually does with it: the flag parses on every subcommand (it is declared `global(true)`), but nothing in `main.rs` reads its value — passing it changes nothing about the output of any command today.

● ● ● *$ smysl --seed-check check fixtures/corpus/F1-incident.smy*

```
fixtures/corpus/F1-incident.smy: 21 records, 8 units, 0 diagnostic(s)
```

Byte-for-byte identical to the same command without the flag. That is not a bug to route around in this manual — it is the accurate state of the flag, and the guarantee it is meant to name is enforced today by a different, coarser mechanism: `cargo xtask determinism`, a CI gate rather than a runtime flag. It runs every registered pure operation twice, under eight environment permutations — two locales, two timezones, two hash seeds — and asserts byte-identical output across all sixteen runs per operation:

● ● ● *$ cargo xtask determinism*

```
xtask determinism (rule D)
  matrix: 8 permutations
  5 of 5 operations registered
  pack: identical across 16 runs
  salience: identical across 16 runs
  merge: identical across 16 runs
  derive_thread: identical across 16 runs
  render: identical across 16 runs
ok
```

All five of rule D's named operations are registered and passing in this build. The permutation matrix targets the four things that quietly break determinism in practice and nowhere else: locale-dependent collation and case folding, timezone-dependent date formatting, and hash-seed-dependent iteration order over an unordered collection — the class of bug that passes every test on one machine and fails silently on another. `--seed-check` names the same property `xtask determinism` enforces; today, the gate is where rule D is actually checked, and the flag is the place that guarantee is documented to eventually live.

`cargo xtask check-purity`

A build-time gate rather than anything `smysl` itself runs, and worth knowing about even though no chapter of this manual will ever tell you to type it against a document: it verifies that the library core stays synchronous and network-free, which is a precondition for rule D rather than rule D itself. A pure operation that could reach a socket or block on a runtime would not be reliably reproducible even if its logic were — timing, retries, and network state are exactly the kind of thing determinism cannot promise anything about.

● ● ● *$ cargo xtask check-purity*

```
xtask check-purity (rules A, B)
  dependency tree (--no-default-features): 25 crates, none forbidden
  pure crates: 6 checked
  source scan: 66 files, 7 symbols
ok
```

Two independent checks run, deliberately redundant with each other: a `cargo tree` walk over the facade's `--no-default-features` build (and over each pure crate on its own) asserting that no async runtime, HTTP client, argument parser, or TUI library ever appears in the dependency graph; and a source-level grep across every pure crate for the literal symbols that would betray a socket or a runtime even if no crate dependency ever named one — a thread spawned and a raw `TcpStream` opened by hand would pass the first check and fail the second. Either alone is escapable; both together are the actual guarantee.

WHAT'S NEXT, AND WHY

Verification is complete at this point, in the sense this Part set out to cover: a document's own consistency (Chapter 21), what it is safe for and for whom (Chapter 22), and the tool's guarantees about its own index and its own reproducibility (this chapter). Everything you have built so far — a checked, conformant, correctly-fidelity-reported graph — is still a graph: units, relations, a thread's ordered walk over a selection of them. Part VII turns that graph into something a person actually reads, starting with `render` and the profiles that shape it.

TRY THIS

1. Run `smysl reindex` on a `.smy` surface file. It fails with `SMY-E004: malformed envelope at byte 0`. Now `merge -o store.cbor` that same file and reindex **that** — it reports the record count and an index size. Why does the command only make sense against one of the two forms?
2. Run any pure command twice with `--seed-check` and confirm it exits `0`. Then say what `--seed-check` would have to observe to exit non-zero, and why a flag like this exists at all when CI already runs `cargo xtask determinism`.
3. Delete a store's index entirely and reindex it. Nothing is lost. Now construct the argument for why the **log** could not be treated the same way — what property does the log have that the index does not?

ANSWERS

1. An index is derived state over an append-only **log**, and a surface file is not a log — it is text that parses into records. There is no envelope to index, no byte offsets to record, nothing to be stale against. `reindex` rebuilds a cache beside a binary store; pointed at surface text it is being asked to cache something that has no persistent form to cache.
2. It would have to find that running the same operation over the same bytes produced different output — a clock read, a hash-map iteration order leaking through, an uninitialised salt. The CI gate proves determinism for the **inputs the gate happens to use**; `--seed-check` lets you assert it for **your** input, in your pipeline, on the machine that will actually run it. A guarantee you can re-verify locally is worth more than one you have to take on trust from someone else's CI.
3. Everything in the index is recomputable from the log, so losing it costs time and nothing else. The log is the only place the content exists — it is append-only and content-addressed, which is exactly what makes it authoritative: entries are never rewritten, and each one's identity is a hash of what it contains, so corruption is detectable rather than silent. Derived state can be thrown away because it can be re-derived; the log cannot, because there is nothing to re-derive it from.

WHAT YOU LEARNED

- An index is derived, disposable state, recoverable from the log alone — which is why corruption recovery and portability both depend on the log never needing its index to be trustworthy.
- `reindex` rebuilds the sidecar; `reindex --verify` compares the existing sidecar against a fresh rebuild without overwriting it, and a genuine mismatch exits `9` after first warning `SMY-W110` on the way in.
- `SMY-E070`, the per-unit hash-mismatch code, is real library code (`Store::verify_against` via `StoreOptions::strict()`) not yet wired to any CLI flag; `reindex --verify`'s whole-index comparison is what the command line actually gives you today.
- `--seed-check` is declared globally but not yet read by any command in this build; `cargo xtask determinism` is where rule D is actually enforced today, across all five registered pure operations and eight environment permutations.
- `cargo xtask check-purity` is a build-time, not a document-time, guarantee — a synchronous, network-free core is a precondition for determinism, checked two independent ways so that neither is escapable alone.

PART VII

Export for Human Consumption

CHAPTER 24

24

render and Profiles

WHY

Every command before this chapter produces more graph: more units, a narrower selection, a named thread. None of it is prose yet. `render` is the one step where a graph plus a thread plus a set of rendering choices becomes text a specific person actually reads — and it is still pure. The same thread, rendered under the same profile, produces the same bytes every time, on every machine, forever. Nothing upstream of `render` cares who reads the output; `render` is where that finally matters, and where the tool has to decide how much to say, in what voice, and whether to let a disagreement stay visible.

What render actually does

`render` takes three things — a store, a thread inside it, and a profile — and produces a fourth: an artifact in one of six formats. The store and the thread say **what** the document claims. The profile says **how** it sounds: how formal, how much apparatus per block, how deep by role, whether a status is a glyph or a word. Changing the profile never changes what a claim says or what it rests on; it only changes how loudly the surrounding apparatus talks about it.

Here is the whole pipeline, run for real against the incident brief from Chapter 8's fixtures — `fixtures/corpus/F1-incident.smy`, which carries an authored `t/brief` thread with three steps and one live rebuttal.

● ● ● *$ smysl render --thread t/brief --profile plain fixtures/corpus/F1-incident.smy*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile plain*

## bottom-line

≈ Pool saturation is the leading cause but is not consistent with the canary.

## support

≈ The eu-west connection pool is saturated.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

## risk

⊢ On the other hand, the canary rules out a pure configuration cause.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

---

**Open contentions:** k/ccm3actwjjti65famnoe6mapo5d
```

Three things to notice before anything else. First, the title line is the thread's own gist — `render` never invents a headline. Second, every claim carries a marker (`≈`, `⊢`) that names its epistemic status; `≈` is `inferred`, `⊢` is `derived`, and the two are never the same glyph. Third, the contested block survived. The finding this thread leads with has a live rebuttal on record, and it appears three times — once against the finding itself and once against each of the two units the rebuttal names — because the renderer attaches a contention note to every unit it touches, not only the one it opens on. Nothing here smoothed the disagreement away, and nothing will, short of asking for that explicitly (rule V2, a few pages on).

> TERM **Profile**
>
> A named bundle of **rendering** choices, entirely separate from the document itself: register (how formal the prose reads), person, verbosity (how much apparatus travels with each block), an audience label, how much level-of-detail each thread role gets by default and per role, and how status, provenance, and contentions are surfaced. A profile never touches what a claim says, what it rests on, or what its status is — only how the surrounding text talks about those facts. `smysl` ships three built in: `plain`, `exec`, `analyst`. You can also write your own (Chapter 26) and pass its file path in place of a name.

WHAT'S NEXT, AND WHY

You've seen one profile render one thread. The rest of this chapter renders the **same** thread under all three built-ins side by side, so the difference a profile makes is visible rather than asserted — then works through the two rules (`V1`, `V2`) that keep a profile honest, `--lod`, and what happens when a store holds more than one thread under the name you asked for.

The same thread, three voices

Run `smysl render --profiles` to see the built-ins at a glance before rendering any of them:

$ smysl render --profiles

```
plain      neutral - · lod L1 · status marker
exec       formal  engineering leadership · lod L1 · status marker
analyst    neutral the person who has to check this · lod L2 · status word
```

PROFILE	AUDIENCE / VERBOSITY	DEPTH (`LOD`)	STATUS SHOWN AS
`plain`	none stated	L1 for every role	a marker — `≈`, `⊢`, …
`exec`	engineering leadership	L1 default; `L0` for `bottom-line`, `risk`, `support`, `ask`	a marker
`analyst`	the person who has to check this	L2 for every role	the word — `[inferred]`, `[derived]`, …

Now the same store, the same thread, the same three blocks — rendered once per profile. `exec` and `analyst` differ from `plain` in exactly the ways the table above predicts, and in no others: none of the three changes which units appear or what they say.

● ● ● *$ smysl render --thread t/brief --profile exec fixtures/corpus/F1-incident.smy*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile exec*
*for engineering leadership*

## bottom-line

≈ Pool saturation is the leading cause but is not consistent with the canary.

## support

≈ The eu-west connection pool is saturated.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

## risk

⊢ On the other hand, the canary rules out a pure configuration cause.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

---

**Open contentions:** k/ccm3actwjjti65famnoe6mapo5d
```

● ● ● *$ smysl render --thread t/brief --profile analyst fixtures/corpus/F1-incident.smy*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile analyst*
*for the person who has to check this*

## bottom-line

[inferred] Pool saturation is the leading cause but is not consistent with the
canary.

*rests on b3:wo4t2c46lq45fnakd6tajlgcac*

## support

[inferred] The eu-west connection pool is saturated.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

*rests on b3:izyuzlt42mqcvgdfb4nfpllxyq*

## risk

[derived] On the other hand, the canary rules out a pure configuration cause.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

*rests on b3:xkys7j42mcuyiaxiyh73xddimr*

---

**Open contentions:** k/ccm3actwjjti65famnoe6mapo5d
```

Read the three side by side and every difference traces to the table. `exec` prints an audience line the plain profile omits, because `exec` names one and `plain` does not. `analyst` spells `[inferred]` and `[derived]` out instead of using `≈`/`⊢`, because its `show.status` is `word` rather than `inline-marker` — the same fact, a slower read. `analyst` also carries a `*rests on …*` note under every block: that is `verbosity: full` surfacing each unit's grounds that this three-step thread does not itself render, so a checker can walk to them; `plain`'s `standard` verbosity and `exec`'s `tight` both keep quiet about the same grounds — `tight` in particular budgets exactly one note per block, which is why `exec`'s `support` block shows the contention once where `plain`'s shows it twice. None of the three profiles drops a block, changes a claim's wording, or renders a status two profiles disagree about identically — that last property is not a coincidence; it is rule V1, next.

Choosing between them is an audience decision, not a formatting one. Reach for `plain` by default. Reach for `exec` when the reader is going to spend four seconds on this and needs the headline and the risk, not the trail. Reach for `analyst` when the reader's job is to check the brief rather than act on it — someone who needs to see every status spelled out and know exactly which unfetched grounds they'd have to pull to verify a claim.

> WHAT'S NEXT, AND WHY
>
> The differences above are all inside what a profile is **allowed** to vary. The next two sections are about what it is not: rule V1 says a profile may never make two statuses look the same, and rule V2 says a profile may hide a disagreement from the rendered text but never from the artifact's own record of itself.

Rule V1: a profile cannot flatten status

> TERM **Rule V1**
>
> Every profile must render each of the six kernel statuses (`unfounded`, `speculative`, `inferred`, `derived`, `cited`, `measured`) distinguishably from every other one. This is checked once, at `Profile::load`, before any rendering happens — not per block, not at emit time. The practical effect: there is no `Profile` value in the entire program that flattens status, so no backend anywhere has to re-check it, and none can be talked into skipping the check. A profile that would flatten status never becomes a document; it becomes an error, and the error names which status collided.

You cannot construct this by accident from the CLI — the three built-ins all pass — but you can construct it deliberately in a profile file, which is exactly how you'd discover the rule if you tried to write a profile that hides status to make a report look tidier:

● ● ● *$ cat /tmp/flat.profile*

```
profile flat {
  register: neutral, person: third, verbosity: standard
  show: { provenance: none, status: none, contentions: always }
}
```

● ● ● *$ smysl render --thread t/brief --profile /tmp/flat.profile fixtures/corpus/F1-incident.smy*

```
smysl render: /tmp/flat.profile: SMY-E210: profile flat has no distinct
rendering for unfounded
```

Exit code `3` (check errors) — not `1`. `show.status: none` means every status renders as an empty string, which is indistinguishable from every other status's empty string; `enforce_v1` catches the first one it walks in kernel order, `unfounded`, and refuses to produce a `Profile` at all. Nothing downstream

— no IR, no backend, no artifact — ever sees this profile, because there is nothing downstream of a load that failed. The same rule fires if two statuses happen to render to the same non-empty marker (`markers: { speculative: "!", measured: "!" }` fails the same way, naming `measured` as the one that collided, because it is checked second in kernel order) or if a single status is blanked while the others are fine. There is exactly one way past rule V1: give every status a distinct, non-empty rendering. `word` and `inline-marker` both satisfy it by construction; only a profile that turns one of them off, or hand-edits `markers`, can violate it.

> **WHAT'S NEXT, AND WHY**
>
> Rule V1 is about status never disappearing. Rule V2, next, is about disagreement — and it is enforced differently: not as a load-time refusal, but as a warning that still lets the render through, because suppressing a contention is sometimes a legitimate choice and flattening status never is.

Rule V2 in practice: contentions

> **TERM Rule V2**
>
> A profile may choose not to **print** an open contention in the rendered text (`show.contentions: suppress`, or `--contentions suppress` on the command line) — but the artifact's own metadata must always say that a suppression happened and name which contention it was. The distinction matters: rule V1 stops a flattening profile from ever producing bytes at all, because there is no way to recover from lost status after the fact. Rule V2 lets the bytes through, because a reader who has the artifact in hand can always ask it whether something was hidden — the suppression is never silent, only optional to display.

Same thread, same profile, contentions shown (the default) and then suppressed:

● ● ● *$ smysl render --thread t/brief --profile plain --contentions show fixtures/corpus/F1-incident.smy*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile plain*

## bottom-line

≈ Pool saturation is the leading cause but is not consistent with the canary.

## support

≈ The eu-west connection pool is saturated.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

## risk

⊢ On the other hand, the canary rules out a pure configuration cause.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

---

**Open contentions:** k/ccm3actwjjti65famnoe6mapo5d
```

● ● ● *$ smysl render --thread t/brief --profile plain --contentions suppress fixtures/corpus/F1-incident.smy*

```
smysl render: SMY-W211: 1 open contention(s) suppressed by profile plain
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile plain*

## bottom-line

≈ Pool saturation is the leading cause but is not consistent with the canary.

## support

≈ The eu-west connection pool is saturated.

## risk

⊢ On the other hand, the canary rules out a pure configuration cause.

---

<!-- SMY-W211: 1 open contention(s) suppressed by profile plain: k/
ccm3actwjjti65famnoe6mapo5d -->
```

The rendered body is genuinely quieter — no `> **contested**` lines anywhere in it — but two things still say the suppression happened. `stderr` gets `SMY-W211` the moment the render runs, and the Markdown artifact itself ends with an HTML comment carrying the same code and the contention's id, invisible when the file is previewed but present in the bytes on disk. This is not a markdown-specific courtesy: every backend records `W211` and the contention id in whatever form its output has room for (a `<meta>` tag in HTML, a `notes: []`-adjacent field in JSON, a comment in Typst), and it is asserted once, across every backend at once, in `crates/smysl-render/src/backend/mod.rs` (`every_backend_records_suppression_in_its_output`) — a new backend cannot quietly opt out of recording it. This is why suppression is a warning and not a hard refusal: with `--strict`, the same command exits `3` instead of `0`, so a pipeline that wants to treat "someone hid a disagreement" as a build failure can do so on demand, without the tool ever hiding it from the one place that would notice — its own metadata.

--lod: overriding depth regardless of the profile

> TERM **Level of detail (LOD)**
>
> `L0` is a unit's one-line gist; `L1` adds its body paragraph if it has one; `L2` adds a further detail block if it has one. A profile sets a **default** LOD (and can override it per role — `exec` does, dropping `risk`, `support`, and `ask` to `L0`), but `--lod` on the command line caps every block at that level regardless of what the profile asked for. It is a cap, not a floor: if the profile already asks for less than the cap, the cap changes nothing.

`t/brief`'s three units carry only a gist — there is no body text to reveal — so LOD is invisible on it. `t/story`, the postmortem narrative in `fixtures/corpus/F3-narrative.smy`, has real body paragraphs, and shows the cap doing something. Rendered under `plain` (default `L1`, so the body already shows):

● ● ● *$ smysl render --thread t/story --profile plain fixtures/corpus/F3-narrative.smy (excerpt)*

```
## setup

† A routine Thursday rollout of release 4.2 to the eu-west shard.

The 4.2 rollout began at 09:14 on a Thursday and reached the eu-west shard
forty minutes later. Nothing in the release notes touched the connection pool,
and the canary had been clean for six hours, so the on-call engineer approved
the promotion without waiting for the extended soak. ...
```

Cap the same render at `L0` and the bodies vanish, leaving only the gist per step — the cap wins over the profile's own `L1` default, which is the whole point of a cap that applies "regardless of what the profile says":

● ● ● *$ smysl render --thread t/story --profile plain --lod L0 fixtures/corpus/F3-narrative.smy (excerpt)*

```
## setup

† A routine Thursday rollout of release 4.2 to the eu-west shard.
[^1]

## complication

† Next, latency alarms fired ninety minutes later, and nobody believed them.
[^2]
```

Reach for `--lod` when you want one render at a fixed depth without writing a whole new profile just to change one number — a quick "give me the headlines only" pass over a thread you'd normally read at `L1` or `L2`.

Disambiguating which thread

Threads are keyed by `(id, owner)`, not by id alone — a store can legitimately hold two threads named `t/brief` if two different agents each derived or authored their own reading of the same graph under that name. `render --thread t/brief` with nothing else is then a guess between them, and this tool does not guess between two agents' readings of the same graph — that is exactly the kind of flattening the whole format exists to prevent. It refuses instead.

Construct the situation for real: derive a second `t/brief` from the same `F1-incident` store, owned by a different agent (`thread --derive`, covered in full in Chapter 20, always defaults its derived thread's

id to `t/derived-<schema>` specifically so it never collides with an authored one by accident — colliding here is deliberate, via an explicit `--id`):

● ● ● *$ smysl thread --derive brief --id t/brief --as human:priya --format surface fixtures/corpus/F1-incident.smy > /tmp/f1-two-briefs.smy*

```
@thread t/brief { schema: brief, owner: human:vladimir, ts: [0, 0] }
~ Auth p95 tripled in eu-west; pool saturation is leading but contested.
  bottom-line → f/root-cause
  support → c/pool-saturation
  risk → c/canary-clean

@thread t/brief { schema: brief, owner: human:priya, ts: [0, 0] }
~ Pool saturation is the leading cause but is not consistent with the canary.;
The canary rules out a pure configuration…
  bottom-line → f/root-cause
  support → d/p95
  support → c/regression
  support → c/pool-saturation
  risk → c/canary-clean
```

The store now holds two `t/brief` threads with different owners and different steps. Ask for `t/brief` without saying whose, and `render` refuses rather than picking one:

● ● ● *$ smysl render --thread t/brief --profile plain /tmp/f1-two-briefs.smy*

```
smysl render: /tmp/f1-two-briefs.smy holds 2 matching threads; narrow with --
thread or --as
smysl render:   t/brief (human:priya)
smysl render:   t/brief (human:vladimir)
exit 2
```

`--as` breaks the tie:

● ● ● *$ smysl render --thread t/brief --as human:priya --profile plain /tmp/f1-two-briefs.smy (excerpt)*

```
# Pool saturation is the leading cause but is not consistent with the canary.;
The canary rules out a pure configuration…

*brief · profile plain*

## bottom-line

≈ Pool saturation is the leading cause but is not consistent with the canary.

## support

† The 95th percentile of request latency over a one-minute window.
[^1]
```

Exit `2` is `usage`, not `failure` — the store and profile were both fine; the command line itself did not say enough to pick one thread over the other. The fix is always the same shape: add `--as <owner>` (or a more specific `--thread`, if two different ids happen to share a prefix you were matching loosely). This is the same category of refusal as rule V1 — a decision the tool will not make silently on your behalf — expressed as a `clap` usage error instead of a load-time one, because the ambiguity lives in which thread you meant, not in what a profile is allowed to render.

WHAT'S NEXT, AND WHY

You can now render one thread three ways, override its depth, control whether a disagreement shows in the text, and pick the right thread out of a store that holds more than one. The other axis of `render` is **format** rather than **voice** — Chapter 25 runs the same thread through every target this build can produce, from a PR-ready Markdown file to a JSON document meant for another program to consume.

TRY THIS

1. Render `F1-incident.smy` under `--profile plain`, then under `--profile analyst`. The same finding comes out as `≈ Pool saturation…` in one and `[inferred] Pool saturation…` in the other, and only the second prints what the claim rests on. Neither is more **correct**. Say which you would put in front of an executive and which in front of an auditor, and defend the choice.
2. Run `render --profile plain --contentions suppress`. The tool emits `SMY-W211` on stderr **and** leaves a comment in the artifact naming the suppressed contention by id. Argue why naming it — rather than counting it — is the difference between rule V2 being honoured and merely gestured at.
3. A profile can hide a contention. It cannot hide a **status**. Explain why the format is willing to let a rendering suppress one and not the other.

ANSWERS

1. `plain` for the executive: the marker is compact and the audience wants the shape of the situation, not the apparatus. `analyst` for the auditor: the status is spelled as a word that cannot be misread, and each claim carries what it rests on, so the reader can walk the argument without the source document. The point of profiles is that this is a **presentation** decision made once, rather than two summaries that can drift apart.
2. Because a count is not actionable and an id is. "1 contention suppressed" tells a reader something was hidden and gives them no way to go look; the id lets them find it in the store and decide for themselves. Rule V2 exists so that a rendered artifact can never read unanimous over a store that is not — and a reader who cannot locate what was hidden is, for practical purposes, reading a unanimous document.
3. Because a contention is a fact **about the document** and a status is a fact **about the claim**. Suppressing a disagreement produces a shorter honest document with a marked omission; suppressing a status would produce a sentence that asserts more than the document does — the reader would see a claim and have no way to tell a measurement from a guess. That is precisely the loss the whole format exists to prevent, so no profile is permitted to cause it: rule V1 requires every status to render distinctly.

WHAT YOU LEARNED

- `render` is a pure function of (store, thread, profile, target): the same inputs always produce the same bytes.
- A profile controls **how** a thread sounds — register, person, verbosity, depth by role, and how status/provenance/contentions are shown — never **what** it says.
- Rule V1 is enforced at profile load: a profile that would render two statuses identically, or any status as nothing, never becomes a `Profile` value and never reaches an artifact (`SMY-E210`, exit `3`).
- Rule V2 lets a profile suppress contentions from the rendered text, but every backend still records the suppression in the artifact's own metadata (`SMY-W211`); `--strict` turns that warning into exit `3`.
- `--lod` caps depth regardless of the profile's own default; it can only lower what a profile asks for, never raise it.
- Threads are keyed by `(id, owner)`. A store holding two threads under one name is refused rather than guessed at (exit `2`, usage); `--as` names the owner and resolves it.

CHAPTER 25

25

Every Target Format

WHY

A brief read by an engineering lead, a brief pasted into a pull request, a brief opened in a browser, a brief projected on a screen, and a brief consumed by another program are five different jobs, not one job with five fonts. `--target` is the axis that answers "who — or what — reads this artifact next," independently of the profile axis from Chapter 24, which answers "in what voice." The two compose freely: any profile, any target.

The six targets

`Target::parse` accepts six names (`md` is an accepted alias for `markdown`, not a seventh target):

`--target`	Extension	Built for
`markdown`	`.md`	Docs, pull requests, chat — anything that already renders Markdown.
`typst`	`.typ`	A print-quality document, chained straight into `typst compile` for a PDF.
`html`	`.html`	A browser view — semantic tags and `data-*` attributes, no styling opinions.
`slides`	`.typ`	A presentation: one Typst slide per block, via the same Typst pipeline as `typst`.
`json`	`.json`	Programmatic consumption — the IR, essentially, as a document another program parses.
`text`	`.txt`	The leanest possible plain rendering: no markup at all, not even Markdown's.

Every target is built from the same IR (Chapter 24's `build_ir` step) by a backend that only ever sees that IR — never the store, never the thread directly. That is a deliberate seam: a backend that could reach back into the store could disagree with another backend about what the document says, which would make "the same thread renders to the same meaning in every format" false. It is asserted directly in `crates/smysl-render/src/backend/mod.rs`: every available target renders every block, keeps every status distinguishable, and records rule V2 suppression identically — the same properties Chapter 24 showed for Markdown hold for all six.

Markdown is the default and already fully shown in Chapter 24 — the same `t/brief` / `plain` render from there is what you get with no `--target` flag at all. The rest of this chapter runs that identical render through the other five.

typst — a print-quality artifact, chained to a real PDF

● ● ● *$ smysl render --thread t/brief --profile plain --target typst fixtures/corpus/F1-incident.smy*

```
#set document(title: "Auth p95 tripled in eu-west; pool saturation is leading
but contested.")
#set text(font: "Libertinus Serif", size: 11pt)
#set par(leading: 0.7em, justify: true)

= Auth p95 tripled in eu-west; pool saturation is leading but contested.

#emph[brief · profile plain]

== bottom-line

#strong[≈] Pool saturation is the leading cause but is not consistent with the
canary.

== support

#strong[≈] The eu-west connection pool is saturated.

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]

== risk

#strong[⊢] On the other hand, the canary rules out a pure configuration cause.

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]

#strong[Open contentions:] k/ccm3actwjjti65famnoe6mapo5d
```

This is not a Markdown file wearing a `.typ` extension — it is Typst source, using `#set`, `#emph`, `#strong`, and `#block` the way the design system in this very manual does. It compiles for real. Piping it straight through the `typst` CLI produces an actual PDF, no manual editing in between:

● ● ● *$ smysl render --thread t/brief --profile plain --target typst fixtures/corpus/F1-incident.smy > brief.typ && typst compile brief.typ brief.pdf*

```
$ typst compile brief.typ brief.pdf
$ ls -la brief.pdf
-rw-r--r--  1 gandalf  wheel  23856 Jul 28 01:35 brief.pdf
```

That is the chain this target exists for: a store you can diff and merge like any other text file, ending in a typeset PDF you can hand to someone who will never see a terminal.

html — a browser view

`html` is compiled in on this build (default features include neither `html` nor `render-html` — more on that below), so building it first requires `--features render-html`:

● ● ● *$ cargo build --features render-html && smysl render --thread t/brief --profile plain --target html fixtures/corpus/F1-incident.smy*

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Auth p95 tripled in eu-west; pool saturation is leading but contested.</
title>
<meta name="smysl-profile" content="plain">
<meta name="smysl-thread" content="t/brief">
<meta name="smysl-contentions-suppressed" content="false">
<meta name="smysl-open-contentions" content="k/ccm3actwjjti65famnoe6mapo5d">
</head>
<body>
<h1>Auth p95 tripled in eu-west; pool saturation is leading but contested.</h1>
<p class="meta">brief · profile plain</p>
<section class="block" data-role="bottom-line" data-status="inferred" data-
uid="b3:js4xzessu5zwjpv2rawtugnuvjuf2o3cys7ko3lsnza3btoldlrq" data-lod="L0">
<h2>bottom-line</h2>
<p><span class="status-marker">≈</span> Pool saturation is the leading cause but
is not consistent with the canary.</p>
</section>
<section class="block" data-role="support" data-status="inferred" data-
uid="b3:cvhirtgs2mpvli2ethhyeo32ufud72l42pa3zxfmsq7dgg722ada" data-lod="L0">
<h2>support</h2>
<p><span class="status-marker">≈</span> The eu-west connection pool is
saturated.</p>
<aside class="contention">k/ccm3actwjjti65famnoe6mapo5d: contested, 2
position(s) on record</aside>
<aside class="contention">k/ccm3actwjjti65famnoe6mapo5d: contested, 2
position(s) on record</aside>
</section>
<section class="block" data-role="risk" data-status="derived" data-
uid="b3:phsoomklkmlq3sjvbe6cyuqy5v3sl6srqxpimogpgmoxvg2pit6q" data-lod="L0">
<h2>risk</h2>
<p><span class="status-marker">⊢</span> On the other hand, the canary rules out
a pure configuration cause.</p>
<aside class="contention">k/ccm3actwjjti65famnoe6mapo5d: contested, 2
position(s) on record</aside>
</section>
<footer><strong>Open contentions:</strong> k/ccm3actwjjti65famnoe6mapo5d</
footer>
</body>
</html>
```

No inline styling, no framework markup — plain semantic HTML with `data-*` attributes carrying exactly the information a stylesheet or a script would need (`data-status`, `data-uid`, `data-lod`), and the metadata the artifact must always carry (rule V2′s suppression flag, the open contention id) landed as `<meta>` tags rather than dropped. This is a target for a browser or for whatever renders on top of the browser, not a finished page — bring your own CSS.

slides — one Typst slide per block

`slides` shares its availability with `typst` (both are Typst-backed; the source ties `Target::Slides.available()` to the same feature flag as `Target::Typst`), and the output is a Typst deck rather than a document — one `#pagebreak()`-separated slide per rendered block, sized for a 25×14 cm screen rather than a page:

● ● ● *$ smysl render --thread t/brief --profile plain --target slides fixtures/corpus/F1-incident.smy*

```
#set page(width: 25cm, height: 14cm, margin: 1.5cm)
#set text(size: 20pt)
#set document(title: "Auth p95 tripled in eu-west; pool saturation is leading
but contested.")

#align(center + horizon)[
  #text(size: 28pt)[Auth p95 tripled in eu-west; pool saturation is leading but
contested.]

  #text(size: 14pt)[brief · plain]
]
#pagebreak()

// slide 1: b3:js4xzessu5zwjpv2rawtugnuvj
#text(size: 14pt)[bottom-line]

#strong[≈] Pool saturation is the leading cause but is not consistent with the
canary.

#pagebreak()

// slide 2: b3:cvhirtgs2mpvli2ethhyeo32uf
#text(size: 14pt)[support]

#strong[≈] The eu-west connection pool is saturated.

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]

#pagebreak()

// slide 3: b3:phsoomklkmlq3sjvbe6cyuqy5v
#text(size: 14pt)[risk]

#strong[⊢] On the other hand, the canary rules out a pure configuration cause.

#block(stroke: 1pt, inset: 6pt)[contested — k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record]


#pagebreak()
#strong[Open contentions:] k/ccm3actwjjti65famnoe6mapo5d
```

This chains through `typst compile` exactly like the `typst` target does — the only differences are the page geometry and the one-block-per-slide layout. If you need to stand in front of a room with this brief, `slides` is the target, not `typst`.

json — for another program

● ● ● *$ smysl render --thread t/brief --profile plain --target json fixtures/corpus/F1-incident.smy*


```
{
  "gist": "Auth p95 tripled in eu-west; pool saturation is leading but
contested.",
  "meta": {
    "profile": "plain",
    "thread": "t/brief",
    "schema": "brief",
    "audience": null,
    "contentions_suppressed": false,
    "open_contentions": ["k/ccm3actwjjti65famnoe6mapo5d"]
  },
  "blocks": [
    {
      "role": "bottom-line",
      "uid": "b3:js4xzessu5zwjpv2rawtugnuvjuf2o3cys7ko3lsnza3btoldlrq",
      "level": "L0",
      "status": "inferred",
      "marker": "≈",
      "connective": null,
      "text": "Pool saturation is the leading cause but is not consistent with
the canary.",
      "notes": []
    },
    {
      "role": "support",
      "uid": "b3:cvhirtgs2mpvli2ethhyeo32ufud72l42pa3zxfmsq7dgg722ada",
      "level": "L0",
      "status": "inferred",
      "marker": "≈",
      "connective": null,
      "text": "The eu-west connection pool is saturated.",
      "notes": [
        {"kind": "contention", "text": "k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record"},
        {"kind": "contention", "text": "k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record"}
      ]
    },
    {
      "role": "risk",
      "uid": "b3:phsoomklkmlq3sjvbe6cyuqy5v3sl6srqxpimogpgmoxvg2pit6q",
      "level": "L0",
      "status": "derived",
      "marker": "⊢",
      "connective": "On the other hand, ",
      "text": "The canary rules out a pure configuration cause.",
      "notes": [
        {"kind": "contention", "text": "k/ccm3actwjjti65famnoe6mapo5d:
contested, 2 position(s) on record"}
      ]
    }
  ]
}
```

This is close to the IR itself, serialized: every field Chapter 24 discussed by name — `status`, `marker`, `level`, `connective`, `notes` with their `kind` — appears as a JSON field with the same name. Reach for this target when the reader is a program: a dashboard that wants to colour by `status`, a notifier that wants to alert on a non-empty `open_contentions`, a second tool in your own pipeline that wants the brief without re-deriving it from the store.

text — the leanest possible rendering

● ● ● *$ smysl render --thread t/brief --profile plain --target text fixtures/corpus/F1-incident.smy*

```
Auth p95 tripled in eu-west; pool saturation is leading but contested.
======================================================================

bottom-line [b3:js4xzessu5zwjpv2rawtugnuvj]
  ≈ Pool saturation is the leading cause but is not consistent with the canary.

support [b3:cvhirtgs2mpvli2ethhyeo32uf]
  ≈ The eu-west connection pool is saturated.
      - k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on record
      - k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on record

risk [b3:phsoomklkmlq3sjvbe6cyuqy5v]
  ⊢ On the other hand, the canary rules out a pure configuration cause.
      - k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on record

open contentions: k/ccm3actwjjti65famnoe6mapo5d
```

No `#`, no `**`, no HTML tags — a title underlined with `=`, indentation instead of headings, a short uid in brackets instead of a footnote. This is the target for a terminal pager, a log line, an email body, or anywhere Markdown's own syntax would just be visual noise the reader has to mentally strip out.

Feature gates: what this build can and cannot produce

`Target::available()` is a compile-time question, and the answer is checked before a backend ever runs — an unavailable target refuses rather than silently substituting a different format:

TARGET	CARGO FEATURE	IN THIS REPO'S DEFAULT BUILD?
`markdown`, `json`, `text`	none — always compiled	Yes, unconditionally.
`typst`, `slides`	`render-typst` → `smysl-render/typst`	Yes — `render-typst` is in the workspace's `default` feature list.
`html`	`render-html` → `smysl-render/html`	No — `render-html` is not a default feature; it must be requested explicitly.

The default build (`cargo build`, no flags) genuinely cannot produce `html` until you ask for it:

● ● ● *$ smysl render --thread t/brief --profile plain --target html fixtures/corpus/F1-incident.smy (default build)*

```
smysl render: render target html is not available in this build
exit 1
```

Rebuilding with `--features render-html` is what produced the real HTML sample earlier in this chapter. The same refusal-not-substitution rule runs the other way too: build with only the bare minimum `--no-default-features` (`--features cli`, which drops `render-typst`) and `typst` refuses on that build the same way `html` refused on the default one, while `markdown` still works untouched:

● ● ● *$ cargo build --no-default-features --features cli && smysl render --thread t/brief --profile plain --target typst fixtures/corpus/F1-incident.smy*

```
smysl render: render target typst is not available in this build
exit 1
```

● ● ● *$ smysl render --thread t/brief --profile plain --target markdown fixtures/corpus/F1-incident.smy (same minimal build)*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.
...
```

An artifact in the wrong format would be more surprising than an error naming the format as unavailable — this is a deliberate design choice (`crates/smysl-render/src/backend/mod.rs`), not an oversight: `emit` checks `target.available()` before matching on the target at all, so there is no code path in which a build silently hands you Markdown when you asked for HTML.

WHAT'S NEXT, AND WHY

Six targets, one IR, one guarantee: whichever format you pick, it says the same thing the others do, with the same statuses, the same contentions, and the same suppression record. Pick the target that matches your actual downstream reader — a person (`markdown`, `html`, `slides`), a typesetter chasing a PDF (`typst`), or another program (`json`) — rather than defaulting to Markdown out of habit. Chapter 26 goes the other direction: instead of picking a shipped profile, you write your own from scratch.

TRY THIS

1. Render `F1-incident.smy` to `markdown`, `text`, `json` and `slides` in turn. The `slides` output is Typst source, not an image, and `json` is a tree rather than prose. Given that all four came from one thread under one profile, say what the **target** axis controls that the profile axis does not.

2. Run `render --target html` on a default build. It reports that the target is not available. Look back at Chapter 4's feature table and say why a missing render backend is a build-time decision rather than a runtime one.
3. Render the same document twice with the same flags and compare hashes. They match. Name the property, and then name one thing you would have to add to `render` to lose it.

ANSWERS

1. The target decides **who or what reads the artifact next** — a person in a terminal, a person in a browser, a projector, another program — and therefore its syntax and structure. The profile decides **in what voice** the same content is put. They are independent on purpose: you can send the analyst voice to a slide deck or the executive voice to JSON, and neither axis has to know about the other.
2. Because the backends are separate code with separate dependencies, and compiling them in is a choice about what you want in your dependency tree. A build without `render-html` genuinely does not contain an HTML renderer, which is a stronger and more auditable statement than a runtime flag that disables one. The command tells you plainly rather than silently emitting something else.
3. Determinism — rule D. `render` is a pure function of the graph, the thread and the profile, so the same inputs give the same bytes forever, and an artifact signed off last year can be regenerated and compared today. You would lose it by adding almost any of the obvious conveniences: a generated-on timestamp in the header, a model call to smooth the prose, or anything that consulted the wall clock or the network.

WHAT YOU LEARNED

- Six targets: `markdown` (default), `typst`, `html`, `slides`, `json`, `text` — `md` is an alias for `markdown`, not a seventh target.
- Every target is built from the same IR by a backend that never touches the store directly, so no format can disagree with another about what the document says.
- `typst` and `slides` chain directly into `typst compile` for a real PDF or deck; verified end to end in this chapter.
- `html` needs `--features render-html`; `typst`/`slides` need `render-typst` (in this repo's default feature set already). An unavailable target refuses with exit `1` rather than substituting another format.
- `json` is the closest thing to the IR itself, and is the target to reach for when the next reader is a program rather than a person.

CHAPTER 26

26

Writing a Custom Profile

WHY

`plain`, `exec`, and `analyst` cover the common cases — a neutral default, a thin brief for people who will act on it, and a full trace for someone who has to check it. They do not cover every case. A regulator who needs every status spelled out in full words with inline citations, in a register your own house style forbids from ever using `exec`'s tight verbosity, is a real audience your three built-ins were never written for. A profile file is how you describe that audience once and reuse it, without any of it touching the graph itself.

The profile file grammar

A profile file is a Hjson-flavoured object, optionally preceded by a `profile NAME {` header (a bare `{ … }` also loads, with a default name of `unnamed`). Every field is optional; a missing field takes the same default `Profile::plain()` uses. Here is the complete field list, straight from `crates/smysl-render/src/profile.rs`:

FIELD	VALUES	DEFAULT
`register`	`formal`, `neutral`, `plain`	`neutral`
`person`	`first`, `second`, `third`	`third`
`verbosity`	`tight` (1 note/block), `standard` (3), `full` (unbounded)	`standard`

FIELD	VALUES	DEFAULT
`audience`	any string	none
`connectives`	`from-relations`, `none`	`from-relations`
`lod.default`	`L0`, `L1`, `L2`	`L1`
`lod.roles.<role>`	`L0`/`L1`/`L2` per role name (`bottom-line`, `risk`, `support`, `ask`, `setup`, …)	falls back to `lod.default`
`show.provenance`	`none`, `inline`, `footnote`	`footnote`
`show.status`	`inline-marker`, `word`, `none` (refused — rule V1)	`inline-marker`
`show.contentions`	`always`, `on-rendered`, `suppress`	`always`
`markers.<status>`	a string per kernel status, overriding the default glyph/word	built from `show.status`

The RFC's own worked example — reproduced verbatim as the built-in `exec` profile — shows the shape end to end:

```
profile exec {
  register: formal, person: third, verbosity: tight
  audience: "engineering leadership"
  lod:  { default: L1, roles: { bottom-line: L1, risk: L0, support: L0, ask: L0 } }
  show: { provenance: footnote, status: inline-marker, contentions: always }
  connectives: from-relations
}
```

Read it field by field: `register: formal` and `verbosity: tight` set the overall voice; `lod.default: L1` is the fallback depth, overridden per role so `risk`, `support`, and `ask` sit at `L0` (headline only) while `bottom-line` stays at `L1`; `show.status: inline-marker` picks glyphs over spelled-out words; `show.contentions: always` means this profile never hides a disagreement, only `--contentions suppress` on the command line can. Nothing here is order-sensitive — Hjson object fields, not a sequence of statements — and any field left out takes `Profile::plain()`'s value for it.

Authoring one from scratch

Suppose the audience is an external compliance reviewer: formal register, full verbosity (every note, every citation inline, nothing held back), status spelled out as full words rather than glyphs, and grounds pulled up to `L2` for the two roles a reviewer actually has to verify (`bottom-line` and `risk`), while `support` stays at `L1`. Connectives off, because a reviewer should read each claim on its own footing rather than have the prose imply a narrative flow between them.

● ● ● *$ cat /tmp/regulator.profile*

```
profile regulator {
  register: formal, person: third, verbosity: full
  audience: "external compliance reviewer"
  lod:  { default: L1, roles: { bottom-line: L2, risk: L2, support: L1 } }
  show: { provenance: inline, status: word, contentions: always }
  connectives: none
  markers: {
    unfounded: "[UNFOUNDED]", speculative: "[SPECULATIVE]", inferred:
"[INFERRED]",
    derived: "[DERIVED]", cited: "[CITED]", measured: "[MEASURED]"
  }
}
```

A profile file has no special extension the tool requires — `--profile` tries a built-in name first, and if that lookup fails, treats the value as a filesystem path and reads it, whatever it's named. Point `--profile` straight at the path:

● ● ● *$ smysl render --thread t/brief --profile /tmp/regulator.profile fixtures/corpus/F1-incident.smy*

```
# Auth p95 tripled in eu-west; pool saturation is leading but contested.

*brief · profile regulator*
*for external compliance reviewer*

## bottom-line

[INFERRED] Pool saturation is the leading cause but is not consistent with the
canary.

*rests on b3:wo4t2c46lq45fnakd6tajlgcac*

## support

[INFERRED] The eu-west connection pool is saturated.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

*rests on b3:izyuzlt42mqcvgdfb4nfpllxyq*

## risk

[DERIVED] The canary rules out a pure configuration cause.

> **contested** — k/ccm3actwjjti65famnoe6mapo5d: contested, 2 position(s) on
record

*rests on b3:xkys7j42mcuyiaxiyh73xddimr*

---

**Open contentions:** k/ccm3actwjjti65famnoe6mapo5d
```

This is a fourth, genuinely distinct voice: `[INFERRED]` / `[DERIVED]` in brackets (the custom `markers` override, not the built-in `word` display's `[inferred]` / `[derived]` — you can restyle the word itself, not only choose word-vs-marker), no connective softening the `risk` block into "on the other hand" the way `plain` and `exec` both did, and every block carries the `*rests on …*` grounds note that only `verbosity: full` unlocks. Nothing about the claims or their status changed from the plain render at the start of Chapter 24 — only how insistently the apparatus around them talks.

Breaking rule V1 on purpose, one more time

The same refusal from Chapter 24 applies to every custom profile, built-in or not — because it is enforced once, in `Profile::load`, before a name is ever attached to a `Profile` value. Ask this new profile file to also hide status, and it fails exactly the way the hand-built `flat` profile did:

● ● ● *$ cat /tmp/regulator-broken.profile*

```
profile regulator {
  register: formal, person: third, verbosity: full
  audience: "external compliance reviewer"
  show: { provenance: inline, status: none, contentions: always }
}
```

● ● ● *$ smysl render --thread t/brief --profile /tmp/regulator-broken.profile fixtures/corpus/F1-incident.smy*

```
smysl render: /tmp/regulator-broken.profile: SMY-E210: profile regulator has no
distinct rendering for unfounded
exit 3
```

There is no register, verbosity, or audience setting anywhere in the grammar that buys an exemption from rule V1 — formal, tight, aimed at a regulator, or otherwise. The check runs against `show.status` / `markers` alone, and it runs before the rest of the profile's intent is even relevant. If your house style genuinely wants a quieter status display than `inline-marker` gives you, the honest move is `word` (spell it out, which reads **more** insistently, not less) or a custom `markers` block with your own six distinct strings — never `none`.

> **WHAT'S NEXT, AND WHY**
>
> You now have full control over every stage this manual has covered so far: creation (Chapters 6–7), enrichment (7–10), operation (11–18), verification (19–21), and — as of these three chapters — export. Part VIII shows the same primitives (`Profile`, `Target`, `build_ir`, `emit`) used as a Rust library rather than through this CLI, for a program that wants to render without shelling out; Chapter 29 chains everything in this manual, `render` included, into one realistic end-to-end scenario.

> **TRY THIS**
>
> 1. `render --profiles` lists three built-ins and their settings: `plain` is neutral at L1 with a status **marker**, `analyst` is L2 with a status **word**. Before writing any profile of your own, say which of those two axes — level of detail, or how status is spelled — you would expect a regulator to care about, and why.

2. Rule V1 says a profile must render every status distinctly. Try to write a profile that maps both `inferred` and `derived` to the same marker, and see what `Profile::load` says. Then explain why this is enforced at **load** time rather than when the offending unit is first encountered.
3. Your house style forbids the terse register `exec` uses, but you want its brevity. Which parts of a profile can you change to get that, and which part of the output is not the profile's to decide at all?

ANSWERS

1. The status spelling. A marker like `≈` is compact and assumes the reader knows the convention; the word `[inferred]` cannot be misread by someone encountering the document once, under obligation, possibly in a dispute. Level of detail matters too, but it trades length against completeness — the status axis trades **nothing**, which is why an audience that must not misunderstand gets words.
2. It refuses to load. Enforcing it at load time means a profile is either valid for every document or rejected outright — the failure cannot depend on which units a particular store happens to contain. The alternative would be a profile that renders a thousand documents correctly and then flattens two statuses on the one document where the distinction mattered, which is the hedge loss from Chapter 1 arriving at the very last step.
3. You can change the register, the verbosity, the level of detail, the status spelling, and whether an audience line is printed. You cannot change what the units **say** — a profile selects and frames gists, it never rewrites them. That boundary is why rendering stays pure: if a profile could reword a claim, the artifact would no longer be a function of the graph, and two renderings of one document could disagree about what it asserts.

WHAT YOU LEARNED

- A profile file is an optional `profile NAME { … }` header over an Hjson object; every field is optional and falls back to `Profile::plain()`'s default.
- Fields: `register`, `person`, `verbosity`, `audience`, `connectives`, a `lod` block (`default` plus per-role overrides), a `show` block (`provenance`, `status`, `contentions`), and a `markers` block overriding individual status renderings.
- `--profile` accepts a built-in name or a filesystem path — no special extension required — and falls back to treating the value as a path only after the built-in lookup fails.
- A hand-authored profile can combine these fields into a voice none of the three built-ins provide, while never changing what a claim says or what it rests on.
- Rule V1 applies identically to every profile, built-in or custom: any profile that would render two statuses the same way, including via `show.status: none`, fails to load (`SMY-E210`, exit `3`) before it can ever produce an artifact.

PART VIII

smysl as a Library

CHAPTER 27

27

The Facade Crate and Feature Flags

Every workflow so far has gone through the `smysl` binary. This chapter is about the thing underneath it — because the binary is not where the work happens.

Why the library is the product

> **WHY**
>
> **The library is the product; the CLI is its first consumer.** This is not a slogan — it is a concrete promise: nothing the CLI does is unreachable from the crate, and no code path is CLI-only. If you can do it with `smysl pack --budget …`, you can do it from Rust, with the same function, the same guarantees, and no process boundary in between.

`src/lib.rs` opens by saying exactly that:

```
//! The library is the product; the CLI is its first consumer (principle P8). Rule
A
//! holds throughout: no CLI capability may be unreachable from here, and no code
path
//! may be CLI-only.
```

The rest of this section is that promise checked, not asserted. Take `pack` — the command Chapter 18 built a whole workflow around. `src/lib.rs` re-exports it from `smysl-pack`:

```
pub use smysl_pack::{
    pack, verify as verify_pack, Constraints, Estimator, Pack, PackError,
PackRequest, Reason,
    Selection, Violation,
};
```

And `cmd_pack` in `src/main.rs` — the function that runs when you type `smysl pack` — calls exactly that function, on exactly the types the facade exports, with nothing standing between the parsed command line and the library call:

```
let sal = smysl::salience(&store,
&SalienceRequest::default().seeded(smysl::view_roots(&store)));
let outcome = smysl::pack(&store, &sal, &req);
```

There is no private `pack_impl` inside the binary that the public `smysl::pack` merely wraps. `req` is a `PackRequest` you could have built yourself; `store` is a `Store` you could have built yourself; `sal` is a `SalienceReport` from a call you could have made yourself. Chapter 28 does exactly that, end to end, with nothing named `main`.

> **TERM Facade crate**
>
> The `smysl` crate itself: a `[lib]` re-exporting the public surface of every other crate in the workspace, plus a `[[bin]]` (`required-features = ["cli"]`) that is a thin shell over it. `src/lib.rs`'s re-export list is the entire public API; nothing else in the workspace is meant to be depended on directly by an embedder.

The feature table

An embedder does not want a terminal UI, an argument parser, or four HTTP clients linked into their service just to call `check` and `pack`. The facade's `[features]` table, from `Cargo.toml`, is how much of the tree you actually pull in — transcribed here exactly, edges and all:

FEATURE	TURNS ON
`default`	`cli`, `local`, `render-typst` — what plain `cargo build` gives you: the binary, local (Ollama) model access, and Typst rendering. **Not** `tui`: `ratatui` and `crossterm` in every default build is a cost an embedder who only calls the library never opted into, so the browser is `--features tui`.
`cli`	`dep:clap`. Without it there is no `[[bin]]` at all — the binary's own manifest entry requires this feature to exist.
`tui`	`dep:smysl-tui`, and `cli` (the TUI is reached through the same binary, so it pulls the parser in too).
`providers`	`dep:smysl-provider` — the provider abstraction: registry, capabilities, ledger. No vendor is wired in by this alone.
`ingest`	`dep:smysl-ingest`, and `providers` (ingest always needs a provider to call, so the dependency is unconditional).

FEATURE	TURNS ON
`local`	`ingest`, plus `smysl-provider/ollama` and `smysl-ingest/ollama` — everything `ingest` and `attest` need, wired to a local Ollama server that never leaves the machine.
`remote`	`ingest`, plus `smysl-provider/{anthropic,openai,gemini,deepseek}` and the matching `smysl-ingest/*` features — the four hosted providers, on both the calling side and the ingest-specific side.
`render-typst`	`smysl-render/typst` — the backend behind `render --target typst`.
`render-html`	`smysl-render/html` — the backend behind `render --target html`.
`exact-pack`	`smysl-pack/branch-and-bound` — what `pack --mode exact` needs to prove optimality instead of packing greedily.
`tls-pure`	`smysl-provider/tls-pure` — a pure-Rust TLS stack for provider connections, keeping C out of the network path the way `blake3`'s `pure` feature keeps it out of the hash path (§13).

Notice what is **not** in that table: `smysl-core`, `smysl-graph`, `smysl-check`, `smysl-pack`, `smysl-thread`, and `smysl-render` are plain, non-optional dependencies in `[dependencies]` — every build gets the kernel, the store, the ten check passes, packing, thread derivation, and the render IR, whatever features you choose. Only the things with a real cost — an argument parser, a terminal UI, a model runtime, a specific render backend — are behind a flag.

A build with nothing turned on

`default-features = false` is the embedder's starting point, and it is worth not taking on faith. Building the facade crate's own library target with every feature off:

● ● ● *$ cargo tree --no-default-features -p smysl*

```
smysl v0.13.0
├── smysl-check v0.13.0
│   ├── smysl-core v0.13.0
│   │   ├── blake3 v1.8.5
│   │   │   ├── arrayref v0.3.9
│   │   │   ├── arrayvec v0.7.8
│   │   │   ├── cfg-if v1.0.4
│   │   │   └── constant_time_eq v0.4.2
│   │   │   [build-dependencies]
│   │   │   └── cc v1.4.0
│   │   │       ├── find-msvc-tools v0.1.9
│   │   │       └── shlex v2.0.1
│   │   └── unicode-normalization v0.1.25
│   │       └── tinyvec v1.12.0
│   │           └── tinyvec_macros v0.1.1
│   └── smysl-graph v0.13.0
│       └── smysl-core v0.13.0 (*)
├── smysl-core v0.13.0 (*)
├── smysl-graph v0.13.0 (*)
├── smysl-pack v0.13.0
│   ├── smysl-core v0.13.0 (*)
│   └── smysl-graph v0.13.0 (*)
├── smysl-render v0.13.0
│   ├── smysl-core v0.13.0 (*)
│   ├── smysl-graph v0.13.0 (*)
│   └── smysl-thread v0.13.0
│       ├── smysl-core v0.13.0 (*)
│       └── smysl-graph v0.13.0 (*)
├── smysl-retrieve v0.13.0
│   ├── bm25 v2.3.2
│   │   └── fxhash v0.2.1
│   │       └── byteorder v1.5.0
│   ├── smysl-core v0.13.0 (*)
│   └── smysl-graph v0.13.0 (*)
└── smysl-thread v0.13.0 (*)
[dev-dependencies]
└── serde_json v1.0.151
    ├── itoa v1.0.18
    ├── memchr v2.8.3
    ├── serde_core v1.0.229
    └── zmij v1.0.23
```

No `tokio`, no `clap`, no `ureq`, no `crossterm`, no `ratatui` — grep the real output for any of them and it comes back empty. That is the point of the flag, and `cargo xtask check-purity` fails the build if any of them ever appears.

What **is** there is worth reading. `blake3` hashes, built with `pure` so there is no C SIMD backend — the hash path is Rust all the way down. (`cc` appears beneath it under `[build-dependencies]`: `blake3`

declares it unconditionally, and `pure` is what stops it being used.) `unicode-normalization` NFC-normalises surface text on parse. And `bm25`, with `fxhash` and `byteorder` behind it, is lexical retrieval — `smysl-retrieve` is a plain dependency rather than an optional one, because its default engine is pure: BM25 with `default-features = false` brings in no runtime, which is a claim `check-purity` enforces rather than asserts.

That is the whole dependency graph of a fully synchronous library: no async runtime, no HTTP client, no argument parser, anywhere.

CHECKED, NOT JUST INTENDED

This is not a documentation promise you have to trust — `cargo xtask check-purity` runs exactly this kind of dependency-tree audit in CI on every change (rule B). The library's shape is a gate, not a description.

The same flag makes the test suite itself meaningfully smaller: `cargo test --no-default-features --lib` builds and runs the crate's own unit tests, including the one Chapter 28 walks through, with nothing beyond `smysl-core`, `smysl-graph`, `smysl-check`, `smysl-pack`, `smysl-thread`, and `smysl-render` compiled at all.

WHAT'S NEXT, AND WHY

You now know **what** is reachable from Rust and **how little** it costs to reach it. Chapter 28 reaches it — a real program, compiled and run against exactly this facade, with `default-features = false` the whole way through.

TRY THIS

1. Chapter 27 claims nothing the CLI does is unreachable from the crate. Pick a command you have used in this book and find the library function behind it. Then say why "the library is the product, the CLI is its first consumer" is a testable claim rather than a slogan.
2. Run `cargo build --no-default-features` and then `cargo build --no-default-features --bin smysl`. Explain the difference in outcome to somebody who has not read Chapter 4.
3. You are asked to add `smysl` to a service that must be certified as unable to send data to a third party. Which feature flags do you enable, and what can you tell the auditor that a runtime flag could not?

ANSWERS

1. Because if it were false, some behaviour would exist only inside `src/main.rs` and could not be reached from a test that imports the crate. The claim is checked by the CLI itself being written against the public API — every subcommand is a thin argument-parsing shell over a library call. The consequence for you is that anything this book demonstrates at the command line is something you can do in-process, with the same guarantees and no subprocess.

2. The first builds the library and quietly skips the binary, because the binary target declares that it requires the `cli` feature and cargo builds only what the enabled features permit. The second asks for the binary by name, so cargo can no longer skip it and reports the missing feature. The library is the crate's primary product; the binary is an optional target layered on top.
3. Neither `remote` nor anything that pulls it in — the default set is `cli`, `tui`, `local`, `render-typst`, and `local` wires ingest to a model running on your own machine. For the strongest statement, build with `--no-default-features` plus only what you need. What you can tell the auditor is that the HTTP client is not in the binary: not disabled, not configured off, but absent from the dependency tree, which is a property they can verify from `cargo tree` rather than take on trust.

WHAT YOU LEARNED

- Rule A is load-bearing, not aspirational: every `cmd_*` function in `src/main.rs` calls straight into a facade re-export — `cmd_pack` calls `smysl::pack`, with no private implementation hiding behind it. Since 0.13 that is checked rather than asserted: `cargo xtask check-purity` fails if anything under `src/` outside `lib.rs` names a sibling crate. It found two bypasses the first time it ran, which is the honest reason the check exists — `cmd_import` was reaching into `smysl-ingest` for the CSV reader, so a consumer holding the facade could not do what `smysl import` does.
- `smysl-core`, `smysl-graph`, `smysl-check`, `smysl-pack`, `smysl-thread`, and `smysl-render` are always present; `cli`, `tui`, `providers`, `ingest`, `local`, `remote`, the two render backends, `exact-pack`, and `tls-pure` are each opt-in.
- `cargo build --no-default-features` produces a dependency tree with no `tokio`, `clap`, `ureq`, `crossterm`, or `ratatui` in it — verified here by `cargo tree`, and checked on every change by `cargo xtask check-purity`.

CHAPTER 28

28

A Minimal Embedding, End to End

Chapter 27 established that nothing stops you from calling `smysl` from Rust directly. This chapter does it — twice: once at the smallest possible scale, and once with two units and a real dependency between them.

> **WHY**
>
> You maintain a service that already produces incident write-ups — a bot that watches alerts, or an internal tool that collects postmortems. It has its own storage, its own web UI, and no interest in acquiring a command-line tool, a TUI, an argument parser, or an async HTTP client just to emit a document other systems can check.
>
> Shelling out to a binary would mean shipping that binary, matching its version to your code, parsing its stdout, and turning its exit codes back into your own error type. Every one of those is a place for the two to drift. Calling the library means the units your service builds are the same units `check` validates, constructed through the same constructors, failing the same way — and the dependency you add is a synchronous crate with no I/O in it at all.

The smallest possible round trip

`src/lib.rs` carries its own proof that the facade alone is enough to build, hash, encode, and decode a unit. It is a real test in the crate — `a_unit_round_trips_through_the_facade_alone` — and it passes under `--no-default-features`, which is the whole point: nothing here needs `cli`, `tui`, or any model feature.

```
let core = UnitCoreBuilder::new(
    KernelType::Claim,
    "p95 auth latency tripled",
    Status::Speculative,
)
.build()
.unwrap();
let uid = canonical_uid(&core);
let bytes = to_cbor(&Record::Unit(core.clone()));
let (decoded, n) = from_cbor(&bytes).unwrap();
assert_eq!(n, bytes.len());
assert_eq!(decoded.as_unit(), Some(&core));
assert!(verify(decoded.as_unit().unwrap(), &uid).is_ok());
```

Walking it line by line is worth doing once, because every later chapter's CLI output is this same sequence with a terminal wrapped around it.

- `UnitCoreBuilder::new(KernelType::Claim, "p95 auth latency tripled", Status::Speculative)` starts a builder for a claim at the lowest non-`unfounded` rung — appropriate, since nothing grounds it yet.
- `.build()` returns a `Result`, and that is deliberate. Building is where the shape rules of `UnitCore::new` are enforced — a missing gist, a `detail` with no `body`, a `derived`/`inferred` unit with empty `grounds`, a `measured`/`cited` unit with no `source`. Get one of those wrong and `.build()` fails with a `ShapeError` carrying the exact `SMY-E0xx` code from Appendix B, before a `UnitCore` value exists at all.
- Once `.build()` succeeds, the value is **already** NFC-normalised and **already** shape-valid — `UnitCore::new`'s whole job is to make that true before it returns, the same invariant the ingest layer relies on when it says "a core is shape-valid from the moment it exists." That is why nothing downstream of `.build()` is fallible in this example.
- `canonical_uid(&core)` hashes the already-canonical bytes. It cannot fail, because there is no invalid input left to reject by this point — the builder already rejected it.
- `to_cbor(&Record::Unit(core.clone()))` encodes to `Vec<u8>` directly, no `Result`, for the same reason: a built `UnitCore` is always encodable.
- `from_cbor(&bytes)` is where fallibility returns, because now the input is **untrusted** again — arbitrary bytes, not a value this process just built. It returns the decoded `Record` and how many bytes it consumed, which is what lets `from_cbor_seq` walk a concatenated log of them without a length prefix.
- `verify(decoded.as_unit().unwrap(), &uid)` recomputes the hash of the decoded core and checks it against the uid you started with. This is the exact check behind `fmt`'s round-trip guarantee and `SMY-E070`/exit `9` (Appendix C) — here, called directly, with no file and no process in between.

> **WHY BUILDING IS FALLIBLE AND ENCODING IS NOT**
>
> This asymmetry is the point of validating on construction (§15.1, guarantee A4): once a `UnitCore` exists, every later operation on it — hashing, encoding, comparing — can be infallible, because "does this shape make sense" was answered exactly once, at the narrowest point, by the builder. Nothing later has to re-check it.

A richer example: two units, grounds, and a check

One unit alone never exercises the interesting rule — rule M, that a claim's status can never outrank the weakest thing it rests on. This example builds two units with a real `grounds` relationship and runs the same `check` pipeline `smysl check` runs on the CLI, entirely in memory:

```
use smysl::{
    canonical_uid, check, CheckOptions, KernelType, Record, SourceKind, SourceRef,
Status,
    Store, UnitCoreBuilder,
};

let evidence = UnitCoreBuilder::new(
    KernelType::Evidence,
    "pool wait rose from 2ms to 310ms",
    Status::Measured,
)
.source(SourceRef::new(SourceKind::Metric, "pool.wait_ms"))
.build()
.unwrap();
let evidence_uid = canonical_uid(&evidence);

let claim = UnitCoreBuilder::new(
    KernelType::Claim,
    "the eu-west pool is saturated",
    Status::Inferred,
)
.grounds([evidence_uid])
.build()
.unwrap();

let store = Store::from_records(vec![
    Record::Unit(evidence.clone()),
    Record::Unit(claim.clone()),
]);

let report = check(&store, CheckOptions::default());
assert!(report.is_clean());
assert_eq!(store.units().count(), 2);
```

Two builder calls are worth pausing on. `.source(...)` on the evidence is not decoration — `Status::Measured` requires it; leave it off and `.build()` fails with `SMY-E032`. `.grounds([evidence_uid])` on the claim is the same story from the other rule: `Status::Inferred` requires non-empty `grounds`, or `.build()` fails with `SMY-E031`. The builder is enforcing the same shape rules Appendix B lists, at the point where you would otherwise author a document by hand and let `smysl check` catch the mistake later — here, you cannot construct the mistake in the first place.

`Store::from_records` takes the two units with no file, no path, and no parsing — a `Store` is a value, not a handle to something on disk. `check(&store, CheckOptions::default())` is the exact function `cmd_check` calls after `load_store` turns a `.smy` file or a CBOR log into the same kind of `Store` value. The report comes back clean because the claim's `Inferred` status does not outrank its one ground's

`Measured` status — rule M, satisfied, with the arithmetic entirely under your control and no subprocess anywhere in the call stack.

HOW THIS WAS VERIFIED

This is not sample code copied from a comment — it is a real `#[test]` that was compiled and run against the `smysl` crate with `cargo test --no-default-features --test ch26_scratch`, and it passed. `--no-default-features` matters here: it is the same proof as Chapter 27's `cargo tree`, done by actually building and running code against the bare facade rather than only inspecting its dependency graph.

WHAT'S NEXT, AND WHY

You have now seen every operation this book covers from both sides: the CLI, which parses a command line and prints a report, and the library underneath it, which takes and returns ordinary Rust values. Chapter 29 goes back to the CLI side and chains everything — authoring, checking, merging, retracting, packing, threading, and rendering — into one continuous piece of work, the way a person actually does it.

TRY THIS

1. The round-trip test in `src/lib.rs` passes under `--no-default-features`. Build the crate that way yourself and confirm no binary appears. Then list what your dependency tree does **not** contain in that configuration, and say which absence you would care about most when adding this to an existing service.
2. The richer example builds two units with a real `grounds` dependency between them. Try building the dependent unit **first**, before the unit it grounds on exists. What does the API let you do, and at what point does the mistake surface?
3. Take the two-unit example and change the dependent unit's status to something its grounds cannot support. Where does that fail — at construction, at encoding, or at `check`? Explain why that is the right place.

ANSWERS

1. No `clap`, no TUI, no HTTP client, no async runtime, no TLS stack. Which matters most depends on your service, but the async runtime is usually the answer: pulling one into a synchronous codebase is invasive in a way an argument parser is not, and a library that quietly required `tokio` would rule itself out of a whole class of programs. The facade is synchronous with no I/O, which is what makes "add this to an existing service" a small decision.
2. The API lets you: a unit's `grounds` names uids, and nothing stops you naming one you have not built. The mistake surfaces when something walks the graph — a closure check, a `check` run, a bundle — and finds the reference unresolved. This is the same behaviour the surface syntax has, and for the same reason: documents are written in the order people think, not in dependency order, and forcing the second would make the API unusable for the case it exists to serve.

3. At `check`, not at construction or encoding. A unit is a value; you can build one that says anything, and encoding it is a mechanical translation that has no opinion. Rule M is a property of a unit **in relation to its grounds**, so it can only be evaluated once the graph is assembled — which is precisely why `check` is a separate pass rather than a constructor guard. The design lets you hold an invalid intermediate state and refuses to let you certify it.

WHAT YOU LEARNED

- `UnitCoreBuilder::new(...).build()` is where shape validation happens, once, at construction — every later operation on a built `UnitCore` (hashing, encoding, comparing) is infallible because of it.
- `to_cbor` / `from_cbor` / `canonical_uid` / `verify` are the same functions behind `fmt`'s round-trip guarantee, callable directly with no file and no subprocess.
- `Store::from_records` builds a store from values in memory; `check` runs the same ten passes against it that `smysl check` runs against a file — both are ordinary function calls, not a wrapper around the binary.
- Both examples in this chapter were compiled and run for real, against the facade, with `--no-default-features` — not printed from memory.

PART IX

A Complete Walkthrough

CHAPTER 29

29

Incident to Report: The Full Pipeline

Every other chapter in this book takes one command at a time. This one does not — it is a single incident, worked start to finish, with every command earning its place in the story rather than being demonstrated for its own sake. The scenario is a real gateway latency incident with two engineers investigating the same page from different angles, and it uses every part of the tool this book has covered: authoring, checking, merging, retracting, packing, threading, rendering, and a closing conformance check.

> **WHY**
>
> A chapter per command teaches you what each one does and leaves out the thing that actually makes the tool worth adopting: that the commands compose, and that the document survives every hop between them.
>
> The scenario here is the ordinary one. Two engineers page in on the same alert, look at different subsystems, and reach conclusions that partly contradict each other. Overnight a metric arrives that disproves one of the theories. In the morning somebody has to hand a leadership audience a brief that fits on a page — and be able to answer, three weeks later in a review, where any sentence in it came from.
>
> Done in prose, that last question has no answer by the second retelling; Chapter 1 measured how fast it stops having one. This chapter is the same incident with the answer still attached at the end.

The page, and the first write-up

`gateway.p99{region=us-east}` alerts. You are on call, and the first thing you do is not open a dashboard for someone else to read later — you write down what you see, as a document, while you still remember why you looked at what you looked at:

```
@doc smysl/0.1 {
  id: v/f8a
  intent: triage
  lang: en
  granularity: { profile: default }
  roots: [c/cause]
}

@evidence e/alpha-trace { status: measured, source: { kind: metric, ref:
"gateway.p99{region=us-east}", captured: 2026-07-22 } }
~ Gateway p99 in us-east rose to 2.4 s between 11:00 and 11:40.

Sampled at ten-second resolution from the edge collector, with the rise confined
to us-east and every other region flat inside its usual band for the whole
forty-minute window.

@claim c/cause { status: inferred, grounds: [e/alpha-trace] }
~ The us-east gateway is exhausting its upstream connection pool.

@evidence e/alpha-pool { status: measured, source: { kind: metric, ref:
"gateway.pool_wait{region=us-east}", captured: 2026-07-22 } }
~ Pool acquisition wait reached 1.9 s at the peak.

@claim c/alpha-scope { status: derived, grounds: [e/alpha-trace] }
~ No region other than us-east moved outside its band.

@claim c/pool-size-32 { status: derived, grounds: [e/alpha-pool] }
~ The pool was sized at 32 connections when the rise began.

@claim c/pool-size-64 { status: derived, grounds: [e/alpha-pool] }
~ The pool was sized at 64 connections when the rise began.

@rel e/alpha-pool --backs--> c/cause
@rel c/pool-size-32 --supersedes--> c/alpha-scope
@rel c/pool-size-64 --supersedes--> c/alpha-scope
```

You do not remember the exact pool size, so you write down both figures you saw quoted in chat, each `supersedes`-ing your scope claim — a decision to reconcile later, not now, and one this format lets you defer honestly rather than silently pick a number. Check it before moving on, the way Chapter 8 taught you to:

● ● ● *$ smysl check alpha.smy*

```
alpha.smy: 16 records, 6 units, 0 diagnostic(s)
```

Clean. Nothing here is **true** yet — `check` never promises that — but the document is internally consistent: every status respects its grounds, every reference resolves.

A second read on the same incident

A teammate is paged too, and investigates independently — no coordination, no "who's looking at what" thread, because none is needed yet:

```
@doc smysl/0.1 {
  id: v/f8b
  intent: triage
  lang: en
  granularity: { profile: default }
  roots: [c/cause]
}

@evidence e/beta-deploy { status: measured, source: { kind: file, ref: "deploy/
gateway-7.1.manifest", captured: 2026-07-22 } }
~ Gateway 7.1 reached us-east at 10:58, two minutes before the rise.

@claim c/cause { status: inferred, grounds: [e/beta-deploy] }
~ The us-east gateway regressed because 7.1 shortened the upstream timeout.

@evidence e/beta-retries { status: measured, source: { kind: metric, ref:
"gateway.retries{region=us-east}", captured: 2026-07-22 } }
~ Upstream retries rose eleven-fold over the same forty minutes.

@claim c/timeout-not-culprit { status: derived, grounds: [e/beta-retries] }
~ Retries were already elevated before 7.1 reached us-east.

@rel e/beta-retries --backs--> c/cause
@rel c/timeout-not-culprit --rebuts--> c/cause { weight: 0.6 }

@thread t/beta-brief { schema: brief, owner: "model:beta" }
~ 7.1 shortened the upstream timeout; the retry evidence is not unambiguous.
  bottom-line → c/cause
  risk → c/timeout-not-culprit
```

Notice: your teammate already drafted their own `brief` thread while writing this up, and already recorded an objection to their own leading theory. Checked clean on its own:

● ● ● *$ smysl check beta.smy*

```
beta.smy: 14 records, 5 units, 0 diagnostic(s)
```

Two independently-authored, individually clean documents about the same incident, both using the label `c/cause` for what each of you believes is **the** root cause — and they disagree. This is exactly what `merge` exists for:

● ● ● *$ smysl merge alpha.smy beta.smy -o merged1.cbor*

```
alpha.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy over
b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
beta.smy: contention k/c3xa27zib4d7t5rme4xekauzh4t over
b3:t65rff76bcbxnwzw4oxzcadthe (2 positions, live-rebuttal)
beta.smy: contention k/cboxvjp36tnst3vmpsoo2mmvhqu over
b3:cjixk2inyftvxj55d53w2ivxej (2 positions, label-collision)
```

Three contentions, three different reasons, and none of them is `merge` failing — they are `merge` doing its job. Read them in order:

- **`supersession-fork`** over the scope claim: your own two pool-size claims both `supersedes` the same target. You raised this yourself, in the first write-up.
- **`live-rebuttal`** over your teammate's root-cause claim: their own `rebuts` edge landed both sides of that disagreement in the same graph.
- **`label-collision`**: `c/cause` in your file and `c/cause` in theirs point at two different uids. Same nickname, two different real claims — exactly the situation `Label` (Appendix D) warns about.

`check` on the result reports no **errors** — a contention is a materialised disagreement, not an invalidity:

● ● ● *$ smysl check merged1.cbor*

```
merged1.cbor: 30 records, 11 units, 0 diagnostic(s)
```

> WHAT'S NEXT, AND WHY
>
> Nothing has been decided yet — the merge recorded three open disagreements rather than picking winners for any of them. The next step is finding evidence that actually resolves one.

New evidence, staged and merged

Twenty minutes later, someone rolls gateway 7.1 back, and the incident channel gets a note about it. There is no model provider configured in this environment to run `smysl ingest` against live, so — as Chapters 10 and 13 do when the same gap comes up — this step is honestly hand-simulated at

exactly the point `ingest` would hand off: a reviewed batch sitting in `.smysl/staged.smy`, waiting for `merge --staged` to confirm it, precisely as rule S requires.

```
@evidence e/rollback { status: measured, source: { kind: file, ref: "deploy/
gateway-7.1-rollback.log", captured: 2026-07-22 } }
~ Rolling gateway 7.1 back at 11:52 returned p99 to its pre-incident baseline
within four minutes.

@claim c/rollback-confirms { status: derived, grounds: [e/rollback] }
~ The rollback's effect confirms the 7.1 timeout change as the cause, not the
connection pool.
```

Committing it folds straight into the same `merge` you already ran:

```
● ● ●  $ smysl merge alpha.smy beta.smy --staged -o merged2.cbor
```

```
alpha.smy: contention k/co7i7feme3q3sed4l63dkcgmmuy over
b3:v5sjn55ujeflrdyu5qycgmtvz3 (2 positions, supersession-fork)
beta.smy: contention k/c3xa27zib4d7t5rme4xekauzh4t over
b3:t65rff76bcbxnwzw4oxzcadthe (2 positions, live-rebuttal)
beta.smy: contention k/cboxvjp36tnst3vmpsoo2mmvhqu over
b3:cjixk2inyftvxj55d53w2ivxej (2 positions, label-collision)
smysl merge: committed 2 staged record(s)
```

```
● ● ●  $ smysl check merged2.cbor
```

```
merged2.cbor: 34 records, 13 units, 0 diagnostic(s)
```

The rollback evidence does not automatically resolve anything in the graph by itself — it is just two more units. What it gives **you** is the confidence to act on the `label-collision` contention: alpha's pool-exhaustion theory is now the one to retire.

> **WHAT'S NEXT, AND WHY**
>
> With the deciding evidence in hand, the next step is not another merge — it is withdrawing the theory the evidence ruled out, and doing that honestly means reporting what else would be affected before touching anything.

Retracting the disproven theory

`retract --dry-run` reports the blast radius without applying anything, and it is safe to run before you have even decided who is retracting:

● ● ● *$ smysl retract --dry-run b3:cjixk2inyftvxj55d53w2ivxej merged2.cbor*

```
merged2.cbor: retracting b3:cjixk2inyftvxj55d53w2ivxej would reach 1 unit(s),
orphaning 0
```

One unit — itself — and nothing orphaned. That is worth reading as information, not just a number: nothing else in the graph was ever built on top of alpha's pool-exhaustion claim. It was a dead end from the start, not a foundation, which is exactly why retiring it now is safe. Applying it for real, without `--dry-run`, hits something instructive:

● ● ● *$ smysl retract b3:cjixk2inyftvxj55d53w2ivxej merged2.cbor*

```
merged2.cbor: retracting b3:cjixk2inyftvxj55d53w2ivxej would reach 1 unit(s),
orphaning 0
smysl retract: origin requires 1 distinct agents, got 0
```

`retract`'s default authority is `origin` — only an agent that **attested** the unit may retract it. Attestations have no surface syntax (they are authored programmatically, never by hand), so a unit written straight into a `.smy` file has none, and `origin` can never be satisfied for it. That is not a bug to work around quietly; it is the tool refusing to guess who has standing to retract something. Naming an issuer and relaxing the authority is the honest way through it:

● ● ● *$ smysl retract b3:cjixk2inyftvxj55d53w2ivxej merged2.cbor --as human:oncall --authority any --reason "rollback confirms 7.1 timeout, not pool exhaustion"*

```
merged2.cbor: retracting b3:cjixk2inyftvxj55d53w2ivxej would reach 1 unit(s),
orphaning 0
merged2.cbor: 1 unit(s) now read as unfounded
```

One thing to know before relying on this in a pipeline: this build's `retract` reports the blast radius and the resulting effective status, but does not itself write the mutated store to `--output` — the retraction lives only in that process's memory. The way to carry it forward is the same way everything else here travels: as a record. `RelKind::Retracts` is an ordinary relation, so a one-line file naming it and folding it back in with `merge` persists exactly what `retract` just reported:

```
@rel b3:cjixk2inyftvxj55d53w2ivxejoderrdj73hnpaprphlpzqpjhyq --retracts-->
b3:cjixk2inyftvxj55d53w2ivxejoderrdj73hnpaprphlpzqpjhyq
```

● ● ● *$ smysl merge merged2.cbor retract.smy -o merged3.cbor && smysl check merged3.cbor*

```
merged3.cbor: 35 records, 13 units, 0 diagnostic(s)
```

> **WHAT'S NEXT, AND WHY**
>
> The store now honestly reflects what happened: one theory retired, its blast radius on record, nothing orphaned. What it is still missing is the thing a postmortem is actually **for** — a conclusion.

A finding, and a budget-bounded pack

Everything so far has been claims and evidence. A postmortem needs a `finding` — the unit type that says what the incident actually concluded, grounded in the claim the evidence now supports:

```
@finding f/root-cause { status: inferred, grounds:
[b3:t65rff76bcbxnwzw4oxzcadthe2s35hisuingd2p6gieh6ayejea] }
~ The gateway 7.1 timeout regression caused the us-east p99 spike; the pool-
exhaustion theory is retracted.
```

● ● ● *$ smysl merge merged3.cbor finding.smy -o merged4.cbor && smysl check merged4.cbor*

```
merged4.cbor: 37 records, 14 units, 0 diagnostic(s)
```

Fourteen units now sit in this store — both original theories, the evidence behind each, the rollback confirmation, the retraction, and the finding. Nobody reading a report wants all fourteen. `pack --focus` fits the graph to a budget around the claim that matters now, and it is the moment to see, concretely, whether retiring the wrong theory actually mattered to what gets shipped:

● ● ● *$ smysl --format surface pack --budget 120 --explain --focus b3:t65rff76bcbxnwzw4oxzcadthe merged4.cbor*

```
b3:t65rff76bcbxnwzw4oxzcadthe @L1  C5  in focus
b3:wo5nb45msupuxfptfxpbh5q4ws @L0  C3  rebuts b3:t65rff76bcbxnwzw4oxzcadthe
b3:34xv6klbhpxqsqsvhm5s3xuegn @L0  C2  ground of b3:t65rff76bcbxnwzw4oxzcadthe
b3:cjixk2inyftvxj55d53w2ivxej dropped: budget
b3:hpffca2bw76gvsrc6tjvr2q6uw dropped: budget
b3:bwgbs3amtoatm36pawri7snlz4 dropped: low-value
...
merged4.cbor: 3 of 13 unit(s), 112 of 120 tokens, greedy mode, gap 0.154
```

Read the three kept lines the way Chapter 18 taught you to: the surviving claim (`C5`, in focus), its rebuttal (`C3` — "retries were already elevated" travels with the claim it objects to, not silently dropped), and its ground (`C2`). Alpha's retracted claim is not merely low-priority here — it is `dropped: budget`, competing on density like anything else and losing, the same as beta's own weaker units. Retraction changed what the graph **means**; a tight, well-chosen budget is what keeps a resolved dead end from crowding out the version worth shipping.

> **WHAT'S NEXT, AND WHY**
>
> A budget-bounded selection is for a prompt or a quick read — it truncates by design. A report a person reads front to back needs an actual order: a thread.

Deriving and rendering the brief

`thread --derive` builds that order deterministically from the graph itself:

● ● ● *$ smysl thread --derive brief --explain --id t/postmortem --as human:oncall merged4.cbor*

```
merged4.cbor: bottom-line  1 of 1..1
merged4.cbor: support      3 of 1..3
merged4.cbor: risk         1 of 0..2
merged4.cbor: ask          0 of 0..1
merged4.cbor: 9 unit(s) not selected
```

Every required role filled — `bottom-line` from the new finding, three units backing it as `support`, the surviving rebuttal carried into `risk`. This is the moment to check something before building further on it, in the spirit of this whole book: a thread derived over units that lost their labels crossing a CBOR merge is correct in memory, but re-serialising it to surface text and re-parsing it is not yet a clean round trip in this build when the units it references have no label — a real rough edge, not a hidden one, and exactly why `check` exists between every step rather than only at the end. The safe path here is the one your teammate already handed you: they filed `t/beta-brief` back when they wrote `beta.smy`, and because a thread's steps are stored by uid from the moment it is authored, it survived every merge since untouched. Render it now that the story is settled:

● ● ● *$ smysl render --thread t/beta-brief --profile exec --target markdown merged4.cbor*

```
# 7.1 shortened the upstream timeout; the retry evidence is not unambiguous.

*brief · profile exec*
*for engineering leadership*

## bottom-line

≈ The us-east gateway regressed because 7.1 shortened the upstream timeout.

A shortened timeout turns a slow upstream into a retried one, and retries are
what fill the queue, so beta reads the same latency curve as a consequence of
the release rather than of the pool being undersized.

> **contested** — k/c3xa27zib4d7t5rme4xekauzh4t: contested, 2 position(s) on
record

## risk

⊢ On the other hand, retries were already elevated before 7.1 reached us-east.

> **contested** — k/c3xa27zib4d7t5rme4xekauzh4t: contested, 2 position(s) on
record

---

**Open contentions:** k/c3xa27zib4d7t5rme4xekauzh4t, k/
co7i7feme3q3sed4l63dkcgmmuy
```

Two things worth noticing. First, the `live-rebuttal` contention over the bottom-line claim survives rendering under `exec` — that is rule V2, and it means a reader of the **exec** brief still sees that the risk section is a live, recorded objection, not editorial hedging. Second, `k/co7i7feme3q3sed4l63dkcgmmuy` — the `supersession-fork` over the pool size — is **still open**. Retiring alpha's root-cause theory did not resolve the smaller disagreement about how big the pool actually was; that is a second, independent decision this report is honestly still waiting on, and the render says so rather than letting it quietly disappear.

> **WHAT'S NEXT, AND WHY**
>
> The report is written. The last thing worth doing before handing it off is asking the store the same question `check` has been answering all along, but at the strength a handoff deserves.

Certifying the store

Every step in this incident went through `check` on the way. The closing move is asking a stronger question of the **final** result: not just "is this valid" but "is this safe to hand to someone else to read, merge, or build on":

```
$ smysl check --conformance C-Full merged4.cbor
```

```
merged4.cbor: C-Full: pass
merged4.cbor: 37 records, 14 units, 0 diagnostic(s)
```

`C-Full` is every conformance class at once — safe to read, safe to consume, safe to author into, safe to merge. The store passes it with the retraction, the finding, and both open contentions all still on record. Nothing about certifying the store required resolving `k/co7i7feme3q3sed4l63dkcgmmuy` first, and that is deliberate: conformance is a statement about the store's **shape**, never about whether every question in it has been answered.

WHAT'S NEXT, AND WHY

This chapter moved quickly through commands the rest of the book gave a full workflow each — Appendix A has the complete flag reference for every one of them when you need the exact syntax again. Appendix B and Appendix C are the same kind of quick lookup for every diagnostic code and exit code this incident could have hit but did not. And where this walkthrough moved past **why** the format is shaped the way it is, or exactly how a construct like `--supersedes--` or `granularity` is spelled, `SMYSL_RATIONALE.typ` and `SMYSL_FORMAT_GUIDE.typ` are the two sibling documents that slow back down.

TRY THIS

1. Work the whole chapter's scenario yourself, from the two write-ups to the final `check --conformance`. Then answer the question the chapter opens with: pick any sentence in the rendered brief and say, using only the store, where it came from and how strong it is entitled to be.
2. At the retraction step, a theory is disproven by new evidence. Run the retraction with `--dry-run` first and note what it says it will reach. Now consider the counterfactual: if this incident had been handled in a chat thread and a shared document, what would have happened to the claims that rested on the disproven theory?
3. The chapter ends with a conformance check rather than a render. Argue for that ordering — why is certifying the store the last step rather than producing the artifact?

ANSWERS

1. Every sentence traces to a unit, every unit carries a status and its grounds, and `trace` walks the chain to whatever measurement or citation sits at the bottom. That is the entire claim of this book made concrete: not that the brief is **better written** than a prose one, but that it is answerable. If you found a sentence you could not trace, it came from the thread's framing rather than a unit, and that is worth noticing too.
2. Nothing would have happened to them, which is the problem. They would have remained in the document, reading exactly as confident as before, because prose has no mechanism by which withdrawing one paragraph reaches the paragraphs that depended on it. Somebody would have had to remember — weeks later, under time pressure — which conclusions rested on the theory that turned out to be wrong. `retract --dry-run` answers that question by walking `grounds`, before you commit to anything.
3. Because the artifact is disposable and the store is not. A rendered brief can be regenerated at any time from the store — `render` is pure, so the same store gives the same bytes forever. The store is the thing that has to be right, and certifying it last means you are asserting the **final** state is consumable, after every merge, retraction and derivation has landed. Checking before the last edit would certify something that no longer exists.

WHAT YOU LEARNED

- Two independently-authored, individually clean documents about the same incident can disagree, and `merge` records exactly how — `supersession-fork`, `live-rebuttal`, and `label-collision` are three different shapes of the same underlying event, none of them an error.
- `retract --dry-run` is always safe and needs no authority; applying a retraction for real defaults to `origin`, which surface-authored units can never satisfy — `--as` and `--authority any` are how a human takes explicit responsibility for one instead.
- A retraction's blast radius is a fact about the graph worth reading even when it is small: zero orphaned units meant the retired theory was never load-bearing.
- `pack --focus` at a real budget is where a retraction's consequences become visible — the retracted claim did not just rank lower, it lost outright to units that still mattered.
- A thread that was authored with labels intact and carried through every merge survives cleanly; deriving a fresh one over units that already lost their labels is correct in memory but not yet guaranteed to round-trip through surface text in this build — check what you built before you build the next thing on it.
- `check --conformance C-Full` on the final store is a statement about safety to hand off, not a claim that every open question — like the still-unresolved pool size — has been settled.

APPENDIX A

A

Full Command and Flag Reference

Every flag below is transcribed from `fn cli()` in `src/main.rs` — the single place the command surface is declared, and therefore the only place this table can drift out of sync with the binary. Each subcommand heading repeats three facts from the command table in `src/main.rs` (`§23`): its one-line description, its **purity** (`pure` — a bit-reproducible function of its inputs; `mixed` — pure except for one option; `model-dependent` — the only kind that can reach off the machine), and the delivery phase that wired it up. All all twenty-two subcommands are covered.

`ui` was described here as a stub through 0.5.0. It is not, and was not: `smysl-tui` is a working crate with its own test suite, the `tui` feature is in the **default** set, and the command runs against any store given a terminal. The claim was written once and then believed — including by me, twice, while planning work around removing it. This appendix's opening paragraph says it cannot drift from the binary; that is the second time it had.

Three of them — `import`, `relink` and `compact` — were absent from this appendix until 0.4.0. They were wired in SM-P15 and the appendix was never extended to match, which is exactly the drift the paragraph above claims this table cannot have. It could, and it did.

Global flags

These apply to every subcommand, in any position on the command line.

FLAG	MEANING
`-C, --config FILE`	Configuration file.

FLAG	MEANING
`-s, --store PATH`	Store path; `-` reads stdin (rule P).
`-o, --output PATH`	Output path; defaults to stdout.
`--format surface|cbor`	Output form; defaults to `cbor` on a non-TTY stdout (rule P).
`--strict`	Treat warnings as errors.
`--offline`	Hard-fail rather than send anything off the machine.
`--no-color`	Disable colour.
`--noprogress`	Disable progress bars, whatever the terminal is.
`--json`	Machine-readable output.
`-q, --quiet`	Suppress non-error output.
`-v, --verbose`	Increase verbosity; repeatable (`-vv` , `-vvv` , …).
`--seed-check`	Assert this invocation is bit-reproducible (rule D).

fmt

Canonicalise surface text and verify the round-trip. Pure · SM-P2.

FLAG	VALUE	MEANING
`--check`	—	Exit 3 if reformatting would change bytes.
`--write`	—	Rewrite files in place.
`FILE …`	positional, 0 or more	Files to format; `-` or none reads stdin (rule P).

check

Run the check pipeline over a store. Pure · SM-P4.

FLAG	VALUE	MEANING
`--conformance`	`CLASS`	Assert the store is consumable at a conformance class.
`--as`	`SCHEMA` (repeatable)	Report fidelity for a consumer implementing these schemas.
`--granularity`	—	Report the granularity distribution of the store.
`--pass`	`NAME` (repeatable)	Run only these passes.
`FILE …`	positional, 0 or more	Stores to check; `-` or none reads stdin (rule P).

pack

Budget-bounded, closure-complete selection. Pure · SM-P9.

FLAG	VALUE	MEANING
`--budget`	`N` (required)	Token budget, counted with the recorded estimator.

FLAG	VALUE	MEANING
`--focus`	`UID` (repeatable)	Units that must reach L1; packing fails if they cannot.
`--lod`	`auto|L0|L1|L2`	Cap every unit at this level.
`--explain`	—	Say which constraint put each unit in.
`--tokenizer`	`ID`	Cost model; recorded in the packinfo either way (D-2).
`--mode`	`greedy|exact`	`exact` proves optimality by branch and bound; needs the `exact-pack` feature.
`--query`	`TEXT`	Focus on what this query finds, instead of naming uids.
`--query-limit`	`N`	How many hits `--query` focuses on; 3 by default.
`PATH`	positional	Store to pack.

merge

Join-semilattice union; materialise contentions. Pure · SM-P6.

FLAG	VALUE	MEANING
`--policy`	`latest|all|contend`	Supersession policy.
`--retraction`	`strict|advisory|ignore`	Retraction policy.
`--staged`	—	Commit `.smysl/staged.smy` into the store.
`--fail-on-contention`	—	Exit 5 when the merged store carries a contention.
`--max-contentions-per-agent`	`N`	Warn when one merge raises more than N contentions.
`STORE …`	positional, 1 or more, required	Stores to merge; `-` reads stdin (rule P).

diff

Partition uids across stores or hops. Pure · SM-P7.

FLAG	VALUE	MEANING
`--hop`	`A..B`	Partition units across a hop range instead of comparing stores.
`--by-agent`	—	Attribute each change to the agents responsible.
`--recipe`	—	Flag whether the prompt changed or the content did (D-8).
`STORE …`	positional, 1 or more, required	One store for `--hop`, two to compare.

trace

Walk provenance or evidential support. Pure · SM-P7.

FLAG	VALUE	MEANING
`UID`	positional, required	The unit to trace from.
`--depth`	`N`	How far back to walk.
`--parents`	—	Causal: attestation parents and supersession.
`--grounds`	—	Evidential: grounds and deps (the default).
`--both`	—	Both walks at once.
`--agents`	—	Name the agents behind each step.
`PATH`	positional	Store to trace within.

view

Define or print a view. Pure · SM-P7.

FLAG	VALUE	MEANING
`--roots`	`UID` (repeatable)	Roots of the view.
`--id`	`NAME`	View identifier.
`--threads`	`ID` (repeatable)	Threads to associate with the view.
`--requires`	`SCHEMA` (repeatable)	Schemas the view requires a consumer to implement.
`PATH`	positional	Store to read.

bundle

Emit the reachable closure of a view. Pure · SM-P7.

FLAG	VALUE	MEANING
`--view`	`ID`	Which view to bundle; the first if omitted.
`--include-retracted`	—	Keep units that have been retracted.
`PATH`	positional	Store to bundle from.

thread

Derive, refine, list, show, or import threads. Mixed (pure except `--refine`, which is not yet wired) · SM-P11.

FLAG	VALUE	MEANING
`--derive`	`analysis|narrative|brief|qa|plan`, 0 or 1 args	Derive a thread from the graph; the schema may follow here or in `--schema`.
`--schema`	`analysis|narrative|brief|qa|plan`	Schema to derive under (§23 spells it this way).

FLAG	VALUE	MEANING
`--only`	—	Emit the thread record alone rather than the store it belongs to.
`--list`	—	List the threads the store already holds.
`--show`	`ID`	Print one thread, step by step.
`--id`	`T`	Thread id for the derived thread.
`--as`	`AGENT`	Owner of the derived thread.
`--scope`	`UID` (repeatable)	Derive over these units only.
`--arity`	`ROLE=N` (repeatable)	Override how many units a role may hold.
`--explain`	—	Say which role each unit took and what repair added.
`PATH`	positional	Store to derive from.

salience

Report derived salience with per-term breakdown. Pure · SM-P8.

FLAG	VALUE	MEANING
`--top`	`N`	Show only the N highest-scoring units.
`--explain`	`UID`	Break one unit's score into its three terms.
`--weights`	`C,R,T`	Override the centrality, corroboration and role weights.
`--seed`	`UID` (repeatable)	Personalise against these units; the view roots by default.
`PATH`	positional	Store to score.

find

Rank units against a query, lexically. Pure · 0.5.0.

FLAG	VALUE	MEANING
`QUERY`	positional, required	What to search for.
`-n, --limit`	`N`	Maximum hits to return; 10 by default.
`--kind`	`TYPE` (repeatable)	Restrict to this kernel type.
`--min-status`	`STATUS`	Restrict to units at or above this status.
`PATH`	positional	Store to search.

The first command whose phase is a release rather than an `SM-Pnn`. The delivery phases ended at SM-P15, and writing `SM-P16` here would invent one to satisfy a naming pattern.

retract

Retract a unit; report the blast radius first. Pure · SM-P6.

FLAG	VALUE	MEANING
`UID`	positional, required	The unit to retract; the display form is resolved as a prefix.
`--dry-run`	—	Report the blast radius without applying anything.
`--as`	`AGENT` (repeatable)	The agent(s) issuing the retraction.
`--reason`	`TEXT`	Why.
`--authority`	`A`	`origin | any | quorum:N`.
`PATH`	positional	Store to retract from.

render

Thread plus profile to artifact. Pure · SM-P12.

FLAG	VALUE	MEANING
`--thread`	`ID`	Thread to render; the store's only thread by default.
`--profile`	`NAME`	Built-in profile name, or a path to a profile file.
`--target`	`markdown|md|typst|html|slides|json|text`	Output format.
`--lod`	`L0|L1|L2`	Cap every block at this level, whatever the profile says.
`--contentions`	`show|suppress`	Override the profile's rule V2 setting.
`--as`	`AGENT`	Whose thread, when several agents hold one under that id.
`--profiles`	—	List the built-in profiles and exit.
`PATH`	positional	Store to render from.

import

Tabular readings to measured units, without a model. Pure · SM-P15.

FLAG	VALUE	MEANING
`--key`	`COL` (repeatable)	Columns naming the reading rather than carrying its value.
`--kind`	`file|metric|tool|url|doc`	Source kind recorded on each unit; `file` by default.
`PATH`	positional, required	Delimiter-separated file to import; `-` reads stdin (rule P).

relink

Re-point references onto superseded units. Pure · SM-P15.

FLAG	VALUE	MEANING
`--dry-run`	—	Report what would be re-pointed without emitting anything.
`PATH`	positional	Store to relink; `-` reads stdin (rule P).

compact

Drop superseded units nothing needs; never in place. Pure · SM-P15.

FLAG	VALUE	MEANING
`--dry-run`	—	Report what would be dropped without emitting anything.
`PATH`	positional	Store to compact; `-` reads stdin (rule P).

ingest

Prose or data to staged units. Model-dependent · SM-P14.

FLAG	VALUE	MEANING
`FILE`	positional	Document to ingest; `-` reads stdin.
`--rung`	`computed|document|web|model`	Trust rung of the source; caps what units may claim (rule T).
`--granularity`	`P`	Granularity profile the units are produced under.
`--path`	`auto|surface|json-ast`	Override the path D-9 would choose.
`--repair`	`N`	Repair attempts before a span degrades to opaque prose.
`--yes`	—	Commit the staged batch instead of exiting 10.
`--dry-run`	—	Report what would be sent and to whom; make no call.

attest

Semantic checks that require a model. Model-dependent · SM-P14.

FLAG	VALUE	MEANING
`--what`	`gist-coverage|warrant-plausibility|granularity`	Which semantic question to ask.

FLAG	VALUE	MEANING
`--sample`	`N` (or `all`)	How many units to ask about; `all` for the whole store.
`PATH`	positional	Store to attest.

providers

List providers, capabilities, and what would egress. Pure · SM-P13.

FLAG	VALUE	MEANING
`--probe`	—	Contact each provider and report what it actually is.
`--models`	—	List each provider's installed models.
`--tasks`	—	Report which tasks would send content off the machine.

usage

Token and cost ledger. Pure · SM-P13.

FLAG	VALUE	MEANING
`--by`	`provider|task|run|model`	How to group the ledger.
`--since`	`MS`	Only calls at or after this epoch-millisecond timestamp.
`--reset`	—	Discard the ledger.

ui

Terminal UI. Pure · SM-P15.

FLAG	VALUE	MEANING
`PATH`	positional	Store to browse; `-` reads stdin (rule P).

No flags of its own beyond the global set — the interface is the terminal, not the command line. It refuses to start when stdout is not a terminal, rather than emitting escape codes into a pipe.

`tui` left the default feature set in 0.6.0. It works and it is tested; what it does not justify is `ratatui` and `crossterm` in every build, including one an embedder installs to call the library and nothing else. Build with `--features tui` to get it; without it the command says so rather than pretending.

reindex

Rebuild the derived index from the log alone. Pure · SM-P3.

FLAG	VALUE	MEANING
`--verify`	—	Compare the rebuilt index against the sidecar instead of writing it.
`PATH`	positional	Store to reindex.

APPENDIX B

B

Full Diagnostic Code Reference

Every diagnostic `smysl` can emit has a stable code, declared once in the `registry!` macro invocation in `crates/smysl-core/src/diag.rs` and never reused. The registry is single-sourced: wire string, severity, group, and one-line meaning all come from that one place, and a workspace test (`registry_matches_appendix_d_size`) asserts the count below stays at 49. Codes are grouped exactly as the source groups them; group membership is reporting structure only; it carries no weight on the wire.

GROUP	COUNT	COVERS
Parse	7	Surface and CBOR parsing; deterministic-encoding and float-quantisation rules.
Identity	7	Dangling references, dependency cycles, hash and uid integrity, index staleness.
Lod	7	Granularity and level-of-detail shape rules — gist, body, detail, closure.
Epistemics	6	Rule M and rule T — status versus grounds, versus the authoring rung.
Merge	6	Retraction, supersession, and the contentions merge materialises.
PackRender	5	Budget feasibility and render-time rule V1/V2 enforcement.
Extension	4	Rule X — unknown schemas and relation kinds at the extension boundary.
Provider	7	The model boundary — reachability, offline, context limits, ingest repair.

Parse — surface and codec

CODE	SEV.	MEANING
`SMY-E001`	error	Surface parse error.
`SMY-E002`	error	Unsupported kernel major version.

CODE	SEV.	MEANING
`SMY-E003`	error	Unsupported format version.
`SMY-E004`	error	Malformed CBOR envelope — or, since 0.3, nesting deeper than 128. The reader walks containers recursively, so an unbounded depth let deeply nested input overflow the stack and **abort the process**, which is worse than an error because it cannot be caught. 128 is far above anything real: the deepest shape the kernel defines is three levels.
`SMY-W014`	warning	Unknown envelope type code — preserved verbatim, skipped semantically. Reported by `check` since 0.2; before that the code existed and nothing emitted it, so an unknown record was preserved in silence.
`SMY-E080`	error	Non-deterministic encoding (key order, indefinite length, non-shortest int, null optional, non-NFC text).
`SMY-E081`	error	Float not binary32 or not quantised to 1/1024.

Identity — hash and integrity

CODE	SEV.	MEANING
`SMY-E060`	error	Dangling reference.
`SMY-E061`	error	Cycle in deps.
`SMY-W062`	warning	Cycle in causes or sequences.
`SMY-E070`	error	Hash mismatch — recomputed uid differs from stored uid.
`SMY-E071`	error	Truncated uid in a canonical record.
`SMY-E072`	error	Ambiguous uid prefix.
`SMY-W110`	warning	Stale or corrupt index — rebuilding.

Lod — granularity and shape

CODE	SEV.	MEANING
`SMY-E020`	error	L1 closure violation — body references a uid absent from deps or grounds.
`SMY-E021`	error	Missing gist.
`SMY-E022`	error	Gist exceeds `l0_max`.
`SMY-E023`	error	`detail` without `body`.
`SMY-W024`	warning	Gist appears to depend on body (heuristic; confirm via `attest`).
`SMY-E040`	error	Multi-assertion body under single-assertion admission.
`SMY-W041`	warning	Body outside `l1_range`.

Epistemics — rules M and T

CODE	SEV.	MEANING
`SMY-E030`	error	Rule M violation — status exceeds weakest ground.
`SMY-E031`	error	`derived` / `inferred` with empty grounds.
`SMY-E032`	error	`measured` / `cited` without source.
`SMY-E033`	error	Rule T violation — status exceeds the ceiling for the attestation's rung.
`SMY-E034`	error	`unfounded` authored.
`SMY-W035`	warning	`measured` with `op: Authored` rather than `Imported`.
`SMY-W036`	warning	Rule M applied at ingest — status lowered to what its grounds support.

Merge — retraction, supersession, contention

CODE	SEV.	MEANING
`SMY-E050`	error	Orphaned grounds — all grounds retracted under strict.
`SMY-E051`	error	Retraction authority not satisfied.
`SMY-W052`	warning	Retracted unit retained under advisory.
`SMY-W053`	warning	Concurrent supersession materialised as a contention.
`SMY-W054`	warning	Label and uid do not correspond one to one — two labels for one unit, or one label for two.
`SMY-W055`	warning	Agent contention rate exceeds `--max-contentions-per-agent`.

PackRender — budget and render-time rules

CODE	SEV.	MEANING
`SMY-E200`	error	Pack infeasible — C3/C4/C5 unsatisfiable; reports minimum feasible budget.
`SMY-E201`	error	Focus unit absent from store.
`SMY-W202`	warning	Greedy mode above `exact_threshold`; optimality gap reported.
`SMY-E210`	error	Rule V1 — profile lacks a rendering for some status.
`SMY-W211`	warning	Rule V2 — contentions suppressed; recorded in output metadata.

Extension — rule X

CODE	SEV.	MEANING
`SMY-W010`	warning	Unknown schema — degraded fidelity (rule X). Two cases: a type this **build** does not know, reported always; and a type the named `--as` consumer does not implement, reported only when one is named. The first covers a kernel type added by a later version, which decodes and round-trips rather than failing.
`SMY-E011`	error	Rule X violation — unrecognised payload dropped on re-emission.
`SMY-E012`	error	Extension schema attempts to weaken a kernel rule.
`SMY-W013`	warning	Unknown relation kind treated as `elaborates` for closure.

Provider — the model boundary and ingest

CODE	SEV.	MEANING
`SMY-E300`	error	Provider unreachable, no fallback configured.
`SMY-E301`	error	Offline violation.
`SMY-E302`	error	Context window exceeded after chunking.
`SMY-W303`	warning	Structured output unsupported; fell back to the surface path.
`SMY-W304`	warning	Span unrepairable; degraded to opaque prose (rule I).
`SMY-W305`	warning	Token count estimated rather than provider-reported.
`SMY-E307`	error	Attributed quote does not occur in the source text.
`SMY-W308`	warning	Attributed quote occurs only loosely — elided or reworded.

Both of the codes this appendix once listed as unreachable have been resolved in 0.6.0, because "documented as unreachable" is a holding pattern rather than a decision.

`SMY-W305` is now emitted by `usage`. The ledger had recorded whether each token count came from the provider or from our own estimate since the provider layer landed, and nothing ever surfaced it — so a total built partly from an estimate looked exactly like one the provider reported. The claim in this appendix that the information "already reaches you through the usage totals line" was simply wrong.

`SMY-W306` is deleted. It described a usage threshold that does not exist and never did, and inventing the feature to justify the code would have been the wrong way round. A code nobody can trigger is worse than a missing one, because a reader waits for it.

APPENDIX C

C

Exit Codes

Twelve codes, `0` through `11`, declared in `ExitCode` in `crates/smysl-core/src/error.rs`. They are part of the contract: stable across minor versions, so a pipeline can branch on exactly what happened rather than parsing stderr. A workspace test (`exit_codes_are_contiguous_and_unique`) asserts the set never grows a gap or a duplicate.

CODE	NAME	MEANS	SEEN FROM
`0`	`Success`	Nothing to report.	Any command that completed cleanly.
`1`	`Failure`	A generic failure — see stderr.	I/O errors, an absent store, a missing thread.
`2`	`Usage`	The command line itself was wrong.	A bad flag value, a missing required argument.
`3`	`CheckErrors`	`check` (or `fmt --check`) found something at or above the failure threshold.	`check`, `fmt --check`, and any command whose input fails to parse.
`4`	`PackInfeasible`	No selection satisfies the budget and focus constraints (`SMY-E200`).	`pack --budget` too small for the mandatory floor.
`5`	`Contentions`	`merge --fail-on-contention` found disagreement.	`merge` with that flag set, when the join raises a contention.

CODE	NAME	MEANS	SEEN FROM
`6`	`Provider`	A model provider failed.	`providers --probe`, `ingest`, `attest` against an unreachable or erroring provider.
`7`	`Offline`	`--offline` blocked a call that would have left the machine.	Any model-dependent command run with `--offline` against a hosted provider.
`8`	`UnsupportedVersion`	The document's format or kernel major isn't supported by this build.	Parsing a store from a future or otherwise incompatible format/kernel major.
`9`	`HashVerification`	A recomputed hash didn't match a stored uid.	`fmt`'s round-trip check, `reindex --verify`, integrity failures.
`10`	`Staged`	Output is staged, awaiting `merge --staged` to confirm it.	`ingest`, unless `--yes` is given.
`11`	`StagedWithCorrections`	The same, and rule M lowered at least one unit — the model claimed more than its grounds support and was corrected. Not a failure. New in 0.2; a script testing `= 10` should test `>= 10`.	`ingest`, including under `--yes`.

APPENDIX D

D

Glossary

A reference restatement of every term boxed elsewhere in this manual — tight, not a first introduction. Where a chapter's own wording differs slightly, trust the chapter for nuance and this page for a quick reminder of what the word means.

TERM Store

Whatever `smysl` is operating on: a `.smy` surface file, a CBOR log (what `merge -o` writes), or `-` for stdin. Almost every command takes one, either as a trailing positional argument or via the global `-s` / `--store` flag.

TERM Purity

A command's classification as **pure** (a bit-reproducible function of its inputs — same bytes in, same bytes out, on any machine, forever), **mixed** (pure except for one option), or **model-dependent** (the only kind that can reach off the machine). Only `ingest` and `attest` are ever model-dependent; `thread` is mixed only because of an unwired `--refine`.

TERM **Canonical form**

The one, unique byte-for-byte spelling of a set of records — quoting decided by content rather than by how you typed it, granularity always expanded to its full field set. Hashes are computed over CBOR, never over surface text, so reformatting never changes a unit's identity, only the bytes a person reads.

TERM **Trust rung**

The rung a unit was **produced** at — `computed`, `document`, `web`, or `model` — which caps the highest status it may ever claim (rule T). `ingest --rung document` is the default: a model reading a document you supplied may propose at most `cited`, never `measured`, however confidently it phrases something.

TERM **Staging**

The holding area a model's output lands in before it becomes part of your store. `ingest` writes candidate units to `.smysl/staged.smy` and exits `10` rather than `0`; nothing from a model reaches a real store without a deliberate `merge --staged` (or `ingest --yes`, accepting the gate ahead of time). The staged file is ordinary surface text, by design — the thing you are asked to approve is the thing you can read.

TERM **Recipe**

A hash of the full conditions of one model call — provider, model, prompt, everything that could change the answer. `usage --by model` groups the ledger by it, so one logical ingest aggregates across vendors instead of looking like unrelated calls.

TERM **Ledger**

A local, append-only record of what was called, how many tokens it cost, and under what recipe — never the content itself. `usage` reads it back; `usage --reset` discards it.

TERM **Label binding**

The record that remembers a label names a particular unit. You write the label as part of the unit; the binding is how it reaches the wire, and it is a separate record because a label is not identity — a label inside hashed content would make renaming one produce a different unit. Two stores binding the same label to different uids are a `label-collision` contention on merge. New in 0.2: before it, labels survived a parse and not a store round trip.

TERM Comment

A line beginning `#` or `//` at column 0, outside any record. Skipped by the parser and not carried by any record, so canonical form cannot reproduce one — `fmt` warns before it drops them. A comment is a comment wherever it appears, including inside a body; a body that needs to open a line with a marker escapes it as `\#` or `\//` (0.6).

TERM Contention

What `merge` materialises instead of silently picking a winner: a record naming the unit in dispute, every position taken on it, and why it was detected — `live-rebuttal` (a `rebuts` edge landed both sides in the same graph), `supersession-fork` (two units both claim to supersede the same target), or `label-collision` (the same label resolves to two different uids across the merged sources). `merge --fail-on-contention` turns any of these into exit `5`.

TERM Blast radius

Everything a retraction would reach — every unit that grounds on the target, recursively — reported **before** anything is applied. `retract --dry-run` only ever reports it; dropping that flag applies the retraction and reports how many units now read as `unfounded`. A small blast radius is itself informative: it says the retracted line of reasoning was a dead end, not a foundation anything else was built on.

TERM Salience

A score in `[0, 1]` built from three named, individually-weighted terms — **centrality** (how much else depends on this unit), **corroboration** (how many independent agents attested it), and **role** (where it sits in a thread) — never a single opaque number. `salience --explain` breaks any one score back into those three terms.

TERM View

A name plus a root set — never a container. Everything reachable from the roots belongs to it at zero copying cost; `bundle` is what turns that reachable closure into a portable, self-contained store.

TERM Conformance class

One of five named tiers `check --conformance` certifies a store against — `C-Read` through `C-Full` — each one a safety statement about what the store may correctly be used for (reading, consuming, producing into, merging), rather than a statement about whether any individual claim is true.

TERM **Fidelity**

What a consumer implementing certain schemas can do with a store: `Full` (everything `requires` names is implemented), `Degraded` (an extension is missing — read what you can), or `Refuse` (the kernel major itself isn't supported — the one case that is not allowed to degrade silently).

TERM **Profile**

A named bundle of **rendering** choices — register, audience, verbosity, how deep to go by role, how to display status — entirely separate from the document itself. `render --profiles` lists the built-ins; a profile that would flatten epistemic status entirely (rule V1) is refused before a single byte is emitted.

TERM **Model boundary**

The line around the only three operations that may ever consult a model: `ingest`, `attest`, and `thread --refine`. Everything on the far side of it — selecting, merging, ordering, ranking, rendering — is a pure function of its inputs, verified in CI rather than merely intended.

TERM **Facade crate**

The `smysl` crate itself: a `[lib]` re-exporting the public surface of every other crate in the workspace, plus a `[[bin]]` that is a thin shell over it. Nothing the CLI does is unreachable from the library — every `cmd_*` function in `src/main.rs` calls straight into a facade re-export, never around it.

TERM **Grounds**

The list naming exactly which units a claim depends on. A claim's status can never outrank the weakest thing in its `grounds` (rule M) — retract a ground and the tool can tell you exactly what falls with it.

AFTERWORD

About the author

Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. A monitoring system that cannot say where a number came from is not a monitoring system; it is a rumour with a dashboard.

That instinct is the whole of what this book describes. `smysl` records where a claim came from, refuses to let confidence rise as it travels, and never invents a verdict it cannot account for — and it never writes the document. The move is the same one observability made thirty years ago, pointed at a newer problem: as soon as several systems hand work to each other, somebody has to be able to ask **where did this come from** and get an answer that is not a guess. Language models made that question urgent. They did not make it new.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not small utilities, but programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistackvm`, `bundcore` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work. `smysl` grew the same way: the question of how to move a claim between two systems without losing what backs it turned out to have no answer worth adopting, so it got one.

A work of love

`smysl` is open source, under the Mozilla Public License — you can read it, fork it, study it, modify it, and pass it on, and the licence asks only that improvements to its own files travel with them. Strictly speaking the licence also lets you sell it; the author would, gently but firmly, disagree with your doing that. `smysl` was not designed as a *for sale* project. It is a work of love made for the people who can least afford to pay for software — the researcher on a battered laptop, the graduate student whose pipeline has to be defensible at a viva, the small team accountable for a system nobody funded properly — and turning it into a commercial product would betray the reason it exists. It carries no analytics, no telemetry, no upsell. Twenty of its twenty-two commands cannot open a socket at all, and the two that can will tell you what they would send before they send it. The binary will never phone home.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

There is a version of the current moment in which machines talk mostly to other machines and people are handed the summary afterwards. This project is a small argument against that version — not by keeping models out, but by insisting that whatever passes between them stays legible to the person who has to answer for it. If this book helps you hand somebody a document they can actually check — or lets you check one that was handed to you — that is enough.

Where to find more

GitHub `@vulogov` — the source for `smysl`, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and pull requests welcome.

LinkedIn `/in/vladimirulogov` — posts on observability, the occasional long-form essay.

YouTube `@vulogov` — talks and walkthroughs from the conference trail.

If the tool ever gets in the way of the work instead of out of it, open an issue on GitHub. The author reads them.

END OF THE BOOK